WAKARU

November 12, 2026 — working notes, results preliminary

Dictionary Learning as Unsupervised Steering-Vector Generation



Keira Wiechecki
Wakaru · kaw504@nyu.edu

Lamb Wakefield
Flock Capital · lamb@flock.holdings

Jade Zaslavsky
Wakaru · jadeofnameless@gmail

vmfunc
Flock Capital · celeste@linux.com

Abstract

Auditing a model for a hidden objective or an unnamed concept requires identifying its internal variables without labels. Supervised probes and difference-of-means vectors outperform sparse autoencoder (SAE) features at detection and steering, but supervised methods only test concepts that an experimenter already suspects. We argue that unsupervised SAEs struggle in part because of the form of their features: each latent either fires or stays silent, absorption makes silence uninformative, and effective steering vectors instead contrast alternative values of a variable. The Ontologizer learns categorical variables directly via stacked layers of competitive classifier heads, each assigning every input to one of k entries associated with a learned atom. Forcing a head to another entry swaps one atom for another, an unsupervised analogue of a difference-of-means vector, so steering can test a discovered variable, though only against arbitrary partitions of the same space, which steer their own cells just as specifically on both SONAR and GPT-2. Once the code is discrete, categorical structure incurs little distortion: on SONAR sentence embeddings, a pure 1900-bit hard code reaches $\mathrm{FVU}_w = 0.158$, whereas a 160-latent TopK SAE at the same bit budget cannot beat 0.236 (though entropy-coded SAEs with variable or wider supports achieve lower error); on GPT-2 small, an 800-bit hard code recovers 97.6% of next-token loss, surpassing a k_{SAE}=32 SAE at similar error. In the soft code, forced entries steer along the supervised difference-of-means direction 0.64× as far as supervised steering and compose nearly additively. However, the learned variables are not yet identifiable: independent seeds match in error to four digits while sharing no heads past the first layer, input-space steps flip other heads as often as random steps of equal norm, and parameter-only decodes fail to describe entries. The one exception is a block of heads rewarded for maintaining consistent entries across two sentences of a document, which reproduces as a block across three seeds (matched NMI 0.41 vs. a 0.005 null), though not yet head by head, and varies more slowly than any linear projection of the embedding. Identifiability rather than fidelity is therefore the binding constraint for auditing and stable goal specification: reconstruction loss alone does not enforce it, and paired-view training is the first objective that improves it.

1 Introduction

Mechanistic interpretability can currently perform supervised steering well or unsupervised steering poorly. The Google DeepMind mechanistic interpretability team has pivoted from ambitious reverse-engineering to pragmatic interpretability evaluated on proxy tasks, and has deprioritized sparse autoencoder (SAE) research: SAE probes did not generalize better out of distribution, and SAEs did not outperform a steering vector built from a single contrastive pair of prompts at suppressing a model's awareness of being evaluated [1]. On current benchmarks, supervised difference-of-means vectors steer better than SAE latents, though still worse than prompting [2]. However, supervised methods only find concepts for which an experimenter already has labeled examples, and a direction optimized to produce a desired output change need not reflect how the model normally computes: a subspace intervention optimized for its effect can instead activate a dormant pathway [3]. Labels are also what has made a supervised direction divisible. The single direction whose ablation removes a model's refusals [4] has been split into vectors that each disable refusal for one topic, but only with the topics labeled in advance and an explicit penalty on each vector's effect on the other topics, without which the vectors revert to the shared direction [5].

Steering vectors can be learned without labels and without a dictionary, by training a perturbation of fixed norm to maximize the change it produces in a later layer's activations on a single prompt [6]. That work reports its vectors largely qualitatively, with neither a quantitative random-direction baseline nor a repeat across initializations, each of which, it notes, may reach a different stationary point. The dominant unsupervised paradigm is the SAE, which approximates a model's hidden state using a few discrete features drawn from an overcomplete dictionary, originally with a `ReLU` activation and a sparsity penalty [7, 8] and more recently with a `TopK` activation and no explicit sparsity loss [9]. The underlying motivation is to decompile a model into a finite state machine, yet such a machine is only as interpretable as an enumeration of its states. Even a mixture of two simple processes (one constant and one a fair coin) requires infinitely many states to track which component is active, so the structure lies in how the components overlap [10]. Ideally, mechanistic interpretability would provide guarantees about the computational process that a model uses, yielding features that are causal, disentangled, and closed-form. Existing SAE features can fail on all three criteria while remaining computationally expensive.

1.1 Problems with SAE features

Features are not disentangled. A representation is disentangled when changing a single underlying factor of variation changes a single factor of the representation [11]. A persistent problem with SAEs is feature absorption [12]: when a narrow feature implies a broader one, sparsity favors folding the broad feature's direction into the narrow latent and leaving the broad latent inactive on those inputs, so a seemingly monosemantic latent fails to fire on subset cases (for instance, a "starts with S" latent that remains silent on "short"). A related failure mode, feature composition [13], merges frequently co-occurring concepts into a single latent so that neither can be modified independently. Nested and hierarchical SAE variants mitigate both

issues (Section 2).

Features are not cleanly causal. Sparsity trades off against reconstruction fidelity [9], and using SAE features for steering often produces inconsistent, context-sensitive effects: adding a decoder direction shifts logits differently across contexts and perturbs unrelated downstream features. While steerability varies across features and can be partially predicted before intervening from decoder-direction crowding and direct logit effects, the most predictive metrics vary across models and dictionaries [14].

Features are not closed-form. A major concern is how well an interpretability technique generalizes out of distribution; ideally, behavior could be inferred from the model's weights alone. Bilinear MLPs make this partly possible because the interaction matrix for any output direction can be eigendecomposed without data, though in deep networks the target output directions still depend on data-driven SAEs [15].

Forward passes are expensive. Although only a handful of features are active for a given input, the entire feature space must be evaluated for each sample [9, 16]. Mixture-of-experts (MoE) SAE variants reduce this cost by dividing the feature space across smaller submodules (Section 2).

1.2 Contrastive classifiers

Less explored is the effect that the structure imposed by an MoE architecture has on the interpretability of feature space. A common approach to interpreting features at scale is to have an LLM generate an explanation from activating examples [17] and score how well an LLM judge, given that explanation alone, distinguishes activating from non-activating contexts [18]. With an unstructured feature space, it quickly becomes intractable to interpret every combination of features in this manner, whereas structured expert routings can be analyzed module by module. The ReLU feed-forward layers of a trained transformer can themselves be partitioned into experts of co-activating neurons, retaining over 95% of task performance while activating only 10–30% of the parameters [19]. Language models also appear to represent a categorical concept as a simplex whose vertices correspond to category values, with hierarchically related concepts placed in orthogonal subspaces so that a hierarchy forms a direct sum of simplices; differences between vertex vectors then act as steering vectors for binary contrasts [20].

We conjecture that MoEfying a representation into classifiers can expose such latent contrastive steering vectors, because swapping a classifier's selected entry applies the difference between two entry vectors. Existing MoE SAEs route similar inputs to the same expert [16] and score an expert as specialized when its features resemble one another and differ from those of other experts [21]. Contrastive learning requires the opposite structure, grouping inputs that lie at opposite ends of a shared axis. The Ontologizer[1] decomposes a representation space into a set of such contrastive classifiers. Each *head* is a k-way classifier over its input, and each of its *entries*

[1] Gene Ontology terms were a source of inspiration for the name, which this work shares with Ontologizer, a tool for Gene Ontology term enrichment analysis [22]; the two are separate projects.

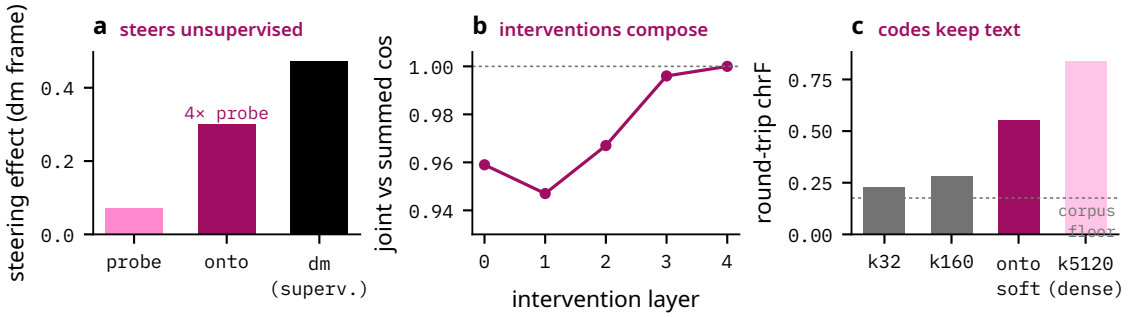


Figure 1 **Steering and reconstruction in the softmax Ontologizer.** (a) Forcing an entry moves decodes along the supervised difference-of-means direction 0.64× as far as supervised steering itself and 4× as far as probe directions (`resid_nc_hm`; Table 17). (b) Joint two-head interventions align with the sum of the individual interventions, and match it exactly in the final layer (`resid_nc`; Section 9). (c) Sparse SAE reconstructions lose most of the text before any intervention is applied (round-trip chrF 0.23–0.28 vs. a 0.18 corpus floor), whereas the soft code preserves it (0.55); the dense `k5120` SAE preserves more text but is not a sparse code (Appendix C.5).

corresponds to a representative vector, its *atom*. An input is encoded as one *assignment* per head and reconstructed from the atoms that those assignments select across a stack of layers, each of which takes as input the residual left unexplained by earlier layers. Interpretability is then expected from *MoEfication* in an extended sense [19]: decomposing the representation space into small classification tasks, one per head, each nameable in isolation, where within-head entry swaps act as steering moves. Figure 1 summarizes the steering and reconstruction behavior of the softmax Ontologizer.

1.3 Disjoint support

In the concatenated-head variant, each head writes to its own block of dictionary coordinates, so atoms from different heads have disjoint supports by construction. Disjoint support is a strong form of orthogonality: for non-negative atoms the two coincide (Eq. (8)), whereas signed vectors can be orthogonal through cancellation while sharing every coordinate. When heads are summed across the full dictionary space, disjoint support can only be encouraged via a penalty and does not form in practice: no trained summed model partitions its dictionary coordinates between heads, and penalizing support overlap on GPT-2 yields a coordinate partition only by shifting atom energy into directions that the decoder discards or shrinks, where it does not affect reconstruction (Section 4.1.4, Appendix G.2). Enforcing disjoint support by concatenation affects each of the four SAE failure modes above differently. In every case, the guarantee holds between heads within a single layer's dictionary space; outputs still pass through a shared linear decoder, which in general maps different heads' blocks onto overlapping subspaces.

Disentanglement. Feature absorption requires both a general latent that turns off when a more specific latent fires and a specific latent whose direction absorbs the general one. Classifier heads prevent the first step because every head selects an entry on every input and cannot simply

turn off, while disjoint dictionary blocks prevent the second step in dictionary space because no entry of one head has nonzero weight on another head's coordinate block. Feature composition is discouraged rather than ruled out. Because h heads of k entries define k^h combined codes from hk atoms, independent factors can occupy separate heads, and theory characterizes when an efficient code separates them. Under a quadratic activity cost and bounded weights [23, 24], a non-negative code assigns two factors disjoint supports when their joint distribution covers a broad enough product range, roughly when every combination of their extreme values occurs [24]. Although that condition applies to the unknown generative factors of the data, our learned codes are consistent with it: among entry pairs from two heads that would occur at least ten times under independence, at most 0.24% have zero count in any layer of a SONAR model. Without activity regularization, summed models do not partition their coordinates (Appendix G.2), whereas concatenated heads enforce the partition by construction. Even so, nothing forces a single head to capture one concept rather than a conjunction of two, and unsupervised learning from marginal data alone cannot recover underlying generative factors without matching inductive biases on both the model and the data [11]. Head structure provides one such architectural bias, and Section 6.5 measures how far it extends.

Causality. One source of steering collateral in SAEs is crowding among decoder directions [14]. With concatenated heads, atoms from different heads never overlap in dictionary space: forcing a head's entry modifies only that head's coordinate block, orthogonal to the outputs of all other heads in the same layer, so interventions on different heads shift the layer's dictionary representation along orthogonal directions and each head's contribution can be isolated directly from its block. Summed heads also compose additively by linearity, but because their dictionary writes overlap, a single head's contribution cannot be separated from the others' in the pooled vector. Disjoint dictionary blocks do not, however, govern how other heads respond when an input is perturbed in input space. When evaluated on summed models by re-encoding a stepped input, an input-space step flips as many other heads as a random step of equal norm, both because heads sit close to their decision boundaries and because downstream layers reclassify the modified residual (Section 6.4). Disjoint dictionary writes eliminate neither mechanism.

Closed form. Statically interpreting a head from its atoms without data requires heads to occupy disjoint coordinates as a weight-level property across all inputs, rather than merely exhibiting low average overlap on a dataset (Section 4.1.4). Summed models violate this condition, with each head's usage spanning 52–98% of dictionary coordinates. Concatenated heads satisfy it by construction, and when each head's decoder block has full column rank, every component of a head's atoms maps nontrivially to the output, unlike the degenerate partitions produced by penalizing overlap (Appendices G.2 and G.3). However, this structural property is a necessary precondition rather than a complete interpretation method: decoding atom directions through the paired decoder has not yielded usable descriptions of entries in the models evaluated so far, none of which used concatenated heads (Section 5.5).

Cost. Disjoint support does not reduce encoder compute. Concatenating heads partitions a dictionary of width e across h heads, so each head stores k atoms of e/h coordinates rather than e (h times fewer dictionary parameters at matched e), and a hard forward pass in the dictionary reduces to h slice lookups that assemble a single e-dimensional vector. However, the classifier still evaluates all hk logits per input, just as an SAE evaluates all of its latents, leaving encoder compute unchanged. Narrow coordinate blocks can also degrade reconstruction. At d_{head}=k=32, where each head's block is only as wide as its k non-negative atoms under constant temperature, concatenated heads had the highest held-out error among the GPT-2 orthant arms ($\text{FVU}_w = 0.416$ vs. 0.154–0.293; Table 7), because k orthogonal non-negative atoms within a head require one coordinate each (Appendix G.3). Giving each head additional dimensions (d_{head}=48 > k=32) and annealing the temperature brings concatenated heads to an unwhitened FVU of 0.186 (0.164 at k=64), outperforming direct atoms at 0.195 (Section 7).

1.4 This document

This document serves as the working record of our experimental campaign. It compares a softmax (soft-code) Ontologizer and a straight-through (hard-code) arm against `TopK` SAEs on SONAR sentence embeddings under a shared whitened reconstruction objective, and evaluates hard-code variants on GPT-2 small residual-stream activations. Section 2 situates the architecture relative to prior work, Section 3 defines our evaluation questions, Section 4 presents the architecture and training objective, and Sections 5–8 report the empirical results.

2 Background

No single line of prior work combines all elements of this architecture, though each design choice has close counterparts in neighboring areas.

Quantizers and discrete latents. A `DictBlock` layer (h heads, each computing a softmax over k entries whose atoms are summed) is a soft additive quantizer [25], generalizing product quantization [26] by allowing every codebook to span the full space rather than a disjoint block of coordinates (concatenated heads recover a product quantizer in dictionary space). Composite quantization [27] lies between the two, holding the summed inner products between codewords from different codebooks constant (where product quantization's disjoint blocks fix them at zero), while optimized product quantization [28] learns the rotation that defines the coordinate blocks. In dictionary-learning terms, a single hard head performs vector quantization (VQ) with one unit-coefficient atom, and a sum of heads lies between VQ and unconstrained sparse coding, selecting exactly one unit-coefficient atom per codebook. Under the gain–shape split (Eq. (14)), each layer acts as gain–shape VQ whose scalar gain is the residual's observed norm rather than a quantized index [29]. Stacking residual layers yields a softmax relaxation of multi-stage residual VQ [30, 31], the quantizer used in neural audio codecs such as SoundStream [32], which selects each stage's codeword by nearest-neighbor search and evaluates codes solely by bitrate and perceptual quality rather than codeword interpretability; residual VQ has also been applied directly to SONAR embeddings using 64 codebooks of 8192 entries [33]. The straight-

through arm (Section 6) makes this connection explicit: using a hard argmax in the forward pass and the softmax gradient in the backward pass—the straight-through estimator [34] in its categorical form [35]—it forms a residual additive quantizer with non-negative atoms whose codewords are selected by learned classifiers rather than nearest-codeword search. Vectors of categorical variables trained with straight-through gradients also serve as the latent state of discrete world models [36, 37], which sample each categorical variable from the softmax distribution that the backward pass differentiates rather than taking its argmax (DreamerV3 additionally mixes in a 1% uniform prior [37]). In those models, however, the discrete codes serve purely as compressed state and are not interpreted as features. Codebook Features [38] is the closest ancestor in mechanistic interpretability: it fine-tunes VQ bottlenecks into a transformer, placing one codebook per attention head before the shared output projection (analogous to concatenated heads with a shared decoder), and steers behavior by substituting codewords, including swapping one state's codeword for another's on a synthetic finite-state task. However, its codewords are selected by cosine similarity to the codebook vectors rather than by a separate classifier, are interpreted individually as binary present-or-absent features, and are evaluated solely on target effect without a structured intervention algebra or a measure of off-target collateral.

Routed and mixture-of-experts dictionaries. Switch SAEs [16] route each activation to one of several expert TopK SAEs to reduce training compute. Their router groups similar inputs together, and experts duplicate some features (5–10% of decoder directions have a near-duplicate in another expert), a phenomenon that Xu et al. characterize as experts collapsing into redundant ensembles and mitigate by activating multiple experts per input [21]. A classifier head layer adopts the dual arrangement, in which every head is active on every input and selects a single entry. In the hard-code stack, heads partition inputs nearly independently (Figure 31), whereas in the reference softmax model, within-head decoded directions repel one another (Section 5.3), so each head forms a contrast set rather than the cluster of similar features that MoE methods treat as a specialized expert. Although MoEfication [19] motivates our terminology, its experts are extracted post hoc from pre-trained layers and multiple experts fire per input; here, entries within a head are mutually exclusive, and the partitions are learned end to end during dictionary training.

Deep clustering. Each head is an unsupervised k-way classifier, connecting our training objective to deep clustering. Specifically, the batch-mean uniformity term KL_m (Eq. (4)) mirrors the anti-collapse regularizer of contrastive clustering [39] and SCAN [40], preventing heads from assigning most inputs to a few entries, while the partition-consensus statistic $\mathrm{KL}_{\mathrm{pwak}}$ (Eq. (22)) pulls each assignment toward a stop-gradient average of its neighbors' assignments, analogous to DEC's self-training target [41] and SCAN's neighbor-consistency loss. Whereas deep-clustering methods learn a single partition of a trainable representation and are evaluated against ground-truth labels, the Ontologizer trains many heads simultaneously to partition a frozen representation through reconstruction loss alone, with the resulting partitions serving directly as the representation code.

Structured SAEs. Sparse autoencoders aim to recover features in superposition [42] as sparse linear combinations of directions from an overcomplete dictionary [7, 8]; our `TopK` baselines are k-sparse autoencoders [43] trained following Gao et al. [9]. One major axis of structural prior in recent SAEs is hierarchical scale: Matryoshka SAEs [13] adapt the nested-prefix loss of Matryoshka representation learning [44] to dictionary learning in order to reduce feature absorption [12], while Tree SAEs [45] enforce explicit parent–child structure by gating child latents on assigned parents. A second axis is subspace grouping: subspace-aware SAEs [46] bundle latents into rank-r blocks and enforce sparsity across blocks so that each active block spans a continuous signed subspace, and block-sparse featurizers [47] motivate this prior by modeling activations as sums of low-dimensional manifolds that single-direction sparse codes must otherwise tile with many separate atoms [48]. A block-sparse unit is thus the structural opposite of a classifier head: blocks are selected sparsely and vary continuously within their subspaces, whereas every head is active on every input and selects one of k discrete atoms. The data model closest to a layer of heads is the Minkowski Representation Hypothesis [49], proposed on preliminary evidence from DINOv2 and motivated by multi-head attention, in which each head outputs a convex combination of its value vectors: an activation is a sum of convex combinations, one from each of a few groups of archetypes, so that the activation space is a Minkowski sum of convex hulls. Two consequences drawn there bear on what follows. Such a sum does not determine its summands, so the groups cannot be recovered uniquely from one layer's activations; and steering toward an archetype, unlike steering along a direction, saturates on arrival, which is the kind of move an entry swap makes. OrtSAEs [50] instead penalize the cosine similarity between each decoder direction and its nearest neighbor. Rung 3 of our ablation ladder (`k32_p5`) incorporates the Matryoshka nested-prefix loss using five equal-width blocks rather than the geometrically doubling prefixes of Bussmann et al. [13], a logarithmic spacing that Matryoshka representation learning found superior to uniform spacing [44]. By contrast, the Ontologizer structures the dictionary by *partitioning features into exhaustive, mutually exclusive heads*. This design mirrors the geometry observed in toy models of superposition, where mutually exclusive features share a low-dimensional subspace with negative internal interference while independent feature sets occupy mutually orthogonal subspaces [42]; a classifier head enforces within-head exclusivity by construction and encourages between-head orthogonality (Section 4.1.4). Outside dictionary learning, Huang and Hahn [51] decompose a residual stream without supervision into mutually independent orthogonal subspaces under the hypothesis that each subspace holds mutually exclusive features, though their subspaces carry continuous activations rather than discrete k-way selections. Post hoc analyses also show that co-occurring LLM SAE latents cluster in decoder space by document genre [52], matching the co-firing structure that `headstruct.py` recovers among SONAR SAE latents in place of categorical groups (Appendix C.1). Each of these structured approaches tests whether architectural inductive biases yield interpretability beyond what code density alone explains. In our autointerp evaluations below, per-latent describability shows no such gain; head structure only shows a suggestive effect in within-head description coherence on the single Ontologizer checkpoint evaluated.

Embedding-space substrate. Training SAEs on dense text embeddings has recently attracted growing interest, including interpreting and discretizing dense-retrieval embeddings [53], aligning sentence-embedding features with human concepts to steer retrieval [54], and constructing interpretable document embeddings from token-level LLM SAEs [55]. All three approaches use single-layer `TopK` or BatchTopK SAEs, with the embedding-level studies focusing on English retrieval encoders. None examines SONAR, where sequence-level generation models such as Large Concept Models operate directly in embedding space [33] and where the paired M2M100 decoder maps reconstructions and interventions back into text without requiring a separately trained inverter. Inverting pooled sentence embeddings otherwise requires training a dedicated decoder that recovers exact text only by iteratively re-embedding and correcting its candidates [56], as exact training-free inversion is known only from per-token causal language model activations [57] rather than bidirectional mean-pooled embeddings. Because `textfid.py` and `steerfid.py` decode in a single greedy pass through a generative decoder that paraphrases and is sensitive to small embedding perturbations [58, 33], our text-space fidelity scores represent conservative lower bounds (Appendix C.8).

Evaluation stance. SAEBench [59] motivates evaluating dictionaries across diverse downstream criteria rather than solely by reconstruction at fixed sparsity. From its eight metrics, our evaluation suite adapts three to SONAR (a downstream fidelity analogue through the paired decoder, `textfid.py`; LLM-judged feature detection, `autointerp.py`; and k_{probe}-sparse probing over 86 language labels, `langprobe.py`) while omitting absorption, RAVEL, unlearning, SCR, and TPP. For automated interpretability, we adopt the detection protocol of Paulo et al. [18]; however, because we evaluate positive examples ranked immediately below the prompt-building examples and report F1 rather than balanced accuracy, our numerical scores are not directly comparable to theirs. Following MDL-SAEs [60], we compare model capacity in bits rather than L0, since an L0 axis favors arbitrarily wide dictionaries with fewer active latents; their description-length metric (summed per-latent usage entropy) corresponds to the realized-bits estimator that we apply to heads (Eq. (36)). On AxBench [2], prompting and fine-tuning outperform all representation-based steering methods across four Gemma-2 sites. Among single-direction methods, only supervised difference of means outperforms SAE latents, whereas linear probes match the strongest concept detectors yet steer poorly, and SAE latents selected via autointerp descriptions lag behind on concept detection. This dissociation between detection and steering motivates evaluating dictionary directions directly by their causal effects. Accordingly, `steerfid.py` measures target effect against off-target collateral at matched perturbation magnitude, following Cunningham et al.'s evaluation of feature edits across intervention magnitudes [7] rather than tuning per-method steering strengths on held-out prompts as in AxBench. Song et al. [61] further argue that cross-seed feature consistency should be a core SAE evaluation criterion, quantified via the mean absolute cosine similarity of one-to-one matched decoder directions across seeds; our Hungarian-matched head NMI (`partition.py`) provides the partition-level analogue for classifier heads, while activation-containment match fractions (`splitting.py`) provide a thresholded activation-based analogue. Wang et al. [62] similarly measure cross-run consistency via each latent's maximum activation correlation with latents

from a second run relative to model-seed and SAE-seed skylines. Paulo and Belrose [63] find that two TopK SAEs trained on the same data in the same order from different initializations share only 42% of their latents (matched one to one on both encoder and decoder directions at cosine ≥ 0.7), fewer as the dictionary widens, so seed dependence is not peculiar to classifier heads. Korznikov et al. [64] show that SAEs trained with the decoder frozen at a random initialization or held near it, or with the encoder frozen at random, score within a few percentage points of fully trained SAEs on autointerp, single-latent sparse probing, and RAVEL, and Heap et al. [65] find that SAEs on randomly re-initialized transformers reach autointerp scores close to those on trained ones in the larger models they test, the two being separated instead by how token-specific the latents are; our permutation nulls shuffle a trained model's own entries, latents, or sample rows, and therefore do not isolate how much training improves over random projections in those metrics. Decoding parameter vectors through SONAR's text decoder to test weight-only interpretability serves as a closed-form instance of dataset-free feature synthesis, an approach that struggles even on planted ground-truth triggers, helping human evaluators identify triggers less than half the time [66]; parameter-only decodes fail in our setting as well. Two elements of our framework have, to our knowledge, no close counterpart in prior work: treating per-head routing as contrastive classification equipped with a direct causal intervention interface, and testing within-head and within-group coherence against size-matched random subsets from the same dictionary across both weight geometry and LLM-generated descriptions (by contrast, Li et al. [52] test global co-occurrence clusterings of SAE latents against permuted labels).

3 Evaluation frame

Many of the comparisons below follow standard SAE evaluation criteria (per-feature monosemanticity, sparse probing, and reconstruction at matched sparsity), and we report those measurements unchanged. However, the goal of the Ontologizer is not to learn a monosemantic dictionary of independent latents, but rather to serve as an *unsupervised generator of steering vectors*. Each head is trained solely via reconstruction loss with the aim of partitioning the representation space into contrastive categories, where within-head entry swaps provide the steering primitive: forcing head i from its argmax entry to entry j' shifts the decoded output along the axis that head i classifies.

Under this framing, our evaluation addresses four primary questions: (a) what reconstruction penalty contrast-set structure incurs (the Pareto frontier and structural ablation ladder); (b) whether trained heads behave as contrastive classifiers in practice (categorical selectivity and within-head semantic repulsion); (c) whether the learned partitions capture *natural* factors that are interpretable, reproducible across runs, and aligned with generative factors of the data (cross-seed reproducibility, language, and topic alignment); and (d) whether entry swaps provide effective steering with consistent target effects and low collateral, comparable to supervised difference-of-means vectors and SAE decoder-direction addition (steering fidelity). Metrics that test whether *individual latents* align with isolated concepts remain informative for the SAE baselines, but are ill-suited to evaluating classifier codes; we note this distinction wherever such metrics appear. Figure 1 illustrates the softmax Ontologizer on three criteria under question (d),

whereas the hard code steers less reliably than the soft code (Section 5.4).

Question (c) depends on a prerequisite that our initial evaluation battery took for granted: an individual head can be assigned a stable interpretation only if the same head emerges across independent training runs. Section 6.5 tests cross-seed reproducibility directly and finds that it holds only partially, and only in the first layer, a limitation that qualifies every per-head result in this document.

4 Methods

4.1 Architecture

We propose a method that encodes samples as classifications mapped to contrastive vectors. This directly constrains a model's representations, and classifications can natively be used for steering. Appendix A summarizes our notation.

The model (Figure 2) consists of l `DictEnc` layers writing into a shared residual accumulator, decoded by a single linear map. Each layer (Figure 3) is composed of a classifier followed by a dictionary. The classifier splits its input into h independent *assignments*, each a distribution over one head's k *entries*, and the dictionary maps each entry to a representative vector (an *atom*). Each layer writes the assignment-weighted sum of its atoms to the accumulator; after each layer the accumulator is decoded and the residual $\mathbf{x} - \mathrm{dec}(\mathbf{r}_\ell)$ is passed to the next layer, where $\mathbf{r}_\ell$ accumulates the ℓ layers before it. Deep supervision [67] constrains every prefix of the stack to reconstruct the input. Because each layer takes the current residual as input, components encoded by earlier layers are removed from the residual passed to later layers, making the code a successive refinement that is coarse-to-fine in distortion. Unlike in a Tree SAE [45], no later entry is tied to an earlier one, so whether the code is also coarse-to-fine in concept is an empirical question.

Because entries within a head are mutually exclusive rather than independent, each dictionary head acts as a contrastive set whose entry swaps are steering moves. An auxiliary loss term minimizing the between-head support overlap ω (Eq. (6)) encourages each head to occupy a disjoint set of dictionary coordinates (Figure 5), and because each head computes a convex combination of its atoms, its outputs lie in the simplex spanned by those atoms (Section 4.1.4).

DictEnc. Each `DictEnc` consists of a multihead bilinear classifier [68, 69] (`BilinearBlock`) with weights $[\mathsf{W}, \mathsf{V}] \in \mathbb{R}^{2\times h\times k\times d}$ ($d+1$ with the constant coordinate of Eq. (14)) and a multihead linear dictionary (`DictBlock`) with weights $\mathsf{D} \in \mathbb{R}^{h\times k\times e}$. Layer ℓ receives the residual $\mathbf{y}_\ell$ left by earlier layers ($\mathbf{x}$ itself at layer 0), from which the classifier takes the normalized shape vector $\mathbf{u}_\ell$ as input (Eq. (14)). Algorithm 1 specifies the single-layer training pass, and Algorithm 3 specifies the full stack and loss computation.

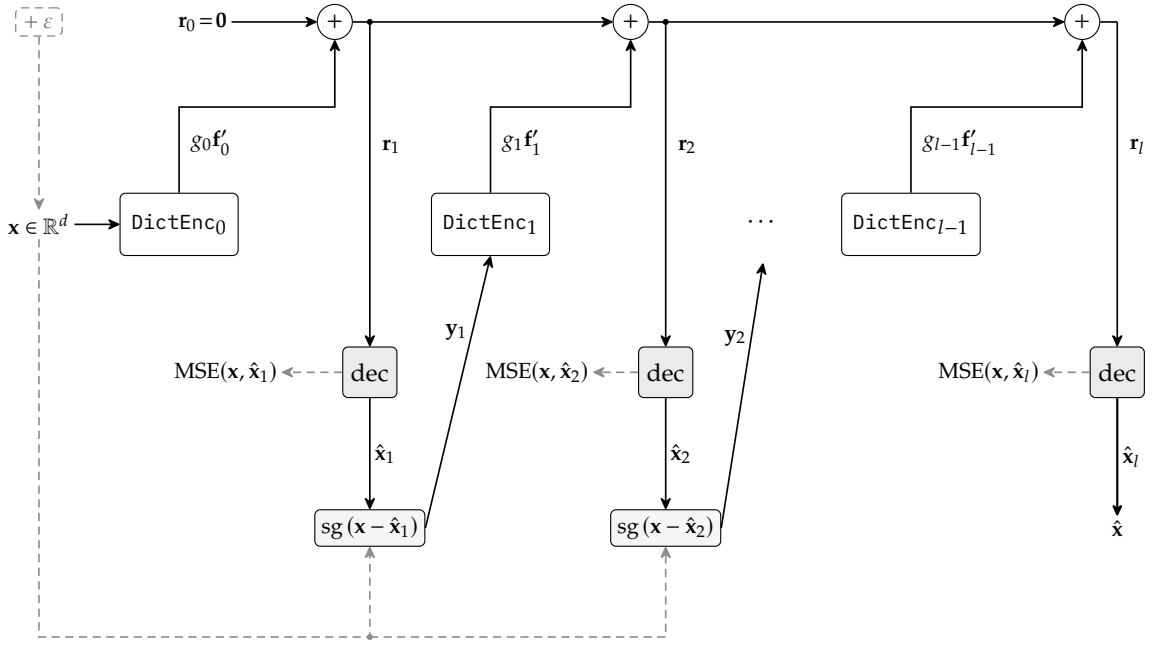


Figure 2 **`Ontologizer` architecture.** An `Ontologizer` stacks l `DictEnc` layers, each writing its gain-scaled pooled output $g_\ell \mathbf{f}'_\ell$ into a shared residual accumulator $\mathbf{r}$. Accumulator states are indexed by the number of layers applied: layer ℓ receives $\mathbf{r}_\ell$ and the residual $\mathbf{y}_\ell = \mathbf{x} - \mathrm{dec}(\mathbf{r}_\ell)$, and produces $\mathbf{r}_{\ell+1} = \mathbf{r}_\ell + g_\ell \mathbf{f}'_\ell$. A single linear decoder dec is shared across all layers, and every prefix reconstruction is supervised against $\mathbf{x}$ via deep supervision. The dashed input-noise hook on $\mathbf{x}$ is disabled in every model reported here; Section 4.1.5 gives the full equations.

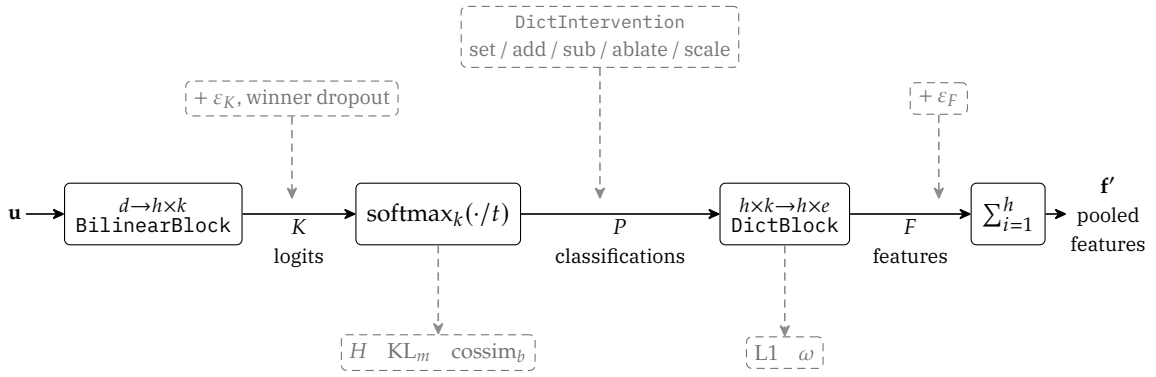


Figure 3 **`DictEnc` architecture.** A multihead bilinear classifier maps the normalized shape vector $\mathbf{u}$ to logits K; assignments $P = \mathrm{softmax}_k(K/t)$ (or their straight-through argmax) weight the dictionary atoms to form per-head features F, which are summed across heads into pooled features $\mathbf{f}'$. Dashed boxes above the main path represent training perturbations and causal interventions; dashed boxes below represent summary statistics that serve as tunable auxiliary loss terms (Sections 4.1.2–4.1.4).

Algorithm 1: Single-layer `DictEnc` training pass (`DictEnc.withStats`). When the router is disabled, $\Gamma = \mathbf{1}$ and $\mathrm{L1}_S = 0$; when fibers are disabled, the coordinates C are omitted; and when `resid_gain` is disabled, the input is not decomposed into gain and shape and $g_n = 1$.

Input: accumulator $R \in \mathbb{R}^{b\times e}$; layer input $E \in \mathbb{R}^{b\times d'}$, whose row n is $[\mathbf{y}_n;\, \mathbf{1}_{n_{\text{const}}}]$: the varying part $\mathbf{y}_n$ (the encoded input at layer 0, the residual above) and n_{const} constant coordinates (`resid_const`)

Data: bilinear classifier (Eq. (1)); router (`scaled`); fiber readouts $A_i \in \mathbb{R}^{r\times d'}$; `DictBlock` (Algorithm 2)

Hyperparameters : logit noise σ_K; fiber bound β; the `DictBlock` hyperparameters

Output: accumulator R; assignments $\mathsf{P} \in \mathbb{R}^{b\times h\times k}$; statistics row $\boldsymbol{\psi}$ (Eq. (22))

begin

$g_n \leftarrow \mathrm{sg}\big[\|\mathbf{y}_n\|\big]; \quad \mathbf{u}_n \leftarrow \big[\mathbf{y}_n/g_n;\, \mathbf{1}_{n_{\text{const}}}\big]$ `// gain and shape`

$\mathsf{K}, \mathrm{L1}_K \leftarrow \mathrm{classifier}(\mathbf{u}_{1:b})$ `// L1 of the gate factor`

$\mathsf{K} \leftarrow \mathrm{noise}_K(\mathsf{K})$ `// Eq. (16) or (17)`

if *router* **then**

$\Gamma, \mathrm{L1}_S \leftarrow \mathrm{router}(\mathbf{u}_{1:b}); \quad \Gamma \leftarrow |\Gamma|$ `// unless router_signed`

else

$\Gamma \leftarrow \mathbf{1}; \quad \mathrm{L1}_S \leftarrow 0$

end

$\mathbf{c}_{ni} \leftarrow \beta \tanh(A_i \mathbf{u}_n/\beta)$ `// fiber coordinates, Eq. (19)`

$F', \mathsf{P}, (\mathrm{L1}, H, \mathrm{cossim}_b, \mathbb{E}_i\kappa_i, \max_i \kappa_i, \mathrm{KL}_m, \omega) \leftarrow$ `DictBlock`$(\mathsf{K}, \Gamma, \mathsf{C})$

$\mathrm{KL}_{\text{pwak}}, \mathrm{L2}_{\text{pwak}} \leftarrow \mathrm{pwak}(\mathsf{P}, \mathbf{u}_{1:b})$ `// zero unless enabled`

$\mathbf{r}_n \leftarrow \mathbf{r}_n + g_n\, \mathbf{f}'_n$ `// row` n `of` R`: the contribution, at the input's gain`

return $R, \mathsf{P}, \boldsymbol{\psi} = (\mathrm{L1}_K, \mathrm{L1}, H, \mathrm{cossim}_b, \mathbb{E}_i\kappa_i, \max_i \kappa_i, \mathrm{KL}_m, \mathrm{KL}_{\text{pwak}}, \mathrm{L2}_{\text{pwak}}, \mathrm{L1}_S, \omega)$

end

Classifier. For a generic input $\mathbf{x} \in \mathbb{R}^d$ (in layer ℓ, the normalized shape vector $\mathbf{u}_\ell \in \mathbb{R}^{d+1}$), the multihead bilinear classifier `BilinearBlock` computes logits $K \in \mathbb{R}^{h \times k}$ via

$$\texttt{BilinearBlock}([\mathsf{W}, \mathsf{V}], \mathbf{x}) := \left(\sum_{q=1}^{d} W_{\cdot\cdot q}\, x_q \right) \odot \left(\sum_{q=1}^{d} V_{\cdot\cdot q}\, x_q \right). \tag{1}$$

Each logit is a quadratic form in $\mathbf{x}$, the low-rank form of bilinear classifiers [68, 69], so the classifier admits closed-form eigendecomposition in the manner of bilinear MLPs [15]. During training, optional Gaussian noise ε_K is added to the logits (Section 4.1.6), and winner dropout sets each head's maximum logit to $-\infty$ with probability p_{drop}.

Assignments and selection. Soft assignments are given by $P = \text{softmax}_k(K/t)$, where the temperature hyperparameter t is annealed during training to sharpen the distributions (schedules in Appendix B.3). Under the *straight-through* selection rule (`ste`), the forward pass selects the one-hot argmax while the backward pass uses $\text{softmax}_k(K/t)$ as a surrogate gradient [34, 35]; intermediate `top`k_{sel} rules apply a softmax to each head's top k_{sel} logits and zero the remainder. Causal interventions act directly on the assignments P.

From P we compute three assignment statistics. The first is per-sample entropy H, measured in bits so that each head saturates at $\log_2 k$ bits (5 bits at k=32),

$$\text{entropy}_k(\mathbf{p}) := -\sum_{j=1}^{k} p_j \log_2 p_j, \qquad H := \mathbb{E}_n\, \mathbb{E}_i \left[\text{entropy}_k(\mathbf{p}_{ni}) \right]. \tag{2}$$

The second is between-sample similarity cossim_b, the mean absolute cosine similarity of the concatenated per-head features over all ordered pairs of samples,

$$\text{cossim}_b := \mathbb{E}_{n,n'} \left| \cos \angle(\mathbf{f}_n, \mathbf{f}_{n'}) \right|, \qquad \langle \mathbf{f}_n, \mathbf{f}_{n'} \rangle = \sum_{i=1}^{h} \gamma_{ni} \gamma_{n'i}\, \mathbf{p}_{ni}^{\top} G_i\, \mathbf{p}_{n'i}, \tag{3}$$

where $\mathbf{f}_{ni} := \mathsf{F}_{ni\cdot} \in \mathbb{R}^e$ is head i's feature vector on sample n, $\mathbf{f}_n = \bigoplus_{i=1}^{h} \mathbf{f}_{ni}$ is their concatenation, γ_{ni} is head i's router scale on sample n (1 without a router; Section 4.1.7), and $G_i = |D_i||D_i|^{\top} \in \mathbb{R}^{k \times k}$ is head i's dictionary Gram matrix, with $D_i := \mathsf{D}_{i\cdot\cdot} \in \mathbb{R}^{k \times e}$ ($G_i = D_i D_i^{\top}$ when the dictionary is signed). Evaluating Eq. (3) in entry space through G_i rather than materializing F yields identical values and gradients while reducing the cost from $O(b^2 he)$ to $O(hk^2 e + b^2 hk)$. Finally, KL_m is the divergence of the batch-mean assignment from uniform, in bits, averaged across heads:

$$\text{KL}_m := \mathbb{E}_i \left[\log_2 k - \text{entropy}_k(\bar{\mathbf{p}}_i) \right], \qquad \bar{\mathbf{p}}_i := \mathbb{E}_n \left[\mathbf{p}_{ni} \right]. \tag{4}$$

In practice an explicit penalty on H is unnecessary, because minimizing cossim_b also reduces per-sample entropy. The batch-mean uniformity term KL_m anchors each head's mean assignment $\bar{\mathbf{p}}_i$ to the uniform distribution: this keeps all k entries active—a head that collapses onto a single entry incurs the full $\log_2 k$-bit penalty—and encourages a uniform assignment $\mathbf{p}_{ni}$ to represent a maximally generic feature vector.

Dictionary. A `DictBlock` maps assignments $P \in \mathbb{R}^{h\times k}$ to per-head features $F \in \mathbb{R}^{h\times e}$, whose row $\mathbf{f}_i$ is head i's output:

$$\texttt{DictBlock}(\mathsf{D}, P)_{i\cdot} = \mathbf{f}_i := \sum_{j=1}^{k} p_{ij}\,\mathbf{a}_{ij}, \qquad \mathbf{a}_{ij} := |\mathsf{D}_{ij\cdot}|, \tag{5}$$

where the atom $\mathbf{a}_{ij}$ is the elementwise absolute value of head i's dictionary vector for entry j ($\mathsf{D}_{ij\cdot}$ itself when the dictionary is signed). Because the atoms are non-negative and $p_{ij} \in [0, 1]$, features are strictly additive; because each head's assignment sums to one, $\mathbf{f}_i$ lies in the simplex whose vertices are the head's atoms (Figure 5), matching the geometry that Park et al. [20] derive for representations of categorical concepts, in which entries represent mutually exclusive alternatives rather than independently firing features. Summed over heads, a layer's output ranges over the Minkowski sum of these simplices, the form that the Minkowski Representation Hypothesis proposes for transformer activations [49]. This lookup takes the form of attention over a learned per-head memory—a softmax over input-dependent entry scores weighting the atoms as values [70, 71, 72]—and, when paired with a linear classifier, coincides with a single update of a modern Hopfield layer at inverse temperature $1/t$ [73]. Algorithm 2 details the full training pass, including the regularizing perturbations and auxiliary statistics computed alongside the lookup.

From the assignments and atoms we compute the between-head support overlap ω, defined as the mean cosine similarity across ordered pairs of distinct heads between their usage-weighted coordinate profiles $\boldsymbol{\pi}_i$:

$$\boldsymbol{\pi}_i := \sum_{j=1}^{k} \bar{p}_{ij}\,|\mathbf{a}_{ij}|, \qquad \omega := \mathbb{E}_{i\neq i'}\,\frac{\langle \boldsymbol{\pi}_i, \boldsymbol{\pi}_{i'}\rangle}{\|\boldsymbol{\pi}_i\|\|\boldsymbol{\pi}_{i'}\|}, \tag{6}$$

with $\bar{\mathbf{p}}_i$ the batch-mean assignment of Eq. (4). The overlap ω vanishes if and only if no two heads share an active coordinate of the dictionary space, and equals one when all heads weight the coordinates in identical proportions (Section 4.1.4). Computing ω requires $O(hke + h^2e)$ operations, independent of the batch size b; weighting by $\bar{\mathbf{p}}_i$ excludes dead entries from the profile and provides a gradient path to the classifier. Figure 4 illustrates the assignments, atoms, and both cosine statistics for a single trained head.

After feature noise ε_F is added to form $\tilde{\mathsf{F}} := \mathsf{F} + \varepsilon_F$ (Section 4.1.6), we also compute the feature sparsity penalty

$$\mathrm{L1}(\tilde{\mathsf{F}}) := \frac{1}{b}\sum_{n=1}^{b}\sum_{i=1}^{h}\sum_{q=1}^{e}|\tilde{f}_{niq}|. \tag{7}$$

Because L1 averages each sample's L1 norm across the batch, it is invariant to b and scales with h and e, so its loss weight is correspondingly small; although L1 decreases under MSE training alone, penalizing it explicitly accelerates convergence.

Disjoint support Between heads, orthogonality of non-negative features acts as a support-partition condition rather than a rotation constraint. Every feature vector $\mathbf{f}_{ni}$ lies in the non-negative orthant of $\mathbb{R}^e$, since $|\mathsf{D}| \geq 0$ and $P \geq 0$. For non-negative vectors $\mathbf{v}, \mathbf{v}'$, the inner product

Algorithm 2: `DictBlock` training pass (`DictBlock.withStats`). At inference time, the four stochastic steps (revival, winner dropout, head dropout, and fiber dropout) and feature noise are omitted. Every statistic is stop-gradiented unless its corresponding loss term is enabled.

Input: logits $\mathsf{K} \in \mathbb{R}^{b\times h\times k}$; router scales $\Gamma \in \mathbb{R}^{b\times h}_{\geq 0}$ (all ones without a router); fiber coordinates $\mathsf{C} \in \mathbb{R}^{b\times h\times r}$ (none without fibers)

Data: dictionary $\mathsf{D} \in \mathbb{R}^{h\times k\times e}$; fiber bases $U_{ij} \in \mathbb{R}^{e\times r}$

Hyperparameters : temperature t; feature noise σ_F; winner dropout p_{drop}; revival rate p_{rev} and threshold τ_{rev}; head dropout p_{head}; fiber dropout p_{fib}; entries kept per head k_{sel} (1 under `ste`/`argmax`, k under softmax, and the rule's own value under `top`k_{sel}, e.g. 4 for `top4`)

Output: pooled features $F' \in \mathbb{R}^{b\times e}$; assignments $\mathsf{P} \in \mathbb{R}^{b\times h\times k}$; statistics $(\text{L1}, H, \text{cossim}_b, \mathbb{E}_i \kappa_i, \max_i \kappa_i, \text{KL}_m, \omega)$

begin

- $\mathbf{a}_{ij} \leftarrow |\mathsf{D}_{ij\cdot}|$ `// atoms; D_ij. itself if signed`
- **if** $k_{\text{sel}} < k$ **then** `// revive starved entries`
  - $\text{use}_{ij} \leftarrow$ share of rows on which entry j is among head i's top k_{sel} logits
  - **foreach** (n, i) *with probability* p_{rev} **do**
    - draw j uniformly from $\{j : \text{use}_{ij} < \tau_{\text{rev}}\, k_{\text{sel}}/k\}$, if any
    - raise entry j's logit on row n to head i's maximum `// tie the leader`
  - **end**
- **end**
- each (n, i): $\max_j[\mathsf{K}]_{nij} \leftarrow -\infty$ with probability p_{drop} `// winner dropout`
- $\mathsf{P} \leftarrow \text{select}_k(\mathsf{K}/t)$ `// softmax, top`k_{sel} `or straight-through`
- $H \leftarrow \mathbb{E}_{n,i}\, \text{entropy}_k(\mathbf{p}_{ni})$ `// Eq. (2)`
- $\bar{\mathbf{p}}_i \leftarrow \mathbb{E}_n\, \mathbf{p}_{ni}$; $\mathbf{o}_i \leftarrow \text{sg}[\bar{\mathbf{p}}_i]$ `// usage; head origin`
- **foreach** (n, i) **do** `// head dropout`
  - $\mathbf{p}'_{ni} \leftarrow \mathbf{o}_i$ with probability p_{head}, else $\mathbf{o}_i + (\mathbf{p}_{ni} - \mathbf{o}_i)/(1 - p_{\text{head}})$
- **end**
- each (n, i): $\mathbf{c}_{ni} \leftarrow \mathbf{0}$ with probability p_{fib} `// fiber dropout`
- $\mathbf{f}_{ni} \leftarrow \gamma_{ni} \sum_j p'_{nij}\, \mathbf{a}_{ij}$ `// Eq. (5); interventions act on` $\mathbf{p}'$
- $\tilde{\mathsf{F}} \leftarrow \text{noise}_{\text{featvar}}(\mathsf{F})$ `// Eq. (18)`
- $\text{cossim}_b \leftarrow \mathbb{E}_{n,n'} |\cos\angle(\mathbf{f}_n, \mathbf{f}_{n'})|$, with $\mathbf{f}$ built from P, not P' `// Eq. (3), through the Grams`
- $\kappa_i \leftarrow \mathbb{E}_{j\neq j'} \cos\angle(\mathbf{a}_{ij}, \mathbf{a}_{ij'})$ `// within-head atom cosine`
- $\text{KL}_m \leftarrow \mathbb{E}_i \left[\log_2 k - \text{entropy}_k(\bar{\mathbf{p}}_i)\right]$ `// Eq. (4)`
- $\boldsymbol{\pi}_i \leftarrow \sum_j \bar{p}_{ij}\, |\mathbf{a}_{ij}|$; $\omega \leftarrow \mathbb{E}_{i\neq i'} \cos\angle(\boldsymbol{\pi}_i, \boldsymbol{\pi}_{i'})$ `// support overlap`
- $\text{L1} \leftarrow \mathbb{E}_n \sum_i \|\tilde{\mathbf{f}}_{ni}\|_1$
- $\mathbf{f}'_n \leftarrow \sum_i \tilde{\mathbf{f}}_{ni}$ `// concatenated, not summed, under ConcatDictBlock`
- **if** *fibers* **then** `// Eq. (19), then any norm cap`
  - $\mathbf{f}'_n \leftarrow \mathbf{f}'_n + \sum_i \gamma_{ni} \sum_j p'_{nij}\, U_{ij}\, \mathbf{c}_{ni}$
- **end**
- **return** F', P, $(\text{L1}, H, \text{cossim}_b, \mathbb{E}_i \kappa_i, \max_i \kappa_i, \text{KL}_m, \omega)$

end

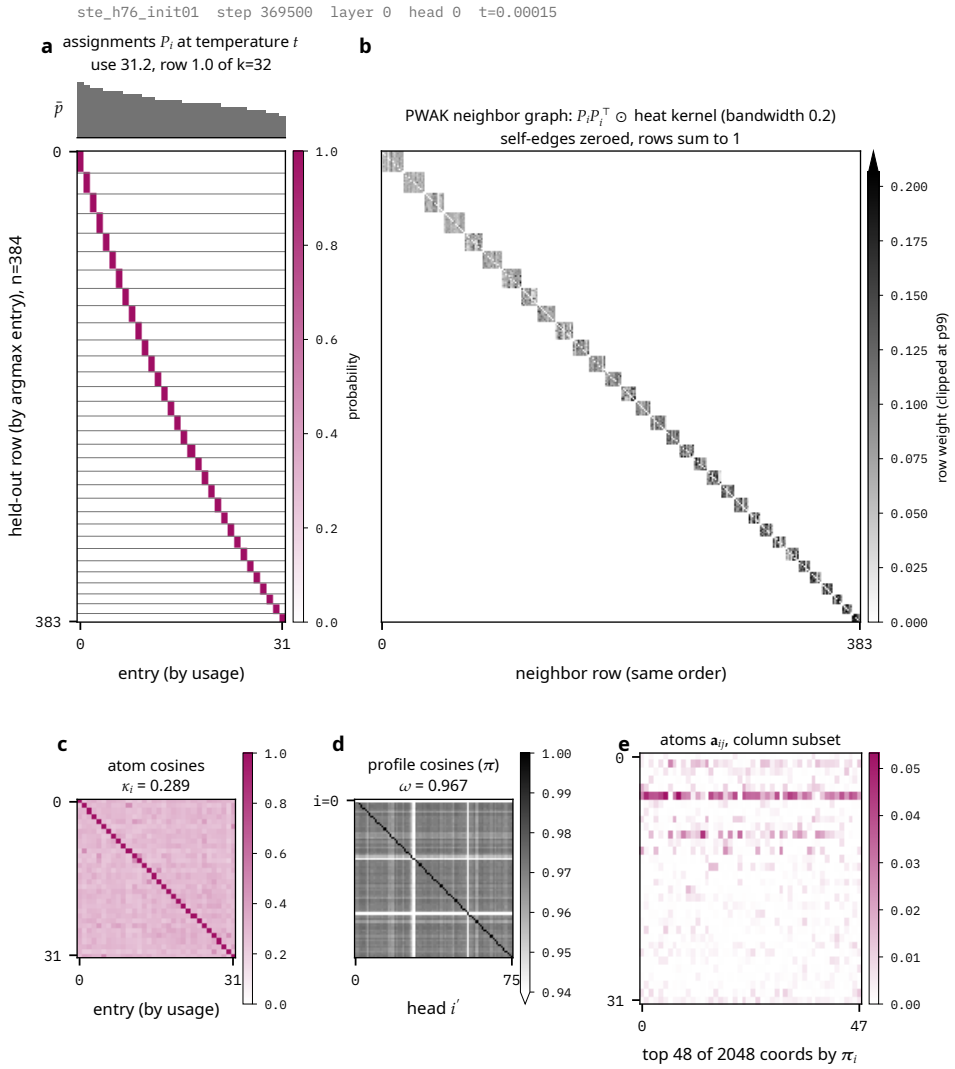


Figure 4 **One trained head, block by block.** Head i=0 of layer 0 in `ste_h76_init01` (step 369,500, t=0.00015) on n=384 held-out rows; one seed. (a) Assignments P_i, with rows sorted by winning entry and entries by mean usage $\bar{p}_{ij}$ (bars). "Use" is the effective entry count across rows, $2^{\text{entropy}_k(\bar{\mathbf{p}}_i)}$, and "row" within a row, 2 raised to the mean per-row entropy (31.2 and 1.0 of k=32, a hard code using nearly every entry). (b) The neighbor graph for KL_{pwak} and L2_{pwak} (Eq. (22)) in the same row order: the elementwise product of $P_iP_i^\top$ and the shape heat kernel $\exp((\langle \mathbf{u}_n, \mathbf{u}_{n'} \rangle - 1)/0.2)$ (excluding the constant coordinate), with self-edges zeroed and rows normalized to unit sum. Hard selection makes this matrix block-diagonal by cell; because neither PWAK term was active in training, it reflects the induced graph alone. (c) Pairwise cosines among head i's atoms (off-diagonal mean $\kappa_i = 0.289$). (d) Pairwise cosines among head coordinate profiles $\boldsymbol{\pi}$ (Eq. (6)) using the n-row mean $\bar{\mathbf{p}}$ (off-diagonal mean $\omega = 0.967$, head i outlined). (e) Head i's atoms on the 48 of e=2048 coordinates where $\boldsymbol{\pi}_i$ is largest (a column subset, not a projection). Each panel has its own scale with zero set to white, except (d), whose scale starts at the 5th percentile of off-diagonal cosines because ω is near 1, so the few cosines below it show as white. Figure 15 shows the softmax counterpart.

$\langle \mathbf{v}, \mathbf{v}' \rangle = \sum_q v_q v'_q$ is a sum of non-negative terms and therefore vanishes if and only if every term vanishes:

$$\langle \mathbf{v}, \mathbf{v}' \rangle = 0 \iff \operatorname{supp}(\mathbf{v}) \cap \operatorname{supp}(\mathbf{v}') = \emptyset. \tag{8}$$

Non-negative sparse PCA and orthogonal non-negative matrix factorization rely on the same equivalence: orthonormal non-negative components partition the coordinates among themselves [74, 75]. Signed vectors admit no such implication, because orthogonality can arise through cancellation even when two vectors share identical support; a penalty on inner products between sets of signed atoms, such as the incoherence penalty that Ramirez et al. [76] apply across per-class dictionaries, can therefore be satisfied without coordinate disjointness. The support overlap ω of Eq. (6) avoids this ambiguity by taking elementwise absolute values of the atoms, whose inner products vanish if and only if their supports are disjoint regardless of atom signs, and by aggregating clean atoms weighted by $\bar{\mathbf{p}}_i$ rather than noisy features $\tilde{\mathsf{F}}$, whose additive noise can drive coordinates negative.

Under softmax selection, this condition constrains the dictionary weights directly rather than merely individual sample activations. Because softmax assignments satisfy $p_{nij} > 0$ for every entry, expanding Eq. (5) yields

$$\langle \mathbf{f}_{ni}, \mathbf{f}_{ni'} \rangle = \sum_{j=1}^{k} \sum_{j'=1}^{k} p_{nij}\, p_{ni'j'} \left\langle \mathbf{a}_{ij}, \mathbf{a}_{i'j'} \right\rangle, \tag{9}$$

a strictly positive linear combination of all k^2 cross-head atom inner products. Consequently, if heads i and i' are orthogonal on even a single sample, $\langle \mathbf{a}_{ij}, \mathbf{a}_{i'j'} \rangle = 0$ must hold for every atom pair (j, j'), partitioning the coordinates of D across heads for all inputs rather than only on the training distribution. This provides the weight-level guarantee required for a static reading of the dictionary, which no signed-feature inner-product penalty can enforce. Dimensionally, such a partition is well within capacity: in the softmax configuration (Table 9), dividing e=2048 coordinates among h=32 heads allocates 64 coordinates per head to represent k=32 atoms, which need not have disjoint supports within a head. Room for between-head disjointness is most of what an overcomplete dictionary width $e > d$ buys.

Two distinct forms of orthogonality are therefore at play, only the between-head one being architectural: disjoint support assigns each head its own coordinate block, whereas *within* a head the k atoms act as contrastive alternatives spread across that block (Figure 5; Section 5.3). When heads are summed, neither property is guaranteed at convergence because ω acts only as a soft penalty, and the geometry is built to make a support partition the cheap solution rather than to force one; similarly, in non-negative matrix factorization, soft overlap penalties reduce coordinate sharing without yielding an exact partition, which instead requires hard coordinate assignment [74, 77, 75, 78]. Empirically, summed models exhibit strong within-head contrast at convergence while failing to partition coordinates between heads (Appendix G.2). In the concatenated-head variant (`ConcatDictBlock`), by contrast, each head writes to a dedicated coordinate slice by construction and ω is identically zero (Section 8, Appendix G.3, and Section 7).

Finally, the noisy per-head features $\tilde{\mathbf{f}}_i$ are summed across heads (or concatenated across slices under `ConcatDictBlock`) to form the pooled layer output $\mathbf{f}' \in \mathbb{R}^e$ written to the accumulator

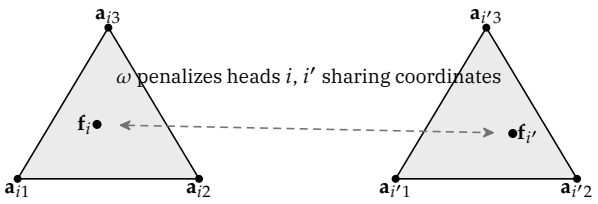


Figure 5 `DictBlock` **head geometry** for h=2, k=3. Each head's output $\mathbf{f}_i$ lies in the simplex spanned by its atoms $\mathbf{a}_{ij}$ (the rows of $|D_i|$), at the convex combination selected by its assignment $\mathbf{p}_i$. Because $|\mathsf{D}| \geq 0$ and $P \geq 0$, features are non-negative and $\cos \angle(\mathbf{f}_i, \mathbf{f}_{i'}) \in [0, 1]$; driving the support overlap ω toward zero encourages heads to occupy disjoint coordinates of $\mathbb{R}^e$ (Section 4.1.4).

$\mathbf{r}$:

$$\mathbf{f}' = \sum_{i=1}^{h} \tilde{\mathbf{f}}_i. \tag{10}$$

Decoder. Each layer's pooled output $\mathbf{f}'_\ell$ is scaled by its input's gain g_ℓ and added to the residual accumulator $\mathbf{r}$, whose state after ℓ layers is

$$\mathbf{r}_\ell = \sum_{\ell'=0}^{\ell-1} g_{\ell'}\, \mathbf{f}'_{\ell'}, \qquad g_\ell := \mathrm{sg}[\mathrm{L2}(\mathbf{y}_\ell)]\,. \tag{11}$$

When gain scaling (`resid_gain`) is enabled, each layer classifies only the direction of its input while g_ℓ restores the residual's norm; otherwise $g_\ell = 1$. At $\ell = 0$ the empty sum gives $\mathbf{r}_0 = \mathbf{0}$, so layer 0 receives the encoded input $\mathbf{y}_0 = \mathbf{x}$ directly, with $g_0 = 1$ identically on unit-norm SONAR embeddings.

A single unbiased linear decoder dec with weights $W_{\mathrm{dec}} \in \mathbb{R}^{d \times e}$, shared across all $\mathtt{DictEnc}_\ell$ layers (or replaced by the identity in direct-atom variants, where $e = d$), projects $\mathbf{r}_\ell$ back to the input space $\mathbb{R}^d$:

$$\hat{\mathbf{x}}_\ell = W_{\mathrm{dec}}\, \mathbf{r}_\ell. \tag{12}$$

The residual $\mathbf{y}_\ell$ entering layer ℓ is stop-gradiented so that earlier layers cannot pass a communication code through the residual instead of reducing reconstruction error. Layer ℓ decomposes $\mathbf{y}_\ell$ into the scalar gain g_ℓ (Eq. (11)) and a normalized shape vector $\mathbf{u}_\ell$ with a constant coordinate appended, which serves as the input to the classifier, router, and fibers:

$$\mathbf{y}_\ell = \mathrm{sg}[\mathbf{x} - \hat{\mathbf{x}}_\ell]\,, \qquad \mathrm{L2}(\mathbf{y}) := \sqrt{\sum_{q=1}^{d} y_q^2}, \tag{13}$$

$$\mathbf{u}_\ell = \frac{\mathbf{y}_\ell}{\mathrm{L2}(\mathbf{y}_\ell) + \epsilon} \oplus [1], \tag{14}$$

where ϵ denotes the floating-point epsilon of the working dtype.

Normalizing stabilizes the classifier's input scale: because a bilinear classifier's logits scale quadratically with input norm, unnormalized inputs would cause logits to explode on large activations (such as raw GPT-2 residuals) and vanish on the smaller residuals passed to deeper layers, whereas the gain–shape split isolates magnitude in g_ℓ. This provides the rescaling invariance of RMSNorm [79] without learned per-coordinate gains, projecting onto the unit sphere rather than RMSNorm's sphere of radius $\sqrt{d}$ and therefore producing small initial logits unless compensated at initialization (Appendix E.1). Appending the constant coordinate $[1]$ yields a $(d+1)$-dimensional input and preserves the sign of $\mathbf{y}_\ell$, which an unbiased homogeneous quadratic form would otherwise lose. Following Pearce et al. [15], this input augmentation endows each bilinear factor with an effective bias while preserving closed-form eigendecomposition, splitting each logit into quadratic interaction, linear, and constant terms.

Noise. During training, Gaussian perturbations may be applied to the inputs X, classifier logits K (ε_K), and dictionary features F (ε_F), controlled by scale hyperparameters σ_X, σ_K, and σ_F. Each operator draws independent standard normal entries of the same shape as its target,

$$\varepsilon \sim \mathcal{N}(0,1), \tag{15}$$

under one of three scaling rules. Unscaled Gaussian noise adds a fixed-variance perturbation,

$$\mathrm{noise}_{\mathrm{normal}}(\mathsf{K}) := \mathsf{K} + \sigma_K \varepsilon; \tag{16}$$

norm-proportional noise scales the perturbation relative to each sample's root-mean-square magnitude,

$$\mathrm{noise}_{\mathrm{batchnorm}}(X) := X + \sigma_X\, \varepsilon \odot \frac{\mathrm{sg}[\mathrm{L2}(X)]}{\sqrt{d}}; \tag{17}$$

and feature-variance noise scales each coordinate's perturbation by its standard deviation across the batch,

$$\mathrm{noise}_{\mathrm{featvar}}(\mathsf{F}) := \mathsf{F} + \sigma_F\, \varepsilon \odot \mathrm{sg}\left[\sqrt{\mathrm{var}_n(\mathsf{F}) + \epsilon}\right]. \tag{18}$$

Here $\mathrm{L2}(X)$ is the per-sample norm of Eq. (13), broadcast across coordinates (Eq. (17) is also used for logit noise ε_K under hard selection). Both adaptive scales are stop-gradiented so that the model cannot suppress noise by shrinking activation or feature variance, and ϵ inside the square root prevents an undefined $0/0$ gradient when a coordinate's batch variance underflows to zero. Because unscaled logit noise enters the softmax as σ_K/t, holding σ_K fixed while annealing t downward would amplify effective noise over training; runs with temperature annealing therefore anneal σ_K proportionally. Noise settings for all models are tabulated in Appendix B.3.

Router and fibers. Two optional branches take the same normalized shape vector $\mathbf{u}$ as input alongside the classifier (Eq. (14), omitting the layer index) and combine with its assignments in the dictionary lookup (Figure 6); Appendix B.3 lists the configurations that enable them.

The *router* (`scaled`) is a second bilinear map from $\mathbf{u}$ to per-head scales $\boldsymbol{\gamma} \in \mathbb{R}^h$, rendered nonnegative via an elementwise absolute value (omitted when `router_signed` is enabled). Because γ_i scales head i's entire output vector, the classifier selects *which* direction the head contributes while the router controls *its magnitude*; a dedicated sparsity penalty $\mathrm{L1}_S$ regularizes $\boldsymbol{\gamma}$ directly,

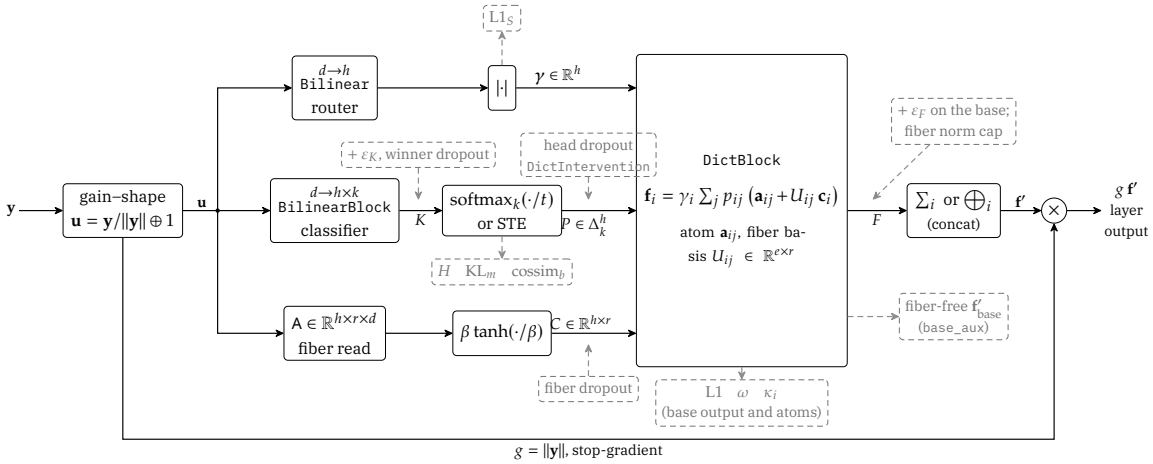


Figure 6 **`DictEnc` with router and fibers.** The gain–shape split passes the normalized shape vector $\mathbf{u}$ to three parallel branches: the classifier (producing assignments P), the router (producing non-negative per-head scales γ), and the fiber readout (producing r continuous coordinates per head, C). The three branches combine in the dictionary lookup (Eq. (19)), and the pooled output is scaled by the input gain g. Dashed boxes above an arrow indicate training perturbations; dashed boxes below indicate auxiliary statistics computed on the fiber-free base output.

whereas the output penalty L1 can be reduced by shrinking either the router scales or the dictionary atoms.

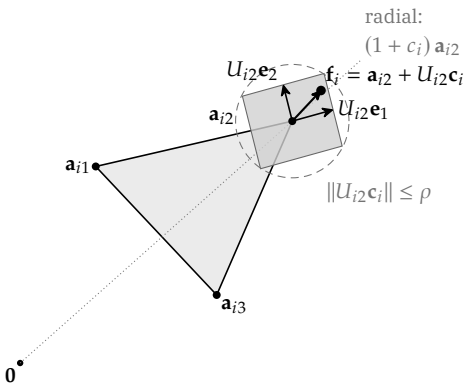
Fibers (`fiber_rank` r) equip each entry with a local basis $U_{ij} \in \mathbb{R}^{e\times r}$ (restricted to the head's coordinate slice under `ConcatDictBlock`), while a linear readout A with per-head matrices A_i projects $\mathbf{u}$ onto bounded coordinates $\mathbf{c}_i = \beta \tanh(A_i\mathbf{u}/\beta)$, yielding

$$\mathbf{f}_i = \gamma_i \sum_{j=1}^{k} p_{ij} \left(\mathbf{a}_{ij} + U_{ij}\, \mathbf{c}_i\right). \tag{19}$$

Under a hard code, the selected atom serves as a discrete base point and the fiber term provides a continuous correction within the subspace spanned by U_{ij} (Figure 7). Without regularization, the continuous correction can dominate reconstruction and render the discrete atom uninformative; we therefore constrain the fiber branch via fiber dropout, a norm cap ($\|U_{ij}\mathbf{c}_i\| \le \rho$ per head, or summed across heads), an auxiliary base-only deep-supervision loss (`base_aux`, which additionally decodes the fiber-free output), or post hoc fiber training on a frozen discrete base. A radial fiber (r=1, without U_{ij}) simply rescales the selected atom along its own direction, and all auxiliary dictionary statistics are evaluated on the fiber-free base output. Finally, head dropout (`p_head_drop`) replaces each head's assignment during training with its stop-gradiented batch mean $\mathbf{o}_i = \mathrm{sg}[\bar{\mathbf{p}}_i]$ (its origin), discouraging co-adaptation across heads and keeping mean-ablation interventions in distribution.

4.2 Training

Embedding model. We embed texts using SONAR [58], a multimodal and language-agnostic sentence embedding model consisting of a translation-trained encoder (mean-pooled into a



$\mathbf{c}_i = \beta \tanh(A_i\mathbf{u}/\beta)$ is read from the layer input;
the selected entry is the discrete base point, the fiber the continuous correction. Fiber-free, the head emits $\mathbf{a}_{i2}$: the base-only output.

Figure 7 **Fiber geometry** for a single head, extending Figure 5. When the head selects the entry with atom $\mathbf{a}_{i2}$, the fiber defines an affine subspace spanned by U_{i2} anchored at $\mathbf{a}_{i2}$, within which the coordinates $\mathbf{c}_i$ position the output. The dashed circle indicates the per-head norm cap ρ, and the dotted ray shows the radial variant, which restricts the correction to the direction of $\mathbf{a}_{i2}$.

1024-dimensional vector) and a paired decoder that generates text from that vector alone in a chosen target language. In our primary corpus cache, each embedding encodes the opening of a web document (up to 512 tokens), longer than the single sentences on which SONAR was trained. Because translations map to nearby vectors, reconstructed or steered embeddings can be decoded into a fixed language and re-encoded to evaluate downstream text fidelity and cycle-consistent steering (Appendices C.5 and C.8). Large Concept Models [33] generate sentence embeddings directly in SONAR space, making SONAR a natural testbed for embedding-level interventions (an edit to a SONAR embedding is an edit to such a model's context or output), although SONAR decoding is known to be sensitive to small embedding perturbations [33]. As a second substrate where sparse dictionary learning is well established and reconstructions can be spliced directly into a transformer forward pass, we also train models on residual-stream activations of GPT-2 small [80].

Loss function. Reconstruction fidelity is trained with a whitened mean-squared error, where $\mathbf{w} \in \mathbb{R}^d$ is the per-dimension inverse variance normalized to mean one (matching our evaluation metric, and implemented equivalently by scaling X and $\hat{X}$ by $\sqrt{\mathbf{w}}$ prior to computing squared error):

$$\mathrm{MSE}(X, \hat{X}) := \frac{1}{bd} \sum_{n=1}^{b} \sum_{q=1}^{d} w_q \left(\hat{x}_{nq} - x_{nq}\right)^2 . \tag{20}$$

The total training objective combines reconstruction error, an optional ghost-gradient term weighted by s_g, and per-layer auxiliary statistics weighted by $\mathbf{s}$ and *summed* (rather than averaged) across the l layers:

$$L := \mathrm{MSE}(X, \hat{X}) + s_g \,\mathrm{MSE}(\hat{X}_g,\, X - \hat{X}) + \sum_{\ell=0}^{l-1} \mathbf{s} \cdot \boldsymbol{\psi}_\ell . \tag{21}$$

Each layer produces the statistics vector (Algorithms 2 and 1)

$$\boldsymbol{\psi}_\ell = \left[\, \mathrm{L1}_K \;\; \mathrm{L1} \;\; H \;\; \mathrm{cossim}_b \;\; \mathbb{E}_i \kappa_i \;\; \max_i \kappa_i \;\; \mathrm{KL}_m \;\; \mathrm{KL}_{\mathrm{pwak}} \;\; \mathrm{L2}_{\mathrm{pwak}} \;\; \mathrm{L1}_S \;\; \omega \,\right]_\ell^\top, \tag{22}$$

where $\mathrm{L1}_K$ is the L1 norm of the classifier's gate factor, κ_i is the mean within-head cosine similarity among head i's atoms, $\mathrm{KL}_{\mathrm{pwak}}$ and $\mathrm{L2}_{\mathrm{pwak}}$ are the two neighborhood partition-consensus statistics, and $\mathrm{L1}_S$ is the router's L1 penalty. The maximum within-head atom cosine $\max_i \kappa_i$ is recorded for a setpoint controller and never penalized directly; Appendix B.4 documents the implementation's full statistics layout, and Appendix B.3 tabulates the loss weights for all models. The ghost-gradient path is disabled throughout ($s_g = 0$), because $\hat{X}_g$ activates only on dictionary coordinates that are identically zero across an entire batch, which never occurs under bilinear classifiers with softmax selection and non-negative atoms.

Under deep supervision [67], $\mathrm{MSE}(X, \mathrm{dec}(R_\ell))$ is evaluated at every prefix $\ell = 1, \ldots, l$ and averaged with equal weight $1/l$, training each layer to reconstruct the residual left unexplained by earlier layers. Supervising every prefix through a shared decoder is the deterministic counterpart of SoundStream's quantizer dropout, which truncates a residual quantizer to a random depth per example [32], and mirrors the nested-prefix objective of Matryoshka SAEs [13] applied across residual layers rather than dictionary prefixes. An optional stop-gradient variant (`deepsup_sg`, disabled in all reported models) detaches the incoming accumulator state at each prefix decode so that each layer receives reconstruction gradients only from its own prefix.

Four auxiliary statistics serve as active loss terms in our standard configurations (Figure 3; Appendix B.3): between-sample similarity cossim_b, evaluated via the per-head Gram matrices to penalize feature-space similarity across samples (which empirically also reduces per-sample entropy H); the batch-mean uniformity term KL_m, which anchors each head's mean assignment to the uniform distribution so that a uniform assignment corresponds to a maximally generic feature; the feature sparsity penalty $\mathrm{L1}$ to accelerate convergence; and the between-head support overlap ω, which penalizes shared active dictionary coordinates across heads (most models reported below were trained with the predecessor of ω; Appendix B.4). Assignment sharpness is governed by the temperature schedule t, while winner dropout and the noise terms ε_K and ε_F regularize assignments and features, respectively.

4.3 Evaluation

We evaluate Ontologizer models against `TopK` SAEs [9] trained on identical cached SONAR embeddings under the same whitened reconstruction objective. Following SAEBench [59], our evaluation suite assesses learned dictionaries across multiple complementary criteria rather than solely by reconstruction error at fixed sparsity, adapting three SAEBench metrics: downstream text fidelity through the paired M2M100 decoder as an analogue of loss recovered, blinded autointerp detection scoring, and sparse probing of language identity. We pair these with evaluations tailored to categorical head structure: reconstruction–capacity Pareto frontiers, categorical-form and semantic-coherence tests of head structure, cross-model feature splitting and cross-seed reproducibility, coefficient shrinkage decomposition, and steering fidelity. A structural-ablation ladder of SAE variants introduces Ontologizer constraints one step at a time to attribute performance differences to specific architectural components. Finally, because

Algorithm 3: `Ontologizer` training pass (`Ontologizer.withStats`) and total loss computation (`Hyperparams.loss`) under residual forwarding (`forward="resid"`), which is used in all reported configurations. When no encoder is used, enc is the identity; in direct-atom configurations, dec is also the identity.

Input: batch $X \in \mathbb{R}^{b \times d}$

Data: encoder enc; layers $\underset{0}{\texttt{DictEnc}}, \ldots, \underset{l-1}{\texttt{DictEnc}}$ (Algorithm 1); decoder dec (Eq. (12)); MSE weights $\mathbf{w}$; loss weights $\mathbf{s}$, s_g

Hyperparameters: input noise σ_X; deep supervision; `deepsup_sg`; `resid_norm`, `resid_const`, `resid_gain`

Output: loss L; per-layer statistics $\boldsymbol{\psi}_0, \ldots, \boldsymbol{\psi}_{l-1}$

begin

- $E \leftarrow \mathrm{enc}\big(\mathrm{noise}_X(X)\big)$ `// Eq. (17)`
- $R \leftarrow \mathbf{0} \in \mathbb{R}^{b \times e}$ `// state 0: no layers applied`
- $E \leftarrow E \oplus [1]$ `// if resid_const`
- **for** $\ell \leftarrow 0$ **to** $l-1$ **do** `// layer` ℓ `reads state` ℓ`, writes state` $\ell+1$
  - $R_{\mathrm{prev}} \leftarrow R$
  - $R, \mathsf{P}_\ell, \boldsymbol{\psi}_\ell \leftarrow \underset{\ell}{\texttt{DictEnc}}(R, E)$
  - **if** *deep supervision* **then** `// a decode per prefix`
    - $\hat{X}_{\ell+1} \leftarrow \mathrm{dec}\big(R - R_{\mathrm{prev}} + \mathrm{sg}[R_{\mathrm{prev}}]\big)$ `//` $\mathrm{dec}(R)$ `without deepsup_sg`
  - **end**
  - **if** $\ell < l-1$ **then**
    - $Y \leftarrow \mathrm{sg}\big[X - \mathrm{dec}(R)\big]$ `// Eq. (13)`
    - $E \leftarrow Y \oplus [1]$ `// if resid_const; the next layer splits it into gain and shape, Eq. (14) (normalized here instead under resid_norm alone)`
  - **end**
- **end**
- **if** *not deep supervision* **then** $\hat{X}_l \leftarrow \mathrm{dec}(R)$
- $\mathrm{MSE} \leftarrow \mathbb{E}_\ell \, \mathrm{MSE}_{\mathbf{w}}(X, \hat{X}_\ell)$ `// Eq. (20); over the prefixes computed, each weighted` $1/l$
- $L \leftarrow \mathrm{MSE} + s_g \, \mathrm{MSE}_g + \mathbf{s} \cdot \sum_{\ell=0}^{l-1} \boldsymbol{\psi}_\ell$ `// Eq. (21);` $\mathrm{MSE}_g = 0$ `without ghosts`
- **return** $L, (\boldsymbol{\psi}_\ell)_{\ell=0}^{l-1}$

end

AxBench [2] found that prompting, fine-tuning, and supervised difference-of-means vectors outperform SAE latents at steering while linear probes do not, we compare our unsupervised steering primitives directly against both difference-of-means and linear-probe directions.

4.4 Data, models, and evaluation

This section summarizes our experimental setup; Appendix B provides the full protocols and defining equations for every evaluation.

Data. Our main SONAR models are trained on a fixed cache of 3,974,400 unit-norm SONAR sentence embeddings (d=1024) drawn from an interleaved 86-language mC4 sample, with a ground-truth language label per row and source text recoverable by row index. In this cache, 80 of the 86 languages (93% of rows) were encoded with SONAR's English source tag rather than their native tag; a companion sentence-split cache (`mc4_sent16`, 44.1M consecutive segments) stores unnormalized embeddings encoded under native language tags (Appendix B.1). Sections 8.1 and 6.5 additionally evaluate residual-stream activations of GPT-2 small [80] leaving block 8 of 12 (d=768), a standard testbed for sparse dictionary learning [9], while Section 7 uses a separate GPT-2 setup whose results are compared only within that section (Appendix B.1).

Objective and score. All models minimize the whitened MSE of Eq. (20), and we report whitened fraction of variance unexplained (FVU_w, Eq. (23)) on the final 32,768 rows of the cache, which are held out from training for all SAEs and straight-through models. Coordinate 837, which is fixed on raw SONAR embeddings by the final LayerNorm hyperplane, encodes $1/\|\mathbf{x}^{\mathrm{raw}}\|$ after L2 normalization and accounts for at most 0.41% of whitened error in any sparse model (Appendix B.1). Although the softmax Ontologizers were trained before that holdout split was introduced, re-evaluating them on freshly encoded out-of-sample rows changes their FVU_w by at most 0.3% (Appendix B.1). Every evaluation uses each run's final training temperature.

Models. The reference softmax Ontologizer (`resid_nc`) stacks l=5 residual layers of h=32 heads with k=32 entries each; a selection-rule sweep varies only its per-head selection operator, and the straight-through arm (Section 6) replaces soft assignments with discrete one-hot codes. Our baselines are `TopK` SAEs following Gao et al. [9], alongside a structural-ablation ladder that introduces Ontologizer constraints one step at a time. Two rungs of that ladder—`g160top1` and `g160softmax`, which apply hard or softmax competition across 160 heads over a shared decoder (`g160top1` fusing a scalar router into its classifier)—are single-layer Ontologizers without bilinear classifiers, residual stacking, or non-negative dictionaries, and are treated as simplified model variants rather than external baselines (Appendix B.2; full configurations in Appendix B.3).

Evaluations. Each evaluation is implemented as a standalone script run on the shared evaluation split (Appendix B.5):

- reconstruction error against code capacity and a Gaussian rate–distortion baseline (`pareto.py`), and coefficient shrinkage via least-squares refitting on frozen active supports (`refit.py`);

- post hoc discovery (`headstruct.py`) of exhaustive, mutually exclusive latent groups in SAEs, and within-head semantic coherence of decoded atoms against size-matched random subsets (`headcoh.py`);
- cross-model feature splitting and cross-seed head matching by sample partition and by output contribution (`splitting.py`, `partition.py`, `headcontrib.py`);
- downstream text fidelity through the M2M100 decoder (`textfid.py`), and language probing by active latents and head partitions (`langprobe.py`, `headlang.py`);
- steering efficacy and collateral in decoded text and in embedding space (`steerfid.py`, `steerembed.py`, `steergeom.py`), along with the additivity of paired head interventions (`compose.py`); and
- automated interpretability detection (`autointerp.py`).

5 Results: the softmax Ontologizer against SAEs

5.1 The price of structure: reconstruction–capacity Pareto

Across all capacities tested, the SAE frontier dominates the softmax Ontologizer's truncated deviation code, which retains the ν entries per head furthest from the mean assignment and pins the rest to the mean (Figure 8b; Appendix B.5). To reach $\mathrm{FVU}_w \approx 0.49$, the Ontologizer requires 320 active coefficients whereas the k_{SAE}=32 SAEs require only 32. Even its coarsest nontrivial truncation, retaining one deviation per head (160 coefficients, $\mathrm{FVU}_w = 0.677$), incurs higher error than the 32-coefficient SAE, leaving the high-sparsity regime inaccessible. Unstructured reconstruction is more accurate at high capacity as well: the dense control reaches $\mathrm{FVU}_w = 1.1 \times 10^{-4}$ using 3.4× fewer active coefficients than the full soft code, achieving roughly 50× lower error at a matched budget. Because these truncations are applied post hoc to a model that was never trained with truncation, they may overstate the intrinsic cost of contrast-set structure: multi-stage residual quantizers similarly degrade when truncated without training for it, whereas training with randomized truncation depths per example avoids that penalty [32], and a residual quantizer trained with a loss on every stage can be cut to a prefix at almost no cost [81]. Furthermore, reconstruction information is distributed across heads rather than concentrated in a few: retaining whole heads instead of per-head deviations performs substantially worse at a matched coefficient count ($\mathrm{FVU}_w = 0.50$ when keeping 8 of 32 heads per layer, vs. 0.094 at ν=8).

Taking the discrete one-hot argmax of the softmax model fails to reconstruct the input at all, yielding $\mathrm{FVU}_w = 17.3$. That error is 77× higher than the Gaussian rate–distortion reference at 800 bits (0.226; Figure 8a), which gives the minimum distortion achievable by an 800-bit code on a Gaussian source with the same empirical covariance (Eq. (25)). Because Gaussian sources maximize rate–distortion error among all distributions of fixed covariance [82, Problem 10.8], the argmax reconstruction sits at least 77× above the data's distortion–rate function; on a selection-sweep run of the same architecture, the gap spans nine orders of magnitude ($\mathrm{FVU}_w = 8.7 \times 10^6$ vs. 0.0007 for the soft code). Consequently, the discrete argmax entries forced by interventions

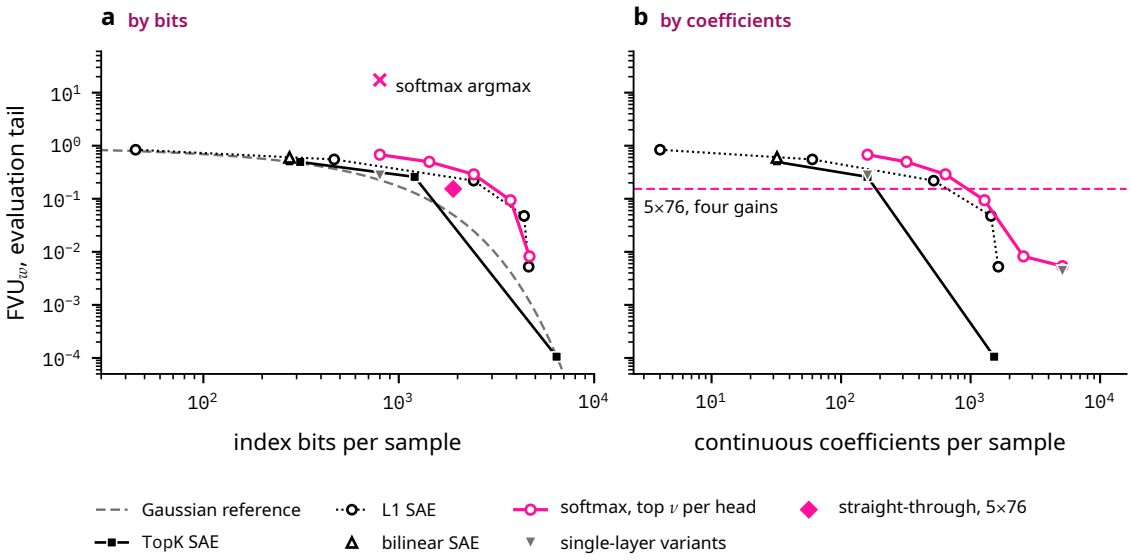


Figure 8 Reconstruction error against code capacity on the shared evaluation tail, plotted against index bits in (a) and active continuous coefficients in (b): truncated codes of the reference softmax model (`resid_nc`), TopK and L1 SAEs (including all five L1 weights in Table 13), single-layer Ontologizer variants, and the straight-through stack (`ste_h76_init01`, Section 6). Each code is plotted only on axes where its size is nonzero: soft codes appear only in (b), the softmax argmax appears only in (a), and the straight-through stack (whose only continuous values are four scalar per-layer gains) appears as a horizontal level in (b). The dashed curve shows the Gaussian rate–distortion reference (Eq. (25)). Index bits represent the entire discrete rate of a hard code (up to those four gains), whereas for codes that also transmit continuous coefficients they measure only support indexing, which allows the k_{SAE}=32 and k_{SAE}=5120 SAEs to reach the Gaussian curve in (a); Figure 29 places all codes on a single operational bit-rate axis. Although the softmax model included this tail in its training set whereas all other models hold it out, re-evaluating on out-of-sample rows that no model saw during training changes every point by at most 0.3%, and changes the dense control's $\mathrm{FVU}_w = 1.1 \times 10^{-4}$ by 0.9% (Appendix B.1). One seed per point, except for the k_{SAE}=32 SAE's seed-43 replica.

on a softmax model do not carry its reconstruction. This mirrors a failure known for soft relaxations of vector quantization: annealed too slowly, the network can learn to invert the soft assignments [83], and trained from scratch, the decoder inverts the relaxation so that no quantization takes place [84]. It motivates the straight-through hard-code arm in Section 6. Notably, the Gaussian reference does not imply that 800 bits are insufficient, since non-Gaussian data can be compressed below that bound.

5.2 Structure decomposition (the ladder)

rung	FVU_w	L0	note
k32 baseline	0.499	32	0 dead
g160top1	0.274	160	1469/5120 dead (29%)
g160softmax	**0.0044**	5120	beats `resid_nc` (0.0054)
k32_p5	0.509	32	nested prefixes, ~2% cost
k32_bl	0.602	32	1920/5120 dead (38%)

Table 1 Converged structural rungs, m=5120, 144.3k steps each.

Soft per-head competition accounts for most of the reconstruction gap relative to unstructured codes: the single-layer soft variant `g160softmax` incurs roughly 40× the error of the dense control ($\mathrm{FVU}_w = 0.0044$ vs. 1.1×10^{-4}). The remaining components of the reference Ontologizer (bilinear classifiers, five residual layers, and a shared 2048-dimensional dictionary space) provide no additional reconstruction benefit on the reference run despite using 2.2× as many parameters. A selection-sweep checkpoint of the same five-layer architecture does reach $\mathrm{FVU}_w = 0.0007$, six times lower than the single-layer variant, and maintains that lead on out-of-sample rows that no model saw during training (0.00067 vs. 0.0044; Appendix B.1). Thus, while a residual stack can outperform a single soft layer out of sample, the reference checkpoint studied in the remainder of this section does not, so any case for it must rest on steering rather than reconstruction.

By contrast, hard per-head selection with a scalar router fused into the classifier (Appendix B.2) incurs only a ~6% penalty relative to an unstructured SAE of equal sparsity ($\mathrm{FVU}_w = 0.274$ for the single-layer hard variant vs. 0.258 for the k_{SAE}=160 SAE). The softmax model's catastrophic argmax error is a training artifact rather than an inherent limitation of discrete head codes: evaluating the soft model under one-hot selection inflates its output norm by 3.7× because individual winning atoms have roughly 4× the norm of the soft mixture, whereas training a five-layer stack directly under argmax selection with no per-head scales reaches $\mathrm{FVU}_w = 0.154$ (Section 6). Replacing the linear encoder of the k_{SAE}=32 SAE with a bilinear encoder increases FVU_w by roughly 20% and leaves over a third of the dictionary dead; however, because the bilinear factors are initialized randomly whereas every linear rung starts tied to its decoder (Appendix B.3), this cost cannot yet be attributed to the bilinear parameterization alone.

Overall, categorical structure appears where it is trained in through hard selection. The single-layer hard variant learns mutually exclusive head-like partitions, whereas soft per-head

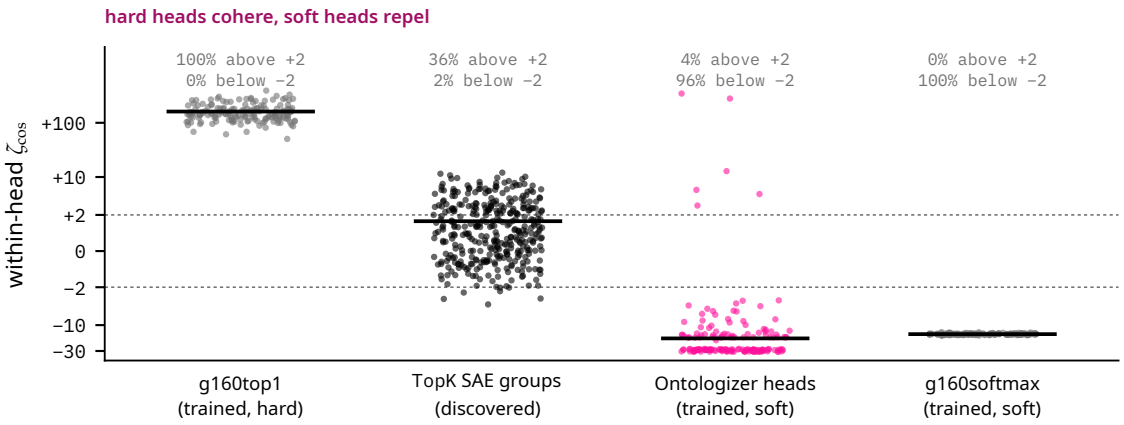


Figure 9 Within-head decode-direction coherence ζ_{cos} for each head or discovered group, standardized against a size-matched null drawn from the same dictionary pool (n_{null}=500). The horizontal axis is linear between the dotted lines at ±2 and logarithmic beyond them; vertical bars mark model means. Soft heads come from `resid_nc` and `g160softmax`, and discovered groups come from the m=11264, k_{SAE}=32 SAE. One seed per model.

codes converge to dense, near-uniform mixtures and the post hoc groups discovered in standard SAEs correspond to co-firing topical clusters (Appendix C.1). The remaining evaluations in the softmax-vs.-SAE battery (feature reliability, text fidelity, language localization, and coefficient shrinkage) are reported in Appendix C.

5.3 Semantic coherence: trained atoms form contrast sets

Evaluated on their trained atoms, hard-selected single-layer heads form coherent directional clusters, whereas soft-selected heads form contrast sets (Figure 9). In the single-layer hard variant `g160top1`, every head decodes to directions with much higher pairwise cosine similarity than random subsets of the same size: winner-take-all selection with an unconstrained positive magnitude allows each head to contribute only one direction per input, placing it in the regime where representing a multi-dimensional feature requires many near-collinear atoms [46] that tile a local region [48, 47]. Post hoc co-firing groups discovered in the `TopK` SAE are mildly coherent, consistent with topical clusters. Both soft architectures exhibit the opposite pattern: in 96–100% of heads, within-head decode directions have lower pairwise cosine similarity than size-matched random subsets, indicating that a soft head's entries act as separated alternatives rather than similar directions. This within-head repulsion does not depend on the choice of inner product: under the whitened inner product used in training, repulsion strengthens in the Ontologizer (mean ζ_{cos} of −24.4 vs. −17.6) and remains essentially unchanged in the single-layer variants (Appendix C.2).

However, soft and hard heads differ in how strongly the model's input–output function constrains these cosines. Because a soft head's assignment weights sum to one, adding a constant vector to every decode direction in one head and subtracting it from another head in the same layer leaves the layer's output unchanged; thus, the function uniquely determines only pairwise differences between a head's decode directions (its swap moves). Because the reconstruction

loss is invariant along those shared per-head offsets, they are determined by initialization and auxiliary regularizers rather than by reconstruction, so raw cosine similarities among soft atoms reflect those parameter-level choices to an unmeasured degree. In `g160top1`, by contrast, unconstrained per-head magnitudes identify each decode direction up to positive scale, making its positive coherence a property of the learned function. Comparing the two regimes on equal terms requires centering decode directions within each head and evaluating against a similarly centered null, since centering alone shifts mean pairwise cosines below zero.

The five-layer hard-code Ontologizer exhibits the same within-head separation in its weights, subject to the same offset caveat because heads within a layer share a single scalar gain. Decoded into output space, its mean within-head atom cosine is 0.023 in layer 0 and −0.015 to −0.020 in layers 1–4, so decoded atoms within each head are roughly orthogonal to slightly negatively correlated; when computed directly in dictionary space, by contrast, the statistic is dominated by the decoder's null space (Appendix G.1). Of the two orthogonality conditions introduced in Section 4.1.4, within-head contrast holds in both soft and hard stacks, whereas between-head coordinate partitioning holds in neither: no summed model partitions dictionary coordinates across heads. The between-head support overlap ω sits at or above a coordinate-shuffled null, with each head's usage spanning 52–98% of dictionary coordinates rather than the ∼3% expected of a 32-way partition (Appendix G.2). Judged by their trained parameters, summed heads therefore form within-head contrast sets that share dictionary coordinates.

5.4 Steering fidelity

We evaluate steering directly in embedding space by stepping from an input along a candidate direction and measuring both target realization (whether the target entry becomes the head's argmax) and collateral (the fraction of other heads whose selected entry changes; Algorithm 4). Unlike downstream feature-shift metrics used in SAE steering evaluations [14], collateral here measures code-internal disturbance across non-target heads. Evaluating in embedding space is necessary because decoding and re-encoding through text introduces substantial round-trip noise: even without an intervention, a text round trip preserves a head's argmax on only 11–13% of rows (Appendix C.8).

In embedding space, soft-model steering reliably realizes target entries: at step strength 0.25, stepping along a target entry's decode direction selects that entry on 82% of rows in the softmax model (Table 2). The hard-code stack is substantially less steerable, realizing 36% of targets at strength 0.25 while already disturbing a larger fraction of other heads (0.66 vs. 0.58), as discrete argmax selection requires crossing finite decision margins. As step strength increases, the hard stack's decode realization saturates near 0.65 (Figure 10b), leaving roughly a third of its interventions unrealized at any step size, and surpasses the soft model only once collateral exceeds 0.85 (where most other heads have flipped as well). Stratifying by target layer reverses the aggregate ranking between decode and gradient directions (Appendix E.5). Furthermore, in both stacks, decode steps realize targets primarily through directions along which the data exhibit minimal variance: rescaled to full step length, the projection of the decode step onto the bottom fifth of the data's variance spectrum achieves 95% (hard) and 100% (softmax) of the full step's realization rate, whereas the projection onto the top half achieves only 20% and 8%,

direction	hard code @0.25 / 1.0	softmax @0.25 / 1.0
decode	0.364 / 0.607	**0.824** / 0.569
grad	0.304 / 0.103	0.453 / 0.477
margin	0.158 / 0.275	—
adjoint, oriented	0.001 / 0.001	0.427 / 0.501
adjoint, oriented per sample	—	0.446 / 0.512
random	0.018 / 0.024	0.020 / 0.037
collateral	0.66 / 0.88	0.58 / —

Table 2 Realization rate (target entry newly selected) by steering direction and step strength across 60–64 entries × 32 held-out rows (chance 0.031). Because collateral varies by at most 0.01 across directions at a fixed step strength, rows within a column compare methods at matched collateral. Hard code denotes the 5×76 stack, whose seed-43 replica agrees within 0.015 across every decode, grad, and random condition. Orientation of the adjoint direction is detailed in Appendix B.5.

respectively (Appendix E.7).

Against supervised steering vectors. Using the same protocol, we evaluate two standard supervised steering baselines for each target entry: the difference-of-means vector between inputs that select the target entry and inputs that do not (corresponding to contrastive activation addition [85], which averages the single-pair difference of activation addition [86] over many examples), and the weight vector of a class-balanced logistic probe [2]. Both supervised baselines are fitted on 8,192 reference rows with the steered evaluation rows held out. Unlike curated contrast pairs that differ only in a target attribute, these corpus subsets are unpaired, so the mean difference also reflects correlated background variation. Because both supervised methods are trained directly on the model's own entry assignments, they serve as an oracle baseline measuring how effectively a supervised direction with full knowledge of the partition can steer into it.

Among the multi-layer models, only the softmax stack's own decode direction outperforms the supervised baselines, realizing target entries on 75% of rows compared to 46–47% for the difference-of-means and probe vectors (Table 3). In the hard-code stack, the target cells are geometrically reachable (supervised vectors realize 91–94% of targets), but the model's own decode direction reaches only 37%. The single-layer hard variant `g160top1` trades realization rate for selectivity (58% vs. 94% realization, at roughly one fifth the collateral), whereas every other model's native direction disturbs non-target heads as much as a random step of equal norm (Figure 10). Oracle knowledge of the partition buys the supervised vectors little additional selectivity (difference-of-means collateral sits at most 0.10 below random), and for standard SAEs the decoder direction already matches the supervised oracle.

Crucially, however, the two arbitrary-partition baselines demonstrate that high target realization in embedding space is a generic consequence of partitioning a vector space rather than evidence that the learned partitions capture meaningful factors. In an untrained 5×76 stack, parametric decode steps rely on random dictionary and decoder weights and remain at chance (0.02), whereas supervised vectors (0.75 and 0.94) and classifier-derived directions (`grad` 0.61

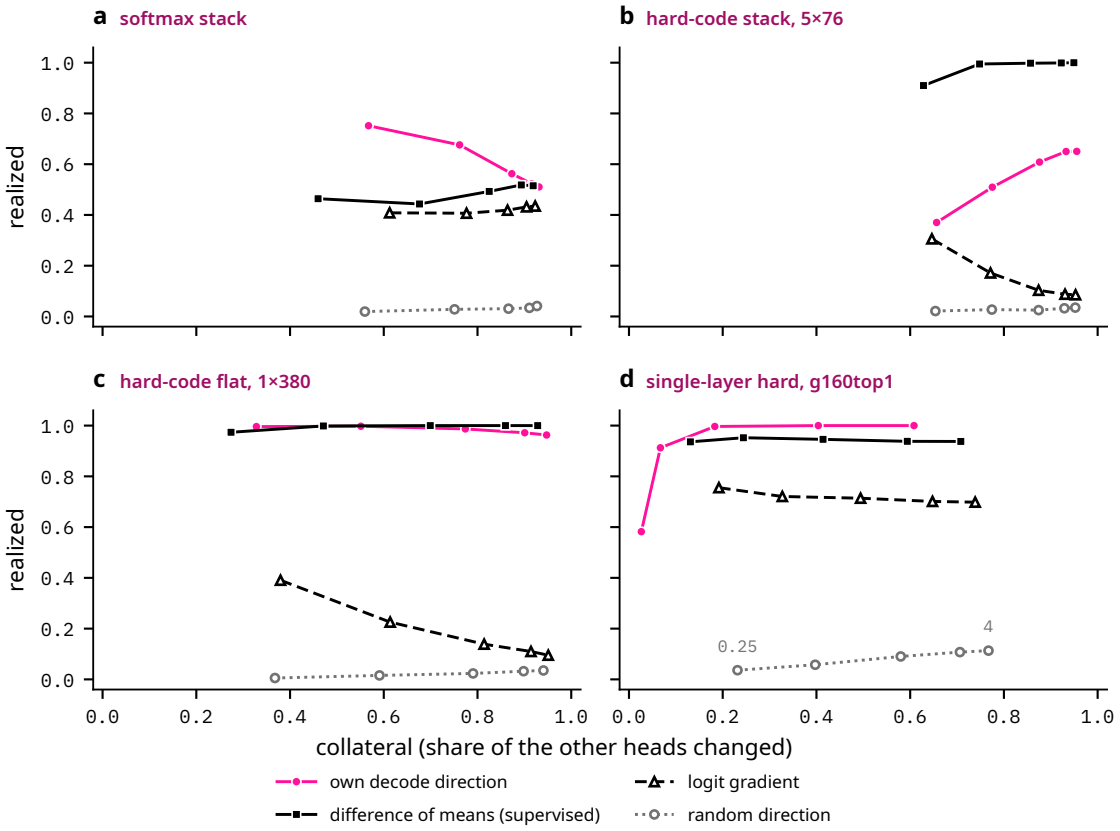


Figure 10 Target realization against collateral as step strength increases along each curve from $0.25\|\mathbf{x}\|$ to $4\|\mathbf{x}\|$, evaluated on the sample draw of Table 3. (a) Softmax stack (`sweep_softmax_shm`). (b) Hard-code stack (`ste_h76_init01`). (c) Flat hard code (`ste_l1_h380_i01_hm1e4`). (d) Single-layer hard variant `g160top1`. At each step strength, native model directions flip as many non-target heads as an isotropic random step in raw embedding coordinates (within ~0.05), except for `g160top1`'s decode direction, which flips substantially fewer; the selective benefit of a direction corresponds to its vertical margin above the random curve. Relative to a covariance-matched random step, the softmax stack's native directions disturb 0.14–0.20 more heads at strength 0.25, whereas the hard codes' directions disturb at most 0.07 more (Appendix E.7). The logistic probe closely tracks the difference-of-means curve (within 0.03 in realization and 0.05 in collateral) and is omitted for clarity.

model	own decode	diff. of means	probe	random
softmax stack	**0.75** / 0.57	0.46 / 0.46	0.47 / 0.51	0.02 / 0.56
hard-code stack	0.37 / 0.66	**0.91** / 0.63	**0.94** / 0.64	0.02 / 0.65
hard-code flat	1.00 / 0.33	0.97 / 0.27	1.00 / 0.31	0.01 / 0.37
untrained stack	0.02 / 0.22	0.75 / 0.22	**0.94** / 0.22	0.01 / 0.22
random-projection heads	0.78 / 0.31	0.87 / 0.30	**1.00** / 0.31	0.02 / 0.32
`g160top1`	0.58 / **0.03**	0.94 / 0.13	0.95 / 0.14	0.04 / 0.23
`g160softmax`	1.00 / 0.30	0.99 / 0.27	1.00 / 0.31	0.01 / 0.34
`TopK` SAE, k_{SAE}=32 (m=11264)	1.00 / 0.21	0.92 / 0.21	0.97 / 0.23	0.00 / 0.25
SAE, bilinear encoder	1.00 / 0.07	0.97 / 0.12	0.99 / 0.11	0.01 / 0.13

Table 3 Comparison of each model's own decode direction against supervised steering vectors fitted to the same target partitions (realized / collateral) at step size $0.25\|\mathbf{x}\|$, across 60–64 targets × 32 held-out rows per model (one seed). Untrained stack denotes the 5×76 hard-code stack at step 0 under the fixed initialization. Random-projection heads use 1×380 locality-sensitive hash heads with k=32 bins (`steerembed.py --model lsh`), whose "own decode" swaps empirical cell centroids ($\mathbf{c}_{ij'} - \mathbf{c}_{ij}$); stepping along the target bin's projection direction or margin gradient achieves 1.00 realization at 0.32 collateral. For SAEs, collateral measures changes across other active latents, so comparisons should be made within each row.

and `margin` 0.85, vs. 0.30 and 0.16 after training) steer reliably into its random partitions at one third of the trained stack's collateral (0.22 vs. 0.66). Training thus aligns the parametric decode step with the classifier's cells while pushing heads closer to their decision boundaries and tripling collateral. Likewise, in a single layer of 380 random-projection heads, empirical cell-centroid swaps realize 0.78 of targets at strength 0.25 (0.67 at native step size) with 0.31 collateral, while the target projection vector, margin gradient, and logistic probe each achieve 1.00 (0.87 for difference of means), matching the trained flat hard code and outperforming the trained five-layer hard stack on both realization and collateral. These arbitrary partitions play the role that control tasks play for probes [87], where a probe's accuracy is read as its margin over the same probe trained on random labels: a steering score counts here only by its margin over the same step into a partition that training did not shape. Evaluated at each method's unnormalized native step size, the difference-of-means vector achieves higher realization in most models, partly by taking longer steps (Appendix E.6). Whereas this evaluation tests steering into a model's own cells, Appendix D.5 evaluates steering toward an external ground-truth attribute—source language—against supervised vectors fitted to language labels.

Individual entries can nonetheless induce coherent semantic or linguistic changes on a single seed: in the reference softmax model, forcing entry 17 of head 8 in layer 1 consistently translates decoded text into Burmese while preserving semantic content.

5.5 Description specificity (autointerp)

Evaluated individually, sparse SAE latents yield specific natural-language descriptions roughly twice as often as Ontologizer entries. Under an identical plain summarization prompt, 51% of k_{SAE}=32 SAE latents receive a specific description (20% naming a language or script), compared to 23% of entries in the reference softmax model (8% naming a language or script). Blinded

detection across the full six-model campaign shows the same ordering (Table 4).

model	density	freq	params	eig	acts	cacts	features
onto (resid_nc)	100%	10.80%	0.039	—	0.473	0.392	225
k32	0.6%	0.61%	0.139	—	0.645	0.603	250
k32_bl	0.6%	2.62%	0.136	0.034	0.578	0.470	75
g160top1	3.1%	5.08%	0.111	—	0.646	0.594	150
g160softmax	100%	0.00%	0.086	—	0.449	0.364	256
k5120 (dense ctrl)	45.5%	17.97%	0.010	—	0.470	0.332	256

Table 4 Mean per-feature blinded detection F1 by description mode (Appendix B.5). Because detection sets are class-balanced, an indiscriminate description that matches every item achieves the always-match baseline of ~0.67, inflating scores for overly generic summaries. Density denotes nominal code density, and `freq` denotes the empirical firing rate above the $2/k$ threshold. For `k32_bl`, the evaluated pool is only 75 live features; no null control was recorded for this campaign.

Across the learned dictionaries evaluated here, per-feature describability tracks activation density rather than model architecture (raw base-model units score lower at comparable sparsity [18]). Plain activation descriptions rank the sparse codes (0.65 for both the k_{SAE}=32 SAE and `g160top1`) above the bilinear SAE (0.58), the reference Ontologizer and dense SAE control (0.47), and the single-layer soft variant (0.45), with nominal density alone accounting for roughly 80% of the variance across runs (Figure 11a). Much of this gap reflects language selectivity on the multilingual corpus. Because mC4 cycles across 86 languages and neither the contrastive background nor the detection negatives are language-matched, a feature whose top activating examples come from a single language can be described simply by naming that language and detected reliably against random negative examples: 63 of the k_{SAE}=32 SAE's 250 evaluated latents and 41 of `g160top1`'s 150 draw all 15 judged top rows from one to three languages and achieve detection F1 of 0.90–0.97, whereas no dense code contains such language-specific features. Across the six runs, mean language purity accounts for 93% of the variance in mean activation-description F1 (and 98% for contrastive descriptions), though across six runs it moves with density. Restricting the comparison to features whose top examples span 13–15 languages (matching random draws from the corpus) narrows the activation-description F1 lead over the reference Ontologizer from 0.17 to 0.10 for the k_{SAE}=32 SAE and to 0.01 for `g160top1`. Likewise, the reference Ontologizer lies directly on the SAEs' log-density trend, and `g160top1`, which sits 0.12 above that trend across all features (Figure 11b), sits only 0.02 above a trend refitted on multilingual features (Appendix D.6). Standard per-feature detection F1 therefore cannot compare architectures fairly across mismatched densities or language distributions, and soft Ontologizers are dense by construction; distribution-based monosemanticity metrics exhibit the same confound, increasing as L0 decreases at fixed dictionary width [88]. Contrastive descriptions (`cacts`), though also evaluated without language-matched backgrounds, mitigate generic-summary inflation: they score below plain activation summaries on every model, dropping least on sparse codes (0.04–0.05) and most on the dense control (0.14).

Generating descriptions by decoding parameter vectors alone without activating examples fails across all models. Parameter-only decodes (`params`) achieve at most 0.14 F1 (on the linear

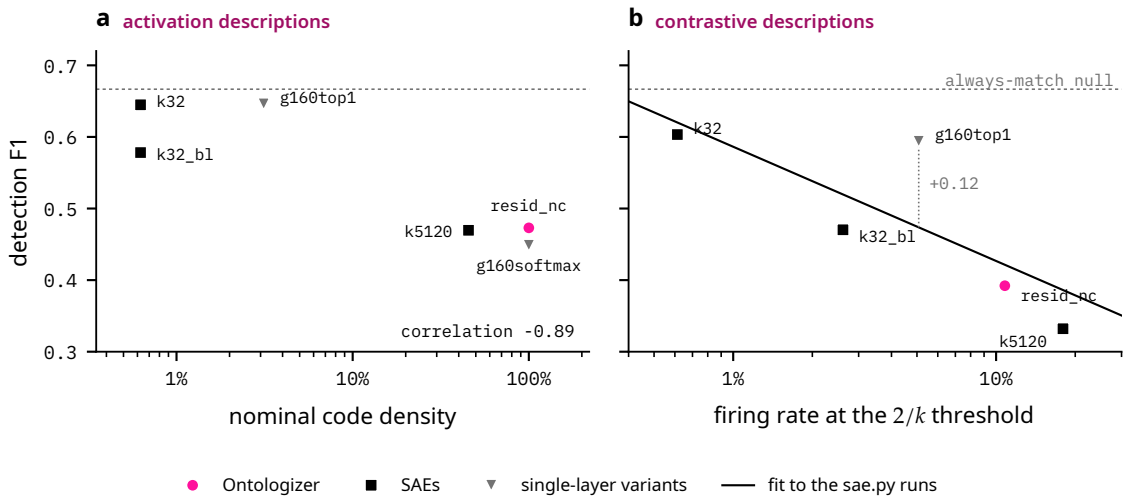


Figure 11 Mean detection F1 across the runs in Table 4 as a function of code density. (a) Plain activation descriptions (`acts`) against nominal code density (the fraction of nonzero coefficients). (b) Contrastive activation descriptions (`cacts`) against empirical firing rate above the $2/k$ threshold, with a log-linear fit across the four `sae.py` runs that cross the threshold (`g160softmax` never crosses $2/k$ and is omitted, and the Ontologizer `resid_nc` is held out of the fit). Because detection sets contain equal numbers of positives and negatives, matching every item yields the always-match baseline of $2/3$ (dotted line). One seed per run.

and bilinear SAEs) and 0.04 on the reference Ontologizer, compared to 0.45–0.65 for activation-based descriptions of the same features. Bilinear eigenfeature decodes (`eig`) score 0.034, but that protocol decodes the top-$|\lambda|$ eigenvector (which for most latents in the bilinear SAE is an inhibitory rather than excitatory direction; Appendix B.2) and thus does not properly test excitatory eigenfeatures. An Ontologizer's parameter decodes are further hindered by shared within-head offsets: in the hard-code stack's topic head, an atom has a mean cosine similarity of 0.849 with its head's mean atom, so raw atom decodes produce a single template sentence with minor variations. Subtracting the origin decode makes the resulting sentences legible, yet they remain undetectable (on a single evaluated head, F1 is 0.079 vs. a 0.021 null, compared to 0.479 vs. 0.273 for contrastive activation descriptions). Fundamentally, a decoded atom yields a single synthetic output instance rather than a membership criterion, and evaluating output-side descriptions with an input-side detection test is misaligned; causal intervention scoring [18] would provide the matched output-side evaluation.

6 A hard-code arm: straight-through selection

Because taking the discrete argmax of the softmax model fails to reconstruct the input (Section 5.1), the discrete entries targeted by interventions do not carry its representations. Under straight-through selection, the forward pass uses the discrete argmax directly, preventing any mismatch between training and discrete evaluation: at every evaluated checkpoint, hard-forward and soft-forward reconstructions agree to four decimal places.

6.1 Capacity, and why the head count rises

A hard code with l layers of h heads and k entries transmits exactly $l \cdot h \log_2 k$ index bits per sample. The Gaussian rate–distortion reference (Eq. (25)) specifies the bit budget required to achieve a given distortion on a Gaussian source with the same covariance, upper-bounding the rate needed for non-Gaussian embeddings:

FVU_w target	bits	heads at k=32, l=5
0.226	800	160 (the softmax architecture)
0.05	1,898	380 (this arm, h=76)
0.0023	4,167	833

Increasing the head count h is computationally cheaper than increasing depth l, because both bit capacity and parameter count scale linearly with h while heads execute in parallel rather than sequentially. Our primary SONAR hard-code arm stacks 5×76 heads of k=32 entries, yielding 1900 index bits and 51.93M parameters. Training classifiers with a hard forward pass on SONAR required three calibration changes, because argmax selection is scale-invariant and a naive port leaves logits near their small initial scale: scaling logit noise proportionally to the logit norm, matching the straight-through surrogate temperature to that scale, and rescaling the weight s_{KL_m} on the batch-mean uniformity term KL_m (Appendix E.1; full configurations in Table 10). By contrast, the GPT-2 straight-through arms used the uncalibrated port settings (t=1, absolute logit noise σ_K=0.02, and s_{KL_m}=10^{-6}; Table 11), so comparisons across the two substrates also reflect different calibrations. Nor did the GPT-2 arms' logits behave alike: in the GPT-2 `ste_h76`, logit norms grew 452-fold until noise flipped only ∼1% of layer-0 assignments, whereas in the concatenated-head seed pair, logits remained near their initial scale and noise flipped 71–90% of layer-0 assignments across every evaluated checkpoint (Appendix E.1).

6.2 Initialization: a third of the error

Because dictionary entries are constrained to be non-negative, the h selected atoms in each layer sum coherently and the layer's output norm grows with h. Multiplied by the per-layer residual gain, this growth compounds geometrically with depth under default initialization: our first hard-code run (`ste_h76`) started at 1.84×10^8 where it should start near 1. Rescaling each dictionary at initialization so that a layer's initial output norm is roughly one tenth of its input norm (`--dict-init-scale 0.1`) cancels the factor of h and initializes the residual stack near the identity:

This single initialization scale reduces held-out FVU_w by one third and brings every head to near-uniform entry usage (Table 5), producing the largest single improvement in the straight-through campaign. It also closes the utilization gap that rescaling the batch-mean uniformity term KL_m and input whitening had each addressed only partially (Appendix E.1). All hard-code results below use this rescaled initialization, except for the per-layer steering and head-health ablations in Appendix E.5.

A discrete code beats `TopK` SAEs up to 160 latents, but not every sparse code. Against the frontier of converged SAEs and the single-layer hard variant, all trained for roughly 144k steps

arm	shape	s_{KL_m}	realized bits	FVU_w	params
`ste_h76` (shipped init)	5×76	10^{-4}	1714 (90.2%)	0.2293	51.93M
`ste_h76_init01`	5×76	10^{-4}	1895 (99.7%)	**0.1535**	51.93M
`ste_h76_i01_s43` (seed 43)	5×76	10^{-4}	1895 (99.7%)	**0.1535**	51.93M
`ste_k2_h380`	5×380, k=2	5×10^{-4}	1887 (99.3%)	0.1844	17.67M
`ste_l1_h380_i01_hm1e4`	1×380	10^{-4}	1898 (99.9%)	0.2750	51.93M
`ste_l1_h380_i01`	1×380	5×10^{-4}	1898 (99.9%)	0.2870	51.93M
`ste_l1_h380_i01_s43`	1×380	5×10^{-4}	1898 (99.9%)	0.2871	51.93M

Table 5 Straight-through arms on the held-out tail under the rescaled initialization (except the first row). Realized bits report the summed marginal entropy of each head's argmax usage over the tail relative to the nominal $l\ h \log_2 k$ budget; this sum upper-bounds the joint information carried per sample, with equality if and only if heads are mutually independent (Eq. (36)).

(Figure 8a), the 5×76 hard stack achieves $\mathrm{FVU}_w = 0.154$ using 1900 index bits and only four continuous per-layer gains (layers 1–4, as layer 0's gain is fixed to 1). Pinning those four gains to their held-out means yields a purely discrete 1900-bit code at $\mathrm{FVU}_w = 0.158$ (Appendix E.4). By comparison, any sparse code that transmits continuous coefficients alongside support indices must split its bit budget between the two, and no quantization of an SAE's coefficients can achieve lower error than an unconstrained least-squares refit on its active support (Eq. (33)). A k_{SAE}=160 `TopK` SAE spends 1,206 bits to index its active set, leaving 4.3 bits per coefficient at a 1900-bit total budget, where its error is bounded below by its support refit of $\mathrm{FVU}_w = 0.236$ (0.258 as trained); at a matched 1900-bit rate, the pure hard code therefore reduces error by at least 33%, and higher-precision coefficients would only increase the SAE's bit rate. Similarly, `g160top1` (which shares hard per-head selection in a single layer of linear classifiers while retaining a scalar ReLU magnitude per winning entry) spends 800 index bits plus 160 coefficients, leaving 6.9 bits per coefficient at 1900 total bits with a refit lower bound of $\mathrm{FVU}_w = 0.273$, so the pure hard code reduces error by at least 42%.

However, SAEs with variable sparsity or wider active supports can outperform the hard stack under entropy coding. When continuous values are quantized and each slot's symbols are entropy-coded (Appendix E.3), a weakly penalized ReLU+L1 SAE (s_{L1}=3×10^{-6}) with coarse quantization reaches $\mathrm{FVU}_w = 0.129$ at an average rate of 1900 bits per row, 17% below the hard stack: skewed latent firing rates reduce its support-indexing cost by roughly one fifth relative to fixed-length indexing, whereas the hard stack uses entries almost uniformly and gains nothing from entropy coding. Because active-set sizes also vary across samples in an L1 SAE, capping every row at a strict 1900-bit budget raises its error to $\mathrm{FVU}_w = 0.152$, roughly matching the fixed-length hard code; if every active support is instead charged its fixed-length combinadic cost, the pure hard code leads all SAEs ($\mathrm{FVU}_w = 0.158$ vs. 0.180). On the sentence-split SONAR cache, a wider `TopK` SAE (256 active out of 11,264 latents) outperforms the hard stacks trained on that cache under both entropy-coded regimes, reaching $\mathrm{FVU}_w = 0.128$ at a 1900-bit average rate and 0.129 under a strict 1900-bit per-row cap, compared to 0.161 and 0.164 for the stronger hard stack; the hard stack leads only when the SAE is restricted to a single global quantization step that must fit every row (0.164 vs. 0.185; Appendix E.3). Finally, for a hard Ontologizer, code

capacity is governed directly by $l \cdot h \log_2 k$: post hoc truncation toward the mean-assignment origin injects untrained mixture weights and degrades reconstruction ($\mathrm{FVU}_w = 0.259$ at ν=1). Per bit, the five-layer stack is less efficient than a single layer, sitting at 3.08× the Gaussian reference at 1900 bits compared to 1.28× for a single layer's 380-bit prefix, and total bit count alone does not fully determine accuracy: replacing 32-entry heads with 380 binary heads (k=2) at the same 1900-bit budget raises error to $\mathrm{FVU}_w = 0.1844$ (Appendix E.2).

6.3 Depth at fixed capacity

Comparing five layers of 76 heads against a single layer of 380 heads isolates the effect of residual depth at identical nominal capacity (1900 bits) and identical parameter count (51.93M, since both dictionary size hke and bilinear classifier size $2hkd$ depend on l and h only through the product $l \cdot h$). Under the rescaled initialization, both models use their entries nearly uniformly (summed marginal head entropy reaches 99.7% and 99.9% of nominal capacity), yet the five-layer stack achieves 1.79× lower reconstruction error ($\mathrm{FVU}_w = 0.1535$ vs. 0.2750). Random seed variation does not account for this difference, as seed replicas agree to four decimal places. Note that summed marginal entropy upper-bounds joint code entropy, and whether the two architectures differ in joint information across heads has not been measured. Although neither run has fully converged and their error ratio narrows gradually over training, the five-layer stack leads at every evaluated checkpoint (Appendix E.4).

Ablations confirm that the benefit of depth does not stem from entry utilization, input conditioning, or the scalar per-layer gains. A single-layer flat model trained on whitened inputs uses entries uniformly yet still incurs 1.7× higher error, and pinning the per-layer gains to their held-out means increases the five-layer stack's FVU_w by only 3.0% (Appendix E.4). Instead, interpolating each layer's subtracted reconstruction toward that of an unrelated sample at evaluation time without retraining causes error to rise sharply well before sample-specific pairing is completely broken, showing that the stack relies on conditioning each stage on the current sample's residual. Whereas a single layer decodes like an additive quantizer [25], it does not encode like one: additive quantization uses beam search to select each codeword against the residual left by previously chosen codewords, whereas a single layer's heads select entries independently in parallel without observing one another's errors. Independent selection is benign in product quantization, where heads occupy orthogonal subspaces, but degrades accuracy when decoded atoms overlap. By contrast, the residual stack acts as a multi-stage residual quantizer in which each layer encodes the error left by earlier layers [32]. Among SAEs, the matching-pursuit SAE has the same structure within a single code: its encoder chooses each latent against the residual its predecessors leave [89]. Because every stack also applies deep supervision to all intermediate prefixes (which has no counterpart in a single layer), whether this improvement requires prefix supervision in addition to residual forwarding has not yet been isolated.

6.4 Entanglement: the cost of the cascade

While residual depth reduces reconstruction error, it substantially degrades input-space addressability. Stepping in embedding space along a target entry's decode direction realizes that

entry on nearly every row in the single-layer flat model (99.6%) while flipping roughly a third of other heads, whereas in the five-layer stack it realizes 37% of targets while flipping 66% of other heads. Restricting stack interventions to layer 0, so that target heads receive the raw input directly just as in the flat model, widens the disparity (realized / collateral at step strength 0.25, and at each model's unscaled native step in the final row; 12 layer-0 targets in the stack and 64 targets in the flat arm):

direction	stack, layer 0	flat
decode	0.049 / 0.680	**0.996** / 0.328
grad	0.281 / 0.656	0.391 / 0.380
margin	0.034 / 0.661	0.076 / 0.379
random	0.008 / 0.655	0.005 / 0.368
native decode	0.044 / 0.678	0.169 / 0.043

High collateral is not specific to the steering direction. In both models, every tested direction flips as many non-target heads as an isotropic random step of equal norm to within 0.05 (Figure 10b, c), and as many as a covariance-matched random step to within 0.025 in the stack and 0.07 in the flat arm (Appendix E.7). This sensitivity arises because hard classifiers sit close to their decision boundaries: in both layer 0 of the stack and the flat model, the median logit margin between the winning and runner-up entries is 1.5×10^{-4}, so a step of $0.25\|\mathbf{x}\|$ crosses many boundaries at once. Linearizing head logits while holding upstream discrete assignments fixed separates direct boundary crossings from downstream cascade effects: layer-0 heads in the stack flip directly at nearly the same rate as heads in the flat arm (0.22 vs. 0.24), indicating that the remaining collateral comes from downstream layers reclassifying the modified residual. Thus, the residual cascade roughly doubles collateral but is not its sole source. Downstream reclassification similarly responds to head ablation by describing the altered input anew rather than restoring the old description (Appendix E.8). At each model's unscaled native step size (the norm of the forced entry swap), the flat arm takes small steps (median $0.028\|\mathbf{x}\|$) and flips 4% of other heads, whereas the stack takes larger steps ($0.13\|\mathbf{x}\|$), realizing 8% of targets across layers and flipping 54% of other heads, matching a random step of equal norm. This behavior mirrors our direct-intervention composition results, where deviations from additivity decrease as fewer downstream layers remain below an intervention (Section 9). Per-layer steering breakdowns, regularizers tested to improve head conditioning (among which only a within-head atom-collinearity setpoint improved steerability), and an HSIC head-independence penalty that proved to reflect finite-sample estimator bias are detailed in Appendices E.5 and E.9.

6.5 Seed reproducibility: the error is determined, the heads are not

Two runs initialized with different random seeds under the rescaled initialization converge to identical held-out reconstruction error: $\mathrm{FVU}_w = 0.1535$ and 0.1535 for the five-layer stack, and 0.2870 and 0.2871 for the flat arm (Table 5). Nevertheless, metrics that evaluate alignment of individual heads show almost no cross-seed agreement:

what is compared	stack	flat	null
head partitions (mean matched NMI)	0.0121	0.0177	0.0048
head contributions (centered cosine)	0.0074	—	0.0015
per-layer subspace, rank 8	= data's	—	see text
atom Grams, entries matched	1.05–1.19× chance	—	within-layer pairing
reconstruction (residual correlation)	0.30	0.34	0.53 (same run)

Hungarian-matched head partitions agree barely above the permutation null (0.0121 in the stack and 0.0177 in the flat arm vs. 0.0048), whereas comparing a single run against its own checkpoint 19,500 steps earlier yields an NMI of 0.38. Centered per-head output contributions show similarly negligible alignment (0.0074 vs. a 0.0015 null). Rank-8 per-layer subspaces overlap across seeds no more than each run overlaps with the data's leading principal directions, and per-head atom Gram matrices align near chance (1.05–1.19× random within-layer pairing) even under optimal entry matching, whereas within a single run each head's Gram matrix matches its earlier checkpoint at 8.5–10.7× chance (Appendix E.10). Thus, individual heads persist across training steps within a run but do not replicate across random seeds. Even the overall reconstructions differ substantially across seeds: whitened residuals correlate at only 0.30 for the stack (and 0.34 for the flat arm), compared to 0.53 between checkpoints 19,500 steps apart within a run and 0.83 between two seeds of the k_{SAE}=32 SAE, which converge to nearly identical reconstructions (`reconseeds.py`). What reproduces across seeds is therefore aggregate distortion and alignment with the data's principal subspaces, rather than individual heads (with one layer-0 exception below). This pattern resembles weak rather than strong universality, in which independently trained networks share an algorithm while differing in the features that implement it [90]; here, however, identical FVU_w arises from substantially different reconstructions in both the flat arm and the stack, so the residual cascade does not cause the cross-seed divergence. Consequently, qualitative and steering findings for specific heads (including the Burmese entry and per-head steering tables) characterize a single seed. Note also that dictionary non-negativity and shared per-layer gains induce substantial baseline alignment, causing uncentered contribution cosines and raw subspace overlaps to appear spuriously high unless compared against structure-matched nulls.

Layer 0 is the sole partial exception across head-level diagnostics, as it is the only layer whose input is not conditioned on upstream discrete selections. Specifically, a single layer-0 head reproduces across seeds: the topic head analyzed in Section 6.6, which forms the strongest layer-0 partition in both seeds (at different head indices) and matches across seeds at NMI 0.41, compared to a median of 0.01 across all other heads. Cross-seed reproducibility is equally low in the softmax Ontologizer, where a matched seed replica aligns on at most 0.1% of entries compared to 3–31% for the k_{SAE}=32 SAE seed pair (Figure 25); note that this thresholded activation-containment metric is stricter than the decoder-cosine [61] and activation-correlation [62] metrics under which language-model SAEs typically report cross-seed agreement near 0.8. Theoretically, static marginal reconstruction objectives cannot identify latent factors without auxiliary side information or structural assumptions matched to the data-generating process [11, 91], and geometrically, a Minkowski sum of polytopes, which is the set a layer of heads can reach, does not determine its summands [49]; Appendix E.10 reviews identifiability results for discrete

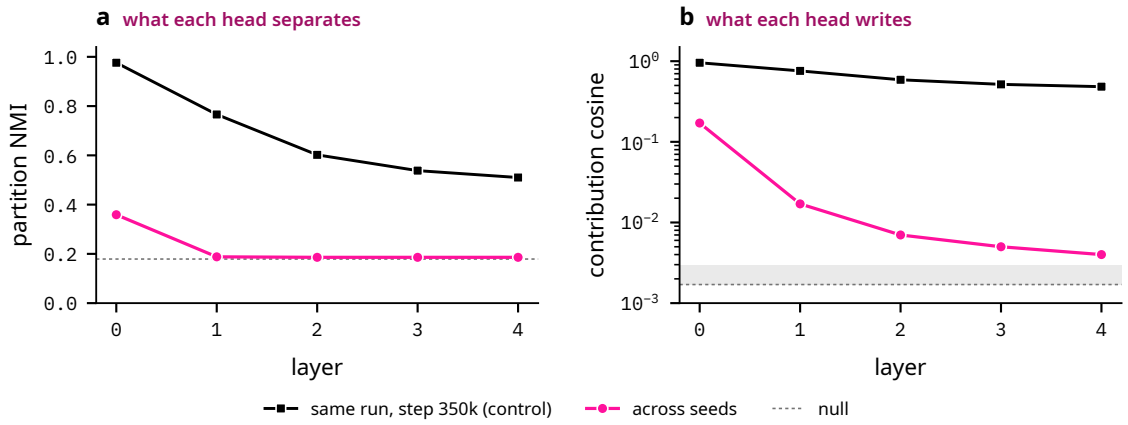


Figure 12 Cross-seed agreement by layer for the GPT-2 concatenated-head seed pair, measured by (a) sample-partition NMI on 32,768 rows and (b) centered output-contribution cosine similarity on 8,192 rows (log scale), evaluated across the two seeds and against the first run's checkpoint at step 350,000. Dotted lines mark permutation nulls; the shaded band in (b) extends from the null mean to the null maximum. One seed pair.

views and temporally paired observations [92, 93, 94, 95, 96, 97, 98] and evaluates next-sentence transition objectives on `mc4_sent16`, producing the one reproducible multi-head block observed so far (Section 6.5.3).

Replication on GPT-2. A matched seed pair trained on GPT-2 small residuals (5×20 concatenated heads, k=256, d_{head}=128, 371,900 steps) exhibits the same layer-wise pattern. We evaluate both partition NMI and centered contribution cosine across the two seeds alongside a within-run positive control comparing the first seed against its own checkpoint at step 350,000 (Figure 12).

Substantial cross-seed agreement appears only in layer 0 (reaching just under one fifth of the within-run control), where every head matches to a layer-0 head in the other seed under both metrics. Layer 1 preserves layer membership under Hungarian matching (19 and 20 of its 20 heads pair with layer-1 heads in the replica) while showing negligible agreement in sample partitions or output contributions, and layers 2–4 lose even layer-level correspondence. Because plug-in NMI between two 256-way partitions on 32,768 samples has a large finite-sample upward bias, the partition null is elevated and cannot resolve tiny effects in deeper layers; against the much tighter contribution null, layers 2–4 sit 2–4× above chance at roughly 1% of the within-run control, an effect that is real but negligible. The excess over the null is larger on GPT-2 than on SONAR (24× vs. 5–11×), almost all of it in layer 0, though these GPT-2 runs also omitted the SONAR logit-scale calibration, and in this seed pair injected logit noise rather than the classifier chose most layer-0 assignments throughout training (Table 11; Appendix E.1).

What layer 0's lead is made of. Three findings bear on layer 0's higher reproducibility, as detailed in Appendices E.11 and E.12. First, layer 0's larger output contributions stem entirely from layer gain rather than atom scale: layer 0 operates on the full unattenuated input whereas its dictionary atoms are no larger than layer 1's (Table 19), and even when conditioned on

matched within-head atom spread, layer 0 reproduces 14× better than deeper layers on GPT-2 and 23× better on SONAR. Second, residual stacking reduces reproducibility at every depth: compared at matched contribution norm against single-layer flat-arm heads (which receive the raw input directly with no downstream layers), the stack's layer 0 reproduces roughly 8× worse and its deeper layers reproduce 19–49× worse, reflecting the reproducibility counterpart of cascade entanglement. Third, within layer 0 the atoms decide: its entries are unequal by atom norm rather than by usage (Table 20). On SONAR, a single outlier head (head 54) contains 8 of layer 0's 11 atoms exceeding three times the median norm, reproduces at 0.844 contribution cosine (vs. 0.249 for the runner-up), and drives the within-layer correlation between cross-seed agreement and maximum atom norm (Table 21); excluding head 54 lowers the layer-0 mean only modestly, from 0.080 to 0.070. On GPT-2, by contrast, no single outlier head dominates: every layer-0 head exhibits a graded tail of large-norm atoms, and cross-seed agreement is narrowly distributed across heads.

The exception: slow heads trained on sentence pairs. A designated block of heads does reproduce across seeds, as a block rather than head by head, when trained with an explicit paired-view objective. Specifically, we trained a single-layer model (h=380, k=32) on pairs of sentences drawn from the same document in `mc4_sent16`, adding to its loss a Fano lower bound on each of its first 38 ("slow") heads' mutual information between the entries it assigns the two sentences (Appendix E.10). Temporal SAEs [99] apply a contrastive version of the same idea to adjacent tokens' codes on a block of SAE latents, without a test across seeds. After three epochs, these 38 slow heads maintain the same entry across adjacent sentences at Cohen's kappa 0.46, compared to 0.07 for the first 38 heads of an unregularized control. Across three random seeds, the slow blocks learn matching partitions: under one-to-one Hungarian matching by NMI (Eq. (29)), every seed pair achieves a mean NMI of 0.412–0.414 (vs. a 0.005 permutation null), every slow head in one seed pairs exclusively with a slow head in the other (chance 10%), and within-language NMI across 84 languages remains at 0.312–0.314 (vs. a 0.041 null), whereas the remaining 342 unregularized heads and all heads of the control model match at only 0.016–0.017. Furthermore, the one-hot slow-head code extracts document-persistent structure beyond linear projections of the embedding: the top 38 canonical slow directions of the discrete slow block achieve a mean held-out cross-sentence correlation of 0.727 and 0.718 (seeds 42 and 43), exceeding the top 38 linear slow-feature projections of the embedding's unit direction (0.636), and maintain that lead after removing per-language means (0.524 and 0.523 vs. 0.462).

Two important caveats bound this result. First, it has been demonstrated only in a single-layer architecture and incurs a 1.8–1.9% increase in FVU_w. Second, it holds for the 38-head block rather than for individual heads. Within a single seed, the slow heads are highly redundant: 34 and 29 of the 38 heads (in seeds 42 and 43) align most strongly with the single slowest linear direction (and the remainder with the second), distinct slow heads within one seed already agree at NMI 0.40, and an arbitrary bijection between two seeds' slow blocks scores 0.397–0.399, so optimal Hungarian matching adds only 0.014. Moreover, pairwise matchings across the three seeds are cycle-consistent for only 6 of the 38 heads (compared to 1 expected by chance). What reproduces across runs is therefore the shared slow subspace rather than individual slow heads. Penalizing dependence between the slow heads' partitions in a fixed order does not separate

them without loss. At a small weight the block becomes graded and no head is identified. At a large weight exactly one head is: over two seeds it agrees with its counterpart at NMI 0.386 and with any other slow head at 0.009, while the remaining 37 heads lose their slow content. The block then explains 0.43–0.44 of the variance of the eight slowest linear directions, where the Fano block explains 0.76–0.77 and a single slow head 0.38–0.43. The Fano block thus carries about twice what one head does, in a form that independence between heads does not recover, and how many distinct document-level factors it captures remains open (Appendix E.10).

6.6 Heads against a known factor: language, and what they encode instead

Source language is the primary ground-truth attribute available across the corpus, which motivates using it as an external probe even though SONAR maps translations to nearby points [58] and thus leaves language as a weak residual signal rather than a dominant axis. Consistent with that geometry, no individual head isolates language identity: the most informative head achieves a language-partition NMI of 0.18 out of an attainable 0.875 (0.165 on the replica seed), and none of the head-health regularizers alters that ceiling (Table 22). Nevertheless, head partitions capture language signal well above chance, and the hard code retains substantially more language information than the softmax reference, scoring 3.5–3.9× higher on null-corrected best-head NMI and reaching 0.42 vs. 0.27 accuracy in a 32-head sparse probe. Individual cells and multi-head cell conjunctions select typological or geographic language neighborhoods (such as Finnish with Estonian, or Japanese with Korean) rather than single languages, matching an embedding space in which languages occupy weakly separated regions rather than orthogonal directions (Appendix E.13).

What the strongest head holds: topic and genre. In a semantic, language-agnostic sentence space, the primary axis of variation should reflect topical content, and the model's dominant head aligns with that prediction. Ranking layer-0 heads by fraction of embedding variance explained (η^2) on 65,536 held-out rows and matching partitions across runs gives:

run	top head	η^2	next head	match across runs
`ste_h76_init01` (seed 42)	h54	0.117	0.062	—
`ste_h76_i01_s43` (seed 43)	h14	0.122	0.068	NMI 0.413 (others 0.010)
`ste_h76` (unfixed init, seed 42)	h54	0.111	0.080	NMI 0.355 (others 0.024)

This topic head appears in every run as the strongest head, explaining nearly twice as much embedding variance as the second-strongest head under the rescaled initialization and 1.4× as much under the original one. Changing the random seed shifts it to a different head index, whereas fixing the seed preserves its index across the initialization rescaling. It is also the sole head whose output contributions reproduce across seeds (Appendix E.12), forming the single exception noted in Section 6.5; however, its cross-seed NMI of 0.41 falls well short of the 0.87 within-run control, indicating that different seeds carve the same variable differently rather than recovering the same partition. Notably, this head also carries as much language signal as almost any head in the stack, ranking first among all 380 heads in seed 43 (NMI 0.165; Table 22) and third in seed 42 (0.161), even though relative to the 0.875 ceiling each entry still pools many languages.

Decoded as deviations from the head's mean atom, its entries correspond to broad document topics and genres, including crime news, geopolitics and parliamentary reporting, encyclopedic place descriptions, manufacturing, marketing, legislation, and entertainment. Contrastive autointerp agrees, with every category of description scoring above its null (Appendix E.13). Evaluating whether heads recover ground-truth semantic variables more generally will therefore require corpora annotated for topic and genre, which mC4 lacks.

7 Hard codes on GPT-2 small: atoms in activation space

A second series of straight-through experiments, conducted on the `headline` branch, evaluates hard codes on GPT-2 small and introduces a downstream functional metric unavailable on static sentence embeddings: the fraction of GPT-2's next-token cross-entropy loss recovered when the reconstructed activation is spliced back into the forward pass, $(\mathrm{CE}_{\mathrm{ablate}} - \mathrm{CE}_{\mathrm{recon}})/(\mathrm{CE}_{\mathrm{ablate}} - \mathrm{CE}_{\mathrm{clean}})$, using mean ablation of the residual stream as the baseline. Because mean ablation sets a lenient floor that compresses small cross-entropy differences near 100% [9], the caption of Table 6 also reports raw cross-entropies. These experiments model the FineWeb residual stream entering block 8 (one block earlier than the GPT-2 site in Section 6.5), centered and scaled to unit mean square, and report unwhitened FVU; results here are compared only within this section against SAEs trained on the identical cache. Held-out evaluations use documents disjoint from the training split (520k tokens for 40M-token runs and 65k tokens for one-epoch runs).

The design. Relative to the SONAR hard-code setup in Section 6, three architectural choices change. First, atoms are parameterized directly as signed vectors in $\mathbb{R}^{768}$ activation space without an intermediate dictionary space or shared decoder, making every atom a direct residual-stream perturbation. Second, every layer (including layer 0) classifies the unit direction of its input residual and scales its output by the observed residual norm clipped at $\|\mathbf{x}\|$, making the forward pass positively homogeneous ($f(\xi\mathbf{x}) = \xi f(\mathbf{x})$ for $\xi > 0$). Third, the straight-through surrogate temperature is annealed early in training (Table 12). With 5×32 heads of k=32 entries, the code transmits 800 discrete bits per token alongside five scalar per-layer gains.

The 800-bit discrete code recovers a larger share of GPT-2's next-token loss than a k_{SAE}=32 `TopK` SAE both on the shared 10M-token training cache (97.0% vs. 96.2%, despite higher unwhitened FVU of 0.217 vs. 0.189) and after one pass over 40M tokens (97.6% at FVU = 0.195; Table 6). However, when the SAE's 32 active coefficients are quantized to ∼5 bits each (556–588 total bits vs. 800 + ∼25 for the Ontologizer), the SAE achieves lower FVU per bit. Across model families, raw FVU therefore does not monotonically order downstream loss recovery, whereas within the Ontologizer family on SONAR, downstream text fidelity varied monotonically with FVU_w (Appendix C.5). Part of this cross-entropy deficit can reflect functional mismatch rather than lost information, as low-rank adaptation of the downstream model to a frozen SAE closes 30–55% of its cross-entropy gap [100]. Expanding each head to 64 entries (960 bits) surpasses the k_{SAE}=32 SAE on both FVU and loss recovery. Across the stack, every layer contributes nontrivially, entry usage remains near uniform, and the observed residual norm closely

model	atoms	code per token	FVU	loss recovered
direct + gain + STE (10M tokens)	signed, in $\mathbb{R}^{768}$	800 bits + 5 gains	0.217	97.0%
direct + gain + STE (40M tokens)	signed, in $\mathbb{R}^{768}$	800 bits + 5 gains	0.195	**97.6%**
concatenated heads + gain + STE	48-dimensional blocks, shared decoder	800 bits + 5 gains	0.186	97.5%
concatenated heads, k=64	48-dimensional blocks, shared decoder	960 bits + 5 gains	**0.164**	**97.9%**
`TopK` SAE, k_{SAE}=32, m=5120	—	32 coeffs + ~276 bits	0.189	96.2%
`TopK` SAE, k_{SAE}=128, m=5120	—	128 coeffs + ~859 bits	0.116	98.6%
soft Ontologizer (1 epoch)	—	4960 mixture weights	0.016	99.85%

Table 6 GPT-2 small, residual stream entering block 8, evaluated on held-out documents. Ontologizers run for one pass over 10M or 40M tokens on a single seed (the 10M direct run's seed-43 replica reaches FVU = 0.219 and 96.9% loss recovered). SAEs train for 10 epochs over the 10M-token cache under `sae.py`'s unwhitened objective; quantizing active coefficients to ~5 bits each gives the k_{SAE}=32 SAE 556–588 total bits at FVU = 0.189–0.190. Clean cross-entropy is 3.806, mean-ablated is 8.283, and the 40M direct code reaches 3.914; FVU is unwhitened here.

tracks the optimal per-sample layer scale. One-epoch ablations confirm that the gain-shape decomposition and direct atoms each improve reconstruction (direct atoms most strongly), that temperature annealing is critical, and that replacing the shared decoder in a twin of the 40M concatenated-head model with separate per-layer decoders reduces FVU from 0.186 to 0.169 (97.3% → 97.7% loss recovered) while reducing cross-layer coupling between co-indexed heads from 97% to 6% (Appendix F.1).

Fibers. Attaching a rank-r local basis to each entry, with fiber coordinates projected linearly from the unit residual, augments each discrete entry with r continuous values per head. Evaluating "base only" removes the fiber contribution at inference to test whether the underlying discrete code preserves a standalone reconstruction:

variant (1 epoch, direct + gain + STE)	extra numbers	FVU (base only)	loss recovered
base	0	0.217	96.95%
rank-4 fiber, fiber dropout 0.5	640	0.059 (0.274)	99.4%
rank-8 fiber, fiber dropout 0.5	1280	0.014 (0.304)	99.9%
rank-4 fiber, joint norm cap 0.2	640	0.180 (0.227)	97.6%
rank-1 radial fiber (gain per entry)	160	0.148 (0.669)	98.5%
frozen base, rank-4 fibers trained after	640	**0.016 (0.215)**	99.9%
unconstrained fibers	—	— (∼10)	—

Low-rank fibers close far more than the FVU gap to the SAE, but because they transmit hundreds of continuous coordinates per token (more than the SAE's 32 active coefficients), this does not constitute a matched-rate comparison. Rather, the experiment tests what training regime prevents continuous fibers from supplanting the discrete base. Under unconstrained joint training, the fibers dominate reconstruction and the discrete base collapses (base-only FVU $\approx$ 10); fiber dropout (0.5) holds base-only FVU at 0.27–0.30 at both ranks, while a joint norm cap (0.2) preserves the base (0.227) only by sacrificing most of the fibers' reconstruction benefit. Freezing a trained discrete base and fitting rank-4 fibers post hoc is best on both counts at its rank: the discrete base remains intact by construction (FVU = 0.215), while the full code matches unconstrained joint fibers (FVU = 0.016, 99.9% loss recovered). Only the rank-8 dropout run reconstructs slightly better (0.014), with twice the coordinates and a base at 0.304. Joint-training regularizers therefore throttle fiber expressivity rather than genuinely protecting the discrete base.

Describability cliff and lexical baseline. Evaluating entries by how accurately a blinded LLM judge separates held-out activating examples from non-members using descriptions generated from activating contexts (balanced accuracy; chance 0.50, document-split nulls 0.496–0.507) reveals a steep decline with layer depth. On the 40M-token models, layer 0 averages 0.70–0.71 across all entries (0.91 on highest-margin members, 0.65 on random members, and 0.55 near the decision boundary), layer 1 drops to 0.56–0.57, and layers 2–4 sit at the 0.50–0.52 null even when provided up to 256 describing examples or 64-token contexts (set-level detection over 10-token bundles retains a weak signal at layer 3, scoring 0.556 vs. a 0.505 null). Moreover, even in layer 0

(where 29–31 of the 32 heads primarily track current-token identity alongside one positional head at 1.3–1.8 bits and two document-level heads), a judge-free lookup baseline conditioned solely on current-token identity outperforms the LLM judge on single cells (0.75 vs. 0.69 on concatenated heads, and 0.74 vs. 0.71 on direct atoms), as a per-token lookup table of mean activations outperformed explanation and simulation for GPT-2's neurons [17]. Conditioning on two-head intersections in layer 0 improves balanced accuracy against single-head siblings by +0.10 on concatenated heads (0.69 → 0.78, driven largely by lexical selectivity) and by +0.01 on direct atoms (0.71 → 0.72).

Steering, edit cost, and the random-projection null. Single-head interventions on the 10M-token models perturb predictions modestly: clamping one layer-0 head to its mean entry across all tokens incurs 0.02–0.075 nats of next-token KL divergence (decreasing from 0.063 to 0.021 on concatenated heads as head dropout increases from 0 to 0.5), comparable to zeroing a single active SAE latent (0.055 nats), while swapping a single head from entry j to j' on the 40M-token models costs 0.001–0.06 nats at unit strength. Evaluated by how closely the steered next-token distribution approaches entry j''s empirical next-token distribution among the head's 31 alternative entries (rank 0 best, ∼15 chance; one head each in layers 0, 2, and 4 across 128 context windows), unit-strength swaps place target j' at mean rank 3.5–6, and 4× swaps achieve rank 5–8 vs. 14–18 for norm-matched random directions. However, this specificity does not imply that the learned entries isolate meaningful model variables: on the 40M concatenated-head model (evaluated across 8 heads per layer at unit strength), stepping between empirical centroids of arbitrary random-projection bins places the target bin at mean rank 0.21, 0.00, 0.44, 1.69, 1.27 across layers 0–4, ahead of the learned cells' centroid steps (0.56, 0.66, 0.75, 1.37, 1.28) in layers 0–2, level with them in layer 4 and behind only in layer 3, and in the range of contrastive atom swaps (0.71–1.94, vs. 14.2–16.9 for random directions). Because the downstream transformation from block 8 to the output logits is locally nearly linear, centroid swaps between any partition's cells steer next-token predictions specifically toward those cells' empirical distributions.

Seed stability, replicated. Under Hungarian matching by partition NMI across two independent seeds, head partitions align above within-run background only in layer 0 (NMI 0.088, vs. 0.050 between unrelated heads within a single run and 0.002 under row permutation) and show no meaningful cross-seed correspondence in layers 1–4 (0.003–0.015). Despite using direct signed atoms, layer-0 gain-shape decomposition, an earlier residual site, and an unwhitened objective, this replicates the finding of Section 6.5: beyond the first layer, which entries belong to the same head depends on the random seed.

8 Dictionary geometry and the non-negativity constraint

Two geometric observations frame this section (Appendix G). First, within-head atom collinearity measured directly in the e-dimensional dictionary space largely reflects unconstrained null-space components: 74–78% of each atom's energy lies in the decoder's null space, so dictionary geometry must be evaluated after decoding into output space (Appendix G.1). Second, the

per-head selection rule influences dictionary geometry less than training duration does, and the final layer becomes largely inert once upstream layers reduce the residual to a small norm (Appendix G.4).

8.1 The orthant forces a collinear dictionary

Dictionary non-negativity constrains geometry directly through the atom Gram matrix rather than through the decoder's null space. Because any two non-negative vectors have a non-negative inner product, vectors with independent $|\mathcal{N}(0,1)|$ coordinates have an expected pairwise cosine similarity of $2/\pi = 0.637$ (with a minimum of +0.58 across 4,000 random draws), compared to 0.000 for signed Gaussian draws. To measure the reconstruction cost of this constraint, we crossed signed vs. non-negative dictionary parameterizations (`--signed`) with dictionary width e on GPT-2 layer-8 residuals using the straight-through stack architecture, training every arm for 371,900 steps (Table 7).

arm	$\bar{\kappa}$ (row cosine)	eff. rank	FVU_w	at step 90k
signed, e=1536	+0.0009	27.37	**0.1540**	0.2910
signed, e=768	+0.0019	26.95	0.1543	0.2942
abs, e=1536	+0.3038	9.15	0.1934	0.3962
abs, e=768	+0.4543	5.41	0.2927	0.4631
concat, d_{head}=32	+0.5353	3.30	0.4155	0.8055

Table 7 Non-negativity × width on GPT-2 layer-8 residuals, held-out tail: geometry from the weights, FVU_w scored from each checkpoint. No collinearity penalty is active in any arm.

Restricting atoms to the non-negative orthant substantially degrades reconstruction. At width e=1536, the non-negative dictionary (`abs`) operates at one third of the signed dictionary's effective rank and incurs 26% higher held-out FVU_w; at e=768, the error penalty reaches 90%. Across all five arms, mean within-head atom cosine $\bar{\kappa}$ monotonically orders converged error (Spearman +1.00 at convergence; +0.90 at step 90k, where `concat` is the sole inversion). Crucially, non-negativity interacts strongly with dictionary width: halving e from 1536 to 768 increases FVU_w by only 0.2% with signed atoms but by 51% with non-negative atoms, so an overcomplete dictionary ($e > d$) primarily provides geometric room to mitigate orthant collinearity rather than additional representational capacity (Appendix G.1). This undercuts the claim in Section 4.1.4 that $e > d$ serves to enable between-head coordinate disjointness—a property that no summed model achieves (Appendix G.2). Over training, the FVU_w gap between `abs` and `signed` at e=1536 widened from 8% at step 20k to 50% at step 200k before narrowing to 26% at step 371.9k (Figure 13), suggesting slower optimization alongside a persistent gap at the end of a full run.

Dropping the absolute-value constraint produces a decorrelated, near-full-rank dictionary and lower reconstruction error directly, achieving without auxiliary penalties what the within-head collinearity setpoint was introduced to encourage. Signed atoms sacrifice two properties: subtractive atom components cannot be read directly as "concept presence," and the L1 norm lower bound $\|F\|_1 \geq h$ under `norm_rows` (which relies on non-cancellation among non-negative

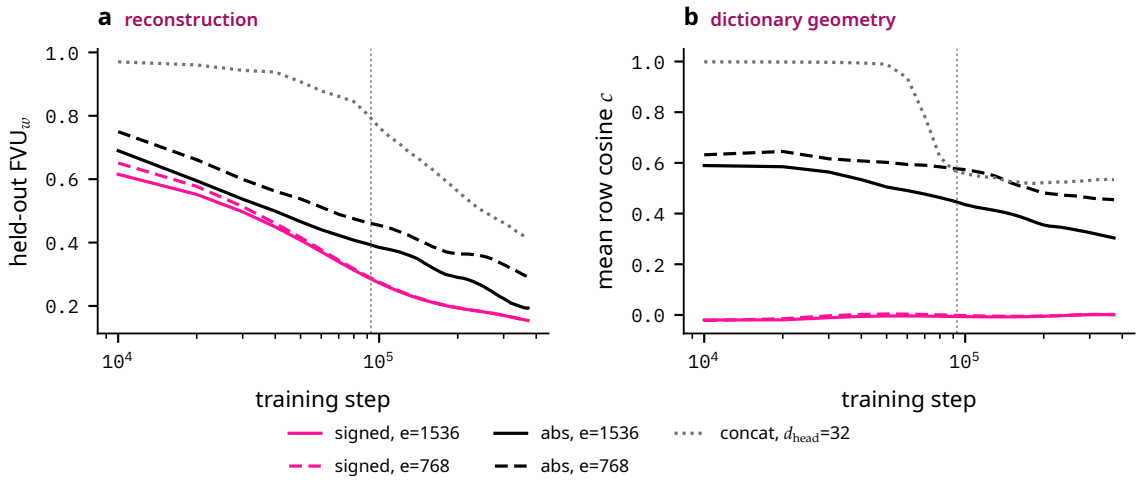


Figure 13 Training trajectories of the orthant-ablation arms across checkpoints: (a) held-out FVU_w and (b) mean within-head row cosine $\bar{\kappa}$. The vertical dotted line marks step 90k. The two signed arms overlap throughout training at both widths, whereas abs trails at both widths and degrades further under a square decoder (e=768).

vectors) no longer holds. Concatenated heads with $d_{\text{head}}=k=32$ under non-negative atoms and constant temperature face a collapsed tradeoff rather than a raised cosine floor and perform worst among these arms (Appendix G.3); providing per-head dimensional headroom ($d_{\text{head}}=48 > k=32$) and annealing the temperature reverses that ranking in Section 7.

9 Formal reading: heads as causal variables

Our steering framework has a natural formalization in the language of causal abstraction [101, 102, 103, 104]. A causal abstraction relates two causal models through a mapping between their states and a mapping between their interventions. Making both models explicit clarifies what each evaluation metric in our battery does and does not estimate.

Definitions. The high-level model is the Ontologizer's forward pass, treated as a causal model over the input and the heads: each head (ℓ, i) is a candidate *variable* with value space $\{1, \ldots, k\}$, the input $\mathbf{x}$ is a parent of every head, and under residual forwarding every earlier head is a parent of every later head because layer $\ell{+}1$'s classifier takes as input the residual produced by layers $\leq \ell$. These causal edges are internal to the code: unlike transcoder circuits [105], which pass through a base model's own sublayers, they describe how the code iteratively refines a single representation rather than how the pretrained encoder computes it. Set-interventions applied via `withArgs` serve as hard interventions in this model: $\text{do}[(\ell, i){=}j']$ forces the head's assignment while leaving all non-descendants unchanged by construction, and marginalizing out the continuous intermediate states of the forward pass defines an exact transformation by construction [101, Thm. 10]. Because a soft head's state is a point on its simplex, discretizing it into one of k categorical values constitutes a lossy value merge [104], so forcing an entry on a

soft head acts as a steering intervention rather than an interchange intervention. The low-level model is the pretrained encoder that produces the input embeddings, and the mapping between states is assign, which maps an embedding $\mathbf{x}$ to its per-head assignments. Whether the head model abstracts the encoder under assign is an approximate-abstraction question [104, Def. 41], and the abstraction cannot be constructive [102] because every head takes the same embedding as input.

What the instruments estimate. Input-space perturbations are not interventions that such an abstraction maps; instead, they measure controllability of the head model rather than whether the encoder computes using those heads [104, Sec. 3.7]. Specifically, the realization rate in `steerembed.py` tests a single coordinate of the condition that a perturbed input's code matches the code under $\mathrm{do}[(\ell, i){=}j']$, so any realization rate below 1 rules out an exact transformation mapping that perturbation to the target intervention. Its collateral metric counts the remaining heads whose assignments change relative to the *unsteered* code, which in a residual stack includes the downstream reclassification that $\mathrm{do}[(\ell, i){=}j']$ itself induces. Similarly, the cycle-consistency gain in `steerfid.py` evaluates a text-level implementation across the closed loop $\mathrm{assign} \circ \mathrm{encode} \circ \mathrm{decode}$. Because it patches nothing from a separate source input, it measures a normalized activation gain net of a random control rather than interchange-intervention accuracy [106], and the loop's ∼13% unsteered round-trip survival rate (Section 5.4) caps how often an exact intervention could be detected through text. Cross-seed matching tests whether the assign maps of two independent training runs agree up to entry permutation. In principle, the softmax parameterization leaves per-head entry ordering and any logit shift or scale absorbed by the temperature unidentified; consequently, for heads to reflect intrinsic structure of the data rather than artifacts of a single run, both the sample partitions and the decoded directions must reproduce up to entry permutation. Section 6.5 tested both criteria on hard codes across SONAR and GPT-2. Beyond layer 0, no individual head reproduces, nor do our layer-level diagnostics show cross-seed agreement beyond each model's shared alignment with the data's principal directions. Testing whether a full layer reproduces as a variable merge of its heads [104, Def. 37], in which individual heads form seed-dependent factorizations, requires a whole-layer comparison that translates one seed's layer code into another's. Layer 0 reproduces only partially, and on SONAR almost entirely through a single large-atom head.

Why the cascade costs reproducibility. Under this causal framing, the residual stack's lower cross-seed reproducibility at matched head size (Section E.11) and its higher input-space intervention collateral (Section 6.4) arise from the same mechanism. Every head computes a function of its own layer input, and every head past layer 0 takes as input the residual left by upstream heads; consequently, any change in the upstream decomposition alters the input distribution that downstream heads partition. Even layer 0, which operates directly on the raw input, is only partially constrained: because its heads are trained jointly with downstream layers, which partition of the input minimizes total loss depends on how subsequent layers model the remaining residual. A single-layer architecture avoids both dependencies: at matched head size, its heads reproduce more consistently across seeds and realize input-space interventions

far more reliably, with half the collateral. Nevertheless, single-layer hard heads are still not independently addressable, because hard-code classifiers lie close to their decision boundaries in both stacked and flat architectures; residual depth therefore compounds an entanglement already present within each layer.

Metric hygiene. Geometric claims about decoded directions should not depend on an arbitrary choice of inner product, a point emphasized by Park et al. in their analysis of language-model representations under an estimated causal inner product [20]. Our within-head repulsion finding is invariant to this choice: under the inverse-variance inner product of the whitened training objective (which whitens the data rather than estimating a causal inner product), within-head repulsion in the softmax Ontologizer strengthens ($\zeta_{\cos}$ of -24.4 vs. -17.6 under the Euclidean inner product), while the coherence profiles of both comparison models remain unchanged. However, this measurement does depend on the parameterization: because a soft head's assignments sum to one, its decoded directions are determined by the forward function only up to a shared per-head offset, meaning that uncentered cosines characterize the trained atom parameters rather than the input–output function alone (Section 5.3). Two additional methodological rules emerged from our analysis. First, geometry must be evaluated in output space rather than in dictionary space, whose decoder null-space component is unconstrained by reconstruction (Section G.1) and represents the same kind of causally disconnected subspace that can render subspace patching misleading [3]. Second, a layer's scalar residual gain must be separated from its atom norms before attributing effects to head magnitude: across layers, differences in output contribution magnitude reflect the layer-wide residual gain rather than larger individual atoms (Section E.11).

Composition. Interpreting heads as causal variables provides little benefit if interventions on two heads interact unpredictably. Using `compose.py`, we measure the additivity of decoded intervention deltas on `resid_nc` under the whitened metric, reporting medians across head pairs:

stratum	pairs	cos	rel. err	later layers
same-layer L0	28	0.959	0.350	4
same-layer L1	28	0.947	0.354	3
same-layer L2	28	0.967	0.259	2
same-layer L3	28	0.996	0.092	1
same-layer L4	28	1.000	0.019	0 (numerical floor)
cross-layer (earlier = L0/L1)	42	0.97–0.98	0.20–0.23	
cross-layer (earlier = L2/L3)	18	1.00	0.02–0.04	

Three patterns emerge from these measurements. (i) Joint two-head interventions align closely with the sum of the corresponding single-head interventions across all strata (median cosine ≥ 0.95), confirming that `withArgs` set-interventions compose approximately additively. (ii) Deviations from exact additivity arise entirely from downstream reclassification: relative error decreases monotonically with the number of remaining downstream layers and reaches

the numerical floor in the final layer (where the measured 0.019 reflects float32 rounding amplified by decoder cancellation, dropping to $\sim 3\times10^{-6}$ in float64). (iii) Cross-layer pairs compose *more* additively than same-layer pairs at matched depth because a set-intervention overrides upstream perturbations on the later head, making cross-layer interaction strictly one-way. This behavior contrasts sharply with input-space perturbations (Section 6.4): `withArgs` sets a head's assignment directly and composes cleanly, whereas an *input-space* step perturbs every head simultaneously and flips most of them. Downstream reclassification, which introduces only a 5–35% deviation under direct code-level interventions, becomes the dominant effect of an input-space step.

10 Preliminary conclusions

1. **The trained atoms form contrast sets.** In the softmax Ontologizer, within-head decoded directions repel relative to random subsets in 96% of heads, and in the hard-code stack, within-head decoded atoms are roughly orthogonal to slightly negatively correlated, so each head's entries represent well-separated alternatives. Because the forward function determines within-head atoms only up to a shared per-head offset, this repulsion characterizes the trained parameters; isolating the functional contribution requires comparing atoms centered within each head against a similarly centered null. Neither reconstruction metrics nor per-latent probes capture this contrast-set structure.

2. **Between-head coordinate partitioning does not emerge in summed models.** No trained summed model partitions its dictionary coordinates across heads. Between-head support overlap sits at or above a coordinate-shuffle null across all summed runs, each head's usage spans 52–98% of dictionary coordinates, and this overlap persists in the subspace projected through the decoder as well as in its null space. Only very long training (8.27M steps) reduced overlap, doing so by sparsifying individual head profiles rather than separating heads onto disjoint coordinates. Consequently, static weight-level analysis cannot assume disjoint heads in summed models; concatenated heads remain the only mechanism that guarantees disjoint support, doing so by construction.

3. **A discrete ontology is viable, though not by discretizing a softmax model.** Taking the one-hot argmax of a trained softmax Ontologizer fails to reconstruct the input, yielding at least 77× higher error than the 800-bit Gaussian reference on `resid_nc` and nine orders of magnitude higher error on `softmax_shm`. By contrast, training a straight-through arm with 2.4× as many heads makes the discrete code the forward pass itself: with its four per-layer gains pinned to their held-out means, it forms a purely discrete 1900-bit code at $\mathrm{FVU}_w = 0.158$. At a matched 1900-bit rate, neither the converged k_{SAE}=160 `TopK` SAE nor the single-layer hard variant can improve on $\mathrm{FVU}_w = 0.236$–0.273 under any coefficient quantization. However, a coarsely quantized, entropy-coded L1 SAE reaches $\mathrm{FVU}_w = 0.129$ at an average rate of 1900 bits per row and 0.152 (comparable to the hard code) when capped at 1900 bits on every row; averaged across rows, the hard code outperforms all SAEs only under fixed-length support indexing (0.158 vs. 0.180), because

its entries are used nearly uniformly whereas an L1 SAE's latents have skewed firing rates. On the sentence-split cache, a wider `TopK` SAE (k_{SAE}=256) outperforms the hard stacks under both average and per-row bit budgets (0.128 and 0.129 vs. 0.161 and 0.164; Appendix E.3). The hard code sits 3.1× above the Gaussian reference for its rate, and so at least that far from the data's distortion–rate function, a limit over long blocks of samples that a code describing each sample alone need not be able to reach [82]. Training this discrete stack required matching the classifier temperature to the logit scale, applying relative logit noise, rescaling the weight on the batch-mean uniformity term KL_m, and rescaling dictionary initialization (which alone reduced held-out error by one third).

4. **The discrete code preserves downstream model performance.** On GPT-2 small, an 800-bit hard code with signed direct atoms in activation space and per-layer gain–shape normalization recovers 97.6% of next-token cross-entropy loss, surpassing a k_{SAE}=32 SAE (96.2%) at nearly the same unwhitened FVU, while increasing head width to 64 entries surpasses that SAE on both metrics. Across model families, FVU does not monotonically order downstream loss recovery. Training continuous low-rank fibers on top of a frozen discrete base recovers nearly all remaining loss (99.9%) while leaving the discrete base assignments unchanged, showing that a discrete ontology and near-lossless reconstruction can coexist at the cost of hundreds of additional continuous coordinates per token.

5. **For soft codes, reconstruction measures structural cost rather than steering utility.** Unstructured codes achieve roughly 50× lower reconstruction error at matched coefficient budgets, with the per-head simplex constraint alone accounting for roughly a 30× penalty. Downstream text fidelity on SONAR is a monotone function of FVU_w and adds no independent evaluation axis.

6. **Residual depth improves reconstruction at the cost of addressability and reproducibility.** Compared to a single-layer model at equal bit budget, parameter count, and near-uniform entry usage, the five-layer stack achieves 1.79× lower reconstruction error, driven by conditioning each stage on the sample's remaining residual error (rather than by utilization, input conditioning, or the scalar gain side-channel, though deep supervision on prefixes has not yet been ablated separately). Conversely, an input-space perturbation flips 66% of unrelated heads in the five-layer stack vs. 33% in the single-layer model, and realizes its target entry on only 37% of rows vs. 99.6%; in both models, off-target collateral matches that of a norm-matched random step because hard-code classifiers sit close to their decision boundaries. Moreover, at matched head contribution size, stacked heads show several-fold lower cross-seed reproducibility than single-layer heads across every layer, including layer 0. Although the single-layer softmax variant `g160softmax` matches the reference checkpoint `resid_nc` in reconstruction, a configuration-matched softmax Ontologizer achieves 6× lower error both on out-of-sample rows that no model saw during training and on the evaluation tail, indicating that `resid_nc`'s lack of a depth advantage was checkpoint-specific.

7. **Steering succeeds in embedding space, whereas text round-tripping obscures it.** Stepping along a decoded atom at strength 0.25 realizes the target entry on 82% of rows in

the softmax model (vs. a 3% chance rate), whereas decoding and re-encoding through text preserves unsteered entries on at most ~13% of inputs, bounding how often exact interventions remain observable after round-tripping. At low collateral, the soft code is more steerable than the hard code. Steerability does not track reconstruction error, and the classifier's input gradient provides an effective steering direction only in layer 0, where the raw input enters the classifier directly.

8. **Individual heads beyond layer 0 do not reproduce across seeds, and layer 0 reproduces only partially.** Independent seeds converge to identical held-out reconstruction error to four decimal places while learning substantially different reconstructions (whitened residual correlation of 0.30, compared to 0.83 between SAE seeds). Past layer 0, independent runs show no above-null head-level agreement across partitions, output contributions, subspaces beyond the data's principal directions, or dictionary Gram matrices on either SONAR or GPT-2 (including under the direct-atom design of Section 7), even though individual heads remain stable across late checkpoints within a single run. Layer 0 reproduces at less than one fifth of the within-run control. Its higher cross-seed agreement relative to deeper layers does not stem from larger atom norms: its atoms are no larger than those of later layers, and its larger output contributions reflect the layer-wide residual gain. Within layer 0, however, heads with unusually large atom norms account for the reproducible signal—on SONAR, almost entirely a single head. This lack of identifiability is the central limitation for our steering framework: a head that does not recur across runs cannot be assigned a stable meaning, qualifying every per-head claim in this study. The sole exception to date is a block of heads explicitly trained for temporal consistency: on pairs of sentences from one document, a Fano mutual-information objective causes a block of 38 slow heads to reproduce across three seeds (Hungarian-matched NMI of 0.412–0.414 vs. a 0.005 null) while varying more slowly across sentences than any linear projection of the embedding. Even there, reproducibility holds at the block level rather than for individual heads: two slow heads within the same seed agree at NMI 0.40, so individual slow heads are not yet uniquely identified across runs. An ordered penalty on dependence between the slow heads identifies one of them at full weight, but only by stripping the other 37 of slow content and the block of about half of what it carried (Section 6.5.3).

9. **Entries contribute nearly equally except when classifying the raw input.** In layers 1–3, entries within each head carry nearly equal shares of output contribution (Gini 0.06–0.15, with zero dead entries), whereas layer 0's entries carry unequal shares (Gini 0.33–0.59) because its atom norms vary widely within heads, despite near-uniform entry usage. Because the single-layer hard model exhibits the same dispersion, this inequality arises from classifying the unconditioned raw input rather than from residual stacking.

10. **Learned heads prioritize semantic content over source language.** Consistent with SONAR's design as a language-agnostic semantic embedding, the strongest head (by η^2) partitions inputs by broad topic and genre, and is the single head that recurs across random seeds. Source language, the corpus's primary free ground-truth label, served as a diagnostic probe rather than an expected single-head variable: language identity

is distributed across multi-head neighborhoods rather than isolated in a single head, reflecting the weak separation of languages in SONAR space. On that probe, the hard code captures 3.5–3.9× as much language information as the softmax reference, confirming that learned partitions align with known attributes above chance; directly testing whether heads recover natural semantic variables will require topic and genre annotations.

11. **Per-latent metrics mischaracterize classifier codes.** Standard sparse probing ranks the Ontologizer lowest even where partition-level analysis identifies verified causal language structure. Across models, per-feature autointerp detection F1 is roughly 80% explained by activation density, and even more strongly by how many languages a feature's top activating examples span, which covaries with density. When restricted to features spanning as many languages as random draws, the sparse SAE's F1 lead over the Ontologizer narrows to 0.10 or less. Furthermore, parameter-only decodes generate single example sentences rather than category descriptions and become legible only after subtracting the head's mean atom, consistent with simplex models of categorical concepts in which value vectors share a parent offset and separate only through their mutual differences [20].

12. **Non-negativity substantially degrades dictionary geometry and reconstruction.** Constraining dictionary weights to the non-negative orthant induces high atom collinearity, reduces effective rank to one third of a signed dictionary's, and increases held-out FVU_w by 26% at the wide dictionary and by 90% with a square decoder; mean within-head atom cosine exactly orders all five GPT-2 orthant arms by held-out error at convergence. Signed atoms eliminate this collinearity and achieve lower error at both dictionary widths. Moreover, dictionary collinearity statistics computed prior to decoding largely reflect unconstrained decoder null-space components.

13. **Every diagnostic in this architecture requires a shape-matched null.** Non-negativity constraints and layer-dependent residual gains induce nontrivial baseline values across geometric and information-theoretic metrics. Three of our four initial cross-seed reproducibility metrics, the apparent HSIC between-head dependence, raw dictionary-space collinearity measurements, and the apparent effect of head magnitude on cross-seed reproducibility all reversed or vanished once compared against shape-matched nulls.

11 Follow-up investigations

We order the primary open investigations by their importance to unsupervised steering and auditing.

- **Head identifiability first.** Before individual heads can be interpreted or used in audits, we must either make them reproducible across training runs or identify the subset of heads that reproduce. Cross-seed partition comparisons on the softmax Ontologizer at 370k steps show virtually no head-level agreement (Appendix D.4). A controlled k=128 experiment sharpens the source of this sensitivity: two distinct architectures trained from the same random seed (one with a learned decoder and one with direct atoms) learn

matching partitions head for head (matched NMI 0.18, with every head pairing with the co-indexed head), whereas each architecture compared across two seeds reaches only 0.064 vs. a 0.059 null. Because seed replicas iterate through the same training examples in the exact same order, the random seed acts solely through parameter initialization and training noise; those two factors, rather than the dataset or downstream parameterization past the classifier, largely determine how each head partitions the space, echoing findings in toy networks where identical training data yield equivalent algorithms implemented via distinct features [90]. Classical identifiability theory points to two remedies. The first is unsupervised model and feature selection via cross-seed agreement, as in Unsupervised Disentanglement Ranking, which scores models by median cross-seed alignment without ground-truth labels [93] (in latent-variable models, three or more conditionally independent views of a discrete variable identify its distribution up to permutation when the class-conditional distributions satisfy a rank condition [92], though here individual heads serve as candidate learned views rather than fixed ones, and standard sparse-coding uniqueness guarantees require lossless reconstruction [91]). The second is breaking static marginal symmetry using paired observations, where temporal contrast [94], sparse transition priors [95], shared factors across corruptions [96], auxiliary conditioning [97], or contrastive conditional alignment [98] guarantee identifiability up to permutation under their respective generative assumptions. Initial experiments on the sentence-split SONAR cache `mc4_sent16` (Appendix E.10) evaluate this second approach. Linear ridge regression from $\mathbf{x}_t$ to the next sentence $\mathbf{x}_{t+1}$ reaches $\mathrm{FVU}_{\mathrm{next}}$=0.896 (roughly 10% of next-sentence variance explained linearly, which does not upper-bound nonlinear predictors), and adjacent sentences exhibit strong directional alignment (raw cosine AUC 0.923 vs. 0.685 in whitened distance, where norm variance dominates), yet heads trained on single sentences switch entries on 92.2–98.1% of adjacent pairs and retain their entry barely above chance (Cohen's kappa at most 0.081 across all heads). Directly penalizing cross-sentence entry switches on 38 designated slow heads collapses their usage entropy to 0.04–0.09 bits for penalty weights $s_{\mathrm{pair}} \geq 10^{-4}$. Replacing that penalty with a Fano lower bound on each slow head's cross-sentence mutual information prevents collapse and yields a 38-head slow block that reproduces across three seeds, though not yet head by head (Section 6.5.3). Decorrelating the slow block, whose heads agree with one another nearly as strongly as they agree across seeds, was the first remedy tried, and independence between heads' partitions did not separate them without loss: a weak ordered penalty grades the block without identifying a head, and a strong one identifies a single head and leaves the others nothing (Appendix E.10). Immediate next steps for paired-view training include separating the slow heads by a criterion that keeps what the block carries jointly, extending the objective to triples of spaced sentences within a document to supply the three conditionally independent views required by classical rank conditions [92], characterizing what the slow heads encode beyond source language, and applying the paired-view objective to a causal language model over multi-turn conversation states. Parallel single-sentence reproducibility checks remain open as well: training full-length (4.4M-step) cross-seed replicas of `resid_nc_hm` and of `g160softmax`; testing whether regularization can extend

the reproducibility observed in large-atom heads; testing whether the binary (k=2) arm's one-bit heads reproduce more reliably than 32-way heads; isolating why the layer-0 topic head recurs when other heads do not (at k=128, a dominant layer-0 head recurs at one third to one half of the k=32 topic head's agreement); testing whether per-head router gains drive agreement, since on GPT-2 adding a router doubles contribution agreement while deactivating the final two layers; and training a single-layer GPT-2 arm to replicate the size-matched comparison of Section E.11 on a second substrate.

- **Consolidating GPT-2 hard-code features on the main branch.** Direct atoms have been ported to the main branch (Appendix F.1), whereas `resid_first` and `gain_clip` have not, so the GPT-2 runs of Section 7 currently reproduce only on the `headline` branch. On SONAR, a 5×54 stack with k=128 entries per head (1,890 bits, matching the 1,900-bit k=32 stack) converges to roughly twice the error ($\mathrm{FVU}_w = 0.31$–0.32 vs. 0.1535, two seeds per arm), with direct atoms outperforming decoded atoms by 0.009. Because both k=128 arms achieve full entry utilization, this gap does not reflect dead entries; however, the temperature, learning rate, and weight on the batch-mean uniformity term KL_m were carried over directly from k=32 and must be retuned before attributing the degradation to k. Direct atoms also make set-interventions act directly in activation space by adding $g_\ell(\mathbf{a}_{ij'} - \mathbf{a}_{ij})$ at layer ℓ, an intervention whose steering behavior remains to be measured.

- **Per-layer decoders vs. width-matched shared decoders.** Early one-epoch direct-atom pilots on GPT-2 showed less than a 0.007 improvement in unwhitened FVU from separate per-layer decoders, initially suggesting little benefit. On a twin of the 40M-token concatenated-head model (Appendix F.1), however, giving each layer its own decoder reduces held-out FVU from 0.186 to 0.169 (97.3% $\rightarrow$ 97.7% loss recovered) and reduces cross-layer coupling between co-indexed heads from 97% to 6% (near the 3% chance floor), demonstrating that the strong coupling arose from projecting head i's slice through the same shared decoder block across all five layers. Because using l=5 separate decoders multiplies decoder parameter count fivefold, a shared-decoder control matched in total decoder width (d_{head}=240) is still required.

- **Two-stage fiber training on SONAR.** Training low-rank fibers on top of a frozen discrete base preserved the base code's assignments and error on GPT-2 while matching the overall reconstruction of unconstrained joint fiber training. Although fibers are implemented on the main branch, no SONAR run has yet used them; this two-stage recipe should be evaluated on SONAR against SAEs at matched continuous capacity (where rank-1 radial fibers require 160 continuous values per token).

- **Re-evaluating ablations under the rescaled initialization.** Our input-space entanglement diagnostics and the gain and blend ablations were run under the original dictionary initialization, which started 1.84×10^8 off at step 0 and left 10% of code capacity unused; these ablations should be repeated under the rescaled initialization.

- **Scaling the steering evaluation.** We have evaluated supervised steering vectors against model-derived directions on the models' own partitions (Section 5.4) and, for `resid_nc_hm`,

on external language steering (Appendix D.5). Several steering comparisons remain open: running the external language-steering benchmark on the hard-code stack and the single-layer hard arm; testing bilinear eigenfeatures as steering directions; measuring across-context consistency of entry-swap effects, which for SAE features is partially predictable prior to intervening [14]; and weighing the single-layer hard arm's near-perfect target realization, halved collateral, and higher cross-seed reproducibility against its 1.79× higher reconstruction error. Reducing input-space collateral appears to need larger classifier margins more than better steering directions. Even with multi-layer residual reclassification excluded, input-space collateral is largely direction-blind: linearized within a single layer, stepping along any Ontologizer's decoded atom flips other heads at the same rate as a random step of equal norm, and in decoded text, ablating a head's contribution perturbs outputs no more than a norm-matched random step. Because hard-code classifiers lie close to their decision boundaries (a within-layer linearized random step of $0.25\|\mathbf{x}\|$ flips 30% of same-layer heads), distance from the boundary looks like the lever. Consistent with this hypothesis, `ste_k2_h380` (five layers of 380 binary heads) exhibits substantially lower collateral: the same $0.25\|\mathbf{x}\|$ step is predicted to flip 13% of other heads to first order and empirically flips 17–19% across the full stack for all steering directions (random included), vs. 66% in the 32-entry stack. Although its raw logit margins are 34× larger than the 32-entry stack's, raw margin scale alone is not the cause, as raw margins doubled between steps 370k and 739k while the first-order flip rate remained unchanged because overall logit norms scaled proportionally. Neither training duration nor the KL_m weight explains the reduction: at step 370k (matching the 32-entry stack's full training horizon), both the first-order flip prediction and empirical collateral were already at 13% and 17–19%, and the batch-mean uniformity weight matches the 32-entry stack's effective per-head weight (since KL_m averages across heads, a weight of 5×10^{-4} over 380 heads applies the same per-head penalty as 10^{-4} over 76 heads). However, because `ste_k2_h380` was trained on a single seed at half the batch size (and thus under noisier per-step gradients), a batch-matched multi-seed comparison is still needed to isolate the effect of k.

- **Task reconstruction.** A central goal for reproducible heads is identifying the classification task each head performs. Head-level autointerp, which summarizes a head from side-by-side examples across all of its entries and scores blinded assignment accuracy against sibling entries, found no above-chance signal across 20 unscreened heads per condition (Appendix D.6). By contrast, when scored by graph modularity on a nearest-neighbor graph of each layer's input residual, partitions in layers 1–4 of the softmax Ontologizer are two to three times as modular as size-matched random regroupings of their entries, whereas neither post hoc discovered groups nor the trained heads of a Matryoshka SAE [13] exceed null modularity on matched prefix-residual graphs (evaluated on one checkpoint per model; Appendix C.3). Remaining task-reconstruction experiments include measuring mutual information between head partitions and metadata label libraries (prioritizing topic and genre from labeled corpora or unsupervised topic clusters to match SONAR's primary semantic axes, alongside length, formatting or noise, and script as secondary attributes), re-running head-level autointerp on heads that pass cross-seed reproducibil-

ity screening with sufficient sample size to resolve above-null accuracy, and evaluating invariance across parallel translations in FLORES to separate semantic-content heads from surface-form heads. Each protocol should be run identically on stacked Ontologizer heads and on the single-layer `g160softmax` heads.

- **Hierarchy as MoE routing.** Measuring conditional mutual information between successive layers' partitions would test whether residual depth constructs compositional tasks, while evaluating `forward="labels"` under the rescaled initialization would test our residual-conditioning explanation from the opposite direction. Pairwise effective information [107] upper-bounds direct edge weights in the causal graph of Section 9: in the hard-code stack, forcing an upstream head to a live entry shifts any single downstream head's distribution by roughly 0.01 out of a possible 5 bits across all depths, though this causal effect is 11–16× larger than the pair's observational mutual information. Whether upstream interventions exert diffuse weak effects or strong joint effects across multiple heads, which pairwise statistics cannot detect, remains open.

- **Mechanism of the collinearity setpoint.** The collinearity setpoint controller, which improved soft-model steerability, currently acts on a dictionary-space cosine largely dominated by decoder null-space freedom; this controller should be tested on decoded atom cosines and on signed dictionaries.

- **Penalizing head overlap in output space.** When trained as a dictionary-space penalty, support overlap ω reaches its target solely by shifting head supports into the decoder's null space, or into near-null singular directions when the decoder is square (Appendix G.2). To take effect on reconstructions, an overlap penalty must act on decoded atoms or on direct atoms that have no decoder. Our k=128 direct-atom runs used s_ω=10^{-6}, which proved too weak to reduce ω (on GPT-2, ω responds only from $s_\omega \geq 10^{-5}$). Because raw activation coordinates are not privileged axes (product quantizers perform substantially better after a learned orthogonal rotation than on raw coordinates [28]), forcing heads onto disjoint output coordinates is a restrictive and basis-dependent constraint, though one that cannot be satisfied via null-space directions. Concatenated heads remain the only reliable route to disjoint support.

- **Logit-scale control for straight-through training.** Without explicit per-head logit normalization (`logit_norm`), classifier logit norms rescale during training to offset temperature annealing after an initial transient (Appendix F.1). Future straight-through runs should normalize logits per head, regulate temperature t via a logit-spread/t setpoint controller rather than a hand-tuned schedule, or add a logit-scale regularizer analogous to the commitment loss that anchors encoder outputs to codebook vectors in VQ-VAE [84].

- **Rate-aware training.** Our operational rate–distortion comparisons evaluate models that were never trained for entropy coding: the hard stack uses entries nearly uniformly (maximizing per-head index entropy), and none of the SAEs were trained with quantized coefficients. Training both families directly against entropy-coded rate (for example, penalizing entry-usage entropy in hard codes and sweeping quantization-aware SAEs

across the 1,900-bit regime) would compare the architectures themselves rather than fixed-rate against unquantized models.

- **Preventing dead entries in single-layer hard models.** The 29% dead-latent rate in `g160top1` stems from within-head winner-take-all starvation, which persisted despite enabling Gao et al.'s dead-latent auxiliary loss [9] (`aux_k` of 512 throughout training), because auxiliary reconstruction gradients do not force a starved latent to win its head's argmax. Alongside winner dropout, two established remedies warrant testing: annealing per-head selection from top-k_{sel} down to top-1 over the first half of training, analogous to k-sparse autoencoders reducing their active budget from 100 to the target sparsity over the first half of training to prevent unit starvation at low k_{SAE} [43], and re-initializing underutilized entries from current-batch residuals as in neural audio codecs [32] (already supported by neuron resampling in `sae.py`).

- **Training for progressive truncation.** Adding a deviation-sparsity penalty during soft-model training would test whether the soft Ontologizer's Pareto frontier can shift toward lower capacity, analogous to multi-sparsity training for progressive `TopK` SAE codes [9].

- **Completing the structural ablation ladder and baselines.** Combining head grouping with nested prefixes (`groups+prefixes`, rungs 2a+3) has now been trained for the modularity control above, as `m5120_g160top1_p5` ($\text{FVU}_w = 0.294$), though it has not yet been added to the full evaluation ladder. Remaining baseline runs include the single-layer variants at m=11264, a mid-range `TopK` sweep ($k_{\text{SAE}} \in \{512, 2048\}$), and random-support and non-negative least-squares refit controls, the latter matching the positivity-constrained refit of Gao et al. [9].

- **Closed lines of inquiry.** Several candidate modifications require no further experiments: enforcing between-head independence via an HSIC penalty (unregularized heads already partition nearly independently, and the biased HSIC estimator induces genuine head freezing because only a constant head escapes its positive bias floor; Appendix D.1), decoder sparsification and decoder bias terms (which produce no interpretable structure and alter error by $< 1\%$), forwarding classification codes alongside the residual, explicit input whitening, and rescaling batch-mean uniformity weights across architectural shapes (the latter two being subsumed by rescaled initialization). Finally, the layer-4 head freezing in `resid_nc` is explained by residual saturation and is unchanged by the evaluation-probe correction, leaving only two anomalously coherent heads (L2h25 and L3h28) for individual inspection.

Reproducibility

All of the code is available at `https://github.com/WakaruLabs/ontologize`. The core evaluation protocols are implemented as root-level scripts (`sae.py`, `pareto.py`, `headstruct.py`, `headcoh.py`, `splitting.py`, `textfid.py`, `langprobe.py`, `refit.py`, `steerfid.py`, `compose.py`, `autointerp.py`), orchestrated by the `sae_ladder.sh`, `sae_evals.sh`, and `autointerp_run.sh`

runners as idempotent, crash-resumable systemd user units with rate-metered judge calls. The straight-through arm and its diagnostics reside in `experiments/ste-arm/` (`train_ste.py`, `steerembed.py`, `blendablate.py`, `gainablate.py`, `inputgeom.py`, `dictgeom.py`, `lastlayer.py`, `partition.py`, `headcontrib.py`, `entryshare.py`, `bigatoms.py`, `headlang.py`, `headscript.py`, `headeta.py`, `embedgeom.py`, `decodehead.py`, `coordfvu.py`, `make_zca.py`), with every run's command and superseded analyses recorded in `experiments/ste-arm/notes.md`. The sentence-split cache and paired-sentence experiments reside in `experiments/sentence-split/` and `experiments/next-sentence/` (`eval_next.py`, `transitions.py`, `train_pair.py`). The GPT-2 hard-code experiments of Section 7 reside in `experiments/gpt2/` (`train_onto.py`, `diagnose.py`, `seedstab.py`, with notes, configurations, diagnostics, and loss curves in `DIRECT_GAIN_STE.md` and `results/README.md`) and use the `headline` branch. A curated test suite verifies each protocol's statistics on synthetic inputs with planted structure. Evaluation artifacts are stored under `data/out/sonar/` and `data/out/gpt2_l8/`. At most three straight-through training runs fit concurrently on a single 24 GB GPU.

Acknowledgements

Claude Opus 5.5, Claude Fable 5.2, and Gemini 4 Argon were used to perform literature searches, identify followup experiments, adapt experimental pipelines, record results, and edit prose for clarity. Claude Opus 5.5 was used to assist formalizing disjoint support. The authors directed the work, reviewed the results, and are responsible for the content. Language models used for the evaluations themselves are described in Appendix B.5.

References

[1] Neel Nanda et al. *A Pragmatic Vision for Interpretability*. 2025. URL: https://www.alignmentforum.org/posts/StENzDcD3kpfGJssR/a-pragmatic-vision-for-interpretability.

[2] Zhengxuan Wu et al. "AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 67035–67080. arXiv: 2501.17148 [cs.CL]. URL: https://arxiv.org/abs/2501.17148.

[3] Aleksandar Makelov, Georg Lange, and Neel Nanda. *Is This the Subspace You Are Looking for? An Interpretability Illusion for Subspace Activation Patching*. 2023. arXiv: 2311.17030 [cs.LG]. URL: https://arxiv.org/abs/2311.17030.

[4] Andy Arditi et al. "Refusal in Language Models Is Mediated by a Single Direction". In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024. arXiv: 2406.11717 [cs.LG]. URL: https://arxiv.org/abs/2406.11717.

[5] Xin Tong et al. *MEUV: Achieving Fine-Grained Capability Activation in Large Language Models via Mutually Exclusive Unlock Vectors*. 2025. arXiv: 2509.12221 [cs.LG]. URL: https://arxiv.org/abs/2509.12221.

[6] Andrew Mack and Alexander Matt Turner. *Mechanistically Eliciting Latent Behaviors in Language Models*. AI Alignment Forum. 2024. URL: https://www.alignmentforum.org/posts/ioPnHKFyy4Cw2Gr2x/mechanistically-eliciting-latent-behaviors-in-language-1.

[7] Hoagy Cunningham et al. *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. 2023. arXiv: 2309.08600 [cs.LG]. URL: https://arxiv.org/abs/2309.08600.

[8] Trenton Bricken et al. "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning". In: *Transformer Circuits Thread* (2023). URL: https://transformer-circuits.pub/2023/monosemantic-features/index.html.

[9] Leo Gao et al. *Scaling and evaluating sparse autoencoders*. 2024. arXiv: 2406.04093 [cs.LG]. URL: https://arxiv.org/abs/2406.04093.

[10] James P. Crutchfield. "Way More than the Sum of Their Parts: From Statistical to Structural Mixtures". In: *Entropy* 28.1 (2026), p. 111. DOI: 10.3390/e28010111.

[11] Francesco Locatello et al. "Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 4114–4124. arXiv: 1811.12359 [cs.LG]. URL: https://proceedings.mlr.press/v97/locatello19a.html.

[12] David Chanin et al. "A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders". In: *Advances in Neural Information Processing Systems*. Vol. 38. 2025, pp. 91489–91526. arXiv: 2409.14507 [cs.CL]. URL: https://arxiv.org/abs/2409.14507.

[13] Bart Bussmann et al. "Learning Multi-Level Features with Matryoshka Sparse Autoencoders". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 6077–6101. arXiv: 2503.17547 [cs.LG]. URL: https://arxiv.org/abs/2503.17547.

[14] Evan Duan. *Pre-Intervention Prediction of Sparse Autoencoder Steering Side Effects*. 2026. arXiv: 2606.08365 [cs.LG]. URL: https://arxiv.org/abs/2606.08365.

[15] Michael T. Pearce et al. "Bilinear MLPs enable weight-based mechanistic interpretability". In: *International Conference on Learning Representations*. 2025. arXiv: 2410.08417 [cs.LG]. URL: https://arxiv.org/abs/2410.08417.

[16] Anish Mudide et al. *Efficient Dictionary Learning with Switch Sparse Autoencoders*. 2025. arXiv: 2410.08201 [cs.LG]. URL: https://arxiv.org/abs/2410.08201.

[17] Steven Bills et al. *Language Models Can Explain Neurons in Language Models*. OpenAI. 2023. URL: https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html.

[18] Gonçalo Paulo et al. "Automatically Interpreting Millions of Features in Large Language Models". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025. arXiv: 2410.13928 [cs.LG]. URL: https://arxiv.org/abs/2410.13928.

[19] Zhengyan Zhang et al. “MoEfication: Transformer Feed-forward Layers are Mixtures of Experts”. In: *Findings of the Association for Computational Linguistics: ACL 2022*. Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 877–890. DOI: 10.18653/v1/2022.findings-acl.71. URL: https://aclanthology.org/2022.findings-acl.71/.

[20] Kiho Park et al. “The Geometry of Categorical and Hierarchical Concepts in Large Language Models”. In: *International Conference on Learning Representations*. 2025. arXiv: 2406.01506 [cs.CL]. URL: https://arxiv.org/abs/2406.01506.

[21] Zhen Xu et al. *Beyond Redundancy: Diverse and Specialized Multi-Expert Sparse Autoencoder*. 2025. arXiv: 2511.05745 [cs.LG]. URL: https://arxiv.org/abs/2511.05745.

[22] Sebastian Bauer et al. “Ontologizer 2.0 — a multifunctional tool for GO term enrichment analysis and data exploration”. In: *Bioinformatics* 24.14 (2008), pp. 1650–1651. DOI: 10.1093/bioinformatics/btn250.

[23] James C. R. Whittington et al. “Disentanglement with Biological Constraints: A Theory of Functional Cell Types”. In: *International Conference on Learning Representations*. 2023. arXiv: 2210.01768 [q-bio.NC]. URL: https://arxiv.org/abs/2210.01768.

[24] Will Dorrell et al. “Range, not Independence, Drives Modularity in Biologically Inspired Representations”. In: *International Conference on Learning Representations*. 2025. arXiv: 2410.06232 [q-bio.NC]. URL: https://arxiv.org/abs/2410.06232.

[25] Artem Babenko and Victor Lempitsky. “Additive Quantization for Extreme Vector Compression”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2014, pp. 931–938. URL: https://openaccess.thecvf.com/content_cvpr_2014/html/Babenko_Additive_Quantization_for_2014_CVPR_paper.html.

[26] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. “Product Quantization for Nearest Neighbor Search”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128.

[27] Ting Zhang, Chao Du, and Jingdong Wang. “Composite Quantization for Approximate Nearest Neighbor Search”. In: *Proceedings of the 31st International Conference on Machine Learning*. Vol. 32. Proceedings of Machine Learning Research 2. PMLR, 2014, pp. 838–846. URL: https://proceedings.mlr.press/v32/zhangd14.html.

[28] Tiezheng Ge et al. “Optimized Product Quantization”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36.4 (2014), pp. 744–755. DOI: 10.1109/TPAMI.2013.240.

[29] Michal Aharon, Michael Elad, and Alfred Bruckstein. “K-SVD: An Algorithm for Designing Overcomplete Dictionaries for Sparse Representation”. In: *IEEE Transactions on Signal Processing* 54.11 (2006), pp. 4311–4322. DOI: 10.1109/TSP.2006.881199.

[30] Biing-Hwang Juang and A. H. Gray Jr. “Multiple Stage Vector Quantization for Speech Coding”. In: *ICASSP ’82. IEEE International Conference on Acoustics, Speech, and Signal Processing*. Vol. 7. 1982, pp. 597–600. DOI: 10.1109/ICASSP.1982.1171604.

[31] Christopher F. Barnes, Syed A. Rizvi, and Nasser M. Nasrabadi. “Advances in Residual Vector Quantization: A Review”. In: *IEEE Transactions on Image Processing* 5.2 (1996), pp. 226–262. DOI: 10.1109/83.480761.

[32] Neil Zeghidour et al. *SoundStream: An End-to-End Neural Audio Codec*. 2021. arXiv: 2107.03312 [cs.SD]. URL: https://arxiv.org/abs/2107.03312.

[33] LCM team et al. *Large Concept Models: Language Modeling in a Sentence Representation Space*. 2024. arXiv: 2412.08821 [cs.CL]. URL: https://arxiv.org/abs/2412.08821.

[34] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. 2013. arXiv: 1308.3432 [cs.LG]. URL: https://arxiv.org/abs/1308.3432.

[35] Eric Jang, Shixiang Gu, and Ben Poole. “Categorical Reparameterization with Gumbel-Softmax”. In: *International Conference on Learning Representations*. 2017. arXiv: 1611.01144. URL: https://arxiv.org/abs/1611.01144.

[36] Danijar Hafner et al. “Mastering Atari with Discrete World Models”. In: *International Conference on Learning Representations (ICLR)*. 2021. arXiv: 2010.02193. URL: https://arxiv.org/abs/2010.02193.

[37] Danijar Hafner et al. “Mastering diverse control tasks through world models”. In: *Nature* 640.8059 (2025), pp. 647–653. DOI: 10.1038/s41586-025-08744-2.

[38] Alex Tamkin, Mohammad Taufeeque, and Noah D. Goodman. “Codebook Features: Sparse and Discrete Interpretability for Neural Networks”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 47535–47563. URL: https://proceedings.mlr.press/v235/tamkin24a.html.

[39] Yunfan Li et al. “Contrastive Clustering”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 10. 2021, pp. 8547–8555. DOI: 10.1609/aaai.v35i10.17037.

[40] Wouter Van Gansbeke et al. “SCAN: Learning to Classify Images without Labels”. In: *European Conference on Computer Vision (ECCV)*. 2020, pp. 268–285.

[41] Junyuan Xie, Ross Girshick, and Ali Farhadi. “Unsupervised Deep Embedding for Clustering Analysis”. In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. 2016, pp. 478–487. arXiv: 1511.06335.

[42] Nelson Elhage et al. “Toy Models of Superposition”. In: *Transformer Circuits Thread* (2022). arXiv: 2209.10652. URL: https://transformer-circuits.pub/2022/toy_model/index.html.

[43] Alireza Makhzani and Brendan Frey. “k-Sparse Autoencoders”. In: *International Conference on Learning Representations (ICLR)*. 2014. arXiv: 1312.5663. URL: https://arxiv.org/abs/1312.5663.

[44] Aditya Kusupati et al. “Matryoshka Representation Learning”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022. arXiv: 2205.13147.

[45] Tue M. Cao et al. *Tree SAE: Learning Hierarchical Feature Structures in Sparse Autoencoders*. 2026. arXiv: 2605.07922 [cs.LG]. URL: https://arxiv.org/abs/2605.07922.

[46] Seyed Arshan Dalili and Mehrdad Mahdavi. *Subspace-Aware Sparse Autoencoders for Effective Mechanistic Interpretability*. 2026. arXiv: 2606.06333 [cs.LG]. URL: https://arxiv.org/abs/2606.06333.

[47] Thomas Fel et al. *Structuring Sparsity: Block-Sparse Featurizers Capture Visual Concept Manifolds*. 2026. arXiv: 2606.25234 [cs.CV]. URL: https://arxiv.org/abs/2606.25234.

[48] Usha Bhalla et al. *Do Sparse Autoencoders Capture Concept Manifolds?* 2026. arXiv: 2604.28119 [cs.LG]. URL: https://arxiv.org/abs/2604.28119.

[49] Thomas Fel et al. "Into the Rabbit Hull: From Task-Relevant Concepts in DINO to Minkowski Geometry". In: *International Conference on Learning Representations*. 2026. arXiv: 2510.08638 [cs.CV]. URL: https://arxiv.org/abs/2510.08638.

[50] Anton Korznikov et al. *OrtSAE: Orthogonal Sparse Autoencoders Uncover Atomic Features*. 2025. arXiv: 2509.22033 [cs.LG]. URL: https://arxiv.org/abs/2509.22033.

[51] Xinting Huang and Michael Hahn. "Decomposing Representation Space into Interpretable Subspaces with Unsupervised Learning". In: *International Conference on Learning Representations*. 2026. arXiv: 2508.01916 [cs.LG]. URL: https://arxiv.org/abs/2508.01916.

[52] Yuxiao Li et al. "The Geometry of Concepts: Sparse Autoencoder Feature Structure". In: *Entropy* 27.4 (2025), p. 344. DOI: 10.3390/e27040344. arXiv: 2410.19750.

[53] Seongwan Park, Taeklim Kim, and Youngjoong Ko. "Decoding Dense Embeddings: Sparse Autoencoders for Interpreting and Discretizing Dense Retrieval". In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, 2025, pp. 26468–26485. DOI: 10.18653/v1/2025.emnlp-main.1345. URL: https://aclanthology.org/2025.emnlp-main.1345/.

[54] Wonseok Shin and Songkuk Kim. *Aligning Sentence Embeddings to Human Concepts via Sparse Autoencoders*. 2026. arXiv: 2607.00023 [cs.IR]. URL: https://arxiv.org/abs/2607.00023.

[55] Nick Jiang et al. "Interpretable Embeddings with Sparse Autoencoders: A Data Analysis Toolkit". In: *Proceedings of the 43rd International Conference on Machine Learning*. Vol. 306. Proceedings of Machine Learning Research. PMLR, 2026. arXiv: 2512.10092 [cs.AI]. URL: https://arxiv.org/abs/2512.10092.

[56] John X. Morris et al. "Text Embeddings Reveal (Almost) As Much As Text". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023, pp. 12448–12460. DOI: 10.18653/v1/2023.emnlp-main.765. URL: https://aclanthology.org/2023.emnlp-main.765/.

[57] Giorgos Nikolaou et al. "Language Models are Injective and Hence Invertible". In: *International Conference on Learning Representations*. 2026. arXiv: 2510.15511 [cs.LG]. URL: https://arxiv.org/abs/2510.15511.

[58] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. *SONAR: Sentence-Level Multimodal and Language-Agnostic Representations*. 2023. arXiv: 2308.11466 [cs.CL]. URL: https://arxiv.org/abs/2308.11466.

[59] Adam Karvonen et al. "SAEBench: A Comprehensive Benchmark for Sparse Autoencoders in Language Model Interpretability". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 29223–29264. arXiv: 2503.09532 [cs.LG]. URL: https://arxiv.org/abs/2503.09532.

[60] Kola Ayonrinde, Michael T. Pearce, and Lee Sharkey. *Interpretability as Compression: Reconsidering SAE Explanations of Neural Activations with MDL-SAEs*. 2024. arXiv: 2410.11179 [cs.LG]. URL: https://arxiv.org/abs/2410.11179.

[61] Xiangchen Song et al. *Position: Mechanistic Interpretability Should Prioritize Feature Consistency in SAEs*. 2025. arXiv: 2505.20254 [cs.LG]. URL: https://arxiv.org/abs/2505.20254.

[62] Junxuan Wang et al. *Towards Universality: Studying Mechanistic Similarity Across Language Model Architectures*. 2024. arXiv: 2410.06672 [cs.CL]. URL: https://arxiv.org/abs/2410.06672.

[63] Gonçalo Paulo and Nora Belrose. *Sparse Autoencoders Trained on the Same Data Learn Different Features*. 2025. arXiv: 2501.16615 [cs.LG]. URL: https://arxiv.org/abs/2501.16615.

[64] Anton Korznikov et al. *Sanity Checks for Sparse Autoencoders: Do SAEs Beat Random Baselines?* 2026. arXiv: 2602.14111 [cs.LG]. URL: https://arxiv.org/abs/2602.14111.

[65] Thomas Heap et al. *Automated Interpretability Metrics Do Not Distinguish Trained and Random Transformers*. 2025. arXiv: 2501.17727 [cs.LG]. URL: https://arxiv.org/abs/2501.17727.

[66] Stephen Casper et al. "Red Teaming Deep Neural Networks with Feature Synthesis Tools". In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023, pp. 80470–80516. arXiv: 2302.10894.

[67] Chen-Yu Lee et al. "Deeply-Supervised Nets". In: *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics*. Vol. 38. Proceedings of Machine Learning Research. PMLR, 2015, pp. 562–570. URL: https://proceedings.mlr.press/v38/lee15a.html.

[68] Hamed Pirsiavash, Deva Ramanan, and Charless C. Fowlkes. "Bilinear classifiers for visual recognition". In: *Advances in Neural Information Processing Systems*. Vol. 22. 2009, pp. 1482–1490.

[69] Jin-Hwa Kim et al. "Hadamard Product for Low-rank Bilinear Pooling". In: *International Conference on Learning Representations*. 2017.

[70] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. "Neural Machine Translation by Jointly Learning to Align and Translate". In: *International Conference on Learning Representations*. 2015. arXiv: 1409.0473. URL: https://arxiv.org/abs/1409.0473.

[71] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. “Effective Approaches to Attention-based Neural Machine Translation”. In: *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. Lisbon, Portugal, 2015, pp. 1412–1421.

[72] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. arXiv: 1706.03762. URL: https://arxiv.org/abs/1706.03762.

[73] Hubert Ramsauer et al. “Hopfield Networks is All You Need”. In: *International Conference on Learning Representations*. 2021. arXiv: 2008.02217. URL: https://arxiv.org/abs/2008.02217.

[74] Ron Zass and Amnon Shashua. “Nonnegative Sparse PCA”. In: *Advances in Neural Information Processing Systems*. Vol. 19. 2006, pp. 1561–1568. URL: https://papers.nips.cc/paper/3104-nonnegative-sparse-pca.

[75] Megasthenis Asteris, Dimitris Papailiopoulos, and Alexandros G. Dimakis. “Orthogonal NMF through Subspace Exploration”. In: *Advances in Neural Information Processing Systems*. Vol. 28. 2015, pp. 343–351.

[76] Ignacio Ramirez, Pablo Sprechmann, and Guillermo Sapiro. “Classification and Clustering via Dictionary Learning with Structured Incoherence and Shared Features”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2010, pp. 3501–3508. DOI: 10.1109/CVPR.2010.5539964.

[77] Filippo Pompili et al. *Two Algorithms for Orthogonal Nonnegative Matrix Factorization with Application to Clustering*. 2014. arXiv: 1201.0901 [math.OC]. URL: https://arxiv.org/abs/1201.0901.

[78] Moses Charikar and Lunjia Hu. *Approximation Algorithms for Orthogonal Non-negative Matrix Factorization*. 2021. arXiv: 2103.01398 [cs.DS]. URL: https://arxiv.org/abs/2103.01398.

[79] Biao Zhang and Rico Sennrich. “Root Mean Square Layer Normalization”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019. arXiv: 1910.07467. URL: https://arxiv.org/abs/1910.07467.

[80] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.

[81] Iris A. M. Huijben et al. “Residual Quantization with Implicit Neural Codebooks”. In: *Proceedings of the 41st International Conference on Machine Learning*. 2024. arXiv: 2401.14732 [cs.LG]. URL: https://arxiv.org/abs/2401.14732.

[82] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd ed. Wiley, 2006.

[83] Eirikur Agustsson et al. “Soft-to-Hard Vector Quantization for End-to-End Learning Compressible Representations”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. arXiv: 1704.00648 [cs.LG]. URL: https://arxiv.org/abs/1704.00648.

[84] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. “Neural Discrete Representation Learning”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. arXiv: 1711.00937. URL: https://arxiv.org/abs/1711.00937.

[85] Nina Rimsky et al. “Steering Llama 2 via Contrastive Activation Addition”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 15504–15522. DOI: 10.18653/v1/2024.acl-long.828. URL: https://aclanthology.org/2024.acl-long.828/.

[86] Alexander Matt Turner et al. *Steering Language Models With Activation Engineering*. 2024. arXiv: 2308.10248 [cs.CL]. URL: https://arxiv.org/abs/2308.10248.

[87] John Hewitt and Percy Liang. “Designing and Interpreting Probes with Control Tasks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019. arXiv: 1909.03368 [cs.CL]. URL: https://arxiv.org/abs/1909.03368.

[88] Moghis Fereidouni et al. “Evaluating Sparse Autoencoders for Monosemantic Representation”. In: *Findings of the Association for Computational Linguistics: EACL 2026*. Rabat, Morocco: Association for Computational Linguistics, 2026, pp. 5969–5984. DOI: 10.18653/v1/2026.findings-eacl.313. URL: https://aclanthology.org/2026.findings-eacl.313/.

[89] Valérie Costa et al. “From Flat to Hierarchical: Extracting Sparse Representations with Matching Pursuit”. In: *Advances in Neural Information Processing Systems*. Vol. 38. 2025. arXiv: 2506.03093 [cs.LG]. URL: https://arxiv.org/abs/2506.03093.

[90] Bilal Chughtai, Lawrence Chan, and Neel Nanda. “A Toy Model of Universality: Reverse Engineering How Networks Learn Group Operations”. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 6243–6267. URL: https://proceedings.mlr.press/v202/chughtai23a.html.

[91] Christopher J. Hillar and Friedrich T. Sommer. *When Can Dictionary Learning Uniquely Recover Sparse Data from Subsamples?* 2015. arXiv: 1106.3616 [q-bio.NC]. URL: https://arxiv.org/abs/1106.3616.

[92] Elizabeth S. Allman, Catherine Matias, and John A. Rhodes. “Identifiability of Parameters in Latent Structure Models with Many Observed Variables”. In: *The Annals of Statistics* 37.6A (2009), pp. 3099–3132. DOI: 10.1214/09-AOS689. arXiv: 0809.5032 [math.ST]. URL: https://arxiv.org/abs/0809.5032.

[93] Sunny Duan et al. “Unsupervised Model Selection for Variational Disentangled Representation Learning”. In: *International Conference on Learning Representations*. 2020. arXiv: 1905.12614 [cs.LG]. URL: https://arxiv.org/abs/1905.12614.

[94] Aapo Hyvärinen and Hiroshi Morioka. “Nonlinear ICA of Temporally Dependent Stationary Sources”. In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*. Vol. 54. Proceedings of Machine Learning Research. PMLR, 2017. URL: https://proceedings.mlr.press/v54/hyvarinen17a.html.

[95] David Klindt et al. “Towards Nonlinear Disentanglement in Natural Data with Temporal Sparse Coding”. In: *International Conference on Learning Representations*. 2021. arXiv: 2007.10930 [stat.ML]. URL: https://arxiv.org/abs/2007.10930.

[96] Francesco Locatello et al. “Weakly-Supervised Disentanglement Without Compromises”. In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020. arXiv: 2002.02886 [cs.LG]. URL: https://arxiv.org/abs/2002.02886.

[97] Ilyes Khemakhem et al. “Variational Autoencoders and Nonlinear ICA: A Unifying Framework”. In: *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics*. Vol. 108. Proceedings of Machine Learning Research. PMLR, 2020. arXiv: 1907.04809 [stat.ML]. URL: https://arxiv.org/abs/1907.04809.

[98] Roland S. Zimmermann et al. “Contrastive Learning Inverts the Data Generating Process”. In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021. arXiv: 2102.08850 [cs.LG]. URL: https://arxiv.org/abs/2102.08850.

[99] Usha Bhalla et al. “Temporal Sparse Autoencoders: Leveraging the Sequential Nature of Language for Interpretability”. In: *International Conference on Learning Representations*. 2026. arXiv: 2511.05541 [cs.CL]. URL: https://arxiv.org/abs/2511.05541.

[100] Matthew Chen, Joshua Engels, and Max Tegmark. “Low-Rank Adapting Models for Sparse Autoencoders”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025. arXiv: 2501.19406 [cs.LG]. URL: https://arxiv.org/abs/2501.19406.

[101] Paul K. Rubenstein et al. “Causal Consistency of Structural Equation Models”. In: *Proceedings of the Thirty-Third Conference on Uncertainty in Artificial Intelligence (UAI)*. 2017. arXiv: 1707.00819. URL: https://www.auai.org/uai2017/proceedings/papers/11.pdf.

[102] Sander Beckers and Joseph Y. Halpern. “Abstracting Causal Models”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 33. 01. 2019, pp. 2678–2685. DOI: 10.1609/aaai.v33i01.33012678. URL: https://ojs.aaai.org/index.php/AAAI/article/view/4117.

[103] Atticus Geiger et al. “Causal Abstractions of Neural Networks”. In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato et al. Vol. 34. Curran Associates, Inc., 2021, pp. 9574–9586. URL: https://proceedings.neurips.cc/paper/2021/hash/4f5c422f4d49a5a807eda27434231040-Abstract.html.

[104] Atticus Geiger et al. “Causal Abstraction: A Theoretical Foundation for Mechanistic Interpretability”. In: *Journal of Machine Learning Research* 26.83 (2025), pp. 1–64. URL: https://jmlr.org/papers/v26/23-0058.html.

[105] Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. “Transcoders Find Interpretable LLM Feature Circuits”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024. arXiv: 2406.11944. URL: https://arxiv.org/abs/2406.11944.

[106] Atticus Geiger et al. “Inducing Causal Structure for Interpretable Neural Networks”. In: *Proceedings of the 39th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri et al. Vol. 162. Proceedings of Machine Learning Research. PMLR, 2022, pp. 7324–7338. URL: https://proceedings.mlr.press/v162/geiger22a.html.

[107] Erik P. Hoel, Larissa Albantakis, and Giulio Tononi. “Quantifying causal emergence shows that macro can beat micro”. In: *Proceedings of the National Academy of Sciences* 110.49 (2013), pp. 19790–19795. DOI: 10.1073/pnas.1314922110.

[108] Markus Frohmann et al. “Segment Any Text: A Universal Approach for Robust, Efficient and Adaptable Sentence Segmentation”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024. arXiv: 2406.16678 [cs.CL]. URL: https://arxiv.org/abs/2406.16678.

[109] Amir Hossein Kargaran et al. “GlotLID: Language Identification for Low-Resource Languages”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023. arXiv: 2310.16248 [cs.CL]. URL: https://arxiv.org/abs/2310.16248.

[110] Rupesh K. Srivastava et al. “Compete to Compute”. In: *Advances in Neural Information Processing Systems*. Vol. 26. 2013, pp. 2310–2318.

[111] Thomas Dooms et al. *Bilinear autoencoders find interpretable manifolds*. 2026. arXiv: 2605.08891 [cs.LG]. URL: https://arxiv.org/abs/2605.08891.

[112] Yifu Qiu, Paul-Ambroise Duquenne, and Holger Schwenk. *Unified Vision-Language Modeling via Concept Space Alignment*. 2026. arXiv: 2603.01096 [cs.CV]. URL: https://arxiv.org/abs/2603.01096.

[113] Maja Popović. “chrF: character n-gram F-score for automatic MT evaluation”. In: *Proceedings of the Tenth Workshop on Statistical Machine Translation*. Ed. by Ondřej Bojar et al. Lisbon, Portugal: Association for Computational Linguistics, Sept. 2015, pp. 392–395. DOI: 10.18653/v1/W15-3049. URL: https://aclanthology.org/W15-3049/.

[114] Maja Popović. “chrF deconstructed: beta parameters and n-gram weights”. In: *Proceedings of the First Conference on Machine Translation: Volume 2, Shared Task Papers*. Berlin, Germany: Association for Computational Linguistics, 2016, pp. 499–504. DOI: 10.18653/v1/W16-2341. URL: https://aclanthology.org/W16-2341/.

[115] Matt Post. “A Call for Clarity in Reporting BLEU Scores”. In: *Proceedings of the Third Conference on Machine Translation: Research Papers*. Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 186–191. DOI: 10.18653/v1/W18-6319. URL: https://aclanthology.org/W18-6319/.

[116] M. E. J. Newman. “Modularity and community structure in networks”. In: *Proceedings of the National Academy of Sciences* 103.23 (2006), pp. 8577–8582. DOI: 10.1073/pnas.0601602103.

[117] Tamás Nepusz et al. “Fuzzy communities and the concept of bridgeness in complex networks”. In: *Physical Review E* 77.1 (2008), p. 016107. DOI: 10.1103/PhysRevE.77.016107.

[118] Andreas Tjärnberg et al. “Optimal tuning of weighted kNN- and diffusion-based methods for denoising single cell genomics data”. In: *PLoS Computational Biology* 17.1 (2021), e1008569. DOI: 10.1371/journal.pcbi.1008569.

[119] Joshua Batson and Loic Royer. “Noise2Self: Blind Denoising by Self-Supervision”. In: *Proceedings of the 36th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 524–533. URL: https://proceedings.mlr.press/v97/batson19a.html.

[120] Wan-Duo Kurt Ma, J. P. Lewis, and W. Bastiaan Kleijn. “The HSIC Bottleneck: Deep Learning without Back-Propagation”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34. 04. 2020, pp. 5085–5092. DOI: 10.1609/aaai.v34i04.5950.

[121] Arthur Gretton et al. “Measuring Statistical Dependence with Hilbert-Schmidt Norms”. In: *Algorithmic Learning Theory (ALT 2005)*. Vol. 3734. Lecture Notes in Computer Science. Springer, 2005, pp. 63–77. DOI: 10.1007/11564089_7.

[122] Ronald J. Williams. “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning”. In: *Machine Learning* 8.3–4 (1992), pp. 229–256. DOI: 10.1007/BF00992696.

[123] Andrew Wagenmaker et al. *Steering Your Diffusion Policy with Latent Space Reinforcement Learning*. 2025. arXiv: 2506.15799 [cs.RO]. URL: https://arxiv.org/abs/2506.15799.

[124] Andrew Ilyas et al. “Adversarial Examples Are Not Bugs, They Are Features”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019. URL: https://arxiv.org/abs/1905.02175.

[125] Kevin Ro Wang et al. “Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 Small”. In: *International Conference on Learning Representations*. 2023. URL: https://arxiv.org/abs/2211.00593.

[126] Simon Kornblith et al. “Similarity of Neural Network Representations Revisited”. In: *Proceedings of the 36th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 3519–3529. URL: https://proceedings.mlr.press/v97/kornblith19a.html.

[127] Le Song et al. “Feature Selection via Dependence Maximization”. In: *Journal of Machine Learning Research* 13.47 (2012), pp. 1393–1434. URL: http://jmlr.org/papers/v13/song12a.html.

[128] Thao Nguyen, Maithra Raghu, and Simon Kornblith. “Do Wide and Deep Networks Learn the Same Things? Uncovering How Neural Network Representations Vary with Width and Depth”. In: *International Conference on Learning Representations*. 2021. arXiv: 2010.15327. URL: https://arxiv.org/abs/2010.15327.

[129] Ekdeep Singh Lubana et al. *Priors in Time: Missing Inductive Biases for Language Model Interpretability*. 2025. arXiv: 2511.01836 [cs.LG]. URL: https://arxiv.org/abs/2511.01836.

[130] J. G. Wendel. “A Problem in Geometric Probability”. In: *Mathematica Scandinavica* 11 (1962), pp. 109–111. ISSN: 00255521, 19031807. URL: http://www.jstor.org/stable/24490189.

[131] Sariel Har-Peled, Piotr Indyk, and Rajeev Motwani. “Approximate Nearest Neighbor: Towards Removing the Curse of Dimensionality”. In: *Theory of Computing* 8 (2012), pp. 321–350. DOI: 10.4086/toc.2012.v008a014.

A Notation

This appendix summarizes our notation and lists the symbols used throughout the paper, together with the section, equation, or algorithm where each is defined.

Conventions. Lowercase italic letters denote scalars (t, g_ℓ), bold lowercase letters denote vectors ($\mathbf{x}$, $\mathbf{a}_{ij}$), uppercase italic letters denote matrices (W_{dec}, D_i), and sans-serif letters denote tensors of rank three or more (D, P). A subscripted object takes the font corresponding to its own rank: p_{nij} is a scalar element of P, $\mathbf{p}_{ni}$ is a vector slice, and $D_i := \mathsf{D}_{i\cdot\cdot}$ is a matrix slice; where a lowercase element could be confused with a reserved count, we enclose it in brackets, as in $[\mathsf{K}]_{nij}$. Batching adds a leading dimension: a single sample's assignments are $P \in \mathbb{R}^{h\times k}$, whereas a batch's are $\mathsf{P} \in \mathbb{R}^{b\times h\times k}$. Vectors are column vectors, so matrices act on the left ($W_{\text{dec}}\mathbf{r}_\ell$, with $W_{\text{dec}} \in \mathbb{R}^{d\times e}$ mapping $\mathbb{R}^e$ to $\mathbb{R}^d$). Batches stack samples as rows ($X \in \mathbb{R}^{b\times d}$), so a batched matrix product carries a transpose, $\hat{X} = R\,W_{\text{dec}}^\top$, or is written as function application, $\text{dec}(R)$. Sets are written in calligraphic type ($\mathcal{S}_n$, $\mathcal{G}$). Named statistics, losses, and operators are set in roman type (FVU_w, MSE, KL_m, dec, sg), as are H (entropy) and L (the total loss); the discussion around Eq. (36) follows standard information-theoretic convention by writing random variables in uppercase. A hat denotes a reconstruction ($\hat{\mathbf{x}}_\ell$), a tilde denotes a noised quantity ($\tilde{\mathsf{F}}$, $\tilde{\mathbf{f}}_i$), a bar denotes a mean ($\bar{\mathbf{p}}_i$, $\bar{\kappa}$), and a prime on an index denotes a second index of the same kind (i', j'). We write $\odot$ for the elementwise product and $\oplus$ for concatenation. Cosine similarity is written as the cosine of the angle between two vectors, $\cos\angle(\mathbf{a},\mathbf{b}) := \langle\mathbf{a},\mathbf{b}\rangle/(\|\mathbf{a}\|\|\mathbf{b}\|)$, so that cos takes a single argument; $|\cos\angle|$ alone denotes its absolute value for the vector pair in context. The expectation operator $\mathbb{E}$ averages over the index or distribution named in its subscript: $\mathbb{E}_n$ over the samples in context (a batch in a training statistic, or the scored rows in an evaluation), $\mathbb{E}_i$ over heads, $\mathbb{E}_{i\neq i'}$ over ordered pairs of distinct heads, and $\mathbb{E}_{j\sim\bar{\mathbf{p}}_i}$ under a distribution. When unsubscripted, $\mathbb{E}$ averages over the data. Similarly, Pr denotes an empirical fraction of samples, with $\Pr_n(\cdot)$ giving the fraction on which the condition holds. Finally, $\mathbf{1}[\cdot]$ denotes the indicator function, and sd and var denote the standard deviation and variance, subscripted under the same convention as $\mathbb{E}$.

Layers and states. Layers are indexed by position, $\ell = 0,\dots,l-1$, and states of the residual stack are indexed by the number of layers applied, $0,\dots,l$, so that layer ℓ takes state ℓ as input and produces state $\ell+1$. Under this convention, $\mathbf{r}_0 = \mathbf{0}$ and $\mathbf{y}_0 = \mathbf{x}$, the final reconstruction is $\hat{\mathbf{x}}_l$, and the deep-supervision prefixes are $\hat{\mathbf{x}}_1,\dots,\hat{\mathbf{x}}_l$. Because prose and tables refer to layers by

index (e.g., "layer 0"), a table row for layer ℓ reports the prefix reconstruction through that layer, $\hat{\mathbf{x}}_{\ell+1}$.

Indices and sizes.

index	ranges over	size	
n, n'	samples	b	batch size
ℓ, ℓ'	layers, $0, \ldots, l-1$	l	layers
i, i'	heads of a layer	h	heads per layer
j, j'	entries of a head	k	entries per head
q	coordinates	d, e	input dimension, dictionary width
		r	fiber rank
		d_{head}	a head's slice of e under concatenated heads
		k_{sel}	entries kept by the $\texttt{top}k_{\text{sel}}$ selection rule
		h'	heads kept, probed, or conjoined, out of h
		n_{const}	constant coordinates appended to a layer's input
		m, k_{SAE}	SAE width, and active latents under `TopK`

Throughout the paper, k without a subscript denotes the number of Ontologizer entries per head.

Symbols. The table below lists all symbols grouped by the component of the paper that introduces them.

symbol	meaning	defined
The model		
$\mathbf{x}$, X; $\bar{\mathbf{x}}$	an input (d) and a batch ($b \times d$); the data mean	Eqs. (20), (23)
enc	input encoder (the identity map unless stated otherwise)	Alg. 3
$\mathbf{u}_\ell$, $\mathbf{u}$	layer ℓ's normalized input shape, with the constant coordinate appended, passed to the classifier, router, and fibers ($\mathbf{u}$ within a layer)	Eq. (14)
W, V	the bilinear classifier's weight tensors	Eq. (1)
K, K	classifier logits for one sample ($h \times k$) and for a batch	Eq. (1)
t	classifier temperature, $P = \mathrm{softmax}_k(K/t)$	§4.1.3
P, P; $\mathbf{p}_{ni}$, p_{nij}	assignments for one sample and a batch; head i's distribution and entry j's probability	§4.1.3
$\bar{\mathbf{p}}_i$	head i's mean assignment (entry usage) over the data	Eq. (4)
p_{drop}	winner-dropout probability	§4.1.2
D, D_i	dictionary tensor ($h \times k \times e$) and head i's slice ($k \times e$)	Eq. (5)
$\mathbf{a}_{ij}$	an atom: $\lvert\mathsf{D}_{ij\cdot}\rvert$, or $\mathsf{D}_{ij\cdot}$ when signed	Eq. (5)
G_i	head i's dictionary Gram matrix	Eq. (3)
$\mathbf{f}_i$; F, F	head i's output (e); stacked over heads, and over heads and samples	Eq. (5)
$\mathbf{f}'_\ell$, F'	layer ℓ's pooled output for one sample and for a batch	Eq. (10)
$\boldsymbol{\gamma}$, γ_i, Γ	router scales across heads, for head i, and for a batch	Eq. (19)

symbol	meaning	defined
U_{ij}, $\mathbf{c}_i$; C, C	fiber basis ($e \times r$) and coordinates; stacked over heads and over a batch	Eq. (19)
A, A_i; β, ρ	fiber readout tensor and head i's slice; coordinate bound and norm cap	§4.1.7
g_ℓ, g_n	scalar gain: input norm at layer ℓ or for sample n	Eq. (11)
$\mathbf{r}_\ell$, R	residual accumulator after ℓ layers, for one sample and batched	Eq. (11)
W_{dec}, dec	linear decoder shared across all layers	Eq. (12)
$\hat{\mathbf{x}}_\ell$, $\hat{X}$	reconstruction after ℓ layers; final batched reconstruction	Eq. (12)
$\mathbf{y}_\ell$	residual after ℓ layers (the input to layer ℓ)	Eq. (13)
$\text{sg}[\cdot]$	stop-gradient operator	Eq. (13)
W_{flat}	code-to-output map of a model's flattened code, $\hat{\mathbf{x}} = W_{\text{flat}}\mathbf{p}_{\text{flat}}$, whose columns are the entries' decode directions	§B.5
Noise, statistics, and loss		
ε; ε_K, ε_F	standard normal draw; logit and feature noise terms	§4.1.6
σ_X, σ_K, σ_F	noise scales for inputs, logits, and features	§4.1.6
$\tilde{\mathsf{F}}$, $\tilde{\mathbf{f}}_i$	noised per-head outputs	Eq. (7)
ϵ	floating-point epsilon of the working dtype	Eq. (14)
τ_{rev}	dead-entry revival threshold	Alg. 2
entropy_k, H	entropy of a head's assignment in bits, and its mean over samples and heads	Eq. (2)
cossim_b	between-sample similarity of per-head outputs	Eq. (3)
KL_m	divergence of each head's batch-mean assignment from uniform, averaged over heads	Eq. (4)
κ_i, $\bar{\kappa}$	mean cosine between head i's atoms, and its mean over heads (row collinearity)	Eq. (22); §G.1
$\boldsymbol{\pi}_i$, ω	head i's usage-weighted coordinate profile; between-head support overlap	Eq. (6)
L1, L1_K, L1_S	L1 penalties on features, classifier gates, and router scales	Eqs. (7), (22)
KL_{pwak}, L2_{pwak}	partition-consensus statistics	Eq. (22)
$\boldsymbol{\psi}_\ell$	layer ℓ's statistics row	Eq. (22)
$\mathbf{w}$	per-dimension MSE weights: inverse variance, normalized to mean one	Eq. (20)
$\mathbf{s}$; s_g, s_ω, s_{L1}, …	loss weights: the vector paired with $\boldsymbol{\psi}_\ell$, and individual weights	Eq. (21)
MSE, L	whitened reconstruction error; total training loss	Eqs. (20), (21)
Evaluation and baselines		
FVU_w	whitened fraction of variance unexplained	Eq. (23)
ν, $\mathcal{S}_\nu$; $\mathbf{o}$	deviations retained per head and their support set; mean-assignment origin	Eq. (24)
$\text{FVU}_{w,\text{G}}$, n_{bits}; λ_j, θ	Gaussian rate–distortion reference and bit budget; covariance eigenvalues and reverse water-filling level	Eq. (25)

symbol	meaning	defined
H_{real}	realized bits: summed per-head entropy of entry usage	Eq. (36)
Δ, Δ_j; H_{op}	quantizer step size, shared and for coefficient j; operational rate in bits per sample	Eq. (26)
$\mathcal{G}$; exh, exc	a head or latent group; its exhaustiveness and exclusivity	Eq. (27)
$\kappa(\mathcal{G})$, $\zeta_{\cos}$; $\mathbf{v}_j$	within-group mean decode-direction cosine and its standardized score; a decode direction	Eq. (28)
$(\cdot)^{(1)}$, $(\cdot)^{(2)}$	the two models in a pairwise comparison	Eq. (29)
$\mathbf{q}_i^{(1)}$; $[M^{\text{part}}]_{ii'}$, $[M^{\text{contr}}]_{ii'}$	head i's entry selections across rows; head-matching similarity matrices	Eq. (29)
ϕ_{ni}, size$_i$	head i's output contribution on sample n; its share of whitened variance	Eqs. (29), (30)
$\mathbf{a}_{ij}^{\text{dec}}$, $\bar{\mathbf{a}}_i^{\text{dec}}$	a decoded atom; head i's usage-weighted mean decoded atom	Eq. (30)
recovered	fraction of next-token or decoder loss recovered	Eq. (31)
η^2; C, n_{rows}	fraction of variance explained by a partition; its cells and number of scored rows	Eq. (32)
k_{probe}	sparse-probing budget	§B.5
$\mathbf{z}_n^\star$, $\mathcal{S}_n$; W, $\mathbf{b}$	refitted coefficients on a frozen support; a linear code's decode directions (columns of W) and offset	Eq. (33)
μ_{ni}; j_1, j_2; J_i; α	a head's logit margin between its top two entries; Jacobian of head i's logits; step strength	Eq. (34)
$j^\star$, dir, ξ_n	steering target entry, steering direction, and random control vector	Alg. 4
assign	abstraction map from an input to its code	§9
$\mathbf{z}$, z_{nj}	SAE latent activations	Eq. (27)
W_{enc}, $\mathbf{b}_{\text{enc}}$; W_{dec}, $\mathbf{b}_{\text{dec}}$	SAE encoder weights and bias; SAE decoder (whose columns are decoder directions) and pre-subtracted decoder bias	§B.2
L0, s_{L1}, aux_k	active SAE latents per sample; L1 penalty weight; dead-latent revival count	§B.2; Table 13

B Supplementary methods

B.1 Data and objective

Embedding space and data. All main SONAR models are trained on a fixed cache of 3,974,400 unit-norm SONAR sentence embeddings (d=1024) computed from an interleaved 86-language mC4 sample, paired with a ground-truth language label sidecar. Because the stream order is deterministic, raw source text is recoverable by row index and verified per row against the language sidecar. Only six of the 86 configurations (English twice, French, Spanish, German, and Chinese) were encoded under their native SONAR source-language tags; the remaining 80 configurations (93% of rows) were encoded under the English tag, which SONAR reads as the text's language.

Re-encoding 400 rows per language under their intended native tags (`experiments/lang-tags/retag.py`) shifts the affected rows by $\mathrm{FVU}_w = 0.305$, roughly twice the hard code's reconstruction error, with nearly all of the shift specific to individual rows rather than a per-language offset. Ground-truth language accounts for 6.7% of embedding variance under the English tag vs. 7.4% under the native tags, and reconstruction error on the re-tagged embeddings increases by 4–12% across both the hard-code Ontologizer and the SAEs. Throughout the main SONAR experiments, "SONAR embeddings" refers to embeddings computed under these cached tags, and the language results inherit the difference (Appendix C.6). The embedding distribution lies on the unit sphere without being centered: $\|\bar{\mathbf{x}}\| = 0.564$, forming a spherical cap in which a single shared direction accounts for 0.318 of every sample's squared norm. It is high-dimensional yet anisotropic, with an effective dimension of 108 out of 1024 (participation ratio 268 on the whitened evaluation tail), 943 principal components required to explain 99% of variance, and a mean pairwise cosine of 0.31. Sections 8.1 and 6.5 additionally evaluate residual-stream activations of GPT-2 small [80] leaving block 8 of 12 (`blocks.8.resid_post`; d=768, harvested by `encode_acts.py`), a substrate where sparse dictionary learning is well established [9]. Section 7 uses a separate GPT-2 configuration from the `headline` branch (the unwhitened residual stream entering block 8 on FineWeb, also evaluated by GPT-2 next-token loss recovered), whose results are compared only within that section.

Evaluation. All models minimize the whitened MSE of Eq. (20). We report FVU_w, defined as whitened MSE divided by the whitened variance around a constant mean predictor,

$$\mathrm{FVU}_w := \frac{\sum_n \sum_q w_q \, (\hat{x}_{n,q} - x_{n,q})^2}{\sum_n \sum_q w_q \, (x_{n,q} - \bar{x}_q)^2}, \qquad \bar{\mathbf{x}} := \mathbb{E}_n[\mathbf{x}_n], \tag{23}$$

where $\mathbf{w}$ is the per-dimension inverse variance normalized to mean one, evaluated on the final 32,768 rows of the cache. This evaluation tail is held out from training for all SAEs and all straight-through arms. Because the softmax Ontologizers (`resid_nc` and the selection sweep) predate the holdout split and were trained on the full cache, we encoded the next 131,072 mC4 stream rows following the training corpus under the identical pipeline (`encode_fresh.py`), yielding an out-of-sample set that no model observed during training. On these fresh rows, every `pareto.py` operating point of `resid_nc`, the selection sweep's softmax run, the straight-through stack, and the SAEs shifts by at most 0.3% (one near-exact SAE at $\mathrm{FVU}_w = 10^{-4}$ shifts by 0.9%), and every operational rate in Appendix E.3 shifts by at most 0.0004 in FVU_w, confirming that the softmax models' tail scores are not inflated by training overlap. Where feature activations are required, Ontologizer features are the flattened per-head assignment probabilities $P \in \Delta^{k\,l\cdot h}$ evaluated at the run's final temperature (entry j of head i in layer ℓ corresponds to feature $(\ell h + i)k + j$, with firing threshold $2/k$), and SAE features are latent activations (with firing threshold 0, except that runs with softmax heads use the above-uniform threshold $2/(m/h)$ for h heads to match the entry convention). Evaluating a softmax checkpoint at the default temperature 1.0 rather than its annealed temperature 0.03 yields $\mathrm{FVU}_w \approx 0.91$ even for a healthy model, making it indistinguishable from a collapsed run; every reported score therefore uses each run's final training temperature.

Coordinate 837 and the sentence-split cache (`mc4_sent16`). Because SONAR's encoder applies a final LayerNorm prior to mean pooling, every unnormalized embedding $\mathbf{x}^{\text{raw}}$ lies in the affine hyperplane $\sum_{q=1}^{d}(x_q^{\text{raw}}-\beta_q)/\gamma_q = 0$. In SONAR's pretrained weights, $\gamma_{837} = 0.00492$ is five times smaller than the next smallest scale and $\beta_{837} = -0.340$ is thirty times larger than typical, so coordinate 837 carries 97% of the hyperplane normal's squared length and is determined by the remaining 1,023 coordinates to a root-mean-square error of 1.9×10^{-9} (1.2×10^{-4} in float16 storage). On the L2-normalized cache `mc4_4M`, dividing by $\|\mathbf{x}^{\text{raw}}\|$ turns that fixed coordinate into an almost exact proxy for $1/\|\mathbf{x}^{\text{raw}}\|$ (Pearson correlation 0.996 with $\beta_{837}/\|\mathbf{x}^{\text{raw}}\|$). Although it retains the smallest variance and therefore receives the largest inverse-variance weight ($w_{837} = 27.25$, or 2.66% of total weight, with the top ten coordinates accounting for 5.1%), $1/\|\mathbf{x}^{\text{raw}}\|$ varies smoothly over the sphere. Consequently, every model reconstructs coordinate 837 to $\text{FVU}_{837} \leq 0.088$, lower than its mean coordinate error and accounting for at most 0.41% of whitened error in any sparse model (`coordfvu.py`); zeroing w_{837} at evaluation changes FVU_w by at most +0.0043 and leaves all model rankings unchanged. A companion sentence-split cache, `mc4_sent16` (`experiments/sentence-split/`), segments the documents of `mc4_4M` with SaT [108] (`sat-3l-sm`, capped at 200 characters; the three-layer model slightly outperforms the twelve-layer model on round-trip SONAR chrF2, 0.593 vs. 0.583, by avoiding under-segmented long spans). It encodes up to 16 consecutive segments per document (44,113,823 rows from 3,974,400 documents: 59.1% full sentences, 21.6% newline-split lines, and 19.4% length-capped pieces) under each row's own SONAR source tag (83 of 86 languages mapped to their own NLLB tag), after dropping segments that the fastText identifier GlotLID [109] labels non-linguistic, and stores unnormalized float32 embeddings alongside document-boundary and segment-kind sidecars. Without L2 normalization, coordinate 837 is nearly constant and receives $w_{837} = 837$, or 81.8% of the total inverse-variance weight. In paired 369.5k-step training runs on a 4M-row slice with and without w_{837} (`ste_h76_sent16_w837` vs. `_no837`, alongside matching `k32` SAEs), retaining w_{837} leaves 265 SAE latents dead (16× as many as in the masked run), whereas the SAE's decoder bias absorbs the constant offset so that coordinate 837 contributes only 0.03% of its error compared to 2.0–3.4% for the bias-free Ontologizer. Masking w_{837} resolves this pathology—improving FVU_w on the remaining 1,023 coordinates from 0.1489 to 0.1462 for the Ontologizer and from 0.4596 to 0.4538 for the SAE—and reconstructing coordinate 837 analytically from the LayerNorm hyperplane prior to M2M100 decoding recovers the 0.03 chrF2 lost when it is left unconstrained.

B.2 Models and baselines

Full configurations for all models are tabulated in Appendix B.3.

Softmax Ontologizers. The reference Ontologizer (`resid_nc`) instantiates the architecture of Section 4.1 with softmax selection across l=5 residual layers of h=32 heads and k=32 entries per head. Layers 0–3 trained cleanly (reducing prefix FVU_w from 0.129 to 0.0051), whereas layer 4 froze 22 of 32 heads late in temperature annealing and slightly degrades final reconstruction (Section G.4). A subsequent *selection sweep* holds this architecture fixed and varies only the `DictBlock` selection rule (`top1`, `top2`, `top4`, `top8`, `top16`, and `softmax`), training each variant

both with and without a raised batch-mean uniformity weight s_{KL_m} (`_shm`).

The straight-through arm. The straight-through trainer (`train_ste.py`) imports the configuration of `sonar.py` and overrides `select="ste"`, selecting the one-hot argmax in the forward pass while using $\mathrm{softmax}(K/t)$ as a surrogate gradient in the backward pass. Its architecture, calibration, and initialization are detailed alongside its results in Section 6.

SAEs. Our SAE baselines follow the architecture of Gao et al. [9], computing pre-activations $\mathrm{relu}(W_{\mathrm{enc}}(\mathbf{x} - \mathbf{b}_{\mathrm{dec}}) + \mathbf{b}_{\mathrm{enc}})$ with `TopK` selection of k_{SAE} active latents and reconstructions $\hat{\mathbf{x}} = W_{\mathrm{dec}}\mathbf{z} + \mathbf{b}_{\mathrm{dec}}$. Decoder directions (the columns of W_{dec}) are renormalized to unit norm after every step, the linear encoder is initialized to the decoder's transpose, and dead latents are revived via `aux_k` (512 dead latents weighted by $1/32$, with a latent marked dead after 1000 steps without firing); all models are trained to convergence. Departing from the recipe of Gao et al. [9], our `TopK` pre-activation retains the encoder bias $\mathbf{b}_{\mathrm{enc}}$ (which their `TopK` encoder omits), $\mathbf{b}_{\mathrm{dec}}$ is initialized to the data mean rather than the geometric median, the reconstruction objective is whitened, Adam runs with its default ϵ without weight averaging or decoder-gradient projection, and training makes multiple passes over a fixed embedding cache. ReLU+L1 SAEs replace `TopK` selection with an L1 sparsity penalty and periodic neuron resampling.

The structural-ablation ladder. To attribute empirical differences to individual architectural choices rather than to complete architectures, we train a ladder of SAE variants at m=5120 on the same data and objective, each introducing a single Ontologizer-style constraint:

rung	run	added structure	analogue
1	`k32`	none (`TopK`, k_{SAE}=32)	baseline
2a	`g160top1`	160 heads, hard winner per head	hard heads, router
2b	`g160softmax`	160 heads, softmax per head	heads (soft)
3	`k32_p5`	5 nested prefix losses (matryoshka)	deepsup layers
—	`k32_bl`	bilinear encoder, $[\mathbf{x}-\mathbf{b}_{\mathrm{dec}}; 1]$ factors	bilinear classifier

Rungs 2a and 2b partition the m=5120 latents into 160 heads of 32 entries, and we treat them as simplified Ontologizer variants rather than external baselines. Per-head competition over a shared decoder reduces to a single layer of linear classifiers selecting dictionary entries, omitting the bilinear classifier, the residual stack, and the non-negative dictionary. Specifically, `g160softmax` is a one-layer softmax Ontologizer whose per-head coefficients sum to one, leaving each head without an independent magnitude. Conversely, `g160top1` is a one-layer hard-code Ontologizer with a scalar router fused into its classifier: each head selects one entry by argmax, and the ReLU of the winning logit scales that entry's unit-norm atom. This winning activation plays the role of the per-head router scale (Section 4.1.7), computed directly by the classifier rather than by a separate projection, so the code's 160 continuous coefficients act as router scales alongside its 800 index bits. Fusing the router also trains hard selection without a straight-through estimator, since the winning logit receives reconstruction gradients through its scale

while non-winning logits receive zero gradient. Equivalently, this model is an SAE with a rectified local winner-take-all activation [110] applied within each head. From a vector quantization perspective, both rungs take the form of a single additive quantizer comprising 160 codebooks of 32 atoms that each span the output space [25]: `g160softmax` forms convex combinations within each codebook, whereas `g160top1` selects one atom per codebook scaled by the router gain, making each head a gain–shape vector quantizer [29]. Switch SAEs [16] adopt the transposed design, routing each input to a single expert with multiple active latents inside it, whereas `g160top1` keeps every head active with a single active entry per head. The bilinear variant `k32_bl` computes each latent pre-activation as the product of two linear projections of the input, matching how the bilinear autoencoder of Dooms et al. [111] implements its atomic latents (though defined there via a single squared projection). Whereas Dooms et al. apply no centering or bias to the raw input, `k32_bl` centers the input by $\mathbf{b}_{\text{dec}}$ and appends a constant coordinate so that its latents are not sign-blind; and whereas their model uses no activation function, a Hoyer sparsity penalty, and a tied decoder reconstructing $\mathbf{x}\mathbf{x}^\top$, `k32_bl` applies a `TopK` activation and a standard linear decoder of $\mathbf{x}$, making it a `TopK` SAE with a bilinear encoder. Finally, Rung 3 (`k32_p5`) is a Matryoshka SAE [13] with five equal prefix blocks of 1024 latents, rather than blocks that roughly double in width.

We also train wider SAE baselines at m=11,264 (parameter-matched to the Ontologizer): `k32`, `k160` (coefficient-matched to one deviation per head), and `k5120` (coefficient-matched to the full soft code; here the `TopK` constraint never binds, with measured L0 $\approx$ 1520, so the model reduces to a dense ReLU autoencoder in the regime where `TopK` latents become dense and PCA-like [9]). A second-seed replica (`k32_s43`) is trained for cross-seed stability analysis. Latents of the bilinear encoder admit closed-form eigenfeature analysis: each latent's symmetric interaction matrix $\mathrm{sym}(\mathbf{w}_1\mathbf{w}_2^\top)$ (Pearce et al. [15]) has rank at most 2, and its dominant (top-$|\lambda|$) eigenvector is $\mathbf{w}_1/\|\mathbf{w}_1\| \pm \mathbf{w}_2/\|\mathbf{w}_2\|$, taking the sign of $\mathbf{w}_1 \cdot \mathbf{w}_2$. When the two factor vectors are negatively correlated, as holds for 90% of `k32_bl`'s latents, this dominant eigenvalue is negative: the quadratic form decreases along that eigenvector (suppressing the latent), whereas the positive-eigenvalue eigenvector $\mathbf{w}_1/\|\mathbf{w}_1\| + \mathbf{w}_2/\|\mathbf{w}_2\|$ increases the pre-activation. The `eig` autointerp mode (`sae.eigenfeatures`) uses the top-$|\lambda|$ eigenvector. Under our steering evaluation, the simplified head-based variants provide the primary structural comparison: `g160softmax` isolates the hypothesis of parallel softmax classifiers from the residual stack, bilinear classifiers, and shared dictionary, and `g160top1` isolates hard per-head selection with a fused router.

B.3 Model configurations

Tables 9–13 report the configuration of every evaluated model directly from its saved run artifacts: training hyperparameters from each run's final configuration record, any hyperparameter modified when a run was resumed, and architectural dimensions from its checkpoint.

B.4 The statistics row and its retired entries

The training loop logs each layer's auxiliary statistics as an append-only row: newly added statistics are placed at the end while retired statistics retain their positions, ensuring that

shared by every softmax run	
input	SONAR mC4 embeddings, d=1024, unit norm
shape	l=5 layers × h=32 heads, k=32 entries per head
classifier	bilinear (order 2), no gate
forward	residual; `resid_norm`, `resid_const`; `resid_gain` in the `sweep_*` runs only (`resid_nc` and `resid_nc_hm` predate it); joint deep supervision
optimizer	Adam, lr 5×10^{-5}, gradient clip 1.0, batch 256, 24 epochs
temperature	t=1 $\longrightarrow$ 0.03, geometric over 50k steps, then held
noise	logits normal, σ_K=0.02 $\longrightarrow$ 0.006; features featvar, σ_F=0.1; input off
winner dropout	p_{drop}=0 $\longrightarrow$ 0.1 over the anneal
loss	whitened MSE; s_{cossim_h}=10^{-5}; ghost path off
seed	42

run	selection	e	s_{KL_m}	other	steps
`resid_nc`	softmax	4096	$10^{-6\dagger}$	s_{L1}=10^{-9}, s_{cossim_h}=0 $\longrightarrow$ $10^{-5\dagger}$	372.6k
`resid_nc_hm`	softmax	2048	10^{-6}	s_{L1}=10^{-9}, s_{cossim_h}=10^{-5} $\longrightarrow$ $10^{-6\dagger}$	8.27M‡
`sweep_softmax`	softmax	2048	10^{-6}	s_{cossim_h}=10^{-6}	369.5k
`sweep_top`k_{sel}	top-k_{sel}, $k_{\text{sel}} \in \{1, 2, 8, 16\}$	2048	10^{-6}	s_{cossim_h}=10^{-6}	369.5k
`sweep_top8_relu`	top-8	2048	10^{-6}	ReLU gate on the classifier	369.5k
`sweep_*_shm`	softmax, top-k_{sel}, $k_{\text{sel}} \in \{2, 4, 8, 16\}$	2048	3×10^{-5}	s_{cossim_h}=10^{-6}	369.5k

Table 9 Softmax Ontologizer configurations. †Changed at a resume: `resid_nc` acquired s_{KL_m} and its s_{cossim_h} mid-run, and `resid_nc_hm` lowered s_{cossim_h}. ‡Latest checkpoint; results attributed to `resid_nc_hm` in Appendix D use its 4.38M-step checkpoint. cossim$_h$ is the retired between-head cosine (Appendix B.4).

shared by every SONAR straight-through arm	
input	SONAR mC4 embeddings, d=1024; eval tail held out
selection	straight-through (`ste`): argmax forward, softmax backward
classifier	bilinear (order 2), no gate
dictionary	non-negative, e=2048, shared linear decoder
forward	residual; `resid_norm`, `resid_const`, `resid_gain`; joint deep supervision
optimizer	Adam, lr 10^{-5}, gradient clip 1.0, batch 256, 24 epochs
temperature	t=1.5 × 10^{-4}, constant
noise	logits batchnorm, σ_K=0.3; features featvar, σ_F=0.1; input off
winner dropout	p_{drop}=0.1
loss	whitened MSE; s_{L1}=10^{-9}, s_{cossim_b}=10^{-5}, s_{cossim_h}=10^{-6}; ghost path off
seed	42 unless noted

arm	shape	k	bits	s_{KL_m}	init scale	other	steps
`ste_h76`	5 × 76	32	1900	10^{-4}	—		369.5k
`ste_h76_init01`	5 × 76	32	1900	10^{-4}	0.1		369.5k
`ste_h76_i01_s43`	5 × 76	32	1900	10^{-4}	0.1	seed 43	369.5k
`ste_h76_hm1e6`	5 × 76	32	1900	10^{-6}	—		369.5k
`ste_h76_kcos`	5 × 76	32	1900	10^{-4}	—	row-cosine setpoint 0.5	377.3k
`ste_h76_hsic3`	5 × 76	32	1900	10^{-4}	—	head CKA 10^{-3} (biased)	380.2k
`ste_k2_h380`	5 × 380	2	1900	5 × 10^{-4}	0.1	batch 128	739.0k
`ste_l1_h380`	1 × 380	32	1900	10^{-4}	—		369.5k
`ste_l1_h380_hm5`	1 × 380	32	1900	5 × 10^{-4}	—		369.5k
`ste_l1_h380_zca`	1 × 380	32	1900	5 × 10^{-4}	—	frozen ZCA encoder	353.3k
`ste_l1_h380_enc`	1 × 380	32	1900	5 × 10^{-4}	—	trained linear encoder	369.5k
`ste_l1_h380_kcos`	1 × 380	32	1900	10^{-4}	—	row-cosine setpoint 0.5; batch 256 → 128†	929.4k
`ste_l1_h380_i01`	1 × 380	32	1900	5 × 10^{-4}	0.1		369.5k
`ste_l1_h380_i01_s43`	1 × 380	32	1900	5 × 10^{-4}	0.1	seed 43	369.5k
`ste_l1_h380_i01_hm1e4`	1 × 380	32	1900	10^{-4}	0.1		369.5k
`ste_k128_h54`	5 × 54	128	1890	10^{-4}	0.1	s_ω=10^{-6}‡	369.5k
`ste_k128_h54_direct`	5 × 54	128	1890	10^{-4}	0.1	signed direct atoms, s_ω=10^{-6}‡	369.5k

Table 10 SONAR straight-through configurations. Init scale corresponds to `--dict-init-scale`, which rescales each dictionary at initialization so that a layer writes that fraction of its input norm (— denotes the default initialization). †Changed at a resume. ‡Trained after cossim_h was retired, using s_ω in its place.

shared by every GPT-2 straight-through arm on this branch	
input	GPT-2 small, residual leaving block 8, d=768; document-level holdout
selection	straight-through (`ste`)
classifier	bilinear (order 2)
forward	residual; `resid_norm`, `resid_const`, `resid_gain`; joint deep supervision
optimizer	Adam, lr 5×10^{-5}, gradient clip 1.0, batch 256, 24 epochs
temperature	t=1.0, constant (the naive port's setting, Appendix E.1)
noise	logits normal, σ_K=0.02 (absolute, not relative to the logit scale); features featvar, σ_F=0.1; input off
winner dropout	p_{drop}=0.1
loss	MSE weighted by `gpt2_l8.mse_weights_matched`; s_{L1}=10^{-9}, s_{cossim_b}=10^{-5}, s_{cossim_h}=10^{-6}, s_{KL_m}=10^{-6}

arm	shape	k	dictionary	gate	init scale, seed	steps
`ste_h76`	5×76	32	non-negative, e=1536	none	1.0, 42	371.9k
`ste_h76_e768`	5×76	32	non-negative, e=768	none	1.0, 42	371.9k
`ste_h76_sgn`	5×76	32	signed, e=1536	none	1.0, 42	371.9k
`ste_h76_sgn768`	5×76	32	signed, e=768	none	1.0, 42	371.9k
`ste_h76_cat32`	5×76	32	non-negative, concat, d_{head}=32	none	1.0, 42	371.9k
`ste_h20_cat128`	5×20	256	signed, concat, d_{head}=128	sigmoid	—, 42	371.9k
`ste_h20_cat128-43`	5×20	256	signed, concat, d_{head}=128	sigmoid	—, 43	371.9k
`ste_h20_cat128-sc`	5×20	256	as above, plus router	sigmoid	—, 42	371.75k
`ste_h20_cat128-sc43`	5×20	256	as above, plus router	sigmoid	—, 43	371.75k

Table 11 GPT-2 straight-through configurations on this branch: the orthant arms of Table 7 and the concatenated seed pairs of Section 6.5. The router (`scaled`) is sigmoid-gated and non-negative.

shared by the headline-branch runs of Section 7	
input	GPT-2 small, residual entering block 8, d=768; mean-centered, scaled to unit mean square; FineWeb
shape	l=5 layers × h=32 heads, k=32 entries per head (800 bits) unless noted
selection	straight-through, temperature t=1 → 0.03 over the first 60k steps
classifier	bilinear (order 2)
forward	residual; `resid_first`, `resid_gain`, `gain_clip`, `resid_norm`, `resid_const`; joint deep supervision
optimizer	Adam, lr 2×10^{-4}, batch 256
noise	logits normal, σ_K=0.02 → 0.006; features featvar, σ_F=0.1
winner dropout	p_{drop}=0 → 0.1
loss	unwhitened MSE; s_{L1}=10^{-9}, s_{cossim_b}=10^{-5}, s_{cossim_h}=10^{-6}, s_{KL_m}=10^{-6}
seed	42 unless noted

run	dictionary	data
direct + gain + STE	signed atoms in $\mathbb{R}^{768}$, no decoder	40M tokens, one pass (156k steps)
private heads + gain + STE	non-negative, e=1536 in 48-dimensional blocks, shared decoder	40M tokens, one pass
one-epoch ablations	as in Appendix F.1	10M tokens

Table 12 Configurations of the headline-branch GPT-2 runs (`experiments/gpt2/results/D40_*/config.json`). Fiber variants add `fiber_rank` r with the dropout, cap, or freezing settings listed in Section 7.

shared by every `sae.py` *run*	
input	same SONAR cache and held-out evaluation tail as the Ontologizers, d=1024
objective	whitened MSE, matching the Ontologizers
architecture	pre-subtracted decoder bias; encoder initialized tied to the decoder, except `m5120_k32_bl`, whose two factors are initialized randomly and scaled to match the tied encoder's pre-activation norm
decoder	unit-norm column directions in W_{dec}, except `g160softmax`, whose columns carry unconstrained norm because its coefficients are bounded
dead latents	`aux_k` 512 (`TopK` and grouped runs) or neuron resampling every 5000 steps (L1 runs)
optimizer	Adam, batch 4096

run	m	sparsity	encoder	structure	lr	steps
`m5120_k32`	5120	top-32	linear	—	4×10^{-4}	144.3k
`m5120_k32_s43`	5120	top-32	linear	seed 43	4×10^{-4}	144.3k
`m5120_k32_bl`	5120	top-32	bilinear	—	4×10^{-4}	144.3k
`m5120_k32_p5`	5120	top-32	linear	5 nested prefixes (Matryoshka)	4×10^{-4}	144.3k
`m5120_l1*`	5120	L1, five weights s_{L1} (caption)	linear	—	4×10^{-4}	144.3k
`m11264_k32`	11264	top-32	linear	—	10^{-4}	336k
`m11264_k160`	11264	top-160	linear	—	$10^{-4\dagger}$	336k
`m11264_k5120`	11264	top-5120	linear	— (dense, L0 ≈ 1520)	10^{-4}	22k
`m5120_g160top1`	5120	160 heads, top-1	linear	one-layer hard variant	4×10^{-4}	144.3k
`m5120_g160softmax`	5120	160 heads, softmax	linear	one-layer soft variant	4×10^{-4}	144.3k

Table 13 SAE baselines (above the rule) and single-layer Ontologizer variants (below it; Section B.2). The L1 sweep covers five weights $s_{\text{L1}} \in \{3 \times 10^{-6}, 10^{-5}, 3 \times 10^{-5}, 10^{-4}, 3 \times 10^{-4}\}$, where $*$ denotes the weight string in the run name (e.g., `m5120_l13e-05`). †Not logged; inferred from the step count shared with `m11264_k32`.

shorter rows from earlier runs remain column-aligned and that positional loss weights remain paired with their intended quantities. Accordingly, the statistics vector of Eq. (22) is stored with two retired slots,

$$\big[\,\mathrm{L1}_K \;\; \mathrm{L1} \;\; H \;\; \mathrm{cossim}_b \;\; \mathrm{cossim}_h \;\; \mathbb{E}_i \kappa_i \;\; \max_i \kappa_i$$
$$\mathrm{KL}_m \;\; \mathrm{KL}_{\mathrm{pwak}} \;\; \mathrm{L2}_{\mathrm{pwak}} \;\; \mathrm{cossim}_{\mathrm{flat}} \;\; \mathrm{L1}_S \;\; \omega\,\big],$$

where the two retired entries are recorded as NaN and zeroed prior to the weighted loss sum.

Between-head cosine cossim_h was defined on clean per-head features as the mean absolute cosine across ordered pairs of heads within each sample,

$$\mathrm{cossim}_h(\mathsf{F}) := \mathbb{E}_n \, \mathbb{E}_{i \neq i'} \left| \frac{\langle \mathbf{f}_{n,i}, \mathbf{f}_{n,i'} \rangle}{\|\mathbf{f}_{n,i}\| \|\mathbf{f}_{n,i'}\|} \right|.$$

Although $\mathrm{cossim}_h = 0$ implies disjoint support when features are non-negative (Section 4.1.4), it penalized support overlap only indirectly at $O(bh^2e)$ cost and provided no support guarantee for signed dictionaries, where orthogonality can arise from sign cancellation rather than disjoint coordinates. Moreover, because it depended only on the entries selected on each sample, unselected entries in a hard code remained unconstrained. By contrast, the support overlap ω measures coordinate sharing directly across all used entries, for both non-negative and signed dictionaries, in $O(hke + h^2e)$ time.

Flattened row cosine $\mathrm{cossim}_{\mathrm{flat}}$ was the mean off-diagonal cosine across all pairs of atoms in a layer, both within and between heads. It was introduced to penalize heads whose atoms are internally diverse yet aligned with those of another head, a failure mode not captured by the within-head collinearity κ_i. Because its average is dominated by cross-head pairs (only $k - 1$ of an atom's $hk - 1$ partners belong to the same head), and because zero cross-head cosine on non-negative atoms is equivalent to disjoint support, the usage-weighted per-head-pair overlap ω superseded $\mathrm{cossim}_{\mathrm{flat}}$ as well.

B.5 Evaluation protocols

Each evaluation protocol is implemented as a standalone script sharing a common evaluation split and feature-activation convention; all root-level scripts are unit-tested on synthetic data with planted structure. Figure 14 summarizes the inputs consumed by each script and the evaluation question it addresses.

Capacity Pareto (`pareto.py`). Code capacity is measured by the number of continuous coefficients transmitted per sample together with the index bits naming them. Ontologizer operating points are generated by *top-ν deviation truncation*: within each head, we retain the ν entries with the largest absolute deviation $|p_j - o_j|$ from an origin distribution $\mathbf{o}$, pin the remaining entries to $\mathbf{o}$, and renormalize,

$$\mathcal{S}_\nu := \text{top-}\nu_j \, |p_j - o_j|, \qquad p'_j := \frac{\mathbf{1}[j \in \mathcal{S}_\nu] \, p_j + \mathbf{1}[j \notin \mathcal{S}_\nu] \, o_j}{\sum_{j' \in \mathcal{S}_\nu} p_{j'} + \sum_{j' \notin \mathcal{S}_\nu} o_{j'}}, \qquad \mathbf{o} := \mathbb{E}_n[\mathbf{p}_n], \tag{24}$$

applied to every head so that the code transmits ν continuous coefficients per head. The origin $\mathbf{o}$ is the empirical corpus-mean assignment rather than the uniform distribution. Because model

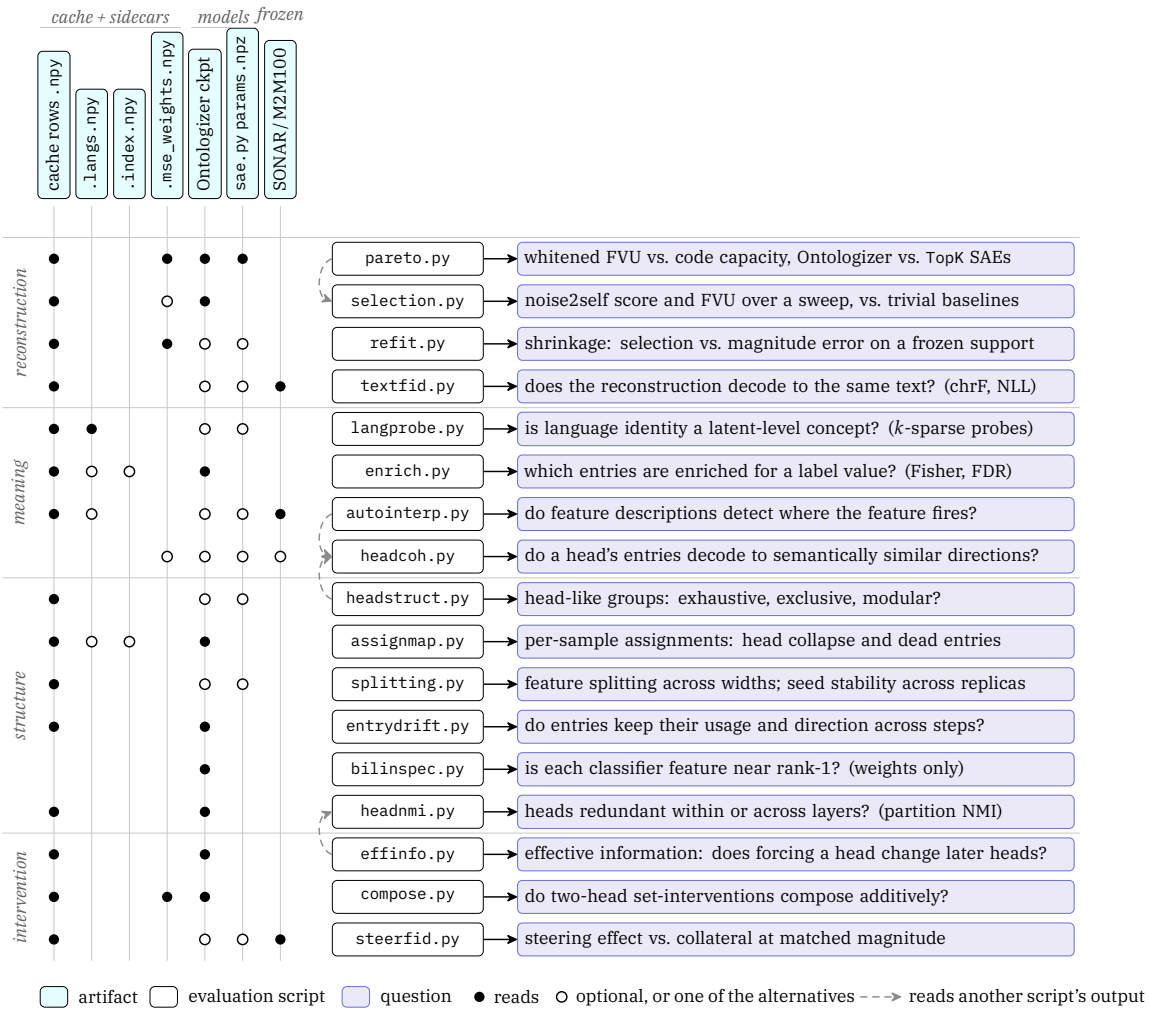


Figure 14 **What validates what.** Columns denote the artifacts consumed by each evaluation script: the embedding or activation cache (whose final rows form the evaluation split held out by `sae.py`) together with its sidecars; Ontologizer checkpoints; `sae.py` baselines; and the frozen SONAR encoder and M2M100 decoder. Filled markers indicate required inputs, and open markers indicate optional or alternative inputs (most scripts accept either model family, whereas `splitting.py` compares a pair of runs such as seed replicas). Dashed arrows trace intermediate outputs produced by one script and consumed by another. The effective-information script `effinfo.py` (in `experiments/effective-information/`) is described alongside `headnmi.py`, which consumes its output.

output is jointly *linear* in the flattened assignments when residual gains are absent (zero initial accumulator, linear decoder, and no g_ℓ scaling, which `resid_nc` predates), $\mathrm{decode}(\mathbf{o}) = \mathbb{E}[\hat{\mathbf{x}}]$, which decodes to generic corpus-mean text, whereas the uniform assignment lands off-manifold (cosine 0.067 with $\bar{\mathbf{x}}$). Notably, `resid_nc` was trained without the batch-mean uniformity term KL_m intended to align the uniform assignment with the corpus mean (Section 4.1.3); this observation motivated adding KL_m, though whether models trained with KL_m achieve that alignment has not been measured directly. The truncation curve spans the endpoints $\nu{=}0$ (the input-independent origin), $\nu{=}k$ (the exact soft model), and end-to-end one-hot argmax ("hard"). An alternative head-level truncation mean-ablates all but the h' lowest-entropy heads per layer, while SAE operating points use the empirical evaluation L0. Index bits count the code length required to identify which coefficients are transmitted. Because transmitted coefficients follow a canonical order, specifying r active slots out of n requires $\log_2 \binom{n}{r}$ bits rather than $r \log_2 n$: $l \cdot h \log_2 \binom{k}{\nu}$ for the deviation code, $l \log_2 \binom{h}{h'}$ for the head-level code, and $\log_2 \binom{m}{\mathrm{L0}}$ at the mean L0 for an SAE (or, for the single-layer hard variant, the active heads plus each active head's winning entry). Index bits alone do not represent the total rate of a code that also transmits continuous coefficients. When comparing such a code against a hard code at matched total rate (index bits plus coefficients at whatever quantization precision equalizes the two totals), we lower-bound its distortion by the least-squares refit on its frozen support (Eq. (33)), which no quantization of the coefficients can improve upon. For a hard code, the $l \cdot h \log_2 k$ index bits constitute its entire rate apart from the scalar per-layer gains (Appendix E.4). We compare each hard rate against a *Gaussian reference*: the minimum distortion achievable at that rate on a memoryless Gaussian source matching the evaluation tail's whitened covariance, given by reverse water-filling [82, Thm. 10.3.3]. Letting $\lambda_1, \ldots, \lambda_d$ denote the eigenvalues of that covariance, a budget of n_{bits} bits per sample yields

$$\mathrm{FVU}_{w,\mathrm{G}}(n_{\mathrm{bits}}) = \frac{\sum_j \min(\theta, \lambda_j)}{\sum_j \lambda_j}, \qquad n_{\mathrm{bits}} = \sum_j \max\Big(0,\ \tfrac{1}{2} \log_2 \frac{\lambda_j}{\theta}\Big), \tag{25}$$

where the water level θ is solved from n_{bits}. This curve serves as a reference rather than a lower bound for two reasons. First, among all sources with a given covariance, the Gaussian maximizes the squared-error rate–distortion function [82, Problem 10.8], so the true distortion–rate function of the embeddings at n_{bits} is *at most* $\mathrm{FVU}_{w,\mathrm{G}}(n_{\mathrm{bits}})$ and may lie well below it. Second, the rate–distortion function is an asymptotic limit over block length, approached by fixed-rate codes that jointly index long blocks of samples [82, Sec. 10.5], whereas our models encode each sample individually (while the converse bound applies at every block length [82, Sec. 10.4]). The comparison is therefore informative in one direction only: a code scoring *above* $\mathrm{FVU}_{w,\mathrm{G}}$ lies at least that far above the data's distortion–rate curve (though not necessarily that far above the optimal per-sample code), and nothing we report can establish that a given bit budget is too small.

Operational rate (`quantrate.py`). To place codes with and without continuous coefficients on a single rate axis, every scalar transmitted by a code is quantized and entropy-coded. A coefficient

z_{nj} with origin o_j and decode direction $\mathbf{v}_j$ is mapped to an integer symbol and reconstructed as

$$\hat{z}_{nj} = o_j + \Delta_j \left\lfloor \frac{z_{nj} - o_j}{\Delta_j} \right\rceil, \qquad \Delta_j = \frac{\Delta}{\|\mathbf{w}^{1/2} \odot \mathbf{v}_j\|}, \qquad H_{\text{op}} = \sum_j H\Big(\Big\lfloor \frac{z_{\cdot j} - o_j}{\Delta_j} \Big\rceil \Big), \tag{26}$$

where $\lfloor \cdot \rceil$ rounds to the nearest integer, so coefficients within $\Delta_j/2$ of o_j map to the origin symbol and every active coordinate incurs approximately equal whitened quantization error ($\Delta^2/12$ at fine step sizes). The origin o_j is 0 for sparse codes and the training-set mean for dense codes (matching $\mathbf{o}$ from Eq. (24) for a soft Ontologizer). The operational rate H_{op} (in bits per sample) encodes each slot's symbol under its own marginal distribution, estimated by plug-in entropy on the evaluation tail with the Miller–Madow correction; codebooks fitted on held-out training rows agree to within 0.5 bits. A grouped top-1 SAE encodes one joint symbol per head representing either silence or its winning entry and quantized level. Ontologizers are quantized inside the forward pass so that each subsequent layer classifies the residual of the quantized prefix; a hard code's selected entries are charged their realized entropy (Eq. (36)), and its per-layer gains are quantized in the same way as coefficients. No parameters are refitted after quantization, and sweeping Δ traces out each model's operational rate–distortion curve. A companion fixed-support rate replaces marginal support entropies with the combinatorial index cost $\log_2 \binom{m}{\text{L0}}$ per sample (or the nominal $l \cdot h \log_2 k$ for a hard code), while still entropy-coding the quantized values.

Categorical form (`headstruct.py`). An Ontologizer head is both exhaustive (firing on every input) and mutually exclusive (selecting a single winner). To test whether flat SAE latents self-organize into analogous categorical units, we split the evaluation rows in half: on the discovery half, latents are clustered under a group-size cap by *paradigmatic* affinity (the product of co-firing profile similarity across the rest of the dictionary and pairwise anti-correlation, gated at PMI < 1); on the held-out half, each discovered group $\mathcal{G}$ is evaluated by its exhaustiveness, exclusivity, and the coefficient of variation of its summed activation,

$$\begin{aligned} \text{exh}(\mathcal{G}) &:= \Pr_n\big(\textstyle\sum_{j\in\mathcal{G}} \mathbf{1}[z_{nj} > 0] \geq 1 \big), \\ \text{exc}(\mathcal{G}) &:= \mathbb{E}_n\big[\textstyle\sum_{j\in\mathcal{G}} \mathbf{1}[z_{nj} > 0] \;\big|\; \sum_{j\in\mathcal{G}} \mathbf{1}[z_{nj} > 0] \geq 1 \big], \end{aligned} \tag{27}$$

benchmarked against size-matched random partitions of the live-latent pool. An ideal categorical head satisfies exh = exc = 1. Head-like structure requires exhaustiveness *above* the null paired with exclusivity *below* the null, whereas topical co-firing clusters produce the opposite signature. Passing `--trained-groups` evaluates a model's explicitly trained heads under the same protocol.

Semantic coherence vs. contrast (`headcoh.py`). Complementing the test of categorical firing form, we evaluate whether a head's entries encode semantically similar meanings or form a contrastive set. Every entry possesses an exact decode direction in SONAR output space (for the Ontologizer, the corresponding column of the linear map W_{flat} satisfying $\hat{\mathbf{x}} = W_{\text{flat}}\mathbf{p}_{\text{flat}}$; for SAEs, the decoder directions), and because SONAR is a semantic sentence space, within-head directional similarity directly reflects semantic similarity among decoded entries. For any head

whose assignments sum to one (including the Ontologizer and `g160softmax`), the forward map determines these columns only up to a per-head vector shift whose shifts across heads sum to zero: because $\mathbf{p}_{\text{flat}}$ sums to one within each head, such shifts leave $\hat{\mathbf{x}}$ invariant. Consequently, raw column cosines reflect the trained parameterization, whereas only within-head differences—the entry-swap directions—are uniquely fixed by the input–output map (Section 5.3). By contrast, standard SAE latents and `g160top1` winners carry unconstrained magnitudes and have no such gauge freedom. For each head or group $\mathcal{G}$, we report the mean pairwise cosine and the top-singular-value energy fraction, standardized as ζ against size-matched random subsets drawn from the same pool (the same layer for Ontologizer heads),

$$\kappa(\mathcal{G}) := \mathbb{E}_{j \neq j' \in \mathcal{G}} \cos \angle(\mathbf{v}_j, \mathbf{v}_{j'}), \qquad \zeta_{\cos}(\mathcal{G}) := \frac{\kappa(\mathcal{G}) - \mathbb{E}_{\mathcal{G}'}[\kappa(\mathcal{G}')]}{\mathrm{sd}_{\mathcal{G}'}[\kappa(\mathcal{G}')]}, \tag{28}$$

where $\mathbf{v}_j$ is entry j's decode direction and $\mathcal{G}'$ ranges over 500 random subsets of equal size; as a metric-hygiene control, both statistics are also recomputed under the training objective's inverse-variance inner product (`--metric whitened`). Values $\zeta_{\cos} \gg 0$ indicate semantic clustering within a head, whereas $\zeta_{\cos} \ll 0$ indicates geometric repulsion among its atoms, as expected for a *contrastive classifier* (arising either from the contrastive differences themselves or from the per-head offsets selected by training). A complementary text-based view encodes autointerp descriptions with SONAR and tests pooled within-head cosine similarity against a label-permutation null. Passing `--assignment` scores post hoc discovered groups.

Between-head support (`headsupport.py`). Along the complementary between-head axis, we test whether distinct heads occupy disjoint coordinate supports, as motivated for non-negative dictionaries in Section 4.1.4. On a trained checkpoint, each head's usage-weighted coordinate profile and the resulting mean pairwise overlap ω follow Eq. (6), with $\bar{\mathbf{p}}_i$ computed as the head's mean assignment over the evaluation tail from a forward pass at the run's final temperature. A *decoded* variant replaces each dictionary atom with $|\mathbf{a}_{ij}^{\text{dec}}|$, where $\mathbf{a}_{ij}^{\text{dec}} := \mathrm{dec}(\mathbf{a}_{ij}) - \mathrm{dec}(\mathbf{0})$ is the output-space direction along which interventions act; for an `sae.py` model, which has no intermediate dictionary space, the profile of a head or group $\mathcal{G}$ is defined as $\boldsymbol{\pi}_{\mathcal{G}} = \sum_{j \in \mathcal{G}} \mathbb{E}[z_j]\,|\mathbf{w}_j^{\text{dec}}|$. Because dense non-negative profiles have positive overlap by chance, ω is benchmarked against a coordinate-permutation null (50 independent random permutations of each head's profile coordinates), which preserves every head's marginal coordinate distribution while destroying cross-head alignment. We also report each head's *spread*, defined as its profile's participation ratio normalized by the ambient dimension, $(\sum_q \pi_{iq})^2/(e \sum_q \pi_{iq}^2)$ (with d replacing e in the decoded variant); an even partition into h disjoint blocks requires a spread near $1/h$.

Feature splitting and seed stability (`splitting.py`, `partition.py`, `headcontrib.py`). Given a coarse model (1) and a fine model (2) evaluated on shared rows, each latent j' of model (2) is classified relative to each candidate parent latent j of model (1) by two-way firing containment at threshold τ: j' is a *child* of j if $\Pr(j \mid j') \geq \tau$ and $\Pr(j' \mid j) < \tau$, where $\Pr(j \mid j')$ denotes the fraction of rows on which j' fires where j also fires, and a *match* if both conditional probabilities are at least τ. Splits (≥ 2 children) are summarized by the children's union coverage of the parent's

firing set and by pairwise child firing overlap. Note that firing containment is an activation-coverage criterion that can admit high-frequency latents as spurious parents [45]. Incorporating decoder geometry, the script also identifies *geometric twins*—latents in model (2) with decoder cosine at least 0.6 to a latent in model (1) that nevertheless rarely co-fire with it ($\Pr(j \mid j') < 0.2$)—and compares each model's nearest-neighbor decoder-cosine distribution against a random baseline. Although the codebase refers to geometric twins as "absorption candidates", they differ from feature absorption as defined by Chanin et al. [12], which is a within-model failure mode evaluated against a ground-truth probe in which the absorbing latent's decoder direction retains only a small component of the general feature direction. When applied to two runs of identical architecture differing only in random seed, the match fractions measure cross-seed feature reproducibility. For hard-code Ontologizers, `partition.py` matches heads across runs by the normalized mutual information (NMI) of their sample partitions (what each head separates), while `headcontrib.py` matches heads by the cosine similarity of their output contributions (what each head writes), defined as each head's whitened decoded output minus $\mathrm{dec}(\mathbf{0})$ and centered across rows (without row centering, the shared non-negative dictionary offset induces spurious cross-run agreement). Letting $q_{ni}^{(1)}$ denote the entry selected by head i of run (1) on row n, $\mathbf{q}_i^{(1)}$ the vector of selections across all rows, and $\boldsymbol{\phi}_{ni}^{(1)}$ the head's whitened decoded contribution on row n, the pairwise head-similarity matrices are

$$\begin{aligned}[M^{\mathrm{part}}]_{ii'} &:= \mathrm{NMI}(\mathbf{q}_i^{(1)}, \mathbf{q}_{i'}^{(2)}),\\ [M^{\mathrm{contr}}]_{ii'} &:= \cos\angle\big(\mathrm{vec}_n[\boldsymbol{\phi}_{ni}^{(1)} - \bar{\boldsymbol{\phi}}_i^{(1)}],\ \mathrm{vec}_n[\boldsymbol{\phi}_{ni'}^{(2)} - \bar{\boldsymbol{\phi}}_{i'}^{(2)}]\big),\end{aligned} \tag{29}$$

where $\mathrm{NMI}(\mathbf{q}, \mathbf{q}') := 2I(\mathbf{q};\mathbf{q}')/(H(\mathbf{q}) + H(\mathbf{q}'))$ under the empirical row distribution, $\bar{\boldsymbol{\phi}}_i$ is the row mean, and $\mathrm{vec}_n[\cdot]$ concatenates per-row vectors across evaluation samples. Each matrix is matched one-to-one via the Hungarian algorithm to maximize summed similarity, and we report the mean over matched head pairs. A permutation null repeats this matching after shuffling run (2)'s rows, while a positive control matches a run against an earlier checkpoint from its own training trajectory. In addition, `headcontrib.py` decomposes each head's variance contribution into its layer's input gain and its decoded-atom spread: because head i's contribution on row n equals the layer gain g_n multiplied by the selected entry's decoded atom $\mathbf{a}_{i,q_{ni}}^{\mathrm{dec}}$,

$$\mathrm{size}_i := \frac{\sum_n \|\boldsymbol{\phi}_{ni} - \bar{\boldsymbol{\phi}}_i\|^2}{\sum_n \|\mathbf{w}^{1/2} \odot (\mathbf{x}_n - \bar{\mathbf{x}})\|^2} \approx \frac{\mathbb{E}_n[g_n^2]\ \mathbb{E}_{j\sim\bar{\mathbf{p}}_i}\|\mathbf{a}_{ij}^{\mathrm{dec}} - \bar{\mathbf{a}}_i^{\mathrm{dec}}\|^2}{\mathbb{E}_n\|\mathbf{w}^{1/2} \odot (\mathbf{x}_n - \bar{\mathbf{x}})\|^2}, \tag{30}$$

where $\bar{\mathbf{p}}_i$ is head i's entry-usage distribution (mean assignment) and $\bar{\mathbf{a}}_i^{\mathrm{dec}}$ is its usage-weighted mean decoded atom, factoring the contribution into mean squared gain times decoded-atom variance up to the gain's small variation. Given a reference seed pair (such as a single-layer control), the script also compares cross-seed agreement at matched head size.

Text fidelity (`textfid.py`). As a SONAR analogue of language-model "loss recovered", we decode both $\mathbf{x}$ and $\mathrm{recon}(\mathbf{x})$ through the M2M100 decoder with target language forced to `eng_Latn`. Prior to decoding, both vectors are rescaled to SONAR's corpus-median mean-pooled norm (0.307) so that the evaluation isolates directional error: native SONAR embeddings are far from

unit norm, the M2M100 decoder is sensitive to vector scale [112], and round-trip decode-then-re-encode fidelity peaks near 0.307 (every M2M100 decode in this paper uses that scale). Surface text agreement between the two decodes is scored by sentence-level chrF2 [113, 114] (character 1–6-grams with whitespace removed and recall weighted twice precision, averaged over rows on a 0–1 scale equal to sacreBLEU's sentence chrF2 [115] divided by 100) and by exact-match rate. We also compute the decoder's teacher-forced per-token negative log-likelihood (NLL) of the *original* embedding's decode conditioned on the original embedding (ceiling), the reconstruction, and the corpus-mean embedding (floor, which decodes to generic text). Loss recovered normalizes the reconstruction's NLL between this floor and ceiling,

$$\text{recovered} := \frac{\text{NLL}_{\text{floor}} - \text{NLL}_{\text{recon}}}{\text{NLL}_{\text{floor}} - \text{NLL}_{\text{ceil}}}, \tag{31}$$

matching the form of GPT-2's cross-entropy loss recovered (Section 7). Within the steering evaluation, baseline text fidelity also sets an upper bound on steering specificity: a code whose unsteered reconstruction fails to preserve decoded text cannot steer text with low collateral.

Language probing (`langprobe.py`, `headlang.py`, `headscript.py`). Following SAEBench's k_{probe}-sparse probing protocol [59], with the corpus's 86 ground-truth language labels in place of its balanced tasks, `langprobe.py` selects the top latents per language by two-sample t-statistic on training rows and evaluates held-out one-vs-rest F1 at k_{probe}=1 (single latent with a fitted threshold) and k_{probe}=4, 16 (logistic probes), alongside a dense probe on the raw embedding as a ceiling, reporting macro-F1 across languages. Note an architectural caveat: sparse single-latent probing tests whether a factor is encoded in individual latent magnitudes, whereas a classifier head encodes a factor in its categorical assignment (partition), which single-latent probes cannot resolve. Distribution-based monosemanticity metrics [88], which credit a unit for responding to multiple concepts provided its response distribution separates them, provide the per-unit analogue that transfers to categorical heads. Accordingly, `headlang.py` evaluates heads directly by computing, for each head, the NMI between its partition and the language label (null-corrected by subtracting a label-shuffled baseline), alongside a held-out linear probe on the concatenated one-hot codes of the top h' heads benchmarked against a dense ridge ceiling. Because NMI is normalized here by the arithmetic mean of the two marginal entropies, its attainable upper bound is $2\min(H_{\text{head}}, H_{\text{lang}})/(H_{\text{head}} + H_{\text{lang}})$, which equals 0.875 for a 5-bit head evaluated against the 6.43-bit language label. The companion script `headscript.py` repeats this analysis for writing-system script labels and additionally scores per-label cells and cross-head conjunctions. Finally, `headeta.py` and `topichead.py` measure the fraction of embedding variance explained by a partition,

$$\eta^2(C) := \frac{\sum_{C_c \in C} |C_c| \, \|\bar{\mathbf{x}}_{C_c} - \bar{\mathbf{x}}\|^2}{\sum_n \|\mathbf{x}_n - \bar{\mathbf{x}}\|^2}, \qquad \mathbb{E}_{C'}\left[\eta^2(C')\right] \approx \frac{|C| - 1}{n_{\text{rows}} - 1}, \tag{32}$$

for a partition C of n_{rows} rows into cells C_c with cell means $\bar{\mathbf{x}}_{C_c}$, reporting both raw η^2 and its excess over the mean across random partitions C' of the same size $|C|$.

Shrinkage (`refit.py`). Following the coefficient-refitting procedure of Gao et al. [9] without its non-negativity constraint, we freeze each sample's active support $\mathcal{S}_n$, solve for the whitened

least-squares coefficients on that support via masked normal equations (forcing inactive slots to zero),

$$\mathbf{z}_n^\star := \underset{\mathbf{z}:\, z_j=0\ \forall j\notin\mathcal{S}_n}{\arg\min} \left\|\mathbf{w}^{1/2}\odot\left(\mathbf{x}_n-\mathbf{b}-W\mathbf{z}\right)\right\|^2, \tag{33}$$

where the columns of W are the code's decode directions and $\mathbf{b}$ is its constant offset, and compare the resulting FVU_w against the unrefitted reconstruction. The gap between raw and refitted FVU_w measures magnitude (shrinkage) error, whereas the refitted FVU_w isolates support selection error. The Ontologizer is evaluated under the same solver via its exact linearity $\hat{\mathbf{x}} = W_{\text{flat}}\mathbf{p}_{\text{flat}}$ (with W_{flat} extracted by decoding constant one-hot codes), taking the top-ν deviation entries as active supports; operating points with support size at least d are excluded as underdetermined. Note a distinction in raw truncation conventions: `refit.py` truncates the soft forward pass's assignments P non-adaptively, whereas `pareto.py` truncates inside the forward pass so that downstream layers reclassify against the truncated prefix residual.

Steering fidelity in text (`steerfid.py`). To evaluate steering fidelity in decoded text, we perturb embeddings along unit-normalized native steering directions (SAE decoder directions, bilinear eigenfeatures, or the Ontologizer's `withArgs` entry-forcing output delta) at matched embedding norm multiples. Steering effect is measured by cycle consistency: the steered text decode is re-encoded through SONAR, and the target feature's activation gain is normalized by its firing-conditioned quantiles relative to the *cycled* unsteered decode so that both terms experience identical round-trip degradation. Collateral is measured as $1-\mathrm{chrF}$ relative to the unsteered decode. Perturbations along random unit directions provide a norm-matched control curve, and *net* effect is defined as feature effect minus that random control.

Steering fidelity in embedding space (`steerembed.py`). To avoid M2M100 round-trip confounding and test causal specificity directly in representation space, `steerembed.py` translates each input along a candidate direction, re-runs the model, and records both whether the target head now selects the intended entry (*realization*) and what fraction of the *other* heads change their winning entry (*collateral*). Five direction families are evaluated: `decode` (the output delta produced by forcing the target entry), `grad` (the input gradient of the target logit), `margin` (the input gradient of the target entry's logit margin over the head's incumbent winner), `adjoint` (the bilinear classifier's dominant eigenvector via `BilinearBlock.rev`), and `random`. Step strength α is specified as a fraction of each sample's norm. We also evaluate each model's *native* step size: the output delta of forcing the entry in an Ontologizer, or the decoder direction scaled by the latent's mean nonzero activation in an SAE. Algorithm 4 gives the full procedure.

Eigenvector orientation. The adjoint direction of a bilinear classifier is an eigenvector of its symmetric interaction matrix, which `eigh` determines only up to sign because a bilinear eigenvector enters the quadratic form through the square of its projection [15]. Orienting each eigenvector by the sign of its eigenvalue with a leading-coordinate tie-break (`nlinear.orient`) realizes 0.427 of targets on the softmax model at strength 0.25, compared to 0.224 under the arbitrary sign returned by `eigh` and 0.446 when oriented per sample. On isotropic inputs, a

Algorithm 4: Embedding-space steering evaluation (`steerembed.py`). For an Ontologizer, a target $j^\star$ is an entry of head i in layer ℓ, and each head's choice is its argmax entry. For an SAE with heads, a target is a latent and each head's choice is its winning latent (with abstention in a top-1 head treated as a distinct outcome); for an SAE without heads, a target is a latent that counts as selected when active, and collateral is the fraction of a sample's other active latents that drop out.

Input: model; held-out rows X; strengths $\alpha \in \{0.25, 0.5, 1, 2, 4\}$; 64 targets per model, stratified by layer, each selected on at least 0.5% of reference rows
Output: realization and collateral per direction and strength, and at the native step
begin
  foreach *target* $j^\star$ **do**
    $X_{j^\star} \leftarrow$ 32 rows on which $j^\star$ is *not* already selected
    record every head's choice on $X_{j^\star}$
    foreach *direction* dir $\in$ {*decode, grad, margin, adjoint, random*} **do**
      $\mathbf{v}_n \leftarrow \mathrm{dir}(\mathbf{x}_n, j^\star)/\|\mathrm{dir}(\mathbf{x}_n, j^\star)\|$ `// unit direction per row`
      foreach *strength* α **do**
        $\mathbf{x}'_n \leftarrow \mathbf{x}_n + \alpha\,\|\mathbf{x}_n\|\,\mathbf{v}_n$; $\mathbf{x}'_n \leftarrow \mathbf{x}'_n\,\|\mathbf{x}_n\|/\|\mathbf{x}'_n\|$ `// rescale to ‖x‖`
        recompute every head's choice on the steered rows
        realization $\leftarrow \mathbb{E}_n\,\mathbf{1}[j^\star$ selected on $\mathbf{x}'_n]$
        collateral $\leftarrow$ share of (row, other head) pairs whose choice changed
      end
    end
    $\boldsymbol{\delta}_n \leftarrow \hat{\mathbf{x}}_n(j^\star$ forced$) - \hat{\mathbf{x}}_n$, or, for an SAE, the latent's decoder direction times $\bar{z}_{j^\star}$, its mean activation when it fires `// native step`
    score $\mathbf{x}_n + \boldsymbol{\delta}_n$, unrescaled, and the control $\mathbf{x}_n + \|\boldsymbol{\delta}_n\|\,\boldsymbol{\xi}_n$ for a random unit $\boldsymbol{\xi}_n$
  end
  return *the means over targets*
end

purely weight-based sign convention could not improve over chance: a step of size α along $\mathbf{v}$ changes $\mathbf{x}^\top B\mathbf{x}$ to first order by $2\alpha\lambda(\mathbf{x}\cdot\mathbf{v})$, whose sign depends on the sample (increasing the logit on roughly half of isotropic Gaussian draws). Weight-only orientation succeeds on SONAR because the embedding cloud is uncentered and anisotropic, so $\mathbf{x}\cdot\mathbf{v}$ is dominated by $\bar{\mathbf{x}}\cdot\mathbf{v}$, which has a consistent sign across the corpus. On the hard-code Ontologizer, however, the oriented eigendirection remains below the random-direction baseline at every step strength.

Linearized collateral (`steergeom.py`). To separate the direct within-layer effect of an input perturbation on non-target heads from downstream cascades through subsequent layers, `steergeom.py` linearizes the target layer's logits while holding all upstream codes fixed. Under residual forwarding, an input perturbation $\mathbf{v}$ shifts the target layer's unnormalized residual input by exactly $\mathbf{v}$. Letting J_i denote the Jacobian of head i's logits at the layer input $\mathbf{e}_n$, head i's winner–runner-up margin and its first-order change under a step of magnitude $\alpha\|\mathbf{x}_n\|$ are

$$\mu_{ni} := [\mathsf{K}]_{nij_1} - [\mathsf{K}]_{nij_2}, \qquad \Delta\mu_{ni} := \alpha\,\|\mathbf{x}_n\|\left(\left(J_i\mathbf{v}_n\right)_{j_1} - \left(J_i\mathbf{v}_n\right)_{j_2}\right), \tag{34}$$

where K denotes the logits and j_1, j_2 are head i's top two entries on row n. For a target in head $i^\star$, the linearized flip fraction is $\mathbb{E}_{n,\,i\neq i^\star}\,\mathbf{1}[\mu_{ni} + \Delta\mu_{ni} < 0]$ evaluated at α=0.25, reported for the `decode`, `grad`, and `random` directions alongside the median margin μ. Two additional summary statistics are computed from the same Jacobians: *off*, the mean magnitude $\|\mathbf{x}_n\|\,|J_i\mathbf{v}_n|$ across non-target heads and entries normalized by that of a random direction (values below 1 indicate reduced alignment with non-target classifier subspaces); and *own*, the median first-order margin gain of the target entry over the incumbent per unit of $\|\mathbf{x}_n\|$, divided by the initial margin gap.

Intervention composition (`compose.py`). To test whether heads compose additively when treated as candidate causal variables, we sample pairs of forced entry interventions $(\ell_1, i_1)\to j_1$ and $(\ell_2, i_2)\to j_2$ (8 random heads per layer, one random entry per head; all within-layer pairs plus 60 cross-layer pairs) across 512 held-out rows, comparing the joint intervention's decode delta $\boldsymbol{\delta}_{12}$ against the sum $\boldsymbol{\delta}_1+\boldsymbol{\delta}_2$ of the single-intervention deltas via the median per-row cosine similarity and the relative error $\|\boldsymbol{\delta}_{12} - (\boldsymbol{\delta}_1+\boldsymbol{\delta}_2)\|/\|\boldsymbol{\delta}_1+\boldsymbol{\delta}_2\|$ in both unwhitened and whitened coordinates. Because the decoder is strictly linear in the flattened assignments, final-layer pairs compose additively by construction; that stratum simultaneously establishes the empirical numerical floor (cancellation from $\|\mathbf{r}\|\approx 31$ to $\|\hat{\mathbf{x}}\|\approx 1$ through the decoder amplifies float32 rounding to $\sim$0.02 relative error across differently compiled variants, whereas float64 evaluation is exact). Any non-additivity above this floor in earlier layers arises from downstream reclassification as subsequent layers re-encode the modified residual.

Autointerp (`autointerp.py`). The automated interpretability pipeline executes in four stages: harvesting (collecting top-activating rows, negative examples, frequency-stratified feature subsets, and parameter-derived decode embeddings), text recovery (deterministic mC4 stream replay by row index, verified per row against the language sidecar), description generation, and blinded detection scoring. Five description modes are supported: `params`, the M2M100 text decode of a parameter-derived embedding (for Ontologizer entries, the origin $\mathbf{o}$ with one

head forced one-hot; for SAE latents, $\mathbf{b}_{\text{dec}} + \alpha\,\mathbf{w}_j^{\text{dec}}$); `pdev`, the M2M100 decode obtained after subtracting the pure-origin embedding decode; `eig`, the M2M100 decode of the bilinear eigenfeature; `acts`, an LLM summary of top-activating text snippets; and `cacts`, a *contrastive* summary in which the LLM receives top-activating snippets alongside random corpus draws and is prompted to state what distinguishes them, preventing the degenerate corpus-level descriptions invited by uncontrasted summaries on noisy multilingual web text. Scoring uses an identical blinded detection protocol over held-out positives and negatives across all modes, reporting per-feature F1 relative to the always-match baseline of F1 ≈ 0.67 on balanced items. Both the `acts` and `cacts` descriptions and the detection judgments are produced by Claude Haiku (`claude-haiku-4-5`) via the Anthropic Messages API with rate metering and bounded reply lengths, so describer and judge are a single model without an independent LLM or human rater; failed API judgments are omitted rather than recorded (avoiding an earlier outage artifact that scored empty responses as F1 $= 0$). Six models are evaluated: the Ontologizer, ladder rungs `k32`, `k32_bl`, `g160top1`, and `g160softmax`, and the dense control `k5120`.

Modularity (`headstruct.py --modularity`). A head's assignments partition samples into soft cells, and soft modularity—Newman's Q [116] with the hard co-membership indicator replaced by the inner product of soft membership vectors, matching the fuzzy modularity of Nepusz et al. [117, Eq. 14]—quantifies the excess affinity concentrated within those cells relative to random assignment at matched node degrees. On each batch of 4,096 rows, given symmetric 15-nearest-neighbor heat-kernel edge weights $\chi_{nn'} = \exp((\cos\angle(\mathbf{y}_n, \mathbf{y}_{n'}) - 1)/0.2)$ between layer inputs $\mathbf{y}_n$ (self-edges removed, symmetrized by union), node degrees $\chi_n = \sum_{n'} \chi_{nn'}$, and total degree $\chi = \sum_n \chi_n$, head i's modularity is

$$Q_i = \frac{1}{\chi} \sum_{n,n'} \left(\chi_{nn'} - \frac{\chi_n \chi_{n'}}{\chi} \right) \mathbf{p}_{ni} \cdot \mathbf{p}_{n'i}, \tag{35}$$

with the graph and degrees restricted to the samples on which the head or group is active so that sparse groups are not penalized for incomplete coverage. This restriction evaluates cells as communities of the induced active subgraph, whereas Newman's subdivision retains the parent graph's degrees [116]. Each Ontologizer layer is scored on its own input ($\mathbf{x}$ at layer 0, and the corresponding residual at deeper layers); prefix block ℓ of a Matryoshka SAE is scored on its prefix residual ($\mathbf{x}$ minus the decoded reconstruction of blocks prior to ℓ, so block 0 is scored on $\mathbf{x} - \mathbf{b}_{\text{dec}}$); and flat models are scored on $\mathbf{x}$, with each SAE group's soft memberships obtained by normalizing member activations within the group. Each layer's mean Q_i is compared against 20 size-matched permutation nulls that randomly regroup entries within the same layer or block across cache rows 524,288–1,048,575 (training rows; post hoc SAE groups are discovered on the preceding slice).

Classifier geometry (`bilinspec.py`). In the trained classifiers (bilinear order 2 without gating), each logit is a single bilinear form over the normalized layer shape, $(\mathbf{w} \cdot \mathbf{u})(\mathbf{v} \cdot \mathbf{u}) = \mathbf{u}^\top B \mathbf{u}$, where $B = (\mathbf{w}\mathbf{v}^\top + \mathbf{v}\mathbf{w}^\top)/2$ is the symmetric interaction matrix of a bilinear unit [15]. Because B has rank at most 2, with nonzero eigenvalues $\lambda_\pm = (\mathbf{w} \cdot \mathbf{v} \pm \|\mathbf{w}\|\|\mathbf{v}\|)/2$ and eigenvectors $\mathbf{w}/\|\mathbf{w}\| \pm \mathbf{v}/\|\mathbf{v}\|$, every scale-invariant statistic of an entry's spectrum is a function of $\cos\angle(\mathbf{w}, \mathbf{v})$.

Across saved checkpoints, `bilinspec.py` computes these quantities directly from the weights on the semantic coordinates of $\mathbf{u}$ (excluding the constant coordinate) without requiring input data, reporting per layer the median and quartiles of $|\cos \angle|$, the fraction of entries whose dominant eigenvalue is negative, and the norm of the constant coordinate's weight relative to the rest of $\mathbf{w}$. These closed-form spectra agree with dense eigendecompositions to 10^{-14}.

Assignment maps (`assignmap.py`). To inspect sample-level assignments directly, we draw a label-stratified sample of evaluation-tail rows (a fixed count per language from the cache sidecar, or per coarse token class for GPT-2: word, capitalized word, continuation, punctuation, digit, newline, or space) and plot their assignment probabilities across each head's k entries, with rows grouped by label and sorted within each group by winning entry and entries ordered by mean usage. Each head is annotated with its "use" and "row" summary metrics (Figure 4). For an SAE, the same rows in identical order are plotted against its latents, with each cell showing a latent's activation as a share of the row's total activation and latents ordered by usage; trained SAE heads are rendered as individual heads, whereas a flat SAE is rendered as a single block with "use" and "row" computed analogously.

Entry drift (`entrydrift.py`). Across a run's saved checkpoints, `entrydrift.py` records each entry's evaluation-tail usage $\bar{p}_{ij}$ at that checkpoint's training temperature together with the cosine similarity between its decoded atom $\mathbf{a}_{ij}^{\mathrm{dec}}$ (the decoded one-hot code minus the decoded zero code) at that checkpoint and at the final checkpoint. Entries with $\bar{p}_{ij} < 1/(4k)$ are considered inactive: because an unused entry receives negligible gradient, its direction freezes and yields a trivial cosine near 1, so drift statistics are summarized over the entries active at the final checkpoint.

Label enrichment (`enrich.py`). Each evaluation-tail row is assigned within every head to its winning entry. For each (entry, label) cell, under the null hypothesis of independence conditioned on the row and column margins, the overlap count follows a hypergeometric distribution. We apply a two-sided exact test (doubling the tail probability on the observed side of the expectation) to every cell whose observed count or expected count is at least 5, adjusting p-values via the Benjamini–Hochberg procedure across all tested cells of all heads; effect size is measured by the $2 \times 2 \log_2$ odds ratio with 0.5 added to each cell count. At the head level, the NMI between the winning entry and the ground-truth label (mutual information divided by the arithmetic mean of the two entropies), minus its mean across five label-shuffled draws, quantifies how much of the head's partition is explained by the label.

Head-partition NMI and effective information (`headnmi.py`, `effinfo.py`). Pairwise similarity between heads' winning-entry partitions on the evaluation tail is measured by chance-adjusted NMI, $(I - I_0)/(\bar{H} - I_0)$, where I is the mutual information, $\bar{H}$ is the arithmetic mean of the two marginal entropies, and I_0 is the Miller–Madow approximation to the mutual information expected between two independent partitions with the same numbers of used entries (empirical permutation nulls agree with I_0 to within 10%). Under this scaling, 0 corresponds to

chance agreement and 1 to identical partitions; heads with fewer than 1.1 effective entries carry no partition and are omitted. Effective information [107] from an upstream source head to a downstream effect head is computed across 4,096 evaluation-tail contexts as the mutual information between an intervention forcing the source head uniformly over the entries it uses on the evaluation tail and the effect head's resulting argmax selection. For a hard code, those forced entries are the source head's native forward states, keeping the intervention on distribution.

Selection surfaces (`selection.py`). On each batch of 2,048 evaluation-tail rows, head i's assignments P_i define the co-classification neighbor graph N_i (Figure 4b) over the layer's input Y_ℓ, obtained by multiplying $P_i P_i^\top$ elementwise with the input heat kernel, zeroing self-edges, and row-normalizing. Each head is scored by how accurately its co-classified neighbors predict each row, $\|Y_\ell - N_i Y_\ell\|^2 / \|Y_\ell - \bar{Y}_\ell\|^2$ with batch mean $\bar{Y}_\ell$—the leave-one-out objective minimized by $\mathrm{L2}_{\mathrm{pwak}}$ (Eq. (22)) using the weighted affinity kernel of DEWAKSS [118]. Although inspired by noise2self [119], this score lacks noise2self's formal guarantee: removing self-edges excludes a row's value from its own linear combination, but the row's input still influences its weights through both its assignment and the heat kernel, so the estimator is not $\mathcal{J}$-invariant in the sense of Batson and Royer [119]. A score of 0 indicates exact neighbor prediction, whereas 1 indicates no improvement over the batch mean. Two baselines are evaluated on the same runs and rows: placing all rows in a single cell (the unpartitioned heat-kernel smoother) and permuting row assignments to form an input-independent random partition with matched cell sizes (three draws per batch); scores are averaged across heads and batches. A strictly $\mathcal{J}$-invariant split variant (`--jinv`) centers each batch's input, applies a random orthogonal rotation, and partitions the coordinates into two halves: one half is used to construct all assignment gates and heat kernels while the other half is predicted, ensuring that predicted coordinates never enter their own weights. Under this split estimator, the kernel bandwidth is selected to minimize the unpartitioned smoother's error, following DEWAKSS [118], and learned partitions are benchmarked against random-projection heads that assign each row to the argmax over k random Gaussian directions—evaluated individually, as the joint gate of 8 heads, and as the joint gate of 64 heads (which approximates their collision kernel).

Diagnostic scripts (`experiments/ste-arm/`). Analyses of the straight-through arm and of dictionary geometry rely on several targeted diagnostic scripts, detailed where their results appear: `inputgeom.py` (geometry of the normalized input vector passed to each classifier), `blendablate.py` (decoupling a layer's residual input from the sample's own upstream reconstruction), `gainablate.py` (isolating the scalar norm side-channel), `dictgeom.py` (within-head atom collinearity and effective rank computed directly from dictionary weights), `lastlayer.py` (final-layer ablations relative to deep-supervision prefixes), `decodehead.py` (decoding a head's atoms as deviations from their mean), `embedgeom.py` (global geometry of the SONAR embedding cloud), `entryshare.py` (measuring inequality in entry output contributions within a layer and attributing it across usage, gain, and atom norm), and `bigatoms.py` (analyzing how large-norm atoms are distributed across heads and whether the heads containing them reproduce across seeds).

C Supplementary results: the softmax Ontologizer against SAEs

C.1 Categorical form

Evaluated on 1M rows with split-half scoring against 20 size-matched nulls, `g160top1`'s trained partition is strictly head-like (exhaustiveness 1.0000 vs. a null of 0.64, exclusivity 1.0000 vs. 1.51, and sum-CV 0.25 vs. 1.03). Its heads match the structure of the mutually exclusive feature groups that Huang and Hahn hypothesize [51]. Although their synthetic feature groups can also remain inactive on a subset of inputs—a state that `g160top1` can represent because a head whose maximum logit is non-positive receives a router scale of zero, unlike an unrouted head—an exhaustiveness of 1.0000 shows that its heads almost never remain inactive in practice. By contrast, post hoc group discovery on every unstructured SAE run (`m11264_k32`, `m5120_k32`, `k32_p5`, and `m11264_k160`) exhibits the opposite signature, with exhaustiveness below the null (ζ from -30 to -50) and exclusivity above it (ζ from $+13$ to $+31$). Spontaneous group structure in SAEs therefore consists of co-firing topical clusters rather than mutually exclusive categorical slots, even when the discovery affinity explicitly selects for anti-correlated same-context pairs. Meanwhile, `g160softmax` never fires sparsely: no coefficient exceeds $2/32$ on any test row. Instead, soft per-head codes converge to dense, near-uniform mixtures, just as the softmax Ontologizer does (where the mean assignment $\mathbf{o}$ lies within 0.06–0.16 bits of uniform in unfrozen layers, and matching the full soft reconstruction requires roughly 16 of the 32 deviations per head). Thus, categorical structure appears precisely when trained in as hard selection; it does not emerge from softmax competition under a reconstruction objective, and enforcing it across 160 heads increases FVU_w roughly 60-fold (0.274 vs. 0.0044). For steering, this implies that soft classifier codes operate through continuous deviations from $\mathbf{o}$ rather than discrete winning entries, unless hard selection is imposed during training—an approach that Section 6 shows incurs a much smaller reconstruction penalty once the head count is increased. Figure 15 illustrates the structure of one soft head in `resid_nc`.

When hard selection is enforced during training, this categorical structure is directly visible at the individual sample level (Figure 16, `assignmap.py`). On both SONAR and GPT-2, layer-0 heads of the straight-through stack assign each row to a single entry (row 1.0) while distributing the dataset across 14–30 effective entries (use), and rows that share a ground-truth label tend to select the same entry: most Cebuano sentences map to a single entry in each of the three SONAR heads shown, and most GPT-2 newline and space tokens map to a single entry per head. By contrast, a k_{SAE}=32 `TopK` SAE evaluated on the same inputs distributes each row across 25–29 effective active latents and spreads the displayed subset across 1,066–1,533 effective latents overall; there, a label such as GPT-2 newlines appears as a set of co-firing latents rather than a single discrete slot.

C.2 Semantic coherence in detail

Under the whitened inner product that the training objective optimizes, the softmax Ontologizer's mean within-head coherence $\zeta_{\cos}$ is -24.4 (vs. -17.6 unwhitened, with the fraction of heads below -2 unchanged at 0.96), the single-layer soft variant's mean is -14.7 (unchanged), and the single-layer hard variant's mean is $+179.3$ (vs. $+161.3$ unwhitened).

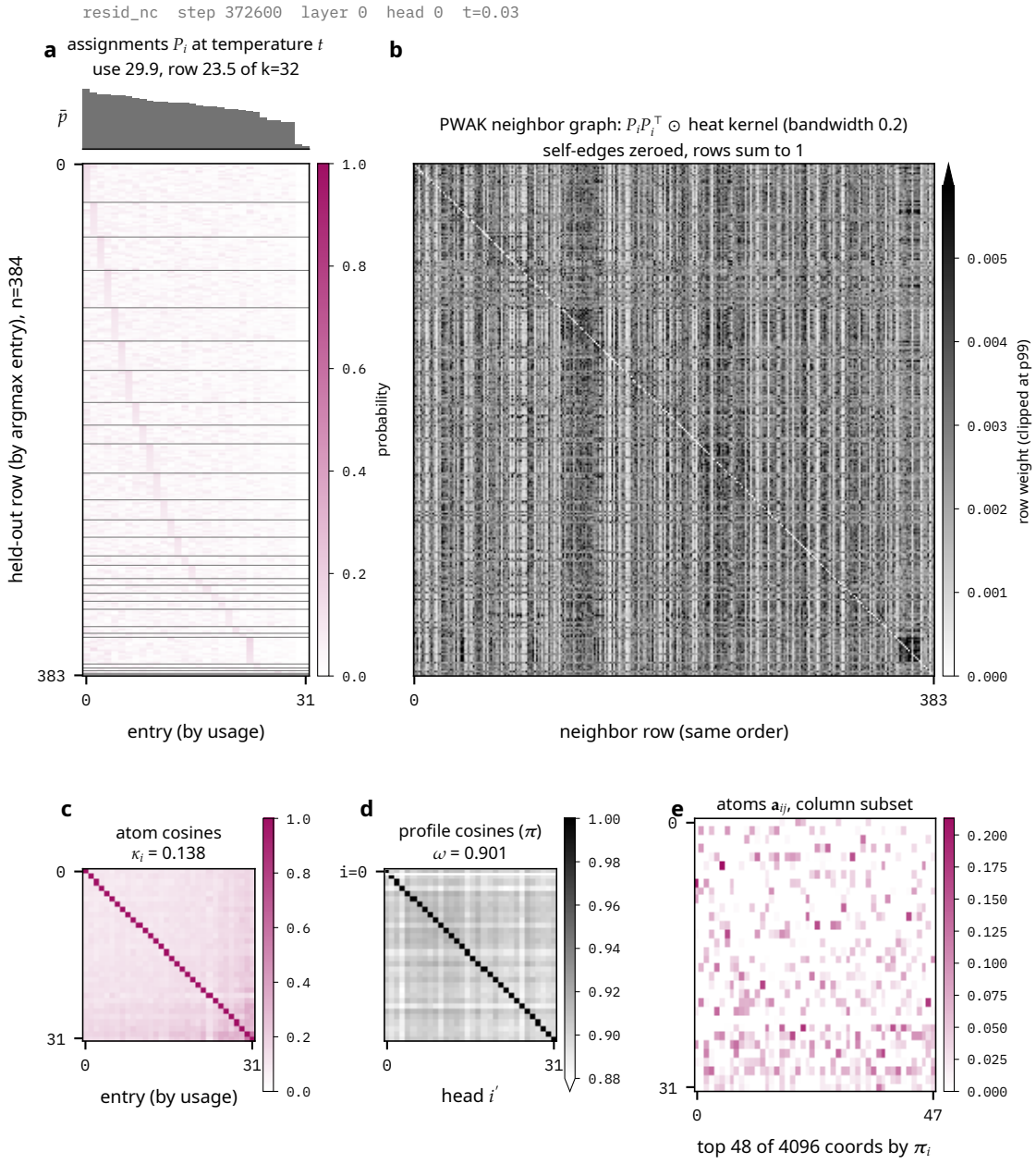


Figure 15 **One softmax head, block by block.** Diagnostic panels corresponding to Figure 4 for head 0 of layer 0 in `resid_nc` (step 372,600, t=0.03, one seed) evaluated on n=384 held-out tail rows. Overall usage is spread across entries (use 29.9 of k=32), but each row receives a near-uniform mixture (row 23.5 in (a)), so the neighbor graph in (b) closely resembles the heat kernel alone, forming hub columns rather than discrete blocks (κ_i=0.138, ω=0.901, e=4096). As in Figure 4, this model was trained without the graph-based auxiliary statistics.

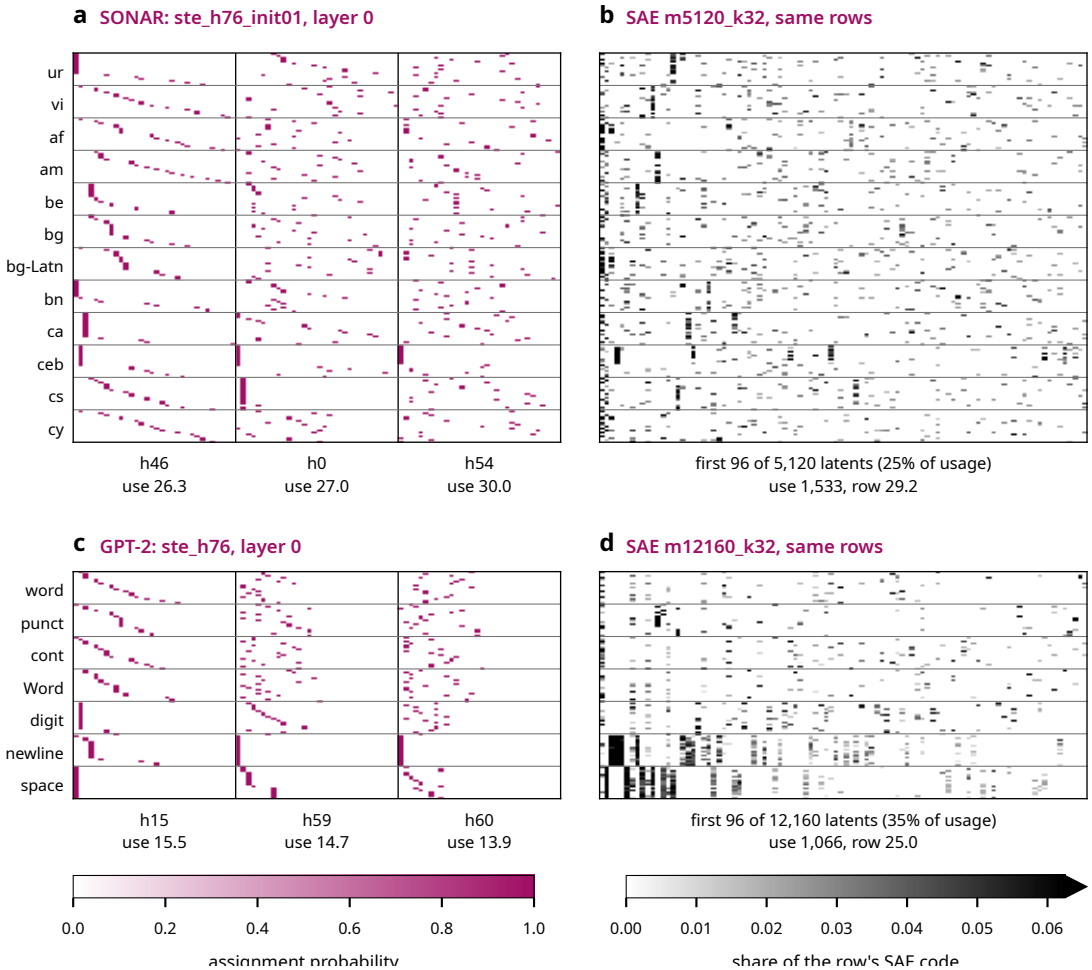


Figure 16 **Hard codes against SAE codes, row by row.** Layer-0 assignments of the straight-through stacks on held-out tail rows (24 rows per label value, one seed per model) for the three heads with the highest null-adjusted NMI with the ground-truth label: (a) SONAR `ste_h76_init01` grouped by language, and (c) GPT-2 `ste_h76` grouped by token class. Entries are sorted by overall usage, and rows are grouped by label and ordered by their winning entries; each head's effective entry usage (use) is reported below, with per-row effective usage (row) equal to 1.0. Panels (b) and (d) show the same rows in the same order encoded by a `TopK` SAE trained on the corresponding cache (`m5120_k32` and `m12160_k32`), displaying each latent's share of a row's code across the 96 latents most used on these rows, with full-code use and row statistics reported below.

Two heads in the reference softmax model (`resid_nc`) exhibit positive clustering rather than repulsion: head 25 in layer 2 ($\zeta_{\cos}$=+346) and head 28 in layer 3 ($\zeta_{\cos}$=+280), followed by heads in the partially frozen layer 4. Sampled entries from these heads also decode to generic text, pointing to partial head collapse—entries that failed to differentiate during training—rather than genuine semantic coherence; we have not inspected them further.

Measuring within-head coherence via text similarity of plain autointerp summaries is uninformative because roughly 77% of Ontologizer entries and 49% of k_{SAE}=32 SAE latents receive generic summaries, inflating the permutation null (yielding ζ=+0.14 for Ontologizer heads). Switching to contrastive descriptions recovers a modest signal: within-head description similarity exceeds the head-membership permutation null for the softmax Ontologizer (ζ=+2.03 across 155 within-head pairs), is marginal for the single-layer hard variant (ζ=+1.55, 62 pairs), and is weaker still for the single-layer soft variant (ζ=+1.09, 217 pairs). Combined with the geometric results, this matches the pattern expected for mutually exclusive values of a shared attribute: decoded entry directions within an Ontologizer head repel one another geometrically while their contrastive descriptions weakly cohere. This metric is also the first to separate the stacked softmax Ontologizer from its single-layer soft counterpart, although the evidence remains suggestive given the limited sample coverage (a single global ζ per model computed over the sampled features).

C.3 Heads as communities of their input

Each head partitions its layer's input distribution into soft cells. To test whether these cells align with local neighborhoods of that input—concentrating more pairwise affinity within cells than expected under a degree-preserving null graph—we compute soft modularity (Eq. (35)) relative to size-matched random regroupings of the same entries (Figure 17, using one checkpoint per model). We compare the softmax Ontologizer `bl_ctl_full` against two Matryoshka SAEs, `m5120_g160top1_p5` (trained heads) and `m5120_k32_p5` (discovered groups), as well as their flat counterparts `m5120_g160top1` and `m5120_k32`.

On the raw input graph itself, none of the models form modular communities: layer 0 of the Ontologizer reaches 1.25× its null modularity, while every SAE sits between 0.76× and 1.06×. When each deeper layer is evaluated on its own residual input, however, layers 1–4 of the Ontologizer achieve 2.1–2.9× the modularity of random entry regroupings. By contrast, Matryoshka SAEs evaluated on matched prefix-residual graphs remain at chance, whether using trained heads (0.96–1.06×) or post hoc discovered groups (0.88–1.03× across blocks with sufficient coverage). Thus, residual modularity does not arise merely from evaluating on residual graphs or from imposing coarse-to-fine prefix losses on a head-structured dictionary. Instead, the key distinction is that each Ontologizer layer directly classifies the residual on which it is evaluated, whereas a Matryoshka block is encoded from the original input and only trained to reconstruct that residual. Because unnormalized modularity increases with depth in both the learned partitions and the nulls (reaching 0.218 vs. 0.105 in layer 4) as assignments sharpen and residual graphs become more clustered, the null-normalized ratio provides the appropriate cross-layer comparison. Finally, low modularity on the raw input is consistent with a factorial representation: a head that isolates one of many independent factors cuts across

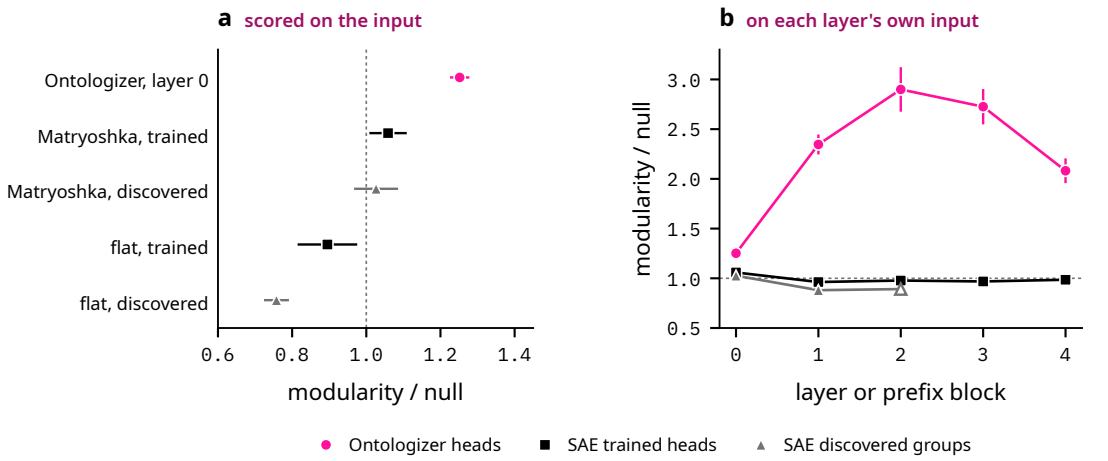


Figure 17 **Ontologizer heads form communities on the residual they classify, whereas SAE heads and groups do not.** Ratio of soft modularity to its size-matched regrouping null (`headstruct.py --modularity`, Eq. (35); one checkpoint per model), with error bars indicating ±2 null standard deviations. (a) Models evaluated on the unreduced input graph: layer 0 of the softmax Ontologizer, block 0 of the two Matryoshka SAEs (evaluated on the centered encoder input $\mathbf{x} - \mathbf{b}_{\text{dec}}$), and the two flat SAEs. (b) Modularity ratio across depth, evaluating each layer or prefix block on the affinity graph of its own residual input. Hollow markers denote blocks whose discovered groups cover fewer than 10% of samples; blocks 3 and 4 for discovered groups are omitted because block 3's null lies within two standard deviations of zero (3% coverage) and block 4 contains only a single group, which its null reproduces identically.

nearest-neighbor structure in the full embedding space, since nearby inputs share most factors without necessarily sharing the one selected by that head.

C.4 Feature reliability

Even in standard SAEs, individual latents are only partially reproducible across random seeds. Comparing two k_{SAE}=32 runs trained with identical architectures and data from different initializations (`k32` vs. `k32_s43`, achieving $\text{FVU}_w = 0.4994$ vs. 0.4967), the fraction of latents with a cross-seed match is 0.31, 0.24, 0.15, and 0.03 at activation-containment thresholds $\tau = 0.3$, 0.5, 0.7, and 0.9, respectively; at moderate containment (τ=0.5), roughly three quarters of individual latents fail to reproduce across initializations. Comparing across dictionary widths (`k32` → `m11264_k32`), 190 of 5,114 active parent latents split at τ=0.7 (with a median of 3 child latents, 0.37 union coverage, and 0.42 child overlap), alongside 441 geometric twins and a heavy-tailed nearest-neighbor decoder-direction cosine distribution (median 0.234 vs. 0.119 for random directions). Feature splitting is therefore present, though topical co-firing remains the more prominent organizational pattern. For an unsupervised steering-vector generator, however, the relevant unit of reproducibility is the head-level partition rather than an isolated feature vector; Section 6.5 evaluates partition reproducibility in the hard-code Ontologizer and shows that it is substantially lower than cross-seed latent matching in SAEs.

C.5 Text-space fidelity (collateral ceiling)

model	chrF	exact	NLL (ceiling 1.560, floor 3.999)
corpus-mean floor	0.176	0	3.999
k32 tier (incl. p5, m11264)	0.23	0.00	3.16–3.18
k32_bl	0.204	0.00	3.365
k160 / g160top1	0.27–0.28	0.00	2.41–2.43
onto soft	0.552	0.070	1.585 (gap 0.025)
g160softmax	0.591	0.094	1.568 (gap 0.008)
k5120	0.841	0.562	1.560 (gap 0.000)

Table 14 Downstream text fidelity on 512 held-out tail rows under forced English decoding. The NLL ceiling (1.560) is shared across all models and corresponds to scoring the unperturbed embedding's decode under the unperturbed embedding.

The relationship between FVU_w and decoded text fidelity is strongly nonlinear: reconstructions at $\mathrm{FVU}_w \approx 0.5$ decode to text that preserves rough topic while altering factual content, scoring only slightly above the corpus-mean floor, whereas both the softmax Ontologizer and `g160softmax` remain close to the likelihood ceiling and preserve nearly all decodable content. Across the eleven Ontologizer runs for which both metrics were evaluated (spanning the selection-rule sweep and the straight-through arms in Table 16), downstream "loss recovered" is strictly monotonic in FVU_w (Spearman rank correlation -1). Text-space fidelity therefore confirms that the whitened embedding objective transfers monotonically to decoded text rather than defining an independent performance axis. This baseline fidelity is particularly consequential for steering: a steering primitive routed through a sparse reconstruction that fails to preserve text (including the entire $k_{\mathrm{SAE}} \leq 160$ SAE tier) starts from roughly 0.75 collateral before any intervention is applied, whereas the dense-mixture classifier codes start near zero.

C.6 Concept (language) localization

Language identity remains distributed across features in every model (Table 15): the highest single-latent macro F1 is 0.121, compared to a dense-probe ceiling of 0.676 (script-distinctive languages such as Chinese reach roughly 0.98 from a single latent, whereas languages sharing a script overlap heavily). Individual entry probabilities in the softmax Ontologizer do not localize language at all, remaining at the always-positive baseline at both 1 and 4 entries. By contrast, the hard-code models' one-hot entries match the best SAE at a single entry (0.121 and 0.120 vs. 0.119), though they trail at 4 and 16 entries. Under a single-feature SAE lens, the soft code's baseline score is an indictment; under a categorical-classifier lens it is expected, since a soft head encodes a factor in its *assignment* distribution rather than in any single entry's scalar probability. Consistent with this view, partition-level analysis of `resid_nc` identified layer-1 heads whose partitions carry mutual information with script and language family, including one entry (entry 17 of head 8 in layer 1) causally verified to produce Burmese decodes. Evaluating head partitions directly with `headlang.py` on a configuration-matched softmax reference and on the hard-code arms (Section 6.6) shows that the softmax reference's strongest head reaches

	86 languages			five languages, F1@1		
model	F1@1	F1@4	F1@16	own tags	it pt nl ja	ca ro sv ko
k160	0.119	**0.393**	**0.473**	0.684	0.841	0.841
k32 (m=5120)	0.089	0.347	0.465	0.624	0.868	0.876
m11264_k32	0.078	0.364	0.471	0.551	0.860	0.871
k32_bl	0.084	0.228	0.368	0.695	0.840	0.786
k32_p5	0.061	0.322	0.456	0.625	0.862	0.870
g160top1	0.066	0.207	0.302	0.592	0.542	0.700
g160softmax	0.067	0.162	0.328	0.432	0.437	0.433
k5120	0.064	0.130	0.237	0.439	0.448	0.447
softmax Ontologizer (entries)	0.022	0.022	0.135	0.333	0.333	0.333
hard-code stack (entries)	**0.121**	0.153	0.362	0.685	0.804	0.806
hard-code flat (entries)	0.120	0.141	0.319	0.609	0.798	0.703

Table 15 Language localization (`langprobe.py`): held-out one-vs-rest macro F1 of the single best latent and of logistic probes on the best 4 and 16 latents across tail rows. The first three columns evaluate all 86 languages, where the always-positive baseline scores 0.023 and a dense linear probe on the raw embedding scores 0.676. The final three columns report single-latent macro F1 within three five-language subsets, where the always-positive baseline scores 0.333: English, French, Spanish, German, and Chinese, which were encoded under their own SONAR source tags (Appendix B.1), alongside two composition-matched subsets encoded under the English tag. The softmax Ontologizer is `resid_nc`, and the hard-code models are `ste_h76_init01` and `ste_l1_h380_i01`.

a null-corrected NMI of only 0.043, roughly one quarter of the hard code's peak. Even in the hard-code stacks, language information is distributed weakly across many heads' partitions rather than concentrated in a few: across both seeds, the 76 most language-informative heads out of 380 are precisely the 76 heads in layer 0 (`headlang.py`), with the strongest head reaching roughly one fifth of the maximum attainable NMI, every deeper head scoring below all layer-0 heads, and removing the top 32 heads leaving most of the recoverable language signal intact in the rest of the code (Appendix E.13).

What the English tag does to localization. Because most of the SONAR cache was encoded under the English source-language tag (Appendix B.1), this tagging artifact strongly shapes Table 15. Each `TopK` SAE learns a single latent that fires jointly on French, Spanish, and German—the three Latin-script languages encoded under their own source tags—separating them from the English-tagged remainder without distinguishing them from one another (yielding F1@1 near 0.49 for each, vs. 0.03–0.15 for Italian, Portuguese, Dutch, Catalan, Romanian, and Swedish). When evaluation is restricted to the five correctly tagged languages, neither French nor German has a dedicated latent or entry in any model (best F1@1 of 0.39–0.50 vs. the 0.33 baseline), whereas English (0.64–0.96) and Chinese (0.86–0.997 in the sparse SAEs and hard codes) do. Moreover, five-language subsets of matching composition that were encoded under the English tag achieve higher single-latent localization (macro F1@1 of 0.70–0.88 vs. 0.55–0.70 for the correctly tagged subset) in every model except `g160top1` and the dense codes, even though a

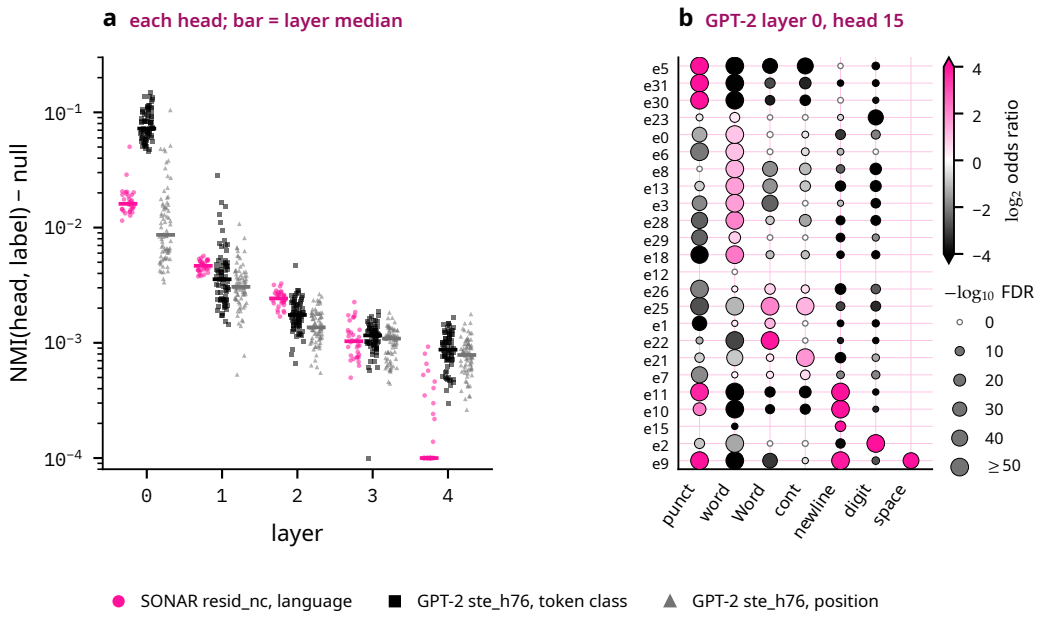


Figure 18 **Label structure is concentrated in layer 0.** (a) NMI between each head's winning entry and a ground-truth label on the held-out tail, minus the mean across five label-shuffled permutations, showing one point per head (values at or below the null mean are plotted at 10^{-4}) and horizontal bars at each layer's median: SONAR `resid_nc` against language, and GPT-2 `ste_h76` against token class and token position. (b) Enrichment across entries and token classes for the GPT-2 layer-0 head with the highest token-class NMI (head 15): marker color encodes the Haldane-corrected $\log_2$ odds ratio, marker area encodes $-\log_{10}$ FDR (capped at 50), and filled circles indicate FDR < 0.05, with entries grouped by the token class for which they are most enriched. One seed per model.

dense linear probe separates the correctly tagged subset at least as accurately (0.95–1.00 vs. 0.92–0.98), indicating that each English-tagged Latin-script language carries a distinct signature. Consequently, most single-latent localization among Latin-script languages in this cache reflects source-tag mismatch rather than intrinsic language separation; whether dictionaries trained on consistently tagged embeddings would allocate dedicated latents to those languages remains untested.

Entry-level enrichment tests the same relationship at the granularity of individual (entry, label) cells (Figure 18, `enrich.py`). In `resid_nc`, 3,698 of 226,763 tested (entry, language) cells are statistically significant at FDR 0.05, and label alignment is concentrated almost entirely in layer 0: the highest null-adjusted head NMI is 0.050 in layer 0, compared to 0.006, 0.003, 0.002, and 0.001 across layers 1–4. The GPT-2 hard code exhibits the same concentration in layer 0 for both token class (0.149 in layer 0 vs. at most 0.029 in deeper layers, with 11,777 of 53,412 tested (entry, token class) cells significant) and token position (0.105 in layer 0 vs. at most 0.011 in deeper layers).

C.7 Shrinkage decomposition

Standard `TopK` SAEs exhibit very small magnitude error: refitting unconstrained least-squares coefficients on each sample's frozen active support reduces pre-refit FVU_w by only 2.2–2.4% for the k_{SAE}=32 variants, 8.5% for `k160`, 0.6% for the bilinear SAE (`k32_bl`), and 1.9% for the Matryoshka SAE (`k32_p5`). Similarly, `g160top1` is nearly perfectly calibrated (0.3% reduction), indicating that its reconstruction gap stems purely from support selection. For the softmax Ontologizer's truncated deviation codes, support-frozen refitting removes 16% of the error at ν=1 and 33% at ν=2, about the fraction $|\mathcal{S}|/d$ (160/1024 and 320/1024 across `resid_nc`'s 5×32 heads) that any $|\mathcal{S}|$ unconstrained coefficients would remove from an isotropic whitened residual. Size-matched random supports remove more error than the models' chosen supports: 19% and 36% for the Ontologizer (ν random entries per head), and 20%, 13%, and 4.6% for `k160`, `g160top1`, and `k32`, compared to 8.5%, 0.3%, and 2.2% on their own supports (`refit.py`; supports of size $\geq d$ are excluded as underdetermined). Because no model's active support benefits more from refitting than a random support of equal size, there is no evidence that the pin-and-renormalize truncation rule mis-scales the retained entries.

Separately, comparing non-adaptive truncation (freezing the soft assignments P from a clean forward pass, truncating them, and decoding linearly) against in-forward truncation (allowing deeper layers to reclassify the residual left by truncated earlier layers) reveals a large gap at ν=8, reducing FVU_w from 0.268 to 0.094—nearly a threefold improvement from downstream compensation. Alongside the larger depth effect measured in Section 6.3, this adaptive re-encoding has direct implications for steering: deeper layers actively compensate for edits to earlier codes, providing robustness to small perturbations while also inducing widespread downstream collateral flips (Section 6.4).

C.8 Steering through the text round trip

Table 16 reports text-space reconstruction fidelity and text-cycle steering across the Ontologizer selection-rule sweep, the straight-through arm, and the k_{SAE}=32 SAE, evaluated on 64 features per run.

Text-cycle steerability does not track reconstruction fidelity. The strongest steerer is `top8` ($\mathrm{FVU}_w = 0.037$), whereas the most faithful reconstruction model (`softmax_shm`) achieves roughly one third of its net steering effect, and every `_shm` variant steers roughly three times more weakly than its twin (`top2` 0.509 vs. `top2_shm` 0.196; `top8` 0.652 vs. `top8_shm` 0.176). The straight-through arm (`ste_h76`) exhibits the weakest response of all: its unadjusted effect remains flat across step magnitudes (0.018, 0.014, and 0.027) and no larger than a random perturbation, while its hit rate (0.055) falls below its random-direction control (0.125). Under the same protocol, the k_{SAE}=32 SAE shows a clear text-cycle effect (net effect 0.402 at magnitude 1.0, with a hit rate of 0.260 vs. 0.016 for random directions).

Two methodological caveats limit this evaluation. First, sampling 16 features is too noisy to support reliable estimates, and even at 64 features the straight-through arm's net effect changes sign depending on the random control draw (shifting from negative at all magnitudes on 16 features to +0.033, +0.045, and −0.020 at 64 features), indicating that its text-cycle signal lies within measurement noise. Second, hit rates are not directly comparable between hard and

run	FVU_w	chrF	loss rec.	net @0.25	@0.5	@1.0	hit @1.0	rand. hit
top8	0.0370	0.416	0.941	0.203	0.404	**0.652**	0.734	0.359
top2	0.3516	0.246	0.521	0.072	0.277	0.509	0.578	0.109
top16	0.0052	0.557	0.987	0.027	0.110	0.363	0.381	0.156
top4_shm	0.0761	0.375	0.892	0.112	0.140	0.248	0.242	0.062
softmax_shm	0.0007	0.706	0.997	0.058	0.128	0.242	0.211	0.094
top2_shm	0.1911	0.301	0.742	0.018	0.071	0.196	0.150	0.016
top8_shm	0.0287	0.437	0.954	−0.015	0.074	0.176	0.330	0.094
top1	0.7190	0.170	0.126	0.047	0.104	0.064	0.102	0.109
`ste_h76`	0.2293	0.295	0.695	0.033	0.045	−0.020	0.055	0.125
SAE `k32`	0.4994	0.228	0.345	0.048	0.187	0.402	0.260	0.016

Table 16 Text fidelity and text-cycle steering (`steerfid.py`, `--n-features 64`) across Ontologizer selection rules and the k_{SAE}=32 SAE under the same protocol. Net effect is feature effect minus the random-direction control. An additional run (`topk4_rev`, $FVU_w = 0.164$) achieves a net effect of 0.636 at magnitude 1.0.

soft codes: because hard-code activations are binary (0 or 1), recording a hit requires that the re-encoded argmax land on the exact target entry, whereas a soft code need only exceed a fractional activation threshold.

The text cycle is too weak to support conclusions. Evaluated on an *unsteered* round trip across the final 512 cache rows, decoding and re-encoding preserves partial semantic similarity to the source row ($\cos \angle(\mathbf{x}, \text{cycle}(\mathbf{x})) = 0.382$, vs. 0.091 to another row's cycle) while shifting embeddings away from the corpus distribution (mean cosine with the corpus mean drops from 0.561 on clean rows to 0.160 after cycling). Crucially, the round trip fails to preserve discrete head assignments: a head's winning entry survives an unsteered cycle on only 12.9% of rows in `ste_h76` and 10.8% in the softmax model, compared to 3.1% by chance, and in the hard code this survival rate drops sharply with depth from 36.2% in layer 0 to 4.8% in layer 4. Even an ideal code-level intervention could therefore register on at most roughly 13% of cycled examples, which is on the same order as the hit rates reported by this evaluation. Several aspects of the cache protocol make this round trip unusually lossy: cache rows correspond to 512-token document openings—exceeding the sequence length at which SONAR auto-encoding degrades—and are decoded into English regardless of source language through a generator that is sensitive to small embedding perturbations [33]. Thus, whereas embedding-space reconstruction remains reliable, text generation obscures most of the code-level signal.

D Supplementary results: training, reproducibility, and steering of the softmax Ontologizer

The experiments in this section originate from the project proposal's evaluation suite and examine the training dynamics, cross-seed reproducibility, and steering behavior of the softmax Ontologizer. Two models appear throughout: `resid_nc` (trained for 372,600 steps), which serves

as the reference softmax model in the comparisons above, and `resid_nc_hm`, which differs from it in three settings (Table 9): half the dictionary width (e=2048 rather than 4096), the batch-mean uniformity term KL_m (weighted by s_{KL_m}) from the start of training, and the weight on the retired between-head cosine. Unless stated otherwise, evaluations of `resid_nc_hm` use its 4.38M-step checkpoint, called its final checkpoint below; the run's latest checkpoint is at 8.27M steps.

D.1 Head freezing and the batch-mean uniformity term

We classify a head as frozen when its top classification share (the fraction of samples its modal entry wins) is at least 0.95 and its assignment churn is at most 0.02. In `resid_nc`, the final layer (layer 4 of 0–4) begins freezing near step 190k—well after temperature annealing concludes—reaching 27 of 32 heads by step 210k, peaking at 28, and remaining frozen at the final checkpoint (Figure 19). As shown in Section G.4, this final layer becomes largely inert once the prefix residual error is small, explaining why the freeze incurs little reconstruction penalty. In `resid_nc_hm`, by contrast, this late-stage freeze never occurs. Instead, the model undergoes only a brief early annealing transient—peaking at step 10k with 22 of 32 frozen heads in layer 1, alongside 16, 11, and 4 of 32 in layers 2, 3, and 0—that resolves completely by step 30k, leaving all layers unfrozen at the final checkpoint. Including the batch-mean uniformity term KL_m therefore prevents late-stage head freezing.

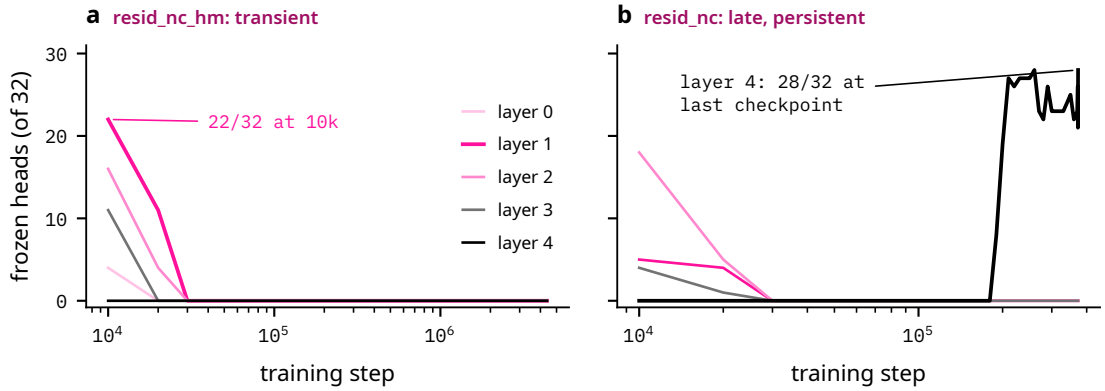


Figure 19 Frozen heads per layer over training. (a) `resid_nc_hm`, trained with the batch-mean uniformity term KL_m, exhibits only an early annealing transient, peaking at 22/32 frozen heads in layer 1 at step 10k and recovering to zero across all layers by step 30k. (b) `resid_nc` develops a persistent post-annealing freeze in layer 4 starting near step 190k that continues through its final checkpoint.

Four 372k-step retrains of the `resid_nc_hm` configuration—an unmodified control and three single-hyperparameter variants—isolate the source of the early transient (Figure 20). The control reproduces the transient closely (22/32 frozen heads in layer 1 at step 10k, recovering to zero everywhere by step 30k) and exhibits no late freeze. Extending the temperature annealing schedule from 50k to 150k steps lowers and broadens the transient (peaking at 14/32 in layer 1 at step 30k and clearing by step 50k). By contrast, raising winner dropout from 0.1 to 0.25 tracks the control to within two heads at the peak, and holding the logit-noise floor constant at its initial value leaves the trajectory unchanged. Thus, the early transient is governed by the temperature schedule alone, and every retrain finishes completely unfrozen.

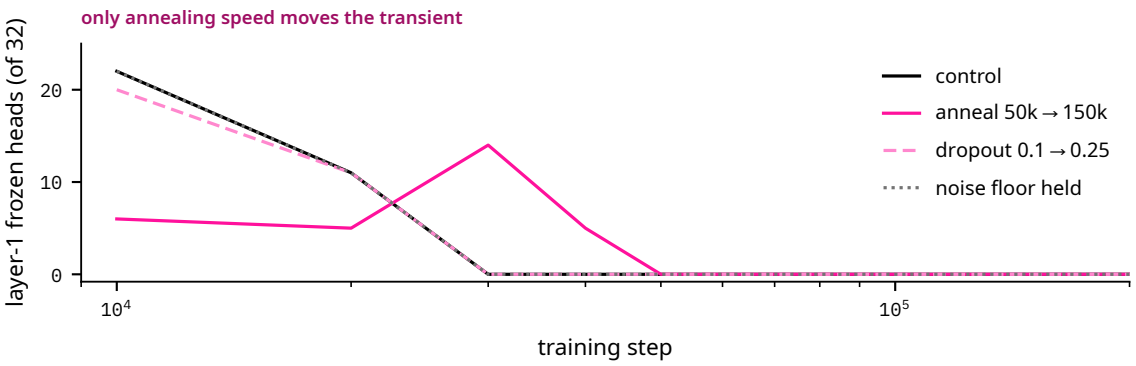


Figure 20 Layer-1 frozen head counts during the early transient across four 372k-step retrains of `resid_nc_hm`. Holding the logit-noise floor constant overlaps the control curve identically, and raising winner dropout tracks it to within two heads at the peak; only extending the temperature annealing schedule shifts the curve, trading peak height for duration.

An HSIC penalty in place of the auxiliary losses. These variants are motivated by the HSIC bottleneck [120], which trains layers without backpropagation to minimize dependence on their inputs while maximizing dependence on ground-truth labels, both measured by normalized HSIC. Because an autoencoder's target is its own input, those two objectives collapse into one and neither is used directly here; instead, reconstruction continues to optimize whitened MSE while a single kernel penalty discourages statistical dependence among heads within a layer. Two 372k-step arms on the `resid_nc_hm` configuration penalize inter-head dependence via the pairwise head CKA of Section E.9 computed under the biased estimator [121]: one arm replaces the four standard auxiliary terms with the CKA penalty, and the other adds the CKA penalty to them. Replacing the auxiliary terms severely degrades head utilization: a persistent late freeze overtakes 77 of 160 heads by the final checkpoint, including all 32 heads of layer 3 from step 80k onward, while tail-mean training MSE lands slightly behind the control (3.7×10^{-5} vs. 3.4×10^{-5}). Adding the penalty alongside the auxiliary terms performs worse on both metrics, yielding a training MSE of 4.4×10^{-5} and leaving 41 of 160 heads frozen at completion after layer 3 freezes completely by step 80k and only partially recovers (Figure 21).

This failure reflects the late-stage freezing mode seen in `resid_nc` rather than the early annealing transient, and it stems directly from estimator bias. Because the biased CKA floor between independent heads decreases with the number of active entries and vanishes for a frozen head (Section E.9), the optimizer can reduce the penalty simply by concentrating or freezing heads, placing the penalty in direct opposition to the batch-mean uniformity term KL_m. Retrained on the current codebase, the replacement arm reproduces this freeze step for step (freezing 6, 29, and 32 of 32 heads in layer 3 at steps 60k, 70k, and 80k, vs. 7, 29, and 32 originally) while its summed CKA penalty drops by 80%. Under the unbiased estimator, by contrast, the same arm freezes at most 5 heads—all in layer 4—through 93k steps (one seed per arm). Once estimator bias is removed, however, the inter-head CKA penalty has very little residual dependence to act upon, and whether it could replace the standard auxiliary terms over a full training run remains untested.

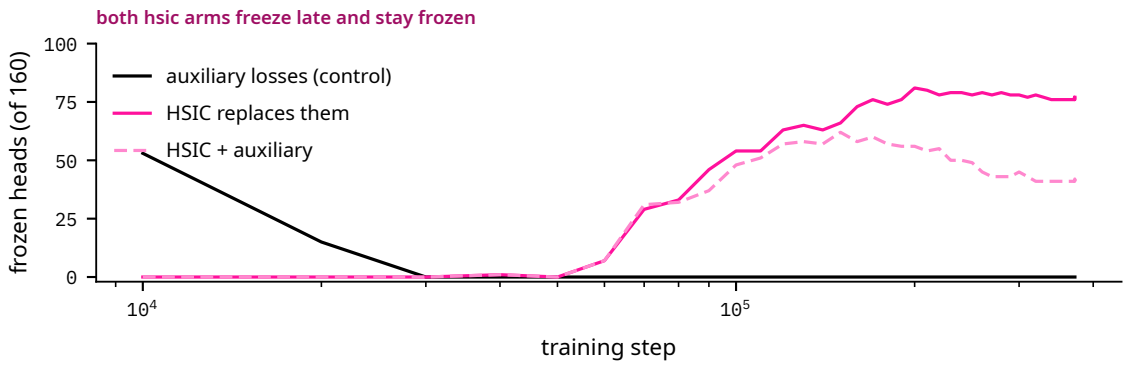


Figure 21 Total frozen heads across all layers over 372k training steps for three loss configurations based on `resid_nc_hm`. The auxiliary-loss control exhibits only the brief early transient before recovering, whereas both HSIC arms—whose head-CKA penalty uses the biased estimator—develop a persistent late-stage freeze that the standard auxiliary terms only partially mitigate.

D.2 Development over training

Tracking `resid_nc_hm` across all 448 saved checkpoints applies the z-score coherence evaluation of Section 5.3 to least-squares refitted end-to-end directions rather than to raw decoded dictionary atoms. These two quantities are not directly comparable: raw decoded atoms in `resid_nc` repel one another ($\zeta_{\cos}$=−17.6), whereas refitted end-to-end directions at the final checkpoint of `resid_nc_hm` cohere strongly ($\zeta_{\cos}$=+72), reflecting differences in both model and measurement (since the end-to-end refit incorporates the shared decoder and cross-layer interactions). Moreover, because a head's directions are determined by the model's function only up to a shared per-head offset (Section 5.3), the trained parameters and a least-squares refit fix that offset differently. Under the refitted metric, coherence rises, peaks, and partially relaxes over training (Figure 22): mean within-head $\zeta_{\cos}$ increases from 1.8 at step 10k to a peak of 119 near step 370k (median 123, with every head exceeding $\zeta_{\cos}$=2), before relaxing over the subsequent 4M steps to 72 at the final checkpoint (median 105, with a fraction 0.79 of heads above $\zeta_{\cos}$=2). Meanwhile, the assignments themselves move in the opposite direction once the early annealing transient resolves: mean per-sample entropy returns to 4.5 out of the 5-bit maximum, and batch-mean KL divergence approaches zero. Relative to initialization, training therefore consolidates the geometry written by the assignments while the assignments themselves remain soft.

Evaluated at the individual entry level on a single seed of `resid_nc`, dictionary directions stabilize after temperature annealing concludes (Figure 23, `entrydrift.py`). Across layers 0–3, the median cosine similarity between an active entry's decoded atom and its final-checkpoint direction rises from 0.40–0.51 at step 10k to 0.73–0.82 at step 60k (immediately after annealing), reaching 0.99–1.00 from step 320k onward. By contrast, layer 4 begins at 0.23, plateaus near 0.53 between steps 60k and 160k, and converges only after freezing near step 190k (Section D.1), retaining 142 active entries out of 1,024. Entry utilization instead dips temporarily at the end of annealing: 40% of layer-0 entries fall below the usage threshold at step 60k, though all but 7% recover by the end of training.

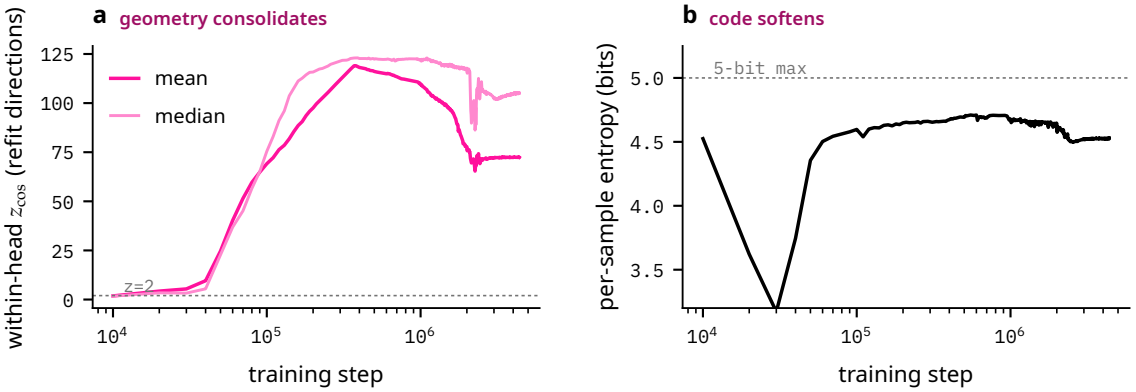


Figure 22 Training trajectory across all 448 saved checkpoints of `resid_nc_hm`. (a) Within-head coherence of refitted end-to-end directions, standardized against size-matched nulls, showing an initial rise, a peak near step 370k, and subsequent partial relaxation. (b) Mean per-sample assignment entropy, which returns toward the 5-bit maximum following the early annealing transient.

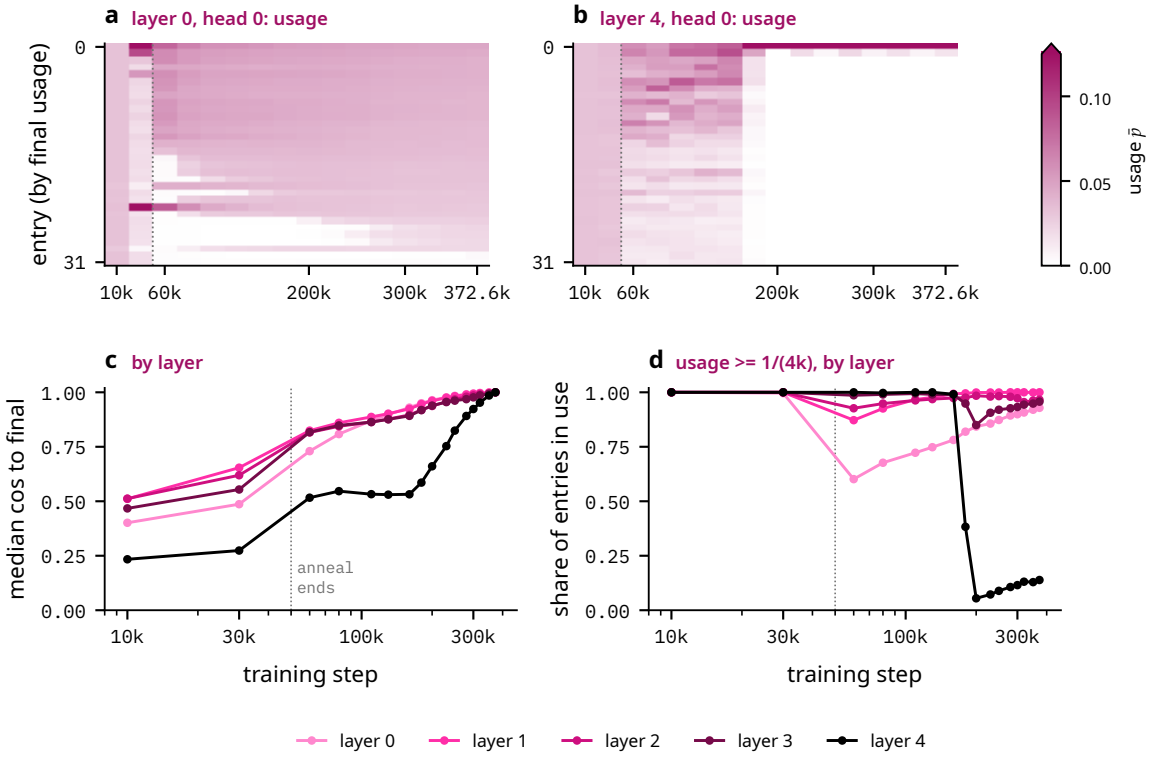


Figure 23 **Entries stabilize after temperature annealing.** Trajectory across the 16 retained checkpoints of `resid_nc` on the held-out tail (one seed). (a, b) Mean usage $\bar{p}_{ij}$ across the 32 entries of head 0 in layers 0 and 4, with one column per checkpoint (not spaced linearly in step) and entries ordered by final usage; the vertical dotted line marks the end of temperature annealing. (c) Median cosine similarity per layer between each active entry's decoded atom and its final direction, computed over entries active at the final checkpoint. (d) Fraction of active entries per layer ($\bar{p}_{ij} \geq 1/4k$).

D.3 Scaling: a star sweep

To test whether within-head coherence improves with model scale, we trained a seven-point star sweep around the baseline h=32, k=32, l=5 architecture, varying one hyperparameter axis at a time for 4 epochs (62k steps, vs. 372k in the standard retrain schedule). None of the swept axes produce an increase in coherence (Figure 24): coherence shows no positive association with parameter count (Spearman correlation +0.14 for mean $\zeta_{\cos}$ and −0.54 for the fraction of heads above $\zeta_{\cos}$=2), entry-usage imbalance increases with scale (Spearman +0.86), and the clearest single-axis trend runs opposite to the hypothesis, with mean $\zeta_{\cos}$ decreasing monotonically as the number of entries per head grows (20.2, 6.3, and 3.7 at k = 16, 32, and 64). The widest run (h=64) fails to converge within this training budget (exhibiting a tail MSE 20× higher than the other runs), so its low score is uninformative. Overall, seven configurations trained on an abbreviated schedule provide limited evidence in either direction; full-length scaling runs remain a necessary control.

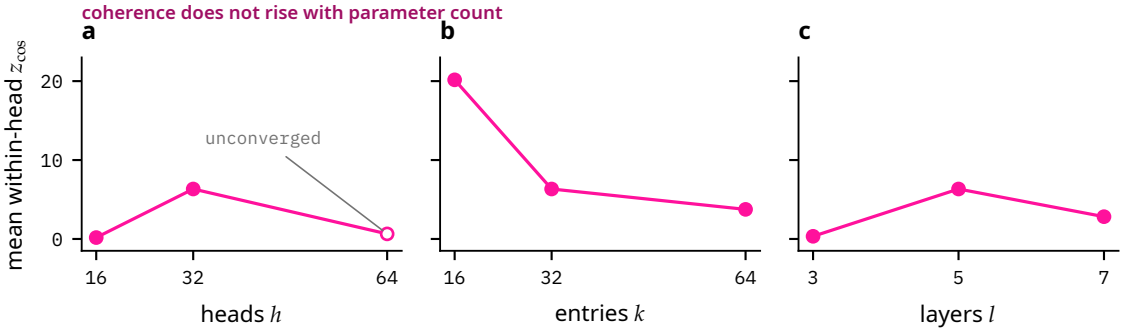


Figure 24 Within-head coherence across each axis of the 4-epoch (62k-step) star sweep: (a) heads h, (b) entries k, and (c) layers l. None of the axes exhibit the predicted increase with scale, and coherence decreases monotonically with k. The open marker in (a) denotes the unconverged h=64 run.

D.4 Seed stability of soft partitions

Comparing a second independent seed of the `resid_nc_hm` configuration against the primary run at a matched training budget (370k steps each) reveals almost no cross-seed correspondence (Figure 25): no pair of heads and at most 0.3% of individual atoms exceed a 0.3 cosine threshold in dictionary geometry, and activation-frame containment over a shared evaluation batch reaches at most 0.001 across all thresholds, compared to 0.31, 0.24, 0.15, and 0.03 between the k_{SAE}=32 SAE (k32) and its seed replica at τ = 0.3, 0.5, 0.7, and 0.9. This near-zero match is not due to metric insensitivity: comparing the primary run against an earlier checkpoint of itself 2,600 steps apart yields 1.000 geometric matching at every threshold and 0.65 activation containment at τ=0.3 (0.20 at τ=0.9). At matched training steps, soft Ontologizer partitions are therefore less reproducible across seeds than individual SAE latents, consistent with the partition-NMI and output-contribution results for the straight-through arm in Section 6.5. Training a second seed to the full 4.4M-step horizon remains an open control.

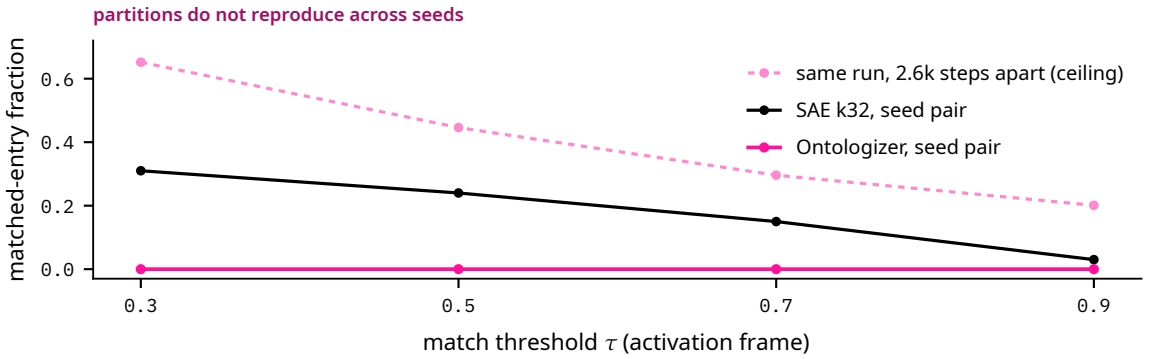


Figure 25 Cross-seed matched-entry fraction under activation-frame containment across thresholds τ. Comparing checkpoints from the same run establishes the metric's detection ceiling (0.65 at τ=0.3); the k_{SAE}=32 SAE pair reproduces partially, whereas the softmax Ontologizer's entries show virtually no cross-seed matching.

D.5 Steering against supervised baselines

On the 4.38M-step checkpoint of `resid_nc_hm` (step 4,380,000), a matched-budget language-steering evaluation compares forced entries against supervised difference-of-means and linear-probe vectors alongside a random-direction baseline (Table 17). As expected, each arm wins the effect frame defined by its own direction. Along the shared difference-of-means direction, forcing an entry shifts decoded embeddings 0.64× as far as supervised difference-of-means steering itself and 4.2× as far as linear-probe steering. However, none of the methods reliably reach the target language distribution: the median hit rate is 0.06 for difference-of-means steering and 0 for the remaining arms. Moreover, every method—including supervised steering—incurs collateral text changes comparable to a random perturbation of equal norm (0.70–0.78), matching the baseline degradation imposed by the text round trip on SAE steering (Section C.8). Decoded text similarity to the unsteered baseline decays similarly across all four arms as intervention magnitude increases (Figure 26): chrF drops from roughly 0.37 at magnitude 0.25 to roughly 0.15 at magnitude 1.0 for both difference-of-means steering and entry forcing, while linear-probe and random perturbations decay in parallel from 0.38–0.40 to 0.19–0.20. A blinded 120-item human-evaluation sheet has been prepared for fluency grading but has not yet been scored.

arm	effect (native)	effect (dm frame)	hit rate	collateral
difference of means	0.474	0.474	0.062	0.752
entry forcing	0.399	0.302	0.000	0.776
linear probe	0.294	0.072	0.000	0.701
random direction	—	—	—	0.705

Table 17 Language steering under matched intervention budgets on `resid_nc_hm` at step 4,380,000, reporting medians across 24 language–magnitude conditions per arm. Effect measures displacement along each arm's own direction (native) and along the shared difference-of-means direction (dm frame); hit rate is the fraction of steered decodes reaching the median projection of genuine target-language rows on the shared frame; and collateral measures content change on unrelated dimensions.

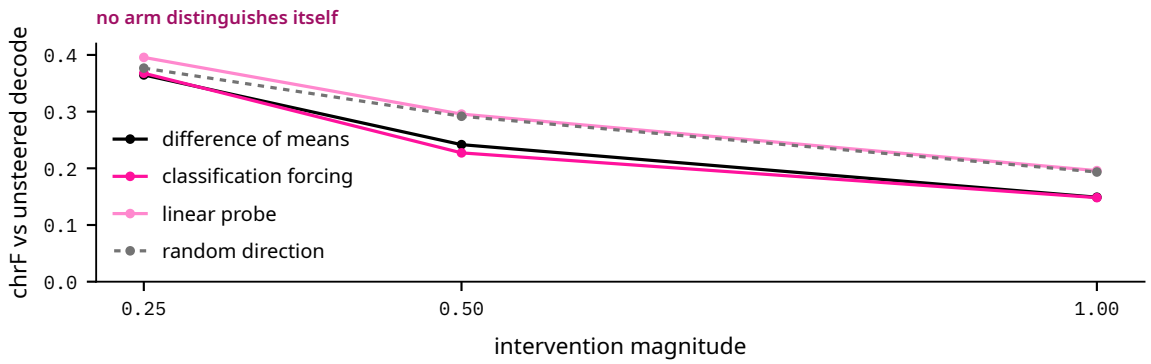


Figure 26 chrF similarity between steered and unsteered decodes as a function of intervention magnitude. All four arms—including random perturbations—degrade decoded text at nearly the same rate, with no method preserving substantially greater similarity at matched magnitude.

An entry score as a training objective. Training a policy over steering directions on frozen `resid_nc_hm`, rewarded by a single entry's quantile-normalized score under a fixed perturbation budget, converges rapidly in the differentiable embedding-space regime, saturating within roughly 40 gradient-ascent steps (Figure 27). Because gradients pass directly through the frozen classifier without text decoding, this regime serves only as an upper bound for decode-cycle optimization—where rewards pass through discrete text generation and require policy-gradient estimation such as REINFORCE [122]—and does not indicate how quickly REINFORCE would converge. Collateral variation in this setup is mechanically bounded as well: the policy adds a direction of fixed norm 0.5 to a unit-norm embedding and renormalizes, which caps the collateral at 0.134, and observed collateral decreases slightly from 0.105 to 0.083 as the policy tilts its direction toward the row and so shortens its effective post-normalization step. More broadly, additive output-space perturbations correspond to the residual intervention form that latent-space steering of diffusion policies improves upon by acting on the generator's input distribution [123], and optimizing inputs directly against a frozen classifier can exploit off-manifold directions along which natural data carry negligible variance [3]. Neither the decode-cycle REINFORCE experiment nor a convergence comparison against paired steering (the model's own set-intervention) has been conducted.

D.6 Head-level autointerp

Because single-feature autointerp metrics evaluate individual directions rather than contrastive partitions, they could in principle understate the interpretability of classifier heads; we test this hypothesis directly with a head-level evaluation on `resid_nc` at step 372,600 (noting the caveat that layer-4 heads in this checkpoint are largely frozen; Section D.1). Partition descriptions generated from decoded dictionary parameters alone fail completely, matching the per-entry result (median accuracy 0.000, identical to the null). Activation-grounded head descriptions also fail to outperform a random null pairing at this sample size (20 heads per condition; median accuracy 0.083 vs. 0.125 for the null, where guessing one value per head scores roughly 0.11). Furthermore, head-level detection accuracy is uncorrelated with the mean describability of a

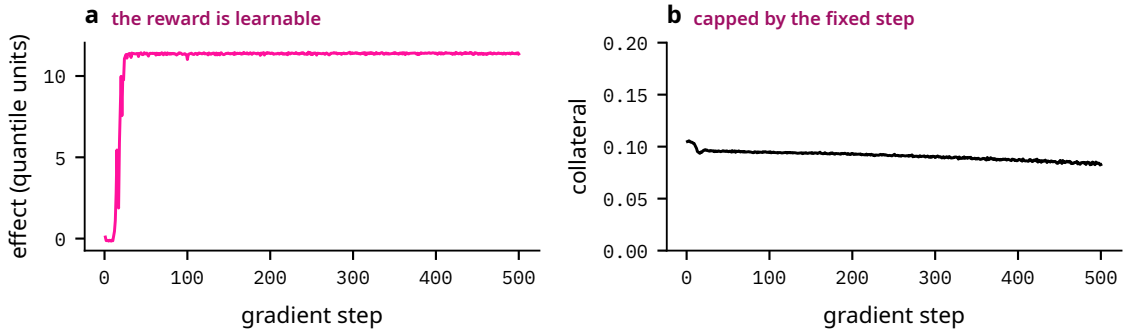


Figure 27 A steering policy trained in embedding space by gradient ascent on a single frozen entry's quantile-normalized score in `resid_nc_hm`. (a) The policy saturates the reward within roughly 40 steps. (b) Collateral—bounded above at 0.134 by the fixed perturbation norm—declines from 0.105 to 0.083.

head's constituent entries (Spearman +0.12 across 30 head–mode pairs: +0.25 with p=0.37 for activation descriptions and +0.05 with p=0.86 for parameter descriptions, 15 heads each). The two modes disagree, and neither yields above-chance head-level detection, so switching from per-entry to head-level scoring does not uncover an interpretability advantage in this sample.

Describability and activation density. Across the six models in Table 4, log nominal activation density correlates at −0.894 with activation-description F1, accounting for roughly 80% of the cross-model variance. Fitting a linear regression of contrastive-description F1 on log empirical firing rate across the four `sae.py` runs with nonzero thresholded firing rates (slope −0.069, correlation −0.766) while holding out the softmax Ontologizer predicts an F1 of 0.421 vs. the observed 0.392 (a residual of −1.8 standard errors); thus, the Ontologizer lies close to the empirical density trend whereas `g160top1` sits 0.120 above it. However, this four-point regression is fragile: the underlying correlation is modest, `g160softmax` is excluded because its firing rate above the $2/k$ threshold is identically zero, and that fixed threshold does not vary monotonically with effective activation density.

Describability and language concentration. Tabulating the number of distinct languages represented among each feature's 15 evaluated rows (`autointerp.py language`; Table 18) separates the sparse models from the dense ones before any text description is generated. Within every sparse model, detection F1 declines sharply as a feature spans more languages (Spearman −0.47 to −0.67 for activation descriptions), whereas within the dense models it shows no trend (−0.06 to +0.03). In `k32` and `g160top1`, a feature's modal language accounts for 28–30% of the held-out positive examples evaluated by the judge, compared to roughly 1% of the negative examples and contrastive background draws. Restricting the log-firing-rate regression of contrastive F1 to multilingual features spanning 13–15 languages reduces `g160top1`'s residual above the trend from +0.120 to +0.018 and shifts the softmax Ontologizer from 0.029 below the trend to +0.049 above it. Across individual features, modal-language purity explains roughly one fifth of the variance in detection F1 (0.20 for activation descriptions and 0.22 for contrastive descriptions), whereas log firing rate explains only 1–4%. Controlling for

language identity directly requires language-matched negative and background examples—implemented in `autointerp.py match` for both contrastive description generation (`describe --mode cacts_lang`) and detection scoring (`score --negatives lang`)—which have not yet been evaluated.

	features by languages spanned					F1, all / 13–15 languages	
model	1–3	4–9	10–12	13–15	purity	`acts`	`cacts`
onto (resid_nc)	0	5	43	177	0.158	0.473 / 0.460	0.392 / 0.380
k32	63	55	68	64	0.466	0.645 / 0.564	0.603 / 0.482
k32_bl	9	9	10	47	0.277	0.578 / 0.502	0.470 / 0.338
g160top1	41	24	32	53	0.440	0.646 / 0.467	0.594 / 0.383
g160softmax	0	5	72	179	0.183	0.449 / 0.443	0.364 / 0.347
k5120 (dense ctrl)	0	2	18	236	0.131	0.470 / 0.470	0.332 / 0.322

Table 18 Detection F1 from Table 4 stratified by the number of distinct languages among each feature's 15 evaluated rows (the 10 examples from which each description was generated and the 5 held-out positives; 15 random rows span an average of 13.8 languages). Purity is the fraction of the 10 description rows belonging to their modal language. One seed per run.

E Supplementary results: the straight-through arm

E.1 Calibration

A naive port of the live softmax configuration (`select="ste"`) trains without numerical failure, but the classifier fails to learn. The root cause is that **the argmax forward pass is scale-invariant, so nothing in the uncalibrated configuration drives the logits upward**: they remain near their initialization scale throughout training (within-head standard deviation 0.0015, close to the $2/(d{+}1) \approx 0.0020$ scale produced by fan-in initialization for a bilinear logit evaluated on a unit-norm shape vector with its constant coordinate, by the degree-two analog of the variance argument for scaled dot-product attention [72, fn. 4]). VQ-VAE encounters the same issue—an encoder scale that the discrete code does not constrain—and addresses it with a commitment loss [84]. Three hyperparameter adjustments follow from this scale invariance. First, *logit noise must be relative*: an absolute scale of $\sigma_K{=}0.02$ is thirteen times the initial logit spread, causing noise to dominate the argmax, whereas $\sigma_K{=}0$ eliminates exploration and causes the codebook to collapse; setting `--noise-k batchnorm` (Eq. (17)) scales the noise by the logit RMS. Second, *the surrogate temperature must match the logit scale*: at the softmax configuration's floor of $t{=}0.03$, the ratio spread$/t$ is 0.05 and the softmax surrogate is nearly flat, so training reduces to adapting the dictionary to a fixed random partition. Third, *the batch-mean uniformity weight s_{KL_m} must be rescaled*: the softmax default of 10^{-6} is calibrated against a whitened MSE near 3×10^{-6}, whereas a hard code trains at an MSE orders of magnitude larger, rendering a 10^{-6} weight effectively inert and allowing heads to collapse. A 4,000-step sweep over the surrogate temperature t (learning rate 10^{-5}, `--noise-k batchnorm --sd-k 0.3`; one seed per arm, which suffices to rank configurations though not to predict final converged error) illustrates the trade-off:

t	spread/t	max p	effective entries/head	raw FVU
0.01	0.15	0.043	23.4	178
0.0015	0.78	0.135	26.4	193
0.0005	3.38	0.681	14.0	59.2
0.00015	16.84	0.974	3.8	7.3

Sharpening the surrogate enables the classifier to train and reduces raw FVU 25-fold, but it trades off directly against entry utilization, which must be maintained by s_{KL_m}. At `sonar.py`'s default learning rate of 5×10^{-5}, every sharp-surrogate arm collapsed abruptly after roughly 1,700 steps, whereas 10^{-5} remains stable. The shipped SONAR arm therefore uses t=1.5×10^{-4}, a learning rate of 10^{-5}, `--noise-k batchnorm --sd-k 0.3`, and s_{KL_m}=10^{-4}, trained for 24 epochs (369,500 steps). Because t is manually matched to a single initial logit-scale measurement, however, effective sharpness drifts as the logit scale evolves: over training, the shipped arm's layer-0 logits shrink 2.9-fold, reducing spread/t from 13 at initialization to 4.5, while relative batchnorm noise flips 20–29% of layer-0 assignments throughout (`logitscale.py`, evaluated on 2,048 tail rows). More generally, scale invariance leaves the logits free to drift in either direction, and the GPT-2 arms—trained at the naive port's uncalibrated settings—exhibit the opposite drift. In GPT-2 `ste_h76`, whose classifier lacks a gate, layer-0 logits grew 452-fold; consequently, its fixed-scale absolute noise flipped more than half of layer-0 assignments through 50,000 steps, but only about 1% from 150,000 steps onward. By contrast, the sigmoid-gated classifiers in the concatenated GPT-2 seed pair remained near their initial scale, so the same absolute noise flipped 71–90% of layer-0 assignments across all 16 evaluated checkpoints. A more principled calibration would either normalize logits per head or adapt t online to maintain a target spread/t setpoint.

E.2 Capacity beyond bits, and training dynamics

Distance from the Gaussian reference. Evaluated relative to the Gaussian rate–distortion reference at each code's bit rate, the full five-layer stack lies further above the reference than a single-layer prefix. Because the optimum for real data may lie below the Gaussian reference, each ratio lower-bounds the code's distance from the optimum at its rate:

code	Gaussian reference	achieved	ratio
380 bits (one layer's prefix)	0.4208	0.537	1.28×
1900 bits, 5×76	0.0499	0.1535	3.08×
1900 bits, 5×380, k=2	0.0499	0.1844	3.70×

Bits are not the whole capacity. At an identical 1900-bit budget, a 5×380 stack of binary heads (k=2) reaches $\mathrm{FVU}_w = 0.1844$, compared to $\mathrm{FVU}_w = 0.1535$ for the 5×76 stack of 32-way heads: whereas bit rate scales as $h \log_2 k$, total codebook size scales as $h \cdot k$ (3,800 vs. 12,160 atoms across five layers). Because the binary arm was also trained at half the batch size (Table 10), this gap reflects both the smaller codebook and higher gradient variance. Residual vector quantizers in neural audio codecs exhibit the same trade-off: at a fixed 80 bits per frame, eight stages of 1,024 codes outperform eighty binary stages [32]. Even so, the binary arm comes

within 17% of the 32-way stack using 2.9× fewer parameters while representing each head as a single binary feature—a format directly amenable to naming.

Realized bits, and what they bound. Following standard information-theoretic convention, this paragraph denotes random variables in uppercase. For a deterministic hard code evaluated on a random input $\mathbf{X}$, let $A_{\ell i}$ denote the entry selected by head (ℓ, i) and let $\mathbf{A}$ denote the joint assignment across all heads. The *realized bits* are defined as the sum of marginal usage entropies across heads,

$$H_{\text{real}} := \sum_{\ell,i} H(A_{\ell i}) = \sum_{\ell,i} \left(\log_2 k - \mathrm{KL}(\bar{\mathbf{p}}_{\ell i} \,\|\, \text{uniform}) \right) \;\geq\; H(\mathbf{A}) = I(\mathbf{X}; \mathbf{A}), \tag{36}$$

where $\bar{\mathbf{p}}_{\ell i}$ is the marginal usage distribution (mean assignment) of head (ℓ, i); in terms of the logged statistic KL_m summed over layers, this equals $h(l \log_2 k - \mathrm{KL}_m)$. Because $\mathbf{A}$ is a deterministic function of $\mathbf{X}$, its joint entropy $H(\mathbf{A})$ equals the mutual information $I(\mathbf{X}; \mathbf{A})$, which is upper-bounded by H_{real} by subadditivity, with equality if and only if all heads are mutually independent. Consequently, H_{real} measures per-head entry uniformity rather than true information content: two codes with identical H_{real} can carry different amounts of information depending on inter-head redundancy, and we do not estimate the joint entropy $H(\mathbf{A})$, which would require density estimation over k^{lh} outcomes.

Training dynamics. In the shipped arm, the logged training MSE (averaged across deep-supervision prefixes) plateaus from step 237k onward, whereas within-head row collinearity continues to fall (0.812 → 0.686, compared to ∼0.64 for random non-negative atoms). Entry usage in the shipped arm collapses early in training and never becomes fully uniform (Figure 28): realized bits drop to 43% of the nominal 1900 bits near step 6k, recover to 85% by step 20k as the rescaled s_{KL_m} penalty counteracts collapse, and finish at 90%. Under the fixed initialization, by contrast, every head reaches near-uniform usage by step 5k (99% of nominal) in both the five-layer stack and the flat arm, and remains there throughout training. We compute realized bits from the logged KL_m as $h(l \log_2 k - \mathrm{KL}_m)$ minus its finite-sample floor: because KL_m is a plug-in estimate on a single batch of b=256 rows, even perfectly uniform usage yields an expected floor of $(k-1)/(2b \ln 2) = 0.087$ bits per layer, which accounts for nearly the entire logged value in the converged fixed-initialization arms. After this correction, the final values match the held-out measurements in Table 5 to within 1 bit on the fixed-initialization arms, while overstating the shipped arm by 14 bits because the correction assumes that all k entries remain live.

E.3 Operational rate: every code quantized

Comparing raw index bits and active coefficient counts cannot directly rank codes that transmit continuous magnitudes against purely discrete codes. Quantizing and entropy-coding all transmitted values (Eq. (26)) places every model on a common operational rate–distortion axis (Figure 29). Evaluating each code at the five-layer stack's 1900-bit budget yields the following FVU_w:

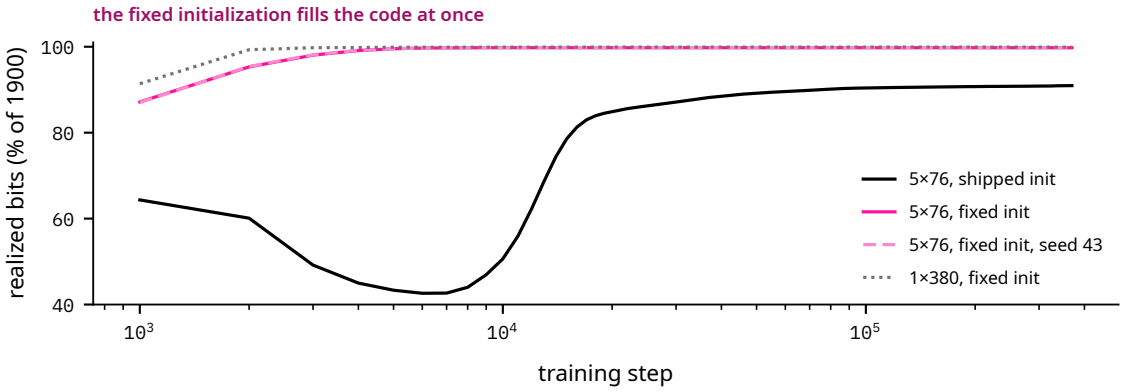


Figure 28 Realized code bits over training as a fraction of the nominal 1900 bits, computed from the logged KL_m minus its single-batch sampling floor. Under the shipped initialization, entry usage drops below half of nominal capacity before recovering to 90% (the plotted 91% slightly over-corrects heads with dead entries); under the fixed initialization, every head in both the five-layer stack and the flat arm reaches near-uniform usage within 5k steps. Realized bits upper-bound the information carried by the code rather than measuring it directly (Eq. (36)).

code	FVU_w unquantized	1900 bits, entropy-coded	1900 bits, fixed supports	1900 bits on every row
straight-through, 5×76	0.1536	**0.155**	**0.158**	0.158
L1 SAE, s_{L1}=3×10^{-6}	0.0052	**0.129**	0.182	**0.152**
softmax Ontologizer	0.0054	0.145	0.239	—
L1 SAE, s_{L1}=10^{-5}	0.048	0.175	0.180	0.200
L1 SAE, s_{L1}=3×10^{-5}	0.219	0.242	0.243	0.250
`TopK` SAE, k_{SAE}=5120	0.0001	0.243	—	0.281
`TopK` SAE, k_{SAE}=160	0.258	0.259	0.259	0.258
`g160top1`	0.274	0.274	0.274	0.274
`g160softmax`	0.0044	0.372	0.364	0.454

Under per-slot entropy coding at an average rate of 1900 bits per row, a coarsely quantized ReLU+L1 SAE (s_{L1}=3×10^{-6}) outperforms the hard stack ($\mathrm{FVU}_w = 0.129$ vs. 0.155), and matches the stack's gain-pinned discrete error of 0.158 at 1,675 bits. Near 1900 bits, this L1 SAE transmits roughly 440 of its ∼1,600 active coefficients at about 4.3 bits per coefficient including support indexing. Because L1 latents fire with skewed frequencies, entropy coding compresses its support indices to roughly 20% below their fixed-length combinatorial cost, whereas the hard stack uses entries almost uniformly (1895 of 1900 bits; Table 5) and therefore saves almost nothing under entropy coding. At the same time, the L1 SAE's code length varies substantially across inputs: near a 1900-bit average rate, per-row bit lengths span 1,406 to 2,399 bits (5th to 95th percentile). By contrast, the hard stack's gain-pinned discrete code uses exactly 1900 bits on every row at fixed length, and under entropy coding its per-row lengths differ only by the few bits induced by minor entry-frequency variations. When we enforce a strict 1900-bit budget on every row—choosing for each row the finest quantization step that fits after reserving roughly 6 bits to specify the step—the s_{L1}=3×10^{-6} SAE reaches $\mathrm{FVU}_w = 0.152$, nearly tied with the stack,

and requires 1,853 bits per row to match the stack's 0.158. Fixed-support codes do not change under per-row capping, whereas other variable-rate codes lose ground (from 0.243 to 0.281 for the dense k_{SAE}=5120 SAE and from 0.372 to 0.454 for `g160softmax`). Whereas average rate is the standard metric for corpus compression, a per-row budget provides a fairer comparison for a fixed-size per-input representation. When supports are charged their fixed-length combinatorial cost, the hard stack outperforms every SAE (0.158 vs. 0.180).

Among fixed-sparsity baselines, the k_{SAE}=160 `TopK` SAE plateaus at its unquantized error from roughly 1900 bits onward—consistent with its unquantized least-squares support floor of 0.236—and already reaches $\text{FVU}_w = 0.275$ at 1,121 bits, indicating that its coefficients convey little beyond support identity; the single-layer hard variant `g160top1` behaves similarly. Quantizing the softmax Ontologizer's assignments around their mean reaches $\text{FVU}_w = 0.145$ at 1900 average bits, with the same value on freshly encoded out-of-sample rows held out from all models. In the hard stack, quantizing the four scalar gains requires 4.8 bits at a step size of two standard deviations (reducing FVU_w from 0.158 to 0.155) and 28 bits to recover the unquantized FVU_w of 0.1536. Finally, none of these checkpoints were trained for an entropy-coded quantization pipeline: uniform usage is the worst case for entropy coding, and none of the SAEs were trained with quantized coefficients, so these curves rank the trained models rather than the architectures themselves.

On the sentence cache. In the recipe-ladder sweep (`experiments/recipe-ladder`), we trained both the 5×76 hard stack (k=32) and a 5×54 direct-atom stack (k=128, signed atoms) on `mc4_sent16` for 50,000 steps each with all four auxiliary loss weights disabled (`--s-bcossim 0 --s-support 0 --s-l1f 0 --s-hm 0`). Evaluating both stacks alongside SAEs trained on the same cache at a 1900-bit budget yields:

code	FVU_w unquantized	averaged over rows	on every row	one step every row fits
direct-atom stack	0.1611	0.161	0.164	**0.164**
hard stack	0.1681	0.172	0.189	0.189
`TopK` SAE, k_{SAE}=256	0.1260	**0.128**	**0.129**	0.185
L1 SAE, $s_{\text{L1}}=4.64 \times 10^{-7}$	0.0526	0.142	0.174	0.499
`TopK` SAE, k_{SAE}=160	0.1872	0.187	0.187	0.188

A wider `TopK` SAE (k_{SAE}=256 active out of 11,264 latents) outperforms both stacks both at 1900 average bits and under a 1900-bit per-row cap using per-row step selection. Because its per-row code lengths vary much less than those of the L1 SAE (1,734 to 2,133 bits from the 5th to 95th percentile, vs. 974 to 3,359 bits), imposing a per-row budget penalizes it very little. The hard stacks lead only when a single global quantization step must fit every row, which penalizes a code according to its longest row: under that constraint, the k_{SAE}=256 SAE averages only 1,107 bits per row, leaving roughly 800 bits unused. By contrast, the k_{SAE}=160 SAE trails the direct-atom stack under every accounting, and trails the 5×76 hard stack only at average rate (0.187 vs. 0.172), not under strict per-row caps (0.187–0.188 vs. 0.189). Each code is evaluated from a single training seed.

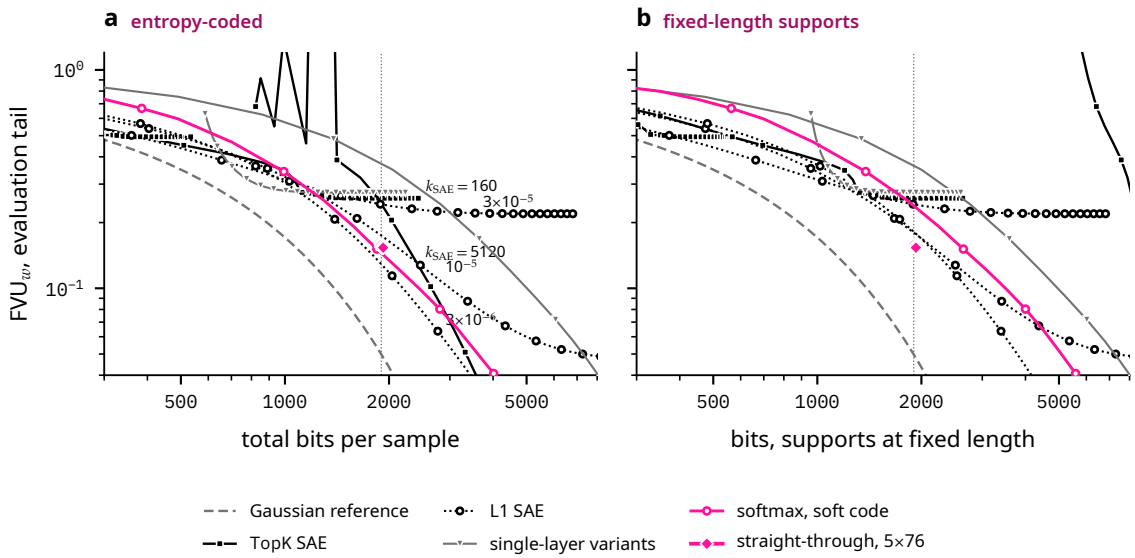


Figure 29 Operational rate–distortion curves on the evaluation tail with all transmitted values quantized across a sweep of step sizes (Eq. (26)): (a) per-slot entropy coding of all symbols, and (b) fixed-length combinatorial support costs with entropy-coded nonzero values. ReLU+L1 SAEs are labeled by s_{L1}; the unlabeled `TopK` curve is k_{SAE}=32, and the single-layer variants are `g160top1` (flat) and `g160softmax` (downward-sloping). The straight-through stack spans 1895–1923 bits. The vertical dotted line marks 1900 bits and the dashed curve shows the Gaussian rate–distortion reference (Eq. (25)). Although the softmax Ontologizer included the evaluation tail during training, re-evaluating on freshly encoded rows held out from all models changes every curve's FVU$_w$ at 1900 bits by at most 0.0004 (Appendix B.1). Each curve represents a single seed.

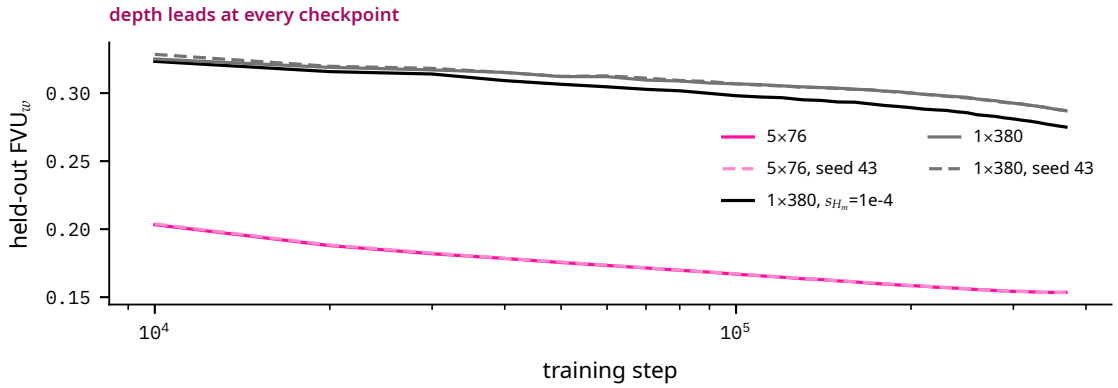


Figure 30 Held-out FVU_w over training for the 5×76 stack and the 1×380 flat arms at matched bit budget and parameter count, with seed replicas. The flat arm trained with the stack's batch-mean uniformity weight (s_{KL_m}=10^{-4}) is the configuration compared in Section 6.3.

E.4 Depth: eliminations and mechanism

Over training. Evaluated on the held-out tail every 10k steps (Figure 30), the five-layer stack outperforms the flat arms from the first checkpoint onward: the error ratio relative to the matched flat arm is 1.59× at step 10k, peaks at 1.83× near step 200k, and finishes at 1.79×. Neither architecture has fully converged. Over the final quarter of training, FVU_w improves by 0.86% in the stack and by 2.3–2.75% in the flat arms, indicating that the gap narrows slowly and that longer runs would be required to determine its asymptote. Seed replicas overlap closely at every checkpoint.

Two earlier, confounded versions of this comparison under the shipped initialization help isolate what residual depth contributes. First, an unadjusted flat arm reached $\mathrm{FVU}_w = 0.5822$ at 484 realized bits (25.5% of nominal), a 2.5× gap relative to the shipped stack. Because KL_m averages across a layer's h heads before summing across layers, per-head uniformity pressure from KL_m scales as s_{KL_m}/h per layer, meaning that the 1×380 flat arm had trained under one fifth of the per-head uniformity pressure of the 5×76 stack; matching per-head uniformity pressure raised realized usage to 1,199 bits and reduced FVU_w to 0.4307. Second, inserting a frozen ZCA whitener (`make_zca.py`) before the flat classifier increased the input's effective dimension from 107 to 901 and drove usage to 1,900 of 1,900 bits (4.999 of 5 bits per head, exceeding the shipped stack's own utilization), yet reconstruction remained 1.7× worse than the stack ($\mathrm{FVU}_w = 0.3842$). Two conclusions survive from these ablations. First, **entry utilization does not explain the benefit of depth**—nor does maximizing realized bits optimize distortion, since reducing s_{KL_m} by 100× in the shipped stack costs 176 realized bits while *improving* FVU_w to 0.2095. Second, **a jointly learned input transform collapses where a frozen whitener succeeds**: prepending a trainable `Linear` encoder causes every head to collapse onto exactly two entries at $\mathrm{FVU}_w = 0.6853$, performing worse than no encoder at all.

The input each classifier receives. Measuring the geometry of the exact input vector passed to each layer's classifier in the shipped arm (`inputgeom.py`) reveals a sharp conditioning transition

after layer 0:

layer	mean direction	mean pairwise cos	effective dim / 1024
0	0.5638	0.3146	107.5
1	0.0932	0.0087	611.3
2	0.1522	0.0233	596.3
3	0.1983	0.0396	548.8
4	0.2348	0.0553	507.1

Layer 0 (like every single-layer flat arm) classifies raw embeddings concentrated in a narrow spherical cap, whereas layers 1–4 classify residuals whose effective dimension spans about half or more of the 1024-dimensional space (507–611). In the shipped arm, layer 0 was the only layer whose heads left most entries unused (10.6 live entries and 216 realized bits, compared to ~375 bits in each deeper layer), providing a within-model control that holds architecture and hyperparameters fixed while varying only the input distribution.

What the cascade contributes: pairing. Without retraining, `blendablate.py` interpolates the subtracted term $\mathrm{dec}(\mathbf{r}_\ell)$ in layer ℓ's input toward a reference distribution with matched marginals but no sample-level pairing—another row's reconstruction (shuffle), the batch mean, or Gaussian noise rescaled to match the shuffled reconstruction's per-sample magnitude:

blend	shuffle	mean	noise	eff. dim L1 (shuffle)	(noise)
0.000	0.2292	0.2292	0.2292	499.1	499.1
0.125	0.2460	0.2374	0.2426	476.0	504.7
0.250	0.3084	0.2652	0.2838	394.2	518.3
0.500	0.6910	0.4100	0.4665	193.4	539.6
0.750	1.8154	0.7497	0.8502	103.0	527.6
1.000	4.6613	1.4302	1.5992	68.5	496.2

Sample-specific pairing is critical well before the residual link is completely severed: a 0.25 blend increases FVU_w by 35%, and substituting another row's reconstruction degrades reconstruction more than magnitude-matched Gaussian noise at every blend level, so the degradation is specific rather than generic perturbation sensitivity. This sweep cannot, however, separate pairing from conditioning: effective dimension drops in lockstep with rising error because a residual is small and nearly isotropic precisely when it subtracts the sample's own coarse reconstruction. That observation reconciles this ablation with the ZCA-whitened flat arm: whitening provides isotropic conditioning without residual pairing, yielding uniform entry usage while improving FVU_w by only ~7%. (Training with `forward="labels"`, which would test this directly by removing the residual stream altogether, has not been run under the fixed initialization. Forwarding upstream assignments *alongside* the residual via `forward="resid_labels"` performed poorly in a three-epoch pilot, finishing at held-out $\mathrm{FVU}_w = 0.826$ vs. 0.249 despite leading by 50× early in training, the signature of a parameter advantage without an information advantage, since the residual is by construction what upstream codes failed to explain.)

The code is not the whole channel. Each layer decomposes its input into a unit direction—which is all that the classifier receives—and a scalar L2 norm (gain) that multiplies the layer's output, allowing one continuous scalar per layer per sample to reach the decoder outside the discrete bottleneck. Pinning each gain to its held-out mean (`gainablate.py`) increases the stack's FVU_w by +3.0% in total, or +0.6% to +0.8% per active layer (layer 0's gain is identically 1 because the cached embeddings are L2-normalized, whereas raw SONAR embeddings are not [58]), compared to the 79% margin by which the matched flat arm trails the stack. Thus, while the continuous gain channel is real, it does not explain the advantage of depth. With its gains pinned, the stack becomes a purely discrete 1900-bit code ($\mathrm{FVU}_w = 0.158$), which is the variant compared against SAEs at matched total rate in Section 6. The gain channel is, however, a nontrivial confound for the ZCA-whitened flat arm, where whitening introduces a varying layer-0 gain whose pinning costs +4.5% in FVU_w, accounting for roughly 40% of whitening's apparent benefit.

The decoder is dense, and not because of the offset. Magnitude-pruning the smallest half of the trained decoder's weights drives FVU_w into the thousands, and its weight distribution closely resembles that of a dense Gaussian matrix. Because dictionary atoms are non-negative, the accumulator's mean vector dwarfs its sample-to-sample variation ($\|\mathbb{E}\mathbf{r}\|/\mathbb{E}\|\mathbf{r} - \mathbb{E}\mathbf{r}\| = 11.6$), and the bias-free decoder attenuates that constant component to match the corpus embedding mean (norm 0.581 vs. 0.564). Adding an explicit decoder bias (`--biased-dec`), however, changes training MSE by only −0.62% at a matched step (that run terminated early at 23% of its schedule, so its held-out $\mathrm{FVU}_w = 0.2464$ is not comparable to the converged arm): fixing $W_{\mathrm{dec}}\,\mathbb{E}\mathbf{r}$ imposes 1,024 linear constraints on a 1024×2048 matrix, or 0.05% of its degrees of freedom. The decoder is dense simply because transporting an approximately 900-dimensional signal requires a dense full-rank linear map.

How the layers' partitions relate. Each head's winning entry partitions the evaluation tail, and Figure 31 compares all pairs among the 380 heads of `ste_h76` under the shipped initialization (`headnmi.py`). Heads partition the inputs nearly independently except within layer 0: mean chance-adjusted NMI within a layer is 0.0075 at layer 0 and 0.0006–0.0028 in layers 1–4, and at most 0.0004 between layers (in the GPT-2 stack, 0.054 at layer 0 and at most 0.004 elsewhere), indicating that deeper layers do not redundantly subdivide partitions created by earlier layers. Causal coupling—measured by effective information, or how much forcing one head shifts another head's assignment distribution—tracks partition overlap only when intervening at layer 0: across head pairs, its Spearman correlation with adjusted NMI is 0.76, 0.50, 0.38, and 0.23 from layer 0 into layers 1–4, respectively, and at most 0.08 from any deeper layer, whose heads shift every downstream head by a comparable 0.003–0.02 bits regardless of partition overlap. Results are reported for a single seed.

E.5 Steering by target layer, and head-health levers

Stratifying the five-layer stack's steering realization rate by target layer (at step strength 1.0, across 12 entries per layer) reverses the aggregate ranking of steering directions:

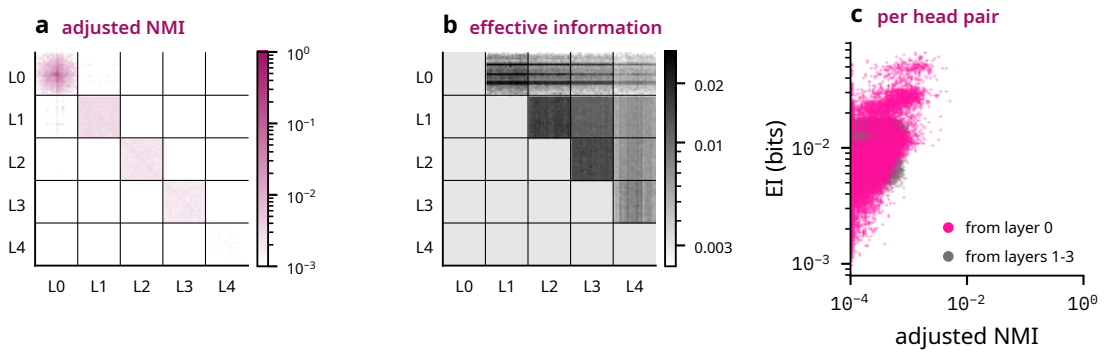


Figure 31 **Partitions overlap and track causal coupling only from layer 0.** Evaluation of the 380 heads of `ste_h76` (shipped initialization) on the evaluation tail, grouped by layer and ordered within each layer by hierarchical clustering. (a) Chance-adjusted NMI between all pairs of heads' winning-entry partitions (0 = chance, log scale; diagonal shaded gray). (b) Effective information from the row head to the column head under uniform interventions over the source head's live entries (non-downstream pairs shaded gray). (c) Effective information vs. adjusted NMI per head pair, stratified by source layer. Evaluated on a single seed.

layer	decode	grad	margin	adjoint	random
0	0.216	**0.583**	0.547	0.237	0.034
1	0.464	0.089	0.367	0.000	0.023
2	0.742	0.130	0.216	0.000	0.055
3	**0.966**	0.130	0.091	0.000	0.034
4	0.362	0.047	0.036	0.005	0.047

At layer 0, where the input perturbation reaches the classifier directly, directions derived from the classifier's own geometry (`grad` and `margin`) outperform `decode` three- to tenfold. At deeper layers, however, classifier-local directions fail: a gradient computed with respect to layer ℓ's residual input is valid only locally, and an input-space perturbation arrives at layer ℓ only after earlier layers have re-encoded and subtracted from it. By contrast, the `decode` direction shifts the input toward the vector emitted when the target entry is active, remaining self-consistent across the residual cascade. Layer 4 is difficult to steer along any direction because its residual is nearly exhausted. Practically, layer 0 is best steered via classifier gradients and layers 1–3 via decoded atoms.

Head-health levers and steering. We tried three training levers on the shipped stack. First, *batch-mean uniformity pressure from* KL_m *does not account for reduced steerability*: a twin trained at $s_{\mathrm{KL}_m}=10^{-6}$ achieves a raw text-cycle steering effect of 0.043 vs. 0.027, both an order of magnitude below `top2` and `top8` models trained at the same weight. Hard argmax selection itself, rather than the KL_m uniformity penalty, therefore accounts for the loss of steerability, while lowering s_{KL_m} substantially degrades entry usage (increasing usage imbalance from 2.71 to 5.00 bits). Second, *head-independence penalties based on HSIC/CKA optimized finite-sample estimator bias rather*

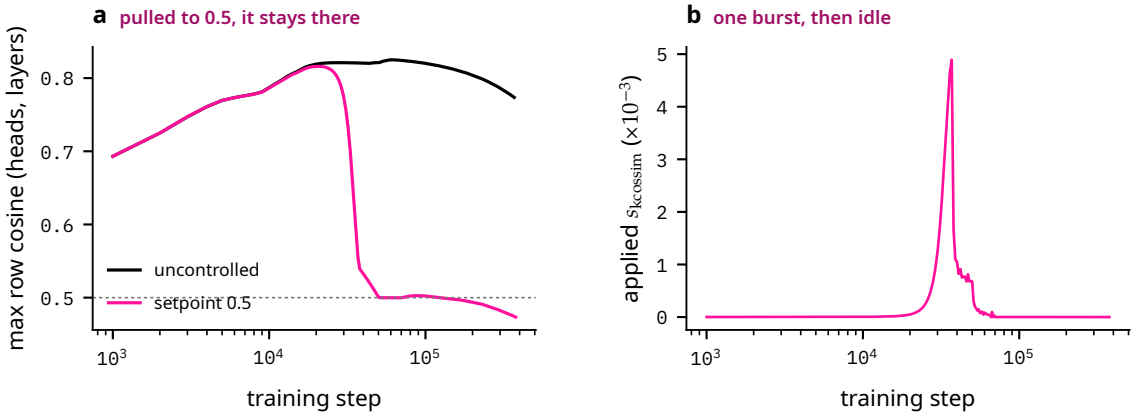


Figure 32 Row-collinearity setpoint controller dynamics on the shipped stack, taken from the training log. (a) Maximum within-head row cosine across heads and layers—the statistic monitored by the controller—compared to the uncontrolled twin; the dotted line marks the 0.5 setpoint. (b) Applied multiplier s_{kcossim}, averaged over 1,000-step windows.

than genuine dependence (Section E.9). Third, *a row-collinearity setpoint is the one lever that improved steerability*: capping within-head atom cosine at 0.5 (compared to 0.774 in the uncontrolled run) increases embedding-space realization across every steering direction (`decode` +15%, `grad` +17%, `margin` +52%, and oriented `adjoint` +122%) while reducing collateral (0.591 → 0.552) and slightly improving held-out FVU_w (0.2157 vs. 0.2293). The setpoint controller acts only transiently during early training (Figure 32): as the maximum within-head atom cosine across heads and layers climbs past 0.8, the multiplier s_{kcossim} rises to 4.9×10^{-3} near step 35k and drives the statistic down to the 0.5 setpoint by step 50k; from roughly step 60k onward, the multiplier projects to zero and the maximum cosine remains at or below 0.5 without further penalty, finishing at 0.473 vs. 0.774 in the uncontrolled run. The setpoint's mechanism is unexplained: the controlled statistic is evaluated in the e-dimensional dictionary space, where 74–78% of each atom's energy lies in the decoder's null space (Section G.1), and the decoded atoms were already nearly orthogonal. That the classifier-derived directions exhibit the largest gains suggests an indirect effect on classifier training rather than on the decoded dictionary, though nothing establishes it. Finally, the setpoint provides no benefit on the single-layer flat arm, where realization is already saturated (0.992 vs. 0.966).

E.6 Supervised vectors at their own step size

Whereas Table 3 compares steering directions at a fixed step size, each native intervention also carries a natural step magnitude: for an Ontologizer, the decoded output change produced by forcing the target entry; for an SAE latent, its decoder direction scaled by its mean nonzero activation; and for the difference of means, the unrescaled mean-difference vector itself, matching the step taken by activation addition. Evaluating each direction at its native step length yields the following rates (reported as realized / collateral, followed in parentheses by the median step norm as a fraction of $\|\mathbf{x}\|$, alongside the collateral of a norm-matched isotropic random step):

model	own (step)	diff. of means (step)	random: own / dm
softmax stack	0.19 / 0.25 (0.064)	0.18 / 0.25 (0.079)	0.25 / 0.30
hard-code stack	0.08 / 0.54 (0.129)	**0.27** / 0.47 (0.064)	0.53 / 0.48
hard-code flat	0.17 / 0.04 (0.028)	**0.54** / 0.12 (0.087)	0.05 / 0.17
untrained stack	0.00 / 0.00 (0.001)	**0.53** / 0.17 (0.162)	0.00 / 0.17
random-projection heads	**0.67** / 0.28 (0.206)	0.55 / 0.20 (0.146)	0.27 / 0.20
`g160top1`	0.41 / **0.01** (0.186)	0.57 / 0.04 (0.053)	0.18 / 0.07
`g160softmax`	0.54 / 0.07 (0.050)	0.57 / 0.13 (0.107)	0.08 / 0.16
`TopK` SAE, k_{SAE}=32 (m=5120)	0.46 / 0.06 (0.072)	**0.77** / 0.15 (0.180)	0.07 / 0.17
`TopK` SAE, k_{SAE}=32 (m=11264)	0.50 / 0.08 (0.076)	**0.82** / 0.17 (0.172)	0.09 / 0.19
SAE, bilinear encoder	0.82 / 0.04 (0.078)	0.88 / 0.08 (0.121)	0.04 / 0.07

Across most models, the difference of means achieves higher realization, though in the SAEs and the flat hard-code arm this comes partly from taking 2–3× larger steps. In the five-layer hard-code stack, however, direction rather than step length is decisive: the supervised vector reaches its target entries more than three times as often using half the step norm (0.27 vs. 0.08 at step 0.064 vs. 0.129). Likewise, on the untrained 5×76 stack at its native step (0.162), the difference of means reaches roughly twice as many targets as on the trained stack (0.53 vs. 0.27) at roughly a third of the collateral (0.17 vs. 0.47), while empirical cell-centroid swaps on random-projection heads reach 0.67 at their native step (0.206). On the softmax stack, the model's native step and the supervised vector perform nearly identically. Finally, `g160top1`'s native intervention is the most selective step achieved by any method, flipping only 1.4% of unrelated heads compared to 18% for a norm-matched random step.

E.7 Data-shaped controls, and where a decode step realizes

The main steering curves benchmark each intervention against an isotropic random step in raw embedding coordinates. However, SONAR's evaluation tail is strongly anisotropic—158 of its 1,024 principal components hold 50% of its variance, and 474 hold 80%—so a unit Euclidean step in raw coordinates is three times as long in Mahalanobis distance (normalized by the typical distance between two tail rows) as a covariance-shaped random step of the same Euclidean length, whereas the objective's diagonal whitening barely registers the difference (1.045 vs. 1.005). To account for this anisotropy, `steerembed.py` evaluates three additional random controls of equal Euclidean norm: an isotropic Gaussian draw shaped by the covariance's square root (shaped), an isotropic draw shaped by its inverse square root (inverse, concentrated along directions of minimal data variance), and a step toward another randomly sampled tail row. Evaluating collateral at step strength 0.25 on the targets and rows of Table 3 gives:

model	isotropic	shaped	to a row	inverse	decode	diff. of means
softmax stack	0.539	0.408	0.410	0.803	0.549	0.424
hard-code stack	0.653	0.634	0.640	0.754	0.658	0.629
hard-code flat	0.367	0.315	0.309	0.521	0.328	0.274

Compared to the covariance-shaped random controls, the softmax stack's `decode` direction

flips 0.14 more unrelated heads, and its gradient, margin, and adjoint directions flip 0.15–0.20 more, though all remain within 0.07 of the isotropic baseline. In the five-layer hard-code stack, every steering direction stays within 0.025 of the data-shaped controls. In the single-layer flat arm, the `decode` direction and the difference of means remain within 0.041 of the data-shaped controls, while the gradient-based directions lie 0.06–0.07 higher, matching the isotropic baseline. Across all models, inverse-covariance steps along low-variance directions disturb the largest fraction of heads.

Splitting the `decode` direction into its projection onto the leading principal subspace (capturing 50% or 80% of corpus variance) and its orthogonal complement—and stepping along each component at the full step's Euclidean length—isolates which spectral band drives target realization. In both five-layer stacks, the component lying in the bottom 20% of variance realizes 95% (hard) and 100% (softmax) of the full step's realization rate, whereas the component in the top 50% realizes only 20% and 8%, respectively; by contrast, the flat hard-code arm realizes targets through either subspace (0.958 from the top-80% subspace alone vs. 0.996 for the full step). Even without splitting, the `decode` vector tilts toward low-variance directions: corpus variance along `decode` is 1.6–1.8× the mean eigenvalue, compared to 6.2–6.7× for a covariance-shaped random step and 7.1–11.7× for the difference of means. This reflects the classifier sensitivity to low-variance directions that Ilyas et al. [124] associate with non-robust features. Because equal Euclidean length is the budget enforced throughout our steering suite, this decomposition characterizes classifier sensitivity per unit raw norm; whether the pretrained SONAR text decoder responds to the low-variance component remains untested. Each model is evaluated on a single seed.

E.8 What a removal does downstream

Ablating individual heads reveals how the residual cascade responds to a change it did not choose (`textstrip.py --removals`). Across 16 randomly sampled heads per layer and 512 tail rows, we replace a head's assignment with the uniform distribution, its marginal mean assignment, its runner-up entry (matching the perturbation applied during winner dropout), or another sample's assignment. We then recompute the output either *live*, allowing downstream layers to reclassify the modified residual, or *frozen*, holding all other head assignments fixed so that only the ablated head's contribution changes. We record two whitened quantities, each expressed as a fraction of the unablated reconstruction's squared error: the excess error introduced by the ablation, and the squared distance between the ablated and unablated reconstructions. In the hard-code stack, live downstream reclassification reduces the excess error of a layer-0 ablation to 0.23× its frozen value (0.48×, 0.68×, and 0.84× at layers 1, 2, and 3, respectively), with the largest reduction for runner-up replacement (0.13× at layer 0), analogous to backup heads compensating for ablated heads in indirect object identification [125]. Yet the live output moves 19× further from the unablated reconstruction than the frozen output does (31×, 19×, and 7× at layers 1, 2, and 3): rather than restoring the original reconstruction, downstream layers re-approximate the input along a different path.

Decoded text shows the same. Evaluating 12 tail rows on the two heads per layer whose uniform ablation perturbs the reconstruction furthest, a live uniform ablation adds no excess

per-token negative log-likelihood when scoring the original input's decoded text relative to the unablated reconstruction (−0.011 nats, vs. +0.036 nats when frozen and +0.016 nats for a random step toward another row of the frozen change's length), even though scoring against the unablated reconstruction's own decoded text registers substantial divergence (+0.139 vs. +0.040 nats). In the reference softmax Ontologizer, downstream layers cancel a layer-0 uniform ablation almost completely: frozen ablation increases error by 505%, whereas live ablation increases it by only 0.5%. These text evaluations execute in `float32` precision on CPU; under JAX's default GPU matrix-multiplication precision, 226 of the hard stack's 4,560 selected entries change on these 12 rows (including 159 of the 912 entries at layer 4), so entry-level comparisons across devices require matched precision. Each model is evaluated on a single seed.

E.9 Head independence: a measurement artifact

An initial head-independence penalty (`--s-hsic-heads`, defined as the mean pairwise RBF-kernel CKA [126] between batch assignments within each layer at the median-heuristic bandwidth) appeared to detect substantial inter-head dependence. In fact, it was measuring estimator bias: standard CKA normalizes the biased V-statistic HSIC estimator [126, Eq. 3], whose expectation is strictly positive even for mutually independent heads:

layer	0	1	2	3	4	mean
unbiased	0.0041	0.0005	0.0005	0.0004	0.0002	**0.0011**
biased	0.0315	0.1062	0.1018	0.0963	0.0976	0.0867

The biased estimator registers chance co-occurrence between independent categorical heads as spurious dependence, with a positive floor that increases as the batch size decreases (0.109 at b=256 and 0.029 at b=1024, vs. 0.0001 across all batch sizes for the unbiased estimator). This bias floor also decreases when a head concentrates onto fewer entries, and vanishes identically when a head freezes onto a constant entry: on synthetic independent heads at b=256 and k=32, the biased estimator yields 0.108 under uniform usage, 0.016 at roughly four effective entries, and 0.026 when half of the heads are frozen (`cka_floor.py`). At the training batch size, this bias floor exceeded genuine inter-head dependence, leaving the penalty nothing to reduce except estimator bias—which decreases only by concentrating or freezing heads. Reconstruction pressure resisted that in the hard-code stack, whereas replacing the softmax Ontologizer's auxiliary losses with this biased penalty froze an entire layer (Section D.1).

We therefore updated `hsic.pairwise_head_cka` to normalize Song et al.'s unbiased HSIC estimator [127, Eq. 5] ("debiased" CKA, as in the minibatch CKA of [128]), whose expectation is identically zero for independent heads at any batch size while still measuring 0.06 and 0.41 at planted factor correlations of 0.5 and 0.8, respectively. Conditioning on language or script explains none of the residue. Overall, **this architecture produces nearly independent heads without explicit regularization**; training under the corrected estimator leaves reconstruction and independence unchanged while reducing language localization (best-head language NMI 0.199 vs. 0.269). Two lessons generalize: first, any candidate regularizer should be verified on a null distribution where it ought to vanish; second, a term's share of the loss value need not reflect its share of gradient norm—at $s_{\mathrm{HSIC}}=10^{-4}$, the penalty accounted for 12% of the scalar

loss, whereas the measured MSE-to-CKA gradient-norm ratio at the classifier (the only module receiving CKA gradients) was 1,265, placing equal gradient magnitude near 8×10^{-4}.

E.10 Seed reproducibility, measure by measure

Partitions. Under Hungarian matching across random seeds, the single best-matched head pair reaches an NMI of 0.42, but the mean across all heads is 0.012 compared to a 0.005 permutation null, and entry containment (`splitting.py`) is 0.000. Neither result is an artifact of the metric: comparing a single run against its own checkpoint 19,500 steps earlier under the same partition NMI yields 0.38 overall and 0.87 at layer 0.

Contributions. Two heads could in principle partition inputs differently while writing similar output contributions, but they do not: matched head contributions have a centered cosine similarity of 0.0074 vs. a 0.0015 null (without centering, the cosine is 0.205, reflecting the shared mean offset of the non-negative dictionary). A standardized re-implementation in `headcontrib.py` (Hungarian-matched decoded, whitened, centered contributions evaluated on 4,096 rows) yields 0.019 vs. a 0.0017 null on the same seed pair; although the exact script configuration of the earlier 0.0074 readout was not recorded, the qualitative conclusion is identical under both.

Subspaces. The top-8 eigenspaces of per-layer head contributions align 25–33× above chance across seeds, yet each seed aligns to the same extent with the *cache's own* principal subspace (for example, at layer 1: 123.7× between seeds vs. 101.3× and 101.2× between each seed and the data), showing that subspace agreement reflects shared data variance rather than architectural identifiability. At layer 4, each seed aligns with the data ∼23× above chance while agreeing with the other seed at only 9× chance, indicating that different seeds select *different* data-aligned subspaces.

Atom Gram matrices. Evaluating within-head atom Gram structure (`atomgram.py`), per-layer mean row collinearity agrees to four digits across seeds (0.2880 vs. 0.2891 at layer 0), but the Gram spectrum is nearly one-parameter: non-negativity forces a dominant eigenvalue equal to $1 + (k-1)\,\kappa_i$ that carries 82% of the spectrum's Frobenius norm. Comparing two heads' Gram matrices entrywise requires a correspondence between their entries. When entries within each head are sorted in canonical order (by mean cosine similarity to the remaining entries in the head), Gram matrices match near chance against a within-layer head-pairing null both across seeds (1.06–1.11×) and within a single run across checkpoints separated by 19,500 steps (1.05–1.14×). That within-run failure is an artifact of the canonical sort rather than head instability: because the Gram matrix is nearly equicorrelated, every entry has nearly the same mean cosine, so canonical sorting is sensitive to minor fluctuations between checkpoints and matches only 33–66% of heads to themselves within a run. Because training never permutes entry indices (dead-entry revival is disabled in both runs), stored index order preserves the true within-run correspondence; under stored order, each head's Gram matrix matches its earlier checkpoint at 8.5–10.7× chance (7.5–10.0× on decoded atoms), and its decoded atoms align at cosine 0.98–0.99 compared to 0.05–0.08 for other heads in the same layer. Across seeds, even when entries are aligned via optimal linear assignment on their centered decoded atoms, which fixes the correspondence exactly, Gram agreement remains near chance (1.05–1.19×) and matched decoded atoms achieve cosines of only 0.07–0.15 vs. 0.06–0.10. Heads therefore

persist within a training run, but do not replicate across random seeds.

Identifiability theory and next-sentence transition probes. This cross-seed disagreement is what classical results for static unsupervised objectives lead one to expect, although it shows that these training runs are unstable, not that every optimum or training procedure must disagree. Without auxiliary supervision or structural constraints matched to the data-generating process, unsupervised disentanglement is unidentifiable from marginal observations alone [11]. Uniqueness guarantees for sparse coding require noiseless linear reconstruction, a dictionary satisfying the spark condition, and sufficient samples across supports, and even then determine the dictionary only up to permutation and scaling [91]; lossy codes fall outside this regime. When three or more discrete views are conditionally independent given a latent class, Kruskal's three-way array theorem identifies the latent distribution up to permutation if the class-conditional view distributions satisfy a rank condition—always when each view's conditionals are linearly independent, and generically whenever each view has at least as many outcomes as latent classes [92]. While Unsupervised Disentanglement Ranking uses pairwise cross-seed similarity to select identifiable models without ground-truth labels [93], in the Ontologizer the heads are learned partitions rather than fixed external views. An established alternative is to train on paired observations—such as adjacent sentences or augmented views—where identifiability up to permutation follows from temporal contrast [94], sparse Laplace transition priors [95], shared latent factors across paired corruptions [96], auxiliary variable conditioning [97], or contrastive alignment of conditional distributions [98].

To test whether adjacent sentences in `mc4_sent16` provide a usable temporal signal, we first fit a linear ridge predictor of $\mathbf{x}_{t+1}$ from $\mathbf{x}_t$ on 500,000 training pairs and evaluated it on 8,683 held-out pairs (`eval_next.py` in `experiments/next-sentence/`, with coordinate 837 masked). The linear baseline achieves $\mathrm{FVU}_{\mathrm{next}} = 0.8962$ (α=10^3; predicting $\hat{\mathbf{x}}_{t+1} = \mathbf{x}_t$ yields 1.418), indicating that $\mathbf{x}_t$ linearly explains 10.4% of $\mathbf{x}_{t+1}$'s whitened variance, and encoding $\mathbf{x}_t$ first through the trained Ontologizer or `k32` SAE reaches 0.9035 and 0.9186 (93% and 78% of the linear signal, respectively). Directly analyzing transition geometry (`transitions.py`) shows that this cross-sentence signal is concentrated in direction rather than norm: raw cosine similarity distinguishes adjacent from random sentence pairs with AUC 0.923 (median cosine 0.680 vs. 0.369), whereas whitened Euclidean distance reaches only AUC 0.685 because 68% of its pair variance arises from sentence-length norm fluctuations. Trained solely for single-sentence reconstruction on `mc4_sent16` (`sent16/ste_h76_init01_no837`, 50,000 steps), the five-layer hard stack fails to capture slow document state: its heads switch entries on 91.6% of adjacent pairs in layer 0 and 92.3–92.5% in layers 1–4 (compared to 96.9% on random pairs), retaining their entry across adjacent sentences barely above chance (Cohen's kappa 0.054 in layer 0 and 0.045–0.047 in layers 1–4, with a maximum of 0.081 for any head). In a one-epoch pilot sweep (`pilot.sh`; 5,011 steps), we trained a single-layer model (l=1, h=380, k=32) on pairs of sentences separated by at least two positions within the same document, adding a cross-sentence disagreement penalty ($1 - \sum_j p_{a,ij} p_{b,ij}$) on its first 38 heads. That penalty had little effect for $s_{\mathrm{pair}} \leq 3 \times 10^{-5}$ and collapsed the regularized heads to 0.04–0.09 realized bits for $s_{\mathrm{pair}} \geq 10^{-4}$, overwhelming the batch-mean uniformity penalty at s_{KL_m}=10^{-4} because a constant

head incurs a bounded KL_m cost while achieving zero disagreement.

Slow heads from a Fano pair term. To prevent collapse, we replaced the disagreement penalty with a Fano lower bound (`pair_term=fano`, `pairloss.py`) on the mutual information between the entries that each slow head assigns to the two paired sentences. Because predicting one sentence's entry from the other's errs precisely when the head switches entries, Fano's inequality upper-bounds the conditional entropy by the binary entropy of the head's switch rate plus that switch rate multiplied by $\log_2(k-1)$; subtracting this quantity from the head's batch marginal entropy and dividing by $\log_2 k$ yields the normalized Fano objective. Under this bound, a collapsed head has zero marginal entropy and therefore scores zero, removing the incentive to collapse. Across the 5,011-step pilot sweep, no slow head collapsed (5.00 realized bits at every s_{pair} up to 10^{-2}), held-out FVU_w stayed within 0.0002 of the unregularized control, and adjacent-pair Cohen's kappa on the slow heads rose from 0.072 to at most 0.097. Temporal SAEs [99] are the nearest SAE counterpart: they add a contrastive term between adjacent tokens' codes on a designated block of high-level latents, and motivate its negatives as preventing the collapse to constant features that our disagreement penalty produced; Temporal Feature Analysis [129] instead splits each activation into a part predicted from its context and a novel residual. Both work at the level of tokens and report single runs, so neither measures whether the slow block reproduces across seeds.

When trained at $s_{\text{pair}}=10^{-3}$ for three epochs over the 3.9M pairs of the first 4M rows (`long.sh`), the 38 slow heads reach a mean Cohen's kappa of 0.463 on adjacent pairs and 0.407 at a separation of twelve sentences by step 45,000 (vs. 0.067–0.072 for the control's first 38 heads), while retaining 4.85 of 5 realized bits per head. Replicas at seeds 43 and 44 follow the same trajectory (0.457 and 0.460 at step 45,000), and the three seeds' slow blocks carve similar partitions. Under one-to-one Hungarian matching by NMI at step 46,250 (`seedmatch.py`, Eq. (29), on the final 32,768 rows), each pair of seeds' 38 slow heads agrees at NMI 0.412–0.414 against a 0.005 null, and every slow head in one seed is matched to a slow head in the other (chance 10%); by contrast, the remaining 342 unregularized heads match at 0.017, and the control's first 38 heads match at 0.016–0.017. Most of this agreement is not merely language identity: evaluated separately within each of 84 languages across 260,453 held-out rows, the slow heads agree at NMI 0.312–0.314 (vs. a 0.041 null), compared to 0.069–0.070 (vs. 0.051) for the control. Regularizing the slow heads costs 1.8–1.9% in held-out FVU_w (0.4679 vs. 0.4593 for seed 42, 0.4668 vs. 0.4586 for seed 43, and 0.4685 vs. 0.4596 for seed 44).

However, this reproducibility holds at the level of the 38-head block rather than for individual heads. Because two slow heads within a single seed already agree with one another at NMI 0.40, a random one-to-one bijection between two seeds' slow blocks achieves nearly the same mean NMI as the optimal Hungarian assignment:

slow heads, NMI	seeds 42, 43	42, 44	43, 44
matched one to one	0.414	0.412	0.412
any one-to-one pairing (mean)	0.399	0.398	0.397
two heads of one seed (mean)	0.400 / 0.398	0.400 / 0.396	0.398 / 0.396
best partner over runner-up (median)	0.004	0.003	0.003

Moreover, the pairwise matchings do not compose transitively: chaining the matching from seed 42 to 43 with that from 43 to 44 agrees with the direct matching from 42 to 44 for only 6 of the 38 slow heads, compared to 1 expected by chance for unrelated matchings (and 2 for the control's first 38 heads). What reproduces across seeds is what the slow heads share with one another; no single slow head is yet identified across runs; likewise, because the within-language NMI is evaluated on these near-arbitrary pairings, it confirms that the shared signal extends beyond language without establishing head-level identifiability. Note that all three seeds traverse the training rows in the same order (as do the unregularized control seeds, which fail to reproduce), and that this experiment uses a single-layer model.

Against linear slow features. Linear slow-feature analysis (SFA) characterizes the highest cross-sentence correlation achievable by any linear projection of the embedding (its unit direction $\mathbf{x}/||\mathbf{x}||$) on these pairs: its directions are the generalized eigenvectors of the pair's symmetrized cross-covariance with respect to the pooled covariance, equivalent to canonical correlation analysis with a single projection shared across both sentences. Fitted on the expected pairs of the first 4M rows (`sfa.py`) and evaluated on the held-out tail, its slowest direction maintains a cross-sentence correlation of 0.935 within a document, 31 directions exceed 0.5, and the 38 slowest directions average 0.636; after removing each language's mean, the slowest direction maintains 0.739, 12 directions exceed 0.5, and the 38 slowest average 0.462. Source language dominates the top of the linear slow spectrum, accounting for 86% of cross-sentence covariance along the eight slowest directions and 54% across the 38 slowest. Applying the same linear SFA procedure to the slow heads' one-hot code (38×32 binary indicators, fitted on the 229,373 rows of the cache's last 262,144 that precede the held-out tail, all past the training prefix, and evaluated on the tail) achieves higher cross-sentence correlation than the linear embedding optimum at every subspace rank:

	mean correlation over the slowest			the same, language means removed		
directions	1	8	38	1	8	38
linear (the embedding)	0.935	0.847	0.636	0.739	0.636	0.462
slow heads' code, seed 42	0.956	0.909	0.727	0.781	0.690	0.524
slow heads' code, seed 43	0.954	0.905	0.718	0.776	0.690	0.523

The slow-head code also spans the leading linear slow directions: ridge regression from the 38 slow heads' one-hot code explains 0.76–0.77 of the held-out variance of the eight slowest linear directions, compared to 0.39–0.40 for any other block of 38 heads in either arm or seed, and 0.64 vs. 0.38–0.40 after removing language means, with the two seeds agreeing within 0.04 at every rank. Nevertheless, the slow heads remain highly redundant: each slow head is best explained by either the first or second linear slow direction (with 34 and 29 of the 38 best explained by the single slowest direction), indicating that the block concentrates at the top of the slow spectrum rather than spreading across distinct slow factors.

Ordered decorrelation of the slow block. To test whether the slow heads can be told apart, we added to the Fano term a penalty on dependence between them (`train_order.py`): the mean, over the 703 pairs of slow heads, of the centered kernel alignment [126] between the two heads' partitions of the batch under the kernel "same entry", estimated with unbiased HSIC [127] and weighted by s_{dec} in units of s_{pair}. In each pair the earlier head is stop-gradiented, so head i is pushed away from heads $0, \ldots, i-1$ and head 0 receives no penalty, with the aim of an order that a second seed would reproduce. Head i feels the penalty at weight $s_{\mathrm{dec}}\, i/703$ against a Fano reward of $1/38$: head 1 at an eighteenth of its reward for s_{dec}=1, and at more than five times it for s_{dec}=100. Both weights were trained for three epochs at seeds 42 and 43, otherwise as the Fano runs. Below, kappa is on same-document pairs two sentences apart and NMI is across the two seeds, both at step 45,000 on the final 32,768 rows (`order.py`), with rows for heads 1–37 giving the mean over those heads; "explained" is the share of held-out variance of the eight slowest linear directions that the ridge regression above recovers from a code, at step 46,250.

seed 42 / seed 43	control	Fano	s_{dec}=1	s_{dec}=100
kappa, head 0	0.066 / 0.058	0.469 / 0.444	0.466 / 0.441	0.467 / 0.446
kappa, heads 1–37	0.064 / 0.062	0.450 / 0.445	0.245 / 0.246	0.036 / 0.036
NMI, head 0 with head 0	0.016	0.378	0.373	0.386
NMI, head 0 with heads 1–37	0.015	0.388	0.188	0.009
explained by head 0 alone	0.03 / 0.02	0.41 / 0.38	0.40 / 0.38	0.41 / 0.39
explained by all 38 heads	0.40 / 0.40	0.77 / 0.76	0.81 / 0.80	0.44 / 0.43
held-out FVU_w	0.4589 / 0.4580	0.4674 / 0.4662	0.4616 / 0.4604	0.4672 / 0.4663

At s_{dec}=1 the block becomes graded rather than separated. Kappa falls with the head index (rank correlation −0.98 and −0.97, from 0.47 at head 0 to 0.09 at head 37 in seed 42), and a head's matched partner in the other seed sits near its own index (rank correlation 0.94). But a head is no closer to the other seed's head of its own index than to that seed's heads one index away (a difference of +0.0013 ± 0.0016 in NMI, where the Fano arm, which has no order, reads +0.0013 ± 0.0020), and the one-to-one matching keeps the exact index for 3 of the 38 heads. At s_{dec}=100 exactly one head is identified: head 0 agrees with the other seed's head 0 at NMI 0.386 and with its other slow heads at 0.009, and every one of seed 42's 38 heads has seed 43's head 0 as its best partner. This is the one head the penalty never touches, and it is the same head in every arm: as slow as a Fano head, and explaining 0.38–0.41 of the eight slowest linear directions on its own. The other 37 heads find no slow content of their own. Their kappa falls over training (0.047, 0.040 and 0.036 at steps 15,000, 30,000 and 45,000 in seed 42), to below that of the control, which has no pair term.

Identification at s_{dec}=100 costs the block about half of what it carried. A single slow head explains 0.38–0.43 of the eight slowest linear directions where the Fano block's 38 heads together explain 0.76–0.77, so those heads, although redundant pair by pair, jointly carry about twice what one of them does. At s_{dec}=100 the 38 heads together explain 0.43–0.44, which is head 0's share plus 0.04, and the eight slowest directions of the block's own code keep a cross-sentence correlation of 0.68, below the embedding's linear optimum of 0.847 that the Fano block exceeds. At s_{dec}=1 the block explains slightly more than the Fano block does (0.80–0.81), with its heads' best-explained directions spread over 9 and 10 linear slow directions rather than 2, at a third of

the reconstruction cost. Neither weight therefore yields a block of individually identified heads: the weak penalty spreads the block without separating its heads, and the strong one separates a single head by leaving the others nothing. What the Fano block holds beyond one head is evidently not independent of that head's partition, since heads barred from depending on head 0 do not acquire it; a finer division of head 0's own cells would be content of that kind, though we have not tested this. Three limits apply: these runs use two seeds, weights between 1 and 100 were not trained to three epochs, and the ridge regression measures only what is linearly readable of linear slow directions (the control's 38 heads, none of them slow, reach 0.40 under it).

E.11 Gain, atom spread, and depth in layer-0 reproducibility

Because layer-0 heads make the largest output contributions and are also the most reproducible across random seeds, cross-seed agreement in layer 0 could simply reflect head magnitude rather than layer depth. To separate these factors, `headcontrib.py` defines a head's size as its centered contribution energy divided by the whitened target variance. Because a head's output contribution on each sample is the product of its layer's scalar input gain and the decoded atom selected on that sample, head size decomposes into squared input gain times squared atom spread (the usage-weighted RMS distance of the head's decoded atoms from their mean); across both stacks, this product matches the measured head size with a log correlation of 1.000.

	GPT-2 concat			SONAR stack		
layer	agreement	gain	atom norm (10^{-4})	agreement	gain	atom norm
0	0.171	122.2	4.6	0.080	1.000	0.043
1	0.017	67.9	5.3	0.005	0.588	0.054
2	0.007	62.3	4.4	0.004	0.490	0.052
3	0.005	59.2	3.7	0.003	0.417	0.051
4	0.004	57.2	3.2	0.003	0.359	0.043

Table 19 Cross-seed head agreement (matched centered contribution cosine) and the two factors of head size, reported as medians across each layer's heads using decoded, whitened atoms. SONAR values compare `ste_h76_init01` vs. `ste_h76_i01_s43` over 4,096 rows.

Across layers, layer 0's larger contributions come entirely from input gain rather than atom magnitude. Layer 0 receives the unreduced input (GPT-2 activations of norm ∼122 vs. residuals of ∼60, and unit-norm SONAR embeddings vs. residuals of 0.36–0.59), whereas its decoded atoms are no larger than layer 1's on either dataset—and on SONAR are smaller than those of layers 1–3 and equal in norm to layer 4's. Because input gain is a single scalar per layer and sample shared by all heads in that layer, cross-layer variation in head size is the layer itself under another name. Controlling for atom spread in a regression of log agreement on log spread and a layer-0 indicator (coefficients of determination 0.979 on GPT-2 and 0.941 on SONAR), layer 0 still reproduces 14× better than deeper layers on GPT-2 and 23× better on SONAR.

Comparison with the flat arm at matched size. Within a residual stack, head size and layer depth do not overlap, so their effects cannot be separated. The single-layer flat arm

(1×380) breaks this confound because all of its heads receive the raw input at unit gain with no downstream layers. On SONAR, cross-seed agreement within the flat arm correlates strongly with log head size (correlation 0.876), rising from 0.008 for its smallest heads (0.057–0.060% of target variance) to 0.150 for heads above 0.071%, and reaching 0.78 and 0.56 for its two largest heads. Because input gain is constant in the flat arm, this relationship reflects atom magnitude alone. However, the five-layer stack falls far below this size–agreement curve at every depth, including layer 0:

stack layer	size, median	stack agreement	flat heads at that size (±10%)	gap
0	0.31%	0.080	0.67 (2 heads)	~8×
1	0.16%	0.005	0.12 (1 head)	~23×
2	0.11%	0.004	0.19 (3 heads)	~49×
3	0.073%	0.003	**0.059** (115 heads)	~**19**×
4	0.039%	0.003	below the flat range	—

Residual depth thus reduces cross-seed reproducibility at matched head size across every layer. Although layer 0 receives the exact same input as the flat arm, its heads reproduce several times worse than size-matched flat heads: in a residual stack, the partition learned by a layer-0 head depends on how subsequent layers reconstruct its residual, so the input distribution alone no longer determines it. Deeper layers degrade even further. The same residual cascade that doubles the collateral flips of a single-head perturbation to 66% of other heads (Section 6.4) also leaves individual head partitions underdetermined by the data. Evaluated along the flat arm's size–agreement curve, the gain reduction from layer 0 to layer 3 accounts for roughly one third of layer 0's log-agreement advantage. Because nearly all flat-arm heads are small, only layer 3 has a dense size match (115 flat heads vs. 1–3 heads for layers 0–2), making this fraction approximate even though its direction is clear. This matched-size comparison is available only on SONAR, as no flat arm was trained on GPT-2.

Within layer 0 the atoms decide. Because SONAR's layer-0 input gain is identically 1, head-size variation within layer 0 reflects atom magnitude alone; there, log agreement correlates with atom norm at 0.62 and with atom spread at 0.64 (and raw agreement cosine correlates with log head size at 0.91), compared to 0.17 and 0.42 in GPT-2's layer 0. Beyond layer 1, atom magnitude has no predictive power on either dataset. Atom spread and atom norm are nearly identical across all layers (with ratios of 0.99–1.08), indicating that each head's usage-weighted mean decoded atom is close to zero and that the decoder cancels the positive offset of the non-negative dictionary before reaching output space.

E.12 Distribution of entry energy within each layer

An entry contributes to the reconstruction only on samples where its head selects it, writing the layer's scalar input gain times its decoded atom; its mean squared contribution over the data is therefore $\mathbb{E}\left[\mathbf{1}[\text{selected}]\,\text{gain}^2\right]\,\|\text{atom}\|^2$. Evaluating entry energy per layer across 32,768 held-out rows (`entryshare.py`) yields the breakdown in Table 20.

run	layer	Gini	top 10% carry	max/ median	from atom norm	within heads
GPT-2 concat, k=256	0	**0.587**	42%	38.7	76%	60%
	1	0.155	16%	30.3	44%	100%
	2–4	0.11–0.12	14%	∼2	∼40%	100%
SONAR stack, k=32	0	**0.326**	32%	46.3	92%	83%
	1–3	0.06–0.10	12–14%	1.5–1.9	16–24%	∼100%
	4	0.195	18%	3.2	77%	87%
SONAR flat, 1×380	—	0.190	23%	64.1	90%	76%

Table 20 Inequality of entry energy within each layer. "From atom norm" reports the covariance share of log entry-energy variance attributable to atom norm (the remainder comes from selection frequency, as input gain conditional on selection contributes ∼0). "Within heads" reports the fraction of variance not explained by head membership. No entry is dead in any layer.

Entry energy is nearly uniform across entries in layers 1–3, whereas layer 0 exhibits substantial inequality driven by atom norms. In layers 1–3, the effective number of entries reaches 95–99.5% of the nominal count and the largest entry carries roughly twice the median energy; the modest variation that remains comes primarily from selection frequency, which is itself nearly uniform (coefficient of variation 0.14–0.18). Layer 0 has similarly even selection frequencies—on SONAR, every entry fires on 2.1–4.5% of rows, compared to uniform $1/k = 3.1\%$—but its atom norms vary five to ten times as widely (CV 0.34–0.52 vs. 0.02–0.10 in deeper layers), predominantly within individual heads. The single-layer flat arm displays a milder version of the same pattern, indicating that atom-norm dispersion arises when classifying the unreduced input. By contrast, inequality in SONAR's layer 4 reflects the opposite tail: a subset of atoms shrinks nearly to zero (minimum-to-median norm ratio 0.017), consistent with the inert final-layer behavior analyzed in Section G.4. Across layers as a whole, total entry energy is dominated by input gain, with layer 0 accounting for 67% of total entry energy on GPT-2 and 49% on SONAR, compared to 4–5% in layer 4.

Distribution of large atoms in layer 0. The two substrates distribute their largest layer-0 atoms very differently across heads (`bigatoms.py`):

	GPT-2 concat, 20 heads	SONAR stack, 76 heads
within a head (atom norm / the head's own median, mean over heads)		
largest	3.7 (1.4–5.1 across heads)	2.0 (1.5–4.3 across heads)
2nd / 8th / 16th largest	3.2 / 2.0 / 1.7	1.7 / 1.2 / 1.0
largest entry's share of head energy	6.1% (uniform 0.4%)	9.9% (uniform 3.1%)
across the layer (atom norm / the layer median)		
atoms > 2×	1030 (20%), in all 20 heads	63 (2.6%), in 28 heads
atoms > 3×	58, in **all 20** heads (at most 15 each)	11, in **4** heads
atoms > 5×	1	5, **all in head 54**
largest head's energy share	15.5% (3× uniform)	9.2% (7× uniform)
usage		
Spearman(atom norm, usage) in heads	+0.81	+0.27 (−0.04 to +0.58)
usage of atoms > 3× / of the rest	0.0062 / 0.0037 ($1/k$ 0.0039)	0.033 / 0.031 ($1/k$ 0.031)
direction		
$\lvert\cos\angle\rvert$ between heads' largest atoms	0.022 (random pairs 0.048)	0.023 (random pairs 0.028)
energy in the data's top 10 PCs	4% (all atoms 13%)	9% (all atoms 3%)

On GPT-2, every layer-0 head exhibits a graded upper tail—its eighth-largest atom remains twice the head median—and large atoms are selected more frequently than average (1.6× uniform usage). Cross-seed agreement varies narrowly across heads (0.135–0.235) and correlates only moderately with atom statistics (Spearman correlation 0.49 with head energy share and 0.34 with largest-atom norm; Table 21). On SONAR, by contrast, large atoms are concentrated in a small subset of heads and fire at roughly the uniform rate. In particular, head 54 contains 8 of the 11 atoms exceeding 3× the layer median and all 5 exceeding 5×, accounts for 9% of total layer-0 energy, and achieves a cross-seed agreement of **0.844** vs. 0.249 for the second most reproducible head. Across SONAR's layer-0 heads, agreement tracks largest-atom norm (Spearman 0.62) and within-head peakedness (0.62) rather than median atom norm (−0.14). Thus, the within-layer correlation between atom size and agreement in SONAR's layer 0 is driven by a few heads with outlier atoms. However, outlier heads do not explain layer 0's overall advantage over deeper layers: excluding head 54 lowers the layer-0 mean agreement only from 0.080 to 0.070, which remains roughly 20× that of layers 1–4. Head 54 is the recurring topic and genre head analyzed in Section 6.6.

The two substrates therefore exhibit qualitatively different layer-0 regimes. On SONAR, a single outlier head dominates every atom metric and is the sole strongly reproducible head, while the remaining heads form a long, nearly flat tail whose agreement scales with within-head atom peakedness. On GPT-2, no individual head stands out: every head contains multiple large atoms, cross-seed agreement spans a narrow band, and the head carrying the largest share of layer-0 energy (head 17) has a flat profile of uniformly large atoms rather than a few outliers yet reproduces no better than neighboring heads; across GPT-2 heads, peakedness is uncorrelated with agreement (Spearman +0.09).

Across heads, these large atoms point along distinct directions: the largest atoms of different

head	agreement	energy share	largest / layer median	atoms > 3×	largest / head median
SONAR stack, layer 0 (uniform share 1.3%)					
54	**0.844**	9.2%	6.81	8	3.41
46	0.249	2.5%	3.21	1	2.24
6	0.181	1.5%	2.76	0	2.83
18	0.150	1.9%	4.50	1	4.34
lowest four	0.008–0.021	1.0–1.1%	1.51–1.84	0	1.48–1.84
GPT-2 concat, layer 0 (uniform share 5%)					
6	0.235	3.1%	4.31	2	4.77
17	0.220	15.5%	4.99	5	1.90
19	0.215	3.4%	3.82	3	4.16
0	0.202	3.1%	3.95	1	4.19
lowest four	0.135–0.141	2.6–3.0%	3.13–4.17	1–2	3.45–4.56

Table 21 Top and bottom layer-0 heads ranked by cross-seed agreement (`bigatoms.py`; matched centered contribution cosine against the seed-43 replica). Spearman correlations with cross-seed agreement across heads (SONAR / GPT-2): head energy share, +0.53 / +0.49; largest-atom norm, +0.62 / +0.34; within-head peakedness (largest / head median), +0.62 / +0.09; median atom norm, −0.14 / +0.16.

heads are nearly orthogonal, with median $|\cos \angle|$ of 0.022–0.023 (below that of random atom pairs). On SONAR, the largest atoms align disproportionately with the top ten principal components of the data (9% of atom energy vs. 3% for typical atoms), consistent with cross-seed agreement concentrating along high-variance data directions; on GPT-2, the reverse holds (4% vs. 13%). All comparisons in this subsection use one seed pair per dataset.

E.13 Probing source language across heads

Table 22 reports single-head NMI and sparse ridge-probe accuracy for held-out source-language identity across 86 languages (dense ridge-probe ceiling 0.721; chance accuracy 0.012).

As expected for a language-agnostic embedding space, no individual head encodes source language outright: the strongest head under the fixed initialization reaches 0.180 out of an attainable 0.875, and none of the auxiliary regularizers lifts any arm past 0.287 (moreover, a single k=32 head has at most $\log_2 32 = 5 < 6.43$ bits and cannot represent all 86 languages). Nevertheless, the learned partitions capture language well above chance, and the hard code retains substantially more language information than the softmax reference: 3.5–3.9× higher null-corrected best-head NMI (0.154–0.169 vs. 0.044) and higher 32-head probe accuracy (0.420–0.430 vs. 0.275), consistently across seeds. Relative to the unfixed initialization, the fixed initialization distributes language across more heads, lowering null-corrected best-head NMI (0.15–0.17 vs. 0.26) while raising 32-head probe accuracy (0.42–0.43 vs. 0.38). Under the unfixed initialization, the five-layer stack also outperforms the flat arm at every head budget (0.385 vs. 0.259 at h'=32).

Completeness. A ridge probe on the top 32 heads—all located in layer 0—reaches 0.42–0.43

arm	shape	best-head NMI	less null	probe h'=1	h'=32
`ste_h76_init01`	5×76	0.180	0.169	0.103	0.420
seed 43	5×76	0.165	0.154	0.089	0.430
softmax reference	5×32	0.055	0.044	0.036	0.275
unfixed initialization					
`ste_h76`	5×76	0.269	0.261	0.165	0.385
+ collinearity setpoint	5×76	0.287	0.279	0.139	0.317
− batch-mean KL_m	5×76	0.085	0.078	0.038	0.321
+ head independence	5×76	0.199	0.189	0.097	0.389
flat	1×380	0.277	0.276	0.102	0.259
+ collinearity setpoint	1×380	0.247	0.246	0.080	0.273

Table 22 Source-language probing across Ontologizer arms (`headlang.py`). Heads are ranked by NMI on the probe-training rows and evaluated on the held-out tail, with each probe's ridge penalty selected on a validation split of the training rows. Because a 5-bit head is compared against the 6.43-bit language label, the maximum attainable single-head NMI is 0.875. Auxiliary head-regularization ablations (including removing the batch-mean uniformity term KL_m) and the flat arm were evaluated only under the unfixed initialization.

accuracy, recovering 73–74% of the 0.57–0.58 accuracy achieved using all 380 heads. However, these 32 heads are not complete in the sense of the IOI circuit criterion of Wang et al. [125], under which removing a complete subset should leave the remainder of the code unable to perform the task: probing the remaining 348 heads still reaches 0.34–0.36, the remaining 44 layer-0 heads alone reach 0.26, and the 304 deeper-layer heads together reach 0.13 (vs. 0.03 for 32 random deeper heads). Language signal is therefore spread across layer 0 rather than concentrated in a few heads. (Although `summary.json`'s `info_share_best` metric appears to favor the flat arm, it normalizes by summed plug-in mutual information whose per-head finite-sample bias differs eightfold between the two architectures, making it unsuitable for cross-architecture comparison.)

Script, cells, and conjunctions. Writing-script identity exhibits the same pattern, with best-head NMI reaching 0.191 of a 0.671 ceiling and the same head (layer 0, head 50) ranking first for both language and script. Individual cells act as high-recall, permissive detectors: the best cell per language achieves recall of 0.74–0.98 at a false-positive rate of 1.6–2.2%, yet no language reaches 0.8 precision. Conjoining top cells from distinct heads does not yield sharp detectors: F1 peaks at h'=2 conjoined cells (7% above a single cell), and maximum precision reaches 0.598 at 4.8% recall. Whereas recall compounds nearly independently across heads (0.234 observed vs. 0.262 predicted under independence), false positives are strongly correlated (precision 0.43 at h'=2 vs. 0.83 under independence) because many heads assign the same confusable inputs to the same neighboring language (for example, grouping Finnish with Estonian or Japanese with Korean). Consequently, the code captures language *neighborhoods* rather than exact language identity.

Embedding geometry of language neighborhoods. This neighborhood structure reflects the residual language signal in SONAR's embedding space. Language accounts for 6.51% of total embedding variance and script for 2.26%. Adjusted for label cardinality (`headeta.py`), ground-truth language remains a stronger partition than 90% of learned heads (51× its random baseline, vs. 5× for the median head and 235× for the strongest head), but languages form weakly separated regions rather than distinct directions: mean pairwise cosine similarity is 0.3596 within a language vs. 0.3160 between languages, with the shared mean direction accounting for 0.3178 and leaving a within-language excess of +0.044 (0.30 standard deviations of the pairwise distribution). Because an unsupervised partition code splits along directions of local residual variance, cuts at a separation of 0.30 standard deviations pass through language clusters rather than around them. Meanwhile, layer-4 heads sit near chance (2× their random baseline), providing an independent view of the inert final layer analyzed in Section G.4.

Entries of the recurring topic head. Parameter decoding and autointerp were evaluated on layer-0 head 54 of the shipped-initialization run, which shares its index with the recurring topic head in the fixed-initialization run (Section 6.6). Decoded as deviations from the head's mean atom, its entries correspond to broad topics and document genres. Blinded contrastive autointerp (against a 0.273 null) groups the entries into four categories, all of which exceed the null: genre or topic (16 entries, F1 0.523), multilingual or code-switched passages (8 entries, F1 0.476), text artifacts such as duplication and mojibake (3 entries, F1 0.395), and script or region (5 entries, F1 0.394).

F Supplementary results: GPT-2 hard codes

F.1 Contribution of each design choice

Single-epoch runs on the 10M-token cache isolate the contribution of each design choice (reporting unwhitened FVU throughout this section):

variant (800 bits unless noted)	FVU	loss recovered
STE alone (latent dictionary + decoder)	0.327	93.3%
+ gain	0.290	94.3%
+ gain, concatenated heads	0.288	94.2%
+ gain, direct atoms	**0.217**	**96.95%**
seed 43	0.219	96.85%
k=64 (960 bits)	0.186	97.6%
k=128 (1120 bits)	0.165	97.9%

Both the gain–shape decomposition and direct atoms reduce reconstruction error, with direct atoms providing the largest single-epoch gain. Part of that advantage reflects faster convergence: over 40M tokens, concatenated heads close the gap and finish slightly ahead (FVU = 0.186 vs. 0.195). Without direct atoms and input gain, adding entries per head causes collapse rather than improvement (FVU = 0.75 at k=64 and 0.80 at k=128), and the other straight-through configurations froze their final layer at 4.8–5.0 bits with every head outputting a constant entry,

whereas the direct-atom model's deepest layer never froze. Across random seeds, the base (direct-atom) 800-bit result reproduces to within 0.002 in FVU. Temperature annealing is likewise critical: straight-through selection at a constant t=1 reaches only FVU = 0.50, and a scheduled soft-to-hard mix reaches 0.39. Finally, placing sparse encoders ahead of the classifier never outperforms the dense classifier: at one epoch, front-ends of width 32 and 640 reach FVU = 0.25–0.29 vs. 0.217 for the dense classifier, and over 40M tokens a 4096-wide, 64-active encoder per head with dead-latent resampling reaches 0.225 vs. 0.195 for the dense front-end, with 88% of its layer-1 latents going dead at no cost to that layer.

Layer-by-layer behavior of the 40M-token runs. Every layer contributes to reconstruction: prefix FVU decreases across layers as 0.472 → 0.347 → 0.278 → 0.231 → 0.195 for the direct-atom code and from 0.451 to 0.186 for concatenated heads. Entry usage remains nearly uniform in every layer (imbalance 0.01–0.16 bits out of 5, and 0.30 bits in the final layer of the concatenated-head code), and the observed residual gain closely approximates the optimal per-sample scale for each layer's contribution (optimal multiplier 0.89–1.09, coefficient of variation 0.05–0.08). Replacing the shared decoder in the 40M-token concatenated-head model with a separate decoder $W_{\mathrm{dec},\ell}$ per layer reduces held-out FVU from 0.186 to 0.169 (97.3% → 97.7% loss recovered, where the shared-decoder side is that model's twin trained with head dropout 0.1 and no winner dropout, which recovers 97.3% against the 97.5% of Table 6's row at the same FVU; prefix FVU of 0.449 → 0.312 → 0.244 → 0.199 → 0.169) and reduces cross-layer coupling between co-indexed heads from 97% to 6% (chance 3%). This confirms that the coupling was an artifact of projecting head i's slice through a single shared 768×48 decoder block across all five layers; because using five decoders multiplies decoder parameters by l=5, a width-matched shared-decoder control (d_{head}=240) remains to be run. Without per-head logit normalization (`logit_norm`), classifier logits also rescale during training to offset temperature annealing after the initial transient. Because these runs rely on two options not implemented on this branch (`resid_first` and `gain_clip`), they reproduce only on the `headline` branch, although direct atoms, concatenated heads, fibers, and head dropout have been ported.

G Supplementary results: dictionary geometry

G.1 Within-head atom collinearity is mostly gauge

The within-head atom cosine κ_i (`cossim_k`, `rowcos`; with $\bar{\kappa}$ denoting its mean across heads) is evaluated on `dicts()` in the e-dimensional dictionary space. Because the $d \times e$ decoder has full rank, $e - d$ dimensions of the dictionary space lie in $\ker(W_{\mathrm{dec}})$; in the SONAR straight-through arm, 74–78% of atom energy resides in that null space, where it neither affects the output nor receives reconstruction gradients. When evaluated on decoded atoms in output space, the same statistic drops from 0.12–0.29 in $\mathbb{R}^e$ to 0.023 in layer 0 and −0.015 to −0.020 in layers 1–4. Geometric dictionary statistics should therefore be computed after decoding into output space.

On GPT-2 layer 8 (e=1536, d=768), four candidate functional explanations for this null-space mass each failed empirical checks: null components across heads add coherently (3.3–11× above the expectation for canceling vectors), ruling out interference debris; the null fraction

does not track feature noise; the per-atom visible energy fraction stays flat at 22% throughout training while the decoder's row space rotates by only 2.6–3.5°, ruling out an active staging subspace; and achieving mixed-sign visible projections requires no non-negative null-space offset. Two baseline calculations also had to be corrected: for $|\mathcal{N}(0,1)|$ atoms, the chance visible energy fraction is 18.26% (not 50%) and the chance alignment with $\mathbf{1}$ is 63.7% (not ∼0), meaning that the trained dictionary is *less* null-heavy than chance; likewise, an apparent 49% share of the reconstruction gradient in $\ker(W_{\text{dec}})$ was a parameterization artifact that drops to 3×10^{-8} with respect to the non-negative atoms $|\mathsf{D}|$. Finally, the apparent share of the null-space component depends strongly on the choice of norm: 2.43% under squared L2, 23.20% under L2, and 56.55% under L1.

Positive span of a random decoder. Wendel's theorem [130] characterizes the probability that non-negativity restricts the output space at initialization. If the e decoder directions (columns of W_{dec}) are drawn from any sign-symmetric distribution in general position (including an i.i.d. Gaussian initialization), they lie in a common open half-space through the origin—so that the cone $\{W_{\text{dec}}\mathbf{r} : \mathbf{r} \geq 0\}$ misses directions in $\mathbb{R}^d$—with probability $2^{1-e} \sum_{q<d} \binom{e-1}{q}$. This probability equals 1 for a square decoder and transitions sharply through $1/2$ at $e{=}2d$ over a window of width $O(\sqrt{d})$ (for $d{=}768$, falling from 0.98 at $e{=}1456$ to 0.02 at $e{=}1616$). Both GPT-2's $e{=}1536$ and the SONAR softmax models' $e{=}2048$ sit directly at this $e{=}2d$ midpoint, though this calculation describes random initialization only.

G.2 Heads share coordinates: no partition

The disjoint-support argument in Section 4.1.4 relies on between-head orthogonality: if non-negative heads are orthogonal, they occupy disjoint coordinate blocks of the dictionary space as a static property of the weights. Because every model here was trained before the support-overlap statistic ω existed (most with its predecessor penalty active instead), we evaluate ω post hoc on their final checkpoints (`headsupport.py`; one seed per row):

model	ω, dictionary (null)	spread	ω, decoded (null)
SONAR `resid_nc` (372.6k steps)	0.818 (0.815)	0.83	0.917 (0.910)
SONAR `resid_nc_hm` (8.27M steps)	0.113 (0.118)	0.13	0.892 (0.878)
SONAR `softmax_shm`	0.731 (0.738)	0.74	0.938 (0.930)
SONAR `top2_shm`	0.862 (0.860)	0.87	0.972 (0.958)
SONAR straight-through, shipped init	0.942 (0.942)	0.94	0.721 (0.719)
SONAR straight-through, 5×76	0.962 (0.961)	0.96	0.980 (0.965)
SONAR straight-through, 1×380	0.977 (0.977)	0.98	0.974 (0.963)
GPT-2 abs, $e{=}1536$	0.739 (0.643)	0.65	0.948 (0.925)
GPT-2 abs, $e{=}768$	0.596 (0.486)	0.52	0.875 (0.852)
GPT-2 signed, $e{=}1536$	0.937 (0.636)	0.64	0.964 (0.943)
GPT-2 signed, $e{=}768$	0.955 (0.950)	0.95	0.965 (0.943)
GPT-2 concat, $d_{\text{head}}{=}32$	0 by construction	—	0.846 (0.788)
`g160top1` (160 trained heads)	—	—	0.648 (0.644)
`g160softmax` (160 trained heads)	—	—	0.983 (0.975)
`k32` discovered groups (359)	—	—	0.899 (0.892)

Across all summed models, no coordinate partition forms between heads. The support overlap ω sits at or above its coordinate-shuffled null everywhere, and each head's profile spans 52–98% of dictionary coordinates rather than the ∼3% required for a 32-way partition. Where ω departs from the null, it lies *above* it: the summed GPT-2 arms concentrate their heads onto the same coordinates, most strongly in the wide signed arm. The sole exception is `resid_nc_hm`, which trains a near-reference configuration (half the dictionary width; Table 9) 22× longer: its head profiles become sparse and ω dips 4% below the null in four of five layers. However, that reduction reflects individual head sparsification from extended training rather than a partition—at a spread of 0.13, each head still uses four times an equal coordinate share—and the reference model at 372.6k steps (`resid_nc`), like `softmax_shm` at `resid_nc_hm`'s width, remains dense and at its null. Thus, the retired predecessor penalty (s_{cossim_h}=10^{-6}, or 10^{-5} for `resid_nc`) did not induce a partition, and none of the models in the table trained with ω itself as a loss term. Concatenated heads remain the only route to disjoint support here, achieving it by construction.

This overlap is not confined to the decoder's null space. Decomposing each atom into its row-space (visible) and null-space projections and scoring each against the same null shows that visible-subspace overlap sits at chance on SONAR and above chance on GPT-2 (0.955 vs. 0.706 in the wide signed arm); moreover, the square-decoder GPT-2 arms (e=768), which have no null space, sit at or above the null directly in dictionary space. Because the decoder's row space is not coordinate-aligned, coordinate support of the visible part is basis-dependent; decoded ω provides the closest basis-free check and yields the same conclusion. Decoded ω itself, however, cannot separate architectures: every model's decoded atoms are dense across SONAR's output coordinates, and all fifteen rows—including trained and discovered SAE groups—fall within 7% of their nulls. The structural property that ω tests exists only in dictionary space, which the SAE variants lack.

Nothing in the training objective favors a coordinate partition. Mixing the dictionary coordinates by any invertible matrix T that preserves atom non-negativity while setting the decoder to $W_{\text{dec}}T^{-1}$ leaves the reconstruction unchanged, so a partitioned dictionary is only one of many achieving the same error. Prior results show that a coordinate partition between variables of a non-negative code is selected by a quadratic activity cost paired with a weight-scale penalty or norm bound [23, 24]. Our models include neither: all configurations train with Adam and zero weight decay (Appendix B.3), and apart from ω and its retired predecessor, no term compares two heads. Because features are non-negative, the feature L1 penalty sums additively over heads and assigns identical cost to mixed and modular codes [24], while cossim_b compares each head only with itself across samples (Eq. (3)). Even under a quadratic cost, two variables partition onto disjoint coordinates only when their joint range is sufficiently rectangular—roughly, when every combination of their extreme values occurs [24]. For categorical heads, that condition corresponds to every pair of entries across two heads co-occurring, which the SONAR data satisfy (`headrange.py`; one seed per model above). Among cells of two heads' winning-entry contingency table that would receive at least ten counts on the evaluation tail under independence, at most 0.24% are empty in any layer of a SONAR model (layer 0 of `sweep_top2_shm`), and most layers contain zero or a handful of empty cells, compared to $\sim 10^{-6}$ under a row-shuffle null. The exception is GPT-2's layer 0, where 2.6–3.4% of tested cells are empty (0.9% with

concatenated heads): because its heads track token class and position (Figure 18), two heads sensitive to token class can assign mutually exclusive entries that never co-occur. Deeper GPT-2 layers cover their joint range as completely as SONAR's do. Where entry pairs exclude each other, their atoms overlap slightly less, rather than more, than those of co-occurring pairs at matched expected counts. On SONAR, the empirical joint range is therefore no obstacle to a partition; what is missing is an energetic incentive to form one.

Directly penalizing ω. Adding ω to the loss does induce a dictionary-space partition, but only in directions that do not affect the reconstruction. On GPT-2 with a signed dictionary (e=1536; three epochs, one seed per penalty weight), dictionary-space ω falls below its null starting at s_ω=10^{-4} (0.61× the null, and 0.15× at s_ω=10^{-3}), at the cost of increasing held-out FVU_w by 10%, 21%, and 43% at s_ω=10^{-5}, 10^{-4}, and 10^{-3}, respectively. In decoded output space, however, overlap does not improve: every penalized arm remains 8–10% above its null, matching the unpenalized baseline. The penalized arms move 73–95% of atom energy into $\ker(W_{\mathrm{dec}})$, giving each head a few large, disjoint coordinates in the null space while leaving the visible projection dense and increasingly overlapping as s_ω grows. A square decoder (e=768) produces the same behavior to within a few percent by developing an effective near-null space instead (condition number 1.0×10^4 unpenalized vs. 5.1×10^5 at s_ω=10^{-3}), with each penalized head placing nearly all of its energy in directions that the decoder shrinks tenfold or more. Penalizing ω on dictionary weights can thus be bypassed through any learned decoder, mirroring the disconnect between regularized and downstream representations documented by Locatello et al. when total-correlation penalties on sampled latents increase the correlation of the mean code [11, §5.1].

G.3 Geometry of concatenated heads

With concatenated heads at d_{head}=32, each head projects to output space through its own 768×32 decoder block: 32 non-negative generators in a 32-dimensional subspace form a simplicial cone with zero per-head null space. Although the geometric lower bound on a head's mean pairwise atom cosine is $(k/d_{\mathrm{head}} - 1)/(k - 1)$, which equals 0 at d_{head}=k=32, achieving κ_i=0 when d_{head}=k requires dedicating one coordinate per atom. Rather than facing a raised cosine floor, the d_{head}=32 concatenated arm faces a rigid expressivity tradeoff. Optimization favored correlated atoms, yielding the highest $\bar{\kappa}$, the lowest effective and numerical rank (rank99 of 17.1 out of 32, vs. 30.2 for summed heads), and the worst reconstruction error among the orthant arms (Table 7). This penalty reflects cone reachability rather than $\bar{\kappa}$ alone: at step 90k, the concatenated arm had slightly lower $\bar{\kappa}$ than `abs/768` (0.571 vs. 0.578) yet substantially higher held-out FVU_w (0.806 vs. 0.463). In the `headline`-branch GPT-2 runs (Section 7), widening each head's slice beyond its entry count (d_{head}=48 > k=32) and annealing classifier temperature relieves this bottleneck, allowing concatenated heads to reach an unwhitened FVU of 0.186 (and 0.164 at k=64), outperforming the direct-atom model's 0.195.

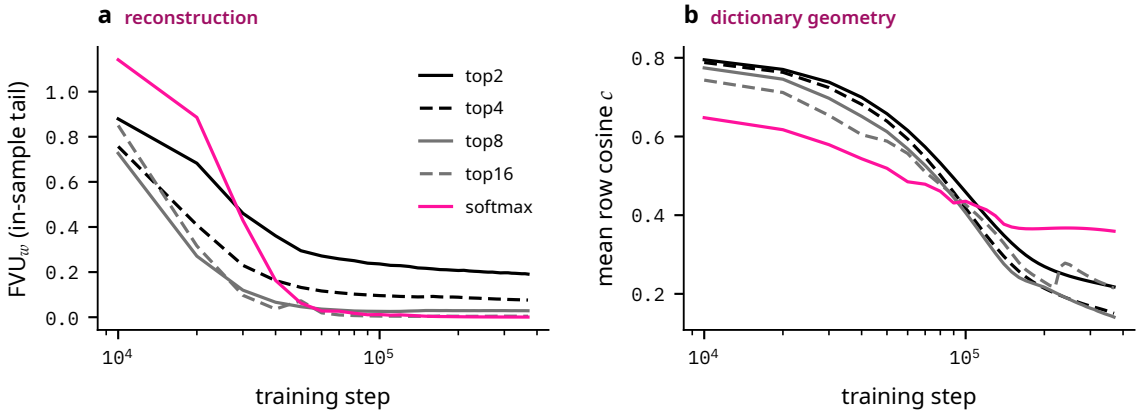


Figure 33 Training trajectories of the `_shm` arms in the SONAR selection sweep: (a) FVU_w on the (in-sample) evaluation tail, scored from each checkpoint, and (b) mean within-head atom cosine $\bar{\kappa}$. The soft arm's $\bar{\kappa}$ flattens near 0.36 while its error continues to fall, whereas the harder arms continue decorrelating their atoms.

G.4 Selection rule, training duration, and final-layer activity

Softer codes do not raise the dictionary's rank. Across all ten arms of the SONAR selection sweep, per-head numerical rank is full (rank99 of 30.7–31.8 out of 32), while effective rank varies sevenfold without varying monotonically with selection softness; instead, the two sweep families (with and without the batch-mean uniformity term KL_m) order the selection rules oppositely at convergence. Tracking each arm over training resolves this reversal: training duration outweighs the selection rule by 3.8× (mean $\bar{\kappa}$ falls from 0.750 to 0.217 between steps 10k and 369.5k, whereas the spread across selection rules at a fixed step averages 0.140), and the within-family orderings cross near step 100k (Figure 33b, showing the `_shm` family). Late in training, the soft arm's $\bar{\kappa}$ plateaus while its reconstruction error continues to fall, whereas the harder arms do the reverse: between steps 150k and 369.5k, `softmax` reduces FVU_w by 85% while $\bar{\kappa}$ changes by only 3%, whereas `top8` changes FVU_w by 3% while reducing $\bar{\kappa}$ by 45%, `top16` leaves FVU_w unchanged while reducing $\bar{\kappa}$ by 29%, and `top2` reduces both by 12% and 34%, respectively (Figure 33). Under a hard code, a head's output *is* one of its atoms, so collinear atoms make the output insensitive to which entry wins; atom collinearity therefore directly costs a hard code capacity, whereas continuous mixture weights allow a soft code to compensate. At convergence, the softest code has the *most* collinear dictionary. Comparing arms at matched FVU_w rather than matched training step reverses that ordering at moderate error: `softmax` has the lowest $\bar{\kappa}$ at $\mathrm{FVU}_w = 0.60$ and 0.40 (0.60 and 0.58, vs. 0.72–0.78 for the harder arms). Only at $\mathrm{FVU}_w = 0.25$ does `top2` drop below `softmax` (0.53 vs. 0.56), and because `top2` reaches that error at step 80k vs. steps 21k–36k for the wider rules, its lower $\bar{\kappa}$ there again reflects training duration. Each arm uses one seed; because these runs trained on the evaluation tail, their FVU_w is in-sample, although the softmax arm's FVU_w changes by only 0.13% on freshly encoded rows that no model saw during training (Appendix B.1).

Inertness of the final layer at low residual error. Whereas within-head atom collinearity decreases monotonically with depth in every hard arm, it spikes in the final layer of the soft arms (reaching $\bar{\kappa}$=0.799 and effective rank 1.54 in layer 4 of `softmax_shm`). To separate collinearity from exhausted residual variance, `lastlayer.py` ablates the final layer's heads in two ways—uniform ablation (emitting each head's mean atom) and zero ablation (emitting zero):

arm	select	L4 $\bar{\kappa}$	L4 eff	prefix FVU_w before L4	L4 adds	zero-ablate
top1	top1	+0.246	10.60	0.7511	+4.3%	+4.5% worse
top2_shm	top2	+0.188	14.89	0.2378	+19.7%	+24.5% worse
top8_shm	top8	—	—	0.0304	+5.5%	+5.8% worse
top16_shm	top16	+0.433	3.46	0.0043	+0.0%	+0.0%
softmax	softmax	+0.263	6.00	0.0023	−1.2%	−1.1% better
softmax_shm	softmax	+0.799	1.54	0.0007	−1.9%	−1.8% better

Whether the final layer helps depends on the residual error entering it rather than on its atom collinearity: layer 4 of `softmax` is not collapsed ($\bar{\kappa}$=0.263) yet is equally unhelpful. Once prefix FVU_w before layer 4 falls below roughly 0.005, the final layer becomes inert or slightly harmful, which accounts for the mildly harmful layer 4 of `resid_nc` (prefix $\mathrm{FVU}_w = 0.0051$) without invoking its frozen heads. Nevertheless, the two low-error regimes end differently: in `top16_shm`, layer 4 shrinks its atom norms to 4% of those in earlier layers and emits virtually nothing, whereas in `softmax_shm`, layer 4 retains comparable atom magnitudes that become highly collinear, emitting a nearly constant vector whose removal slightly reduces reconstruction error (reflecting mild overfitting rather than a useful bias).

Comparison against an unpartitioned kernel smoother. The leave-one-out noise2self partition score corresponding to $\mathrm{L2}_{\mathrm{pwak}}$ can be evaluated post hoc on the selection sweep (Figure 34, `selection.py`), measuring how accurately a sample's co-classified neighbors predict its layer input relative to the same heat-kernel smoother with all samples placed in a single cell (none of the sweep arms trained with $\mathrm{L2}_{\mathrm{pwak}}$ active). Across all arms, gating by a head's partition improves over that smoother by at most 0.020 in layer 0 (`top4_shm`), by at most 0.005 in layers 1–3, and by 0.031 in layer 4 only for `softmax` (where layer 4 is already inert). A random partition with matched cell sizes never improves over the smoother, and a single-entry code (`top1`) performs worse than the smoother above layer 0. Because this leave-one-out score is not $\mathcal{J}$-invariant (Appendix B), the row-permuted partition serves as a floor only for partitions that do not depend on the input: any partition computed from the input—whether trained or not—can improve on the smoother without clustering genuine modes. Each arm uses one seed and was trained on the evaluation tail.

$\mathcal{J}$-invariant evaluation and random-projection baselines. Evaluating the coordinate-split $\mathcal{J}$-invariant estimator (Appendix B) across all 76 heads per layer of `ste_h76_init01` resolves both the bandwidth and self-leakage confounds (`selection.py --jinv`; one seed). Bandwidth selection under the split estimator chooses 0.05 in layers 0 and 4 and 0.03 in layers 1–3; in

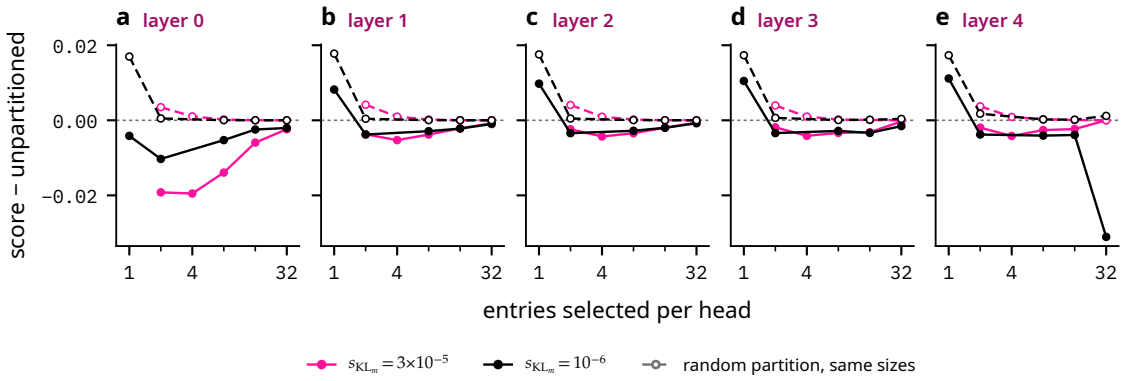


Figure 34 **Learned partitions barely improve over an unpartitioned smoother.** Leave-one-out noise2self score per layer across the SONAR selection sweep as a function of the number of selected entries per head (1 for argmax, 32 for softmax), shown relative to the same run's score when all rows are placed in a single cell (values below 0 indicate an improvement from partitioning), both with and without the batch-mean uniformity term KL_m. Filled markers show mean scores across learned head partitions; hollow markers with dashed lines show row-permuted assignments (random partitions with matched cell sizes). Evaluated with a bandwidth-0.2 heat kernel on each layer's input in batches of 2,048 rows; one seed per arm on the in-sample tail.

layer 0, narrowing the unpartitioned smoother's bandwidth from Figure 34's 0.2 to 0.05 lowers its error by 0.112, exceeding the largest gain achieved by any single head or multi-head gate at bandwidth 0.2 (at most 0.060). Furthermore, a random-projection head acts as an angular locality-sensitive hash [131]: because the probability that it places two rows in the same cell decreases as the angle between them grows, gating a wide heat kernel by random projections sharpens the kernel much like reducing its bandwidth. At bandwidth 0.2, a single random-projection head improves over the smoother by 0.017 under the leave-one-out score but by −0.001 (no gain) under the split estimator, showing that its single-head gain came from self-leakage, whereas the joint gate of eight random heads and the random collision kernel retain gains of 0.042 and 0.047 because they sharpen an over-wide kernel. At the selected bandwidth, single trained heads score worse than the unpartitioned smoother in every layer (by 0.040 in layer 0, 0.065–0.069 in layers 1–3, and 0.030 in layer 4), matching single random-projection heads (0.030 and 0.063–0.068). Likewise, the joint gate across a layer's trained heads improves over the smoother by 0.034 in layer 0 and 0.005–0.006 in layers 1–4, closely tracked by the random-projection collision kernel (0.039 and 0.005–0.009). Because trained heads depend on all input coordinates, comparisons involving trained heads use the leave-one-out estimator; relative to random-projection heads, learned partitions exhibit no structure beyond angular locality. On isotropic inputs, the coordinate-split estimator maintains its null floor down to a bandwidth of 0.05, whereas the leave-one-out estimator drops below the floor by 0.010 at bandwidth 0.2 and by 0.090 at bandwidth 0.02, quantifying its self-prediction leak.

G.5 Bilinear classifier geometry from model weights

Each classifier logit is computed by a single bilinear factor pair $(\mathbf{w} \cdot \mathbf{u})(\mathbf{v} \cdot \mathbf{u})$, so the factor alignment $|\cos \angle(\mathbf{w}, \mathbf{v})|$ characterizes the quadratic form directly. When $|\cos \angle(\mathbf{w}, \mathbf{v})|$ is near 1, the form approaches rank 1 and is proportional to $(\mathbf{w} \cdot \mathbf{u})^2$, which is even in $\mathbf{u}$ and therefore invariant to flipping the input's sign along $\mathbf{w}$. When $|\cos \angle(\mathbf{w}, \mathbf{v})|$ is near 0, the form defines an indefinite saddle that distinguishes whether its two projections share the same sign, yet remains equally even on the semantic coordinates because $(\mathbf{w} \cdot (-\mathbf{u}))(\mathbf{v} \cdot (-\mathbf{u})) = (\mathbf{w} \cdot \mathbf{u})(\mathbf{v} \cdot \mathbf{u})$. Thus, rank changes the shape of the region to which a feature responds rather than its sign symmetry, which only the appended constant coordinate can break (see Appendix B for protocol details). Figure 35 evaluates this alignment directly from the weights of the softmax stack (`resid_nc`) and the hard-code stack (`ste_h76_init01`).

The softmax stack's classifiers converge toward rank-1 forms in intermediate layers, whereas the hard-code stack's classifiers mostly do not. At the final checkpoint, median $|\cos \angle|$ in the softmax stack reaches 0.94 in layer 1 and 0.52–0.80 in layers 2–3, whereas layers 1–3 of the hard-code stack remain at 0.10–0.16 (compared to 0.021 for random vector pairs). Layer 0 reaches 0.54–0.64 in both models, while layer 4 of the softmax stack falls to 0.18. Factor alignment develops relatively late in training: every layer starts at the random floor, with the softmax stack departing between 30k and 50k steps as temperature annealing finishes, and the hard-code stack departing only after 80k steps in layer 0 and 220k steps in layers 3–4 (with layers 1–2 exceeding 0.05 only at the final checkpoint). The dominant eigenvalue is negative for 63–98% of features in the softmax stack and 97–99% in the hard-code stack, meaning that a rank-1 feature takes its largest values where the input is orthogonal to $\mathbf{w}$ rather than aligned with it—implementing the quadratic invariances that bilinear layers learn in place of constants [15]. Finally, the constant coordinate can break the sign symmetry of a rank-1 feature for inputs weakly aligned with $\mathbf{w}$: across layers 1–4, its weight magnitude equals 0.18–0.24 of the norm of the remaining coordinates of $\mathbf{w}$ in the softmax stack, compared to 0.017–0.025 in the hard-code stack. This analysis uses one seed per run and inspects model weights only.

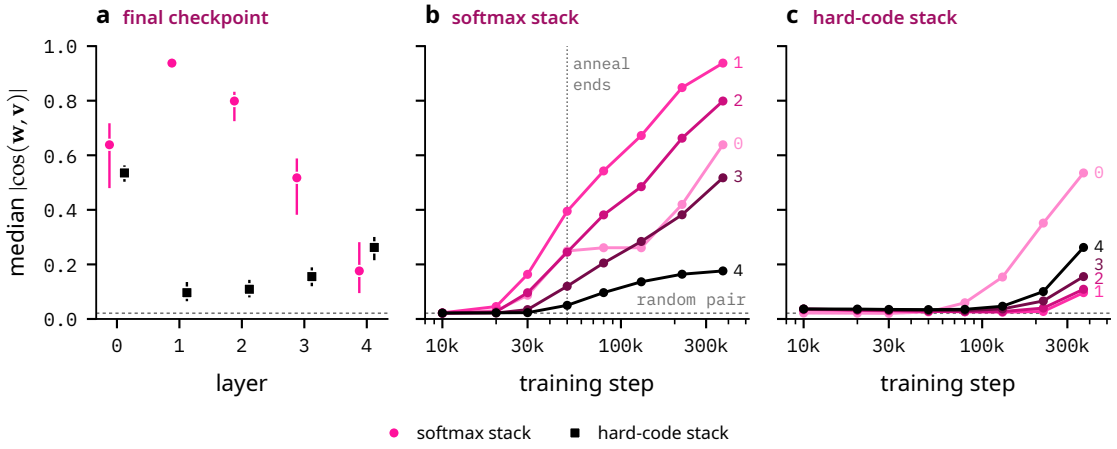


Figure 35 **Bilinear classifiers approach rank 1 in the softmax stack but remain mostly rank 2 in the hard-code stack.** Median factor alignment $|\cos \angle(\mathbf{w}, \mathbf{v})|$ across bilinear feature pairs per layer (`bilinspec.py`; 1 indicates a rank-1 feature). (a) Final-checkpoint medians and interquartile ranges by layer for the softmax stack (`resid_nc`, step 372.6k) and the hard-code stack (`ste_h76_init01`, step 369.5k). (b, c) Evolution over training, with one curve per layer labeled at its right endpoint; temperature annealing in the softmax stack ends at step 50k, whereas the hard-code stack uses a constant temperature. Dashed lines mark the median for a random pair in the 1024-dimensional semantic block (0.021). One seed per run.